MDN JavaScript Documentation

Volume 5 of 9 — Built-in objects: decodeURI() – Iterator

Unofficial print compilation. This volume contains 179 MDN articles, reproduced without changes to their wording.

Source, attribution and licence

All articles in this volume are from **MDN Web Docs**, `https://developer.mozilla.org/en-US/docs/Web/JavaScript`, retrieved on **2026-11-10** from MDN's rendered pages. They are written **by Mozilla Contributors** and licensed under the Creative Commons Attribution-ShareAlike licence, **CC-BY-SA 2.5** or any later version (`https://creativecommons.org/licenses/by-sa/2.5/`). The title and the URL of the specific MDN page are printed at the head of every article; the list of authors of a page is available through that page on MDN. Code samples added to MDN on or after 20 August 2010 are in the public domain (**CC0**, `https://creativecommons.org/publicdomain/zero/1.0/`); older samples are under the MIT licence. Browser compatibility data is from `@mdn/browser-compat-data` 8.1.8 (CC0).

Modifications. This is not an official MDN or Mozilla publication and is not endorsed by Mozilla. The content was re-laid-out for A4 paper: website navigation, banners and interactive widgets were removed or replaced by short print notes; compatibility and specification tables, which the website generates from data, were re-rendered from the same data; in-volume links were turned into PDF links. Wording, code and section order were not edited. Like the original, this compilation is distributed under CC-BY-SA 2.5 or later. “MDN” and “Mozilla” are trademarks of the Mozilla Foundation; its logos and website design are not part of the licensed content and are not used here.

How to read this book

- **Text.** Article wording, headings, section order, notes, tables and code are MDN's, taken programmatically from MDN's rendered pages. Only the layout was changed for paper.
- **Dashed boxes in italics starting with ▷** and the small grey lines under each article title (breadcrumb, source URL, availability) are additions of this print edition, not MDN text.
- **Code** is printed character for character in a monospaced font. A line longer than about 92 characters wraps at the right edge and continues at the left edge of the code box; no characters are added. Labels above a box give the language; **✗ bad example** and **✓ good example** replace MDN's red/green colouring. “code continues” / “code continued” in the page margin mark a code box that runs over a page break.
- **Interactive examples and live samples** cannot run on paper. Their source code is printed; output is shown only where MDN's source states it (for example in `// Expected output` comments).
- **Links.** Dotted underlines mark links. In the PDF, links to articles in this volume jump to them; all other links open the MDN page. Web API, DOM, HTML/CSS, Glossary and Learn pages are outside the scope of this collection (only `/en-US/docs/Web/JavaScript/` is included) and remain links to MDN.
- **Badges** such as Deprecated Experimental Non-standard Optional reproduce MDN's status icons and labels as text.
- **Browser compatibility tables** are rebuilt from MDN's browser-compat-data (the data the website's tables are generated from). Each cell gives the first browser version that supports the feature. **No** (grey cell) = not supported; **?** = unknown; **a–b** = supported from version a, removed in version b (grey cell); **Prev.** = only in a preview release of that browser; ≤ = that version or earlier; small raised numbers refer to the numbered notes below the table (flags, prefixes, partial support and other remarks).
- **“Baseline” availability lines** are condensed from the Baseline status MDN shows at the top of a page.

Contents

decodeURI() 5
decodeURIComponent() 8
DisposableStack 11
DisposableStack() constructor 13
DisposableStack.prototype.adopt() 14
DisposableStack.prototype.defer() 16
DisposableStack.prototype.dispose() 18
DisposableStack.prototype.move() 20
DisposableStack.prototype.use() 24
DisposableStack.prototype[Symbol.dispose]() 25
DisposableStack.prototype.disposed 27
encodeURI() 28
encodeURIComponent() 32
Error 35
Error() constructor 41
Error.captureStackTrace() 43
Error.isError() 46
Error.stackTraceLimit 48
Error.prototype.toString() 50
Error.prototype.name 52
Error.prototype.stack 53
Error: cause 56
Error: columnNumber 59
Error: fileName 60
Error: lineNumber 61
Error: message 63
escape() 65
eval() 67
EvalError 78
EvalError() constructor 79
FinalizationRegistry 82
FinalizationRegistry() constructor 86
FinalizationRegistry.prototype.register() 87
FinalizationRegistry.prototype.unregister() 89
Float16Array 93
Float16Array() constructor 95
Float32Array 97
Float32Array() constructor 99
Float64Array 101
Float64Array() constructor 103
Function 105
Function() constructor 108
Function.prototype.apply() 112
Function.prototype.bind() 116
Function.prototype.call() 123
Function.prototype.toString() 126
Function.prototype[Symbol.hasInstance]() 130
Function.prototype.arguments 132
Function.prototype.caller 134
Function: displayName 137
Function: length 139
Function: name 141
Function: prototype 147
Generator 151
Generator.prototype.next() 153
Generator.prototype.return() 155
Generator.prototype.throw() 158
GeneratorFunction 161
GeneratorFunction() constructor 163
GeneratorFunction.prototype.prototype 164
globalThis 168
Infinity 171
Int8Array 173
Int8Array() constructor 175
Int16Array 177
Int16Array() constructor 179
Int32Array 181
Int32Array() constructor 183
InternalError 185
InternalError() constructor 187
Intl 189
Intl.getCanonicalLocales() 200
Intl.supportedValuesOf() 202
Intl.Collator 213
Intl.Collator() constructor 216
Intl.Collator.supportedLocalesOf() 219
Intl.Collator.prototype.compare() 221
Intl.Collator.prototype.resolvedOptions() 223
Intl.DateTimeFormat 226
Intl.DateTimeFormat() constructor 231
Intl.DateTimeFormat.supportedLocalesOf() 241
Intl.DateTimeFormat.prototype.format() 243
Intl.DateTimeFormat.prototype.formatRange() 245
Intl.DateTimeFormat.prototype.formatRangeToParts() 248
Intl.DateTimeFormat.prototype.formatToParts() 250
Intl.DateTimeFormat.prototype.resolvedOptions() 255
Intl.DisplayNames 259
Intl.DisplayNames() constructor 262
Intl.DisplayNames.supportedLocalesOf() 265
Intl.DisplayNames.prototype.of() 266
Intl.DisplayNames.prototype.resolvedOptions() 269
Intl.DurationFormat 272
Intl.DurationFormat() constructor 274
Intl.DurationFormat.supportedLocalesOf() 278
Intl.DurationFormat.prototype.format() 280
Intl.DurationFormat.prototype.formatToParts() 283
Intl.DurationFormat.prototype.resolvedOptions() 285
Intl.ListFormat 287
Intl.ListFormat() constructor 290
Intl.ListFormat.supportedLocalesOf() 294
Intl.ListFormat.prototype.format() 296
Intl.ListFormat.prototype.formatToParts() 298
Intl.ListFormat.prototype.resolvedOptions() 300
Intl.Locale 302

Intl.Locale() constructor 307
Intl.Locale.prototype.getCalendars() 309
Intl.Locale.prototype.getCollations() 311
Intl.Locale.prototype.getHourCycles() 313
Intl.Locale.prototype.getNumberingSystems() 315
Intl.Locale.prototype.getTextInfo() 316
Intl.Locale.prototype.getTimeZones() 318
Intl.Locale.prototype.getWeekInfo() 320
Intl.Locale.prototype.maximize() 322
Intl.Locale.prototype.minimize() 324
Intl.Locale.prototype.toString() 326
Intl.Locale.prototype.baseName 328
Intl.Locale.prototype.calendar 329
Intl.Locale.prototype.caseFirst 330
Intl.Locale.prototype.collation 332
Intl.Locale.prototype.hourCycle 333
Intl.Locale.prototype.language 335
Intl.Locale.prototype.numberingSystem 336
Intl.Locale.prototype.numeric 338
Intl.Locale.prototype.region 339
Intl.Locale.prototype.script 341
Intl.Locale.prototype.variants 342
Intl.NumberFormat 344
Intl.NumberFormat() constructor 349
Intl.NumberFormat.supportedLocalesOf() 368
Intl.NumberFormat.prototype.format() 370
Intl.NumberFormat.prototype.formatRange() 373
Intl.NumberFormat.prototype.formatRangeToParts() 375
Intl.NumberFormat.prototype.formatToParts() 377
Intl.NumberFormat.prototype.resolvedOptions() 380
Intl.PluralRules 385
Intl.PluralRules() constructor 389
Intl.PluralRules.supportedLocalesOf() 392
Intl.PluralRules.prototype.resolvedOptions() 393
Intl.PluralRules.prototype.select() 396
Intl.PluralRules.prototype.selectRange() 398
Intl.RelativeTimeFormat 400
Intl.RelativeTimeFormat() constructor 403
Intl.RelativeTimeFormat.supportedLocalesOf() 405
Intl.RelativeTimeFormat.prototype.format() 407
Intl.RelativeTimeFormat.prototype.formatToParts() 409
Intl.RelativeTimeFormat.prototype.resolvedOptions() 411
Intl.Segmenter 414
Intl.Segmenter() constructor 416
Intl.Segmenter.supportedLocalesOf() 418
Intl.Segmenter.prototype.resolvedOptions() 420
Intl.Segmenter.prototype.segment() 422
Segments 424
Segments.prototype.containing() 425
Segments.prototype[Symbol.iterator]() 427

isFinite() **430**

isNaN() **432**

Iterator **435**
Iterator() constructor 442
Iterator.concat() 445
Iterator.from() 449
Iterator.zip() 452
Iterator.zipKeyed() 457
Iterator.prototype.chunks() 460
Iterator.prototype.drop() 462
Iterator.prototype.every() 464
Iterator.prototype.filter() 467
Iterator.prototype.find() 469
Iterator.prototype.flatMap() 471
Iterator.prototype.forEach() 475
Iterator.prototype.includes() 476
Iterator.prototype.join() 479
Iterator.prototype.map() 481
Iterator.prototype.reduce() 483
Iterator.prototype.some() 485
Iterator.prototype.take() 487
Iterator.prototype.toArray() 490
Iterator.prototype.windows() 492
Iterator.prototype[Symbol.dispose]() 494
Iterator.prototype[Symbol.iterator]() 496

JavaScript › Reference › Standard built-in objects

decodeURI()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/decodeURI · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`decodeURI()`** function decodes a Uniform Resource Identifier (URI) previously created by `encodeURI()` or a similar routine.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: decodeURI()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const uri = "https://mozilla.org/?x=шеллы";
const encoded = encodeURI(uri);
console.log(encoded);
// Expected output: "https://mozilla.org/?x=%D1%88%D0%B5%D0%BB%D0%BB%D1%8B"

try {
  console.log(decodeURI(encoded));
  // Expected output: "https://mozilla.org/?x=шеллы"
} catch (e) {
  // Catches a malformed URI
  console.error(e);
}
```

Syntax

js

```
decodeURI(encodedURI)
```

Parameters

`encodedURI`

A complete, encoded Uniform Resource Identifier.

Return value

A new string representing the unencoded version of the given encoded Uniform Resource Identifier (URI).

Exceptions

`URIError`

Thrown if `encodedURI` contains a `%` not followed by two hexadecimal digits, or if the escape sequence does not encode a valid UTF-8 character.

Description

`decodeURI()` is a function property of the global object.

The `decodeURI()` function decodes the URI by treating each escape sequence in the form `%XX` as one UTF-8 code unit (one byte). In UTF-8, the number of leading 1 bits in the first byte, which may be 0 (for 1-byte ASCII characters), 2, 3, or 4, indicates the number of bytes in the character. So by reading the first escape sequence, `decodeURI()` can determine how many more escape sequences to consume. If

`decodeURI()` fails to find the expected number of sequences, or if the escape sequences don't encode a valid UTF-8 character, a `URIError` is thrown.

`decodeURI()` decodes all escape sequences, but if the escape sequence encodes one of the following characters, the escape sequence is preserved in the output string (because they are part of the URI syntax):

```
; / ? : @ & = + $ , #
```

Examples

Decoding a Cyrillic URL

```js
decodeURI(
  "https://developer.mozilla.org/ru/docs/JavaScript_%D1%88%D0%B5%D0%BB%D0%BB%D1%8B",
);
// "https://developer.mozilla.org/ru/docs/JavaScript_шеллы"
```

decodeURI() vs. decodeURIComponent()

`decodeURI()` assumes the input is a full URI, so it does not decode characters that are part of the URI syntax.

```js
decodeURI(
  "https://developer.mozilla.org/docs/JavaScript%3A%20a_scripting_language",
);
// "https://developer.mozilla.org/docs/JavaScript%3A a_scripting_language"

decodeURIComponent(
  "https://developer.mozilla.org/docs/JavaScript%3A%20a_scripting_language",
);
// "https://developer.mozilla.org/docs/JavaScript: a_scripting_language"
```

Catching errors

```js
try {
  const a = decodeURI("%E0%A4%A");
} catch (e) {
  console.error(e);
}

// URIError: malformed URI sequence
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/global-object.html#sec-decodeuri-encodeduri

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`decodeURI`	1	12	1	7	1.1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.decodeURI`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `decodeURIComponent()`
- `encodeURI()`
- `encodeURIComponent()`

JavaScript › Reference › Standard built-in objects

decodeURIComponent()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/decodeURIComponent · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`decodeURIComponent()`** function decodes a Uniform Resource Identifier (URI) component previously created by `encodeURIComponent()` or by a similar routine.

Try it

▷ Interactive example on MDN: "JavaScript Demo: decodeURIComponent()". It runs live in the browser there; its source code follows.

js · interactive example source

```
function containsEncodedComponents(x) {
  // ie ?,=,&,/ etc
  return decodeURI(x) !== decodeURIComponent(x);
}

console.log(containsEncodedComponents("%3Fx%3Dtest")); // ?x=test
// Expected output: true

console.log(containsEncodedComponents("%D1%88%D0%B5%D0%BB%D0%BB%D1%8B")); // шеллы
// Expected output: false
```

Syntax

js

```
decodeURIComponent(encodedURI)
```

Parameters

`encodedURI`

An encoded component of a Uniform Resource Identifier.

Return value

A new string representing the decoded version of the given encoded Uniform Resource Identifier (URI) component.

Exceptions

`URIError`

Thrown if `encodedURI` contains a `%` not followed by two hexadecimal digits, or if the escape sequence does not encode a valid UTF-8 character.

Description

`decodeURIComponent()` is a function property of the global object.

`decodeURIComponent()` uses the same decoding algorithm as described in `decodeURI()`. It decodes *all* escape sequences, including those that are not created by `encodeURIComponent`, like `-.!~*'()`.

Examples

Decoding a Cyrillic URL component

```js
decodeURIComponent("JavaScript_%D1%88%D0%B5%D0%BB%D0%BB%D1%8B");
// "JavaScript_шеллы"
```

Catching errors

```js
try {
  const a = decodeURIComponent("%E0%A4%A");
} catch (e) {
  console.error(e);
}

// URIError: malformed URI sequence
```

Decoding query parameters from a URL

`decodeURIComponent()` cannot be used directly to parse query parameters from a URL. It needs a bit of preparation.

```js
function decodeQueryParam(p) {
  return decodeURIComponent(p.replace(/\+/g, " "));
}

decodeQueryParam("search+query%20%28correct%29");
// 'search query (correct)'
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/global-object.html#sec-decodeuricomponent-encodeduricomponent`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`decodeURIComponent`	1	12	1	7	1.1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.decodeURIComponent`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `decodeURI`
- `encodeURI`
- `encodeURIComponent`

JavaScript › Reference › Standard built-in objects

DisposableStack

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`DisposableStack`** object represents a stack of disposers to run when the stack itself is disposed. Disposer functions are executed in reverse order of registration, with strong error handling guarantees. Calling its `move()` method will transfer responsibility for calling the current registered disposers to a new `DisposableStack` and prevent registering any additional disposers.

Description

A `DisposableStack` is not exactly a "stack" in terms of its interface. It has several methods for pushing disposers to it, but it has no way to pop one disposer off. Rather, *all* disposers are popped and executed one-by-one when the stack is disposed.

You register disposable resources to the `DisposableStack` using its `use()`, `adopt()`, or `defer()` methods.

```js
using disposer = new DisposableStack();
const reader = disposer.use(stream.getReader());
```

Then, when the `disposer` goes out of scope, all resources registered to it are disposed in reverse order of registration, unless they have been moved out with `move()`.

It is good practice to *not* extract the resource acquisition expression to a separate statement, no matter how long the expression is. You should always wrap the `use()` or `adopt()` call around the resource acquisition expression to ensure that the resource is registered to the stack immediately.

js · ✗ bad example

```
using disposer = new DisposableStack();
const reader = stream.getReader();
disposer.use(reader);
```

Functionally, these two code snippets are equivalent. However, the first one is less error-prone because the resource is declared and registered in a single line. If someone puts more code between the second and third lines of the second snippet, an error could occur, causing the resource to leak.

Constructor

`DisposableStack()`

Creates a new `DisposableStack` object.

Instance properties

These properties are defined on `DisposableStack.prototype` and shared by all `DisposableStack` instances.

`DisposableStack.prototype.constructor`

The constructor function that created the instance object. For `DisposableStack` instances, the initial value is the `DisposableStack` constructor.

`DisposableStack.prototype.disposed`

Read-only. Returns `true` if the `DisposableStack` has been disposed, or `false` if not.

`DisposableStack.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"DisposableStack"`. This property is used in `Object.prototype.toString()`.

Instance methods

`DisposableStack.prototype.adopt()`

Registers a value that doesn't implement the disposable protocol to the stack by providing a custom disposer function.

`DisposableStack.prototype.defer()`

Takes a callback function to be called when the stack is disposed.

`DisposableStack.prototype.dispose()`

Disposes this stack by calling all disposers registered to it in reverse order of registration.

`DisposableStack.prototype.move()`

Creates a new `DisposableStack` instance that contains the same disposers as this stack, and then marks this stack as disposed, without calling any disposers.

`DisposableStack.prototype.use()`

Registers a value that implements the disposable protocol to the stack.

`DisposableStack.prototype[Symbol.dispose]`

An alias for the `dispose()` method.

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-disposablestack-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`DisposableStack`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`@@dispose`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`DisposableStack`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`adopt`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`defer`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`dispose`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`disposed`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`move`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
`use`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DisposableStack`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `DisposableStack` in `core-js`
- JavaScript resource management
- `Symbol.dispose`
- `using`
- `AsyncDisposableStack`

DisposableStack() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack/DisposableStack · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`DisposableStack()`** constructor creates `DisposableStack` objects.

Syntax

```
new DisposableStack()
```

> **Note:** `DisposableStack()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

None.

Return value

A new `DisposableStack` object.

Examples

Creating a DisposableStack

```
const disposer = new DisposableStack();
disposer.defer(() => console.log("Disposed!"));
disposer.dispose();
// Logs: Disposed!
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-disposablestack-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`DisposableStack`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DisposableStack.DisposableStack`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `DisposableStack` in `core-js`
- JavaScript resource management

JavaScript › Reference › Standard built-in objects › DisposableStack

DisposableStack.prototype.adopt()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack/adopt · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`adopt()`** method of `DisposableStack` instances registers a value that doesn't implement the disposable protocol to the stack by providing a custom disposer function.

Syntax

```js
adopt(value, onDispose)
```

Parameters

`value`

Any value to be registered to the stack.

`onDispose`

A function that will be called when the stack is disposed. The function receives `value` as its only argument.

Return value

The same `value` that was passed in.

Exceptions

`TypeError`

Thrown if `onDispose` is not a function.

`ReferenceError`

Thrown if the stack is already disposed.

Description

The primary purpose of `adopt()` is to register a value that doesn't implement the disposable protocol to the stack. If the value is already disposable, you can use `use()` instead, which automatically uses the value's `[Symbol.dispose]()` method as the disposer.

`adopt(value, onDispose)` is almost the same as `defer(() => onDispose(value))`, but it allows you to declare the resource and register it on the same line. This way, there's minimal chance of an error happening between the resource creation and registration, which will cause the resource to leak.

js · ✓ good example

```
using disposer = new DisposableStack();
const reader = disposer.adopt(stream.getReader(), (reader) =>
  reader.releaseLock(),
);
```

js · ✗ bad example

```
using disposer = new DisposableStack();
const reader = stream.getReader();
// If someone adds code in between these lines and an error occurs,
// the stream will be locked forever.
disposer.defer(() => reader.close());
```

In the same spirit of "make your resource registered as soon as it's declared", you should always wrap your resource acquisition expression in `adopt()`, instead of extracting it to a separate statement.

js · ✗ bad example

```
using disposer = new DisposableStack();
const reader = stream.getReader();
disposer.adopt(reader, (reader) => reader.close());
```

Examples

Using adopt()

This code consumes a `ReadableStream` via a `ReadableStreamDefaultReader`. The reader does not implement the disposable protocol, so we use `adopt()` to register it to the stack.

js

```
{
  using disposer = new DisposableStack();
  const reader = disposer.adopt(stream.getReader(), (reader) =>
    reader.releaseLock(),
  );
  const { value, done } = reader.read();
  if (!done) {
    // Process the value
  }
  // The reader.releaseLock() method is called here before exiting
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack.prototype.adopt`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`adopt`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DisposableStack.adopt`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `DisposableStack.prototype.adopt` in `core-js`
- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.defer()`
- `DisposableStack.prototype.use()`

JavaScript › Reference › Standard built-in objects › DisposableStack

DisposableStack.prototype.defer()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack/defer · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`defer()`** method of `DisposableStack` instances takes a callback function to be called when the stack is disposed.

Syntax

```js
defer(onDispose)
```

Parameters

`onDispose`

A function that will be called when the stack is disposed. The function receives no arguments.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown if `onDispose` is not a function.

`ReferenceError`

Thrown if the stack is already disposed.

Description

The primary purpose of `defer()` is to register a cleanup callback that's not specific to the disposal of a particular resource. If the callback is specific to a resource, you should use `use()` or `adopt()` instead. You can also use `defer` when the resource is not claimed within your code:

```js
function consumeReader(reader) {
  using disposer = new DisposableStack();
  disposer.defer(() => reader.releaseLock());
  // Do something with reader
}
```

Examples

Using defer()

This function sets a simple lock to prevent multiple async operations from running at the same time. The lock is released when the function completes.

```js
let isLocked = false;

async function requestWithLock(url, options) {
  if (isLocked) {
    return undefined;
  }
  using disposer = new DisposableStack();
  isLocked = true;
  disposer.defer(() => (isLocked = false));
  const data = await fetch(url, options).then((res) => res.json());
  return data;
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-asyncdisposablestack.prototype.defer`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`defer`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DisposableStack.defer`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `DisposableStack.prototype.defer` in `core-js`
- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.adopt()`
- `DisposableStack.prototype.use()`

JavaScript › Reference › Standard built-in objects › DisposableStack

DisposableStack.prototype.dispose()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack/dispose · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`dispose()`** method of `DisposableStack` instances disposes this stack by calling all disposers registered to it in reverse order of registration. If the stack is already disposed, this method does nothing.

It performs the same action as `using disposer = new DisposableStack()` at scope exit. It can be used if you need to clean up at a point other than scope exit.

Syntax

```js
dispose()
```

Parameters

None.

Return value

None (`undefined`).

Exceptions

`SuppressedError`

Thrown if multiple disposers in the stack threw an error. If only one error is thrown, it is rethrown as-is. Otherwise, for each additional error, a new `SuppressedError` is created, with the original error as the `suppressed` property, and the new error as the `error` property.

Examples

Disposing a stack

Here we push three disposers to the stack, using the `use()`, `adopt()`, and `defer()` methods. When `dispose()` is called, the disposers are called in reverse order of registration.

Note that usually you don't need to call `dispose()` manually. Declare the stack with `using`, and its `[Symbol.dispose]()` method will be automatically called when the stack goes out of scope.

```js
class Resource {
  dispose() {
    console.log("Resource disposed");
  }
  [Symbol.dispose]() {
    console.log("Resource disposed via Symbol.dispose");
  }
}

{
  const disposer = new DisposableStack();
  const resource = disposer.use(new Resource());
  const resource2 = disposer.adopt(new Resource(), (resource) =>
    resource.dispose(),
  );
  disposer.defer(() => console.log("Deferred disposer"));
  disposer.dispose();
  // Logs in order:
  // Deferred disposer
  // Resource disposed
  // Resource disposed via Symbol.dispose
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management https://tc39.es/proposal-async-explicit-resource-management/#sec-disposablestack.prototype.dispose

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`dispose`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DisposableStack.dispose`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `DisposableStack.prototype.dispose` in `core-js`
- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.adopt()`
- `DisposableStack.prototype.defer()`
- `DisposableStack.prototype.use()`
- `DisposableStack.prototype[Symbol.dispose]()`

JavaScript › Reference › Standard built-in objects › DisposableStack

DisposableStack.prototype.move()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack/move · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`move()`** method of `DisposableStack` instances creates a new `DisposableStack` instance that contains the same disposers as this stack, and then marks this stack as disposed, without calling any disposers.

Syntax

```js
move()
```

Parameters

None.

Return value

A new `DisposableStack` instance.

Exceptions

`ReferenceError`

Thrown if the stack is already disposed.

Description

The primary purpose of `move()` is to enable transferring responsibility for disposal out of the current scope. For example, your function can claim ownership of some resources and dispose them if an error occurs; if everything completes successfully, then you return these resources and transfer ownership to the caller.

When using `move()` to transfer ownership, calling `move()` should be the very last step in your control flow, because there will be no owner in between your code dropping ownership via `move()` and the caller picking up ownership from the return value.

js · ✓ good example

```
let resource1;

function init() {
  using disposer = new DisposableStack();
  resource1 = disposer.use(getResource1());
  // ...
  // Drop ownership immediately before returning
  return disposer.move();
}

// Pick up ownership immediately after returning
using disposer = init();
```

js · ✗ bad example

```
let resource1;

function init() {
  using disposer = new DisposableStack();
  resource1 = disposer.use(getResource1());
  // ...
  const newDisposer = disposer.move();
  // If someone adds code in between these lines and an error occurs,
  // there would be no owner to free resource1
  return newDisposer;
}

using disposer = init();
```

Also be cautious with the following pattern, although using the "good" pattern may be very awkward in many cases:

js

```
function init() {
  using disposer = new DisposableStack();
  const resource1 = disposer.use(getResource1());
  // ...
  return { disposer: disposer.move(), resource1 };
}

const { resource1, ...rest } = init();
// If someone adds code in between these lines and an error occurs,
// there would be no owner to free resource1
using disposer = rest.disposer;
```

`move()` can also be used for conditional disposal in cases where you may sometimes not want to dispose the resources at all. For example:

```js
using disposer = new DisposableStack();
const server = disposer.use(makeServer());
await server.init();
if (server.ready) {
  // Successfully initialized server; it now should live through the rest
  // of the program. Drop its disposer and don't pick it up. It will no
  // longer be disposed at all.
  disposer.move();
}
// If we reach the end of the scope without running disposer.move(),
// then server isn't ready for any reason, and we dispose its resources
// by disposing the disposer.
```

Examples

Claiming ownership of a stack

```js
function consumeStack(stack) {
  using newStack = stack.move(); // newStack now owns the disposers
  console.log(stack.disposed); // true
  console.log(newStack.disposed); // false
  // newStack is disposed here immediately before the function exits
}

const stack = new DisposableStack();
console.log(stack.disposed); // false
consumeStack(stack);
console.log(stack.disposed); // true
```

Allowing resources to be disposed within two code paths

The major use case of `move()` is when you have one or more resources which could either be disposed right here or could be persisted for later use. In this case, you can put the resources in a `DisposableStack` and then call `move()` when you need to persist the resources for later usage.

```js
class PluginHost {
  #disposed = false;
  #disposables;
  #channel;
  #socket;

  constructor() {
    // Create a DisposableStack that is disposed when the constructor exits.
    // If construction succeeds, we move everything out of `disposer` and into
    // `#disposables` to be disposed later.
    using disposer = new DisposableStack();

    // Create an IPC adapter around process.send/process.on("message").
    // When disposed, it unsubscribes from process.on("message").
    this.#channel = disposer.use(new NodeProcessIpcChannelAdapter(process));

    // Create a pseudo-websocket that sends and receives messages over
    // a NodeJS IPC channel.
    this.#socket = disposer.use(new NodePluginHostIpcSocket(this.#channel));
```

(code continues on the next page)

(code continued from the previous page)

```
    // If we made it here, then there were no errors during construction and
    // we can safely move the disposables out of `disposer` and into `#disposables`.
    this.#disposables = disposer.move();

    // If construction failed, then `disposer` would be disposed before reaching
    // the line above. Event handlers would be removed, allowing `#channel` and
    // `#socket` to be GC'd.
  }

  [Symbol.dispose]() {
    if (this.#disposed) {
      return;
    }
    this.#disposed = true;
    // Put `this.#disposables` into a `using` variable, so it is automatically
    // disposed when the function exits.
    using disposables = this.#disposables;

    // NOTE: we can free `#socket` and `#channel` here since they will be
    // disposed by the call to `disposables[Symbol.dispose]()`, below.
    // This isn't strictly a requirement for every disposable, but is
    // good housekeeping since these objects will no longer be useable.
    this.#socket = undefined;
    this.#channel = undefined;
    this.#disposables = undefined;
  }
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-disposablestack.prototype.move`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`move`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DisposableStack.move`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `DisposableStack.prototype.move` in `core-js`
- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.dispose()`

JavaScript › Reference › Standard built-in objects › DisposableStack

DisposableStack.prototype.use()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack/use · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`use()`** method of `DisposableStack` instances registers a value that implements the disposable protocol to the stack.

Syntax

js

```
use(value)
```

Parameters

`value`

The value to register to the stack. Must either contain a `[Symbol.dispose]()` method, or be `null` or `undefined`.

Return value

The same `value` that was passed in.

Exceptions

`TypeError`

Thrown if `value` is not `null` or `undefined`, and does not contain a `[Symbol.dispose]()` method.

`ReferenceError`

Thrown if the stack is already disposed.

Description

The primary purpose of `use()` is to register a value that implements the disposable protocol to the stack, as the equivalent of the `using` declaration. If the value does not implement the disposable protocol (it doesn't have the `[Symbol.dispose]()` method), use `adopt()` instead, passing a callback that calls the resource's cleanup method.

You should make your resource registered as soon as it's declared. This means you should always wrap your resource acquisition expression in `use()`, instead of extracting it to a separate statement.

js · ✗ bad example

```
using disposer = new DisposableStack();
const reader = stream.getReader();
disposer.use(reader);
```

Examples

Using use()

This code consumes a `ReadableStream` via a `ReadableStreamDefaultReader`. The reader is automatically closed when the function completes, assuming it implements a `[Symbol.dispose]()`

method that synchronously releases the lock on the stream.

```js
{
  using disposer = new DisposableStack();
  const reader = disposer.use(stream.getReader());
  const { value, done } = reader.read();
  if (!done) {
    // Process the value
  }
  // The reader.releaseLock() method is called here before exiting
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management https://tc39.es/proposal-async-explicit-resource-management/#sec-disposablestack.prototype.use

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
use	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DisposableStack.use`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `DisposableStack.prototype.use` in `core-js`
- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.adopt()`
- `DisposableStack.prototype.defer()`

JavaScript › Reference › Standard built-in objects › DisposableStack

DisposableStack.prototype[Symbol.dispose]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack/Symbol.dispose · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`[Symbol.dispose]()`** method of `DisposableStack` instances implements the *disposable protocol* and allows it to be disposed when used with `using` or `await using`. It is an alias for the `dispose()` method.

Syntax

```js
disposableStack[Symbol.dispose]()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Declaring a stack with `using`

The `Symbol.dispose` method is intended to be automatically called in a `using` declaration.

```js
{
  using disposer = new DisposableStack();
  const resource = disposer.use(new Resource());
  resource.doSomething();
  // stack is disposed here immediately before the function exits
  // which causes the resource to be disposed
}
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-disposablestack.prototype-@@dispose`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`@@dispose`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DisposableStack.@@dispose`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `DisposableStack.prototype[Symbol.dispose]` in `core-js`
- JavaScript resource management
- `DisposableStack`
- `DisposableStack.prototype.dispose()`

JavaScript › Reference › Standard built-in objects › DisposableStack

DisposableStack.prototype.disposed

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/DisposableStack/disposed · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`disposed`** accessor property of `DisposableStack` instances returns a boolean indicating whether or not this `DisposableStack` has been disposed or moved by doing any of the following:

- Calling its `dispose()` method
- Calling its `move()` method
- Declaring it with `using` and letting the variable go out of scope, which automatically calls the `[Symbol.dispose]()` method.

Examples

Checking if a stack is disposed

```js
const disposer = new DisposableStack();
console.log(disposer.disposed); // false
disposer.dispose();
console.log(disposer.disposed); // true
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-get-disposablestack.prototype.disposed`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`disposed`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.DisposableStack.disposed`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `DisposableStack.prototype.disposed` in `core-js`
- JavaScript resource management

JavaScript › Reference › Standard built-in objects

encodeURI()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/encodeURI · MDN page last modified 2026-09-02

Baseline: Widely available — available across major browsers since July 2015.

The **`encodeURI()`** function encodes a URI by replacing each instance of certain characters by one, two, three, or four escape sequences representing the UTF-8 encoding of the character (will only be four escape sequences for characters composed of two surrogate characters). Compared to `encodeURIComponent()`, this function encodes fewer characters, preserving those that are part of the URI syntax.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: encodeURI()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const uri = "https://mozilla.org/?x=шеллы";
const encoded = encodeURI(uri);
console.log(encoded);
// Expected output: "https://mozilla.org/?x=%D1%88%D0%B5%D0%BB%D0%BB%D1%8B"

try {
  console.log(decodeURI(encoded));
  // Expected output: "https://mozilla.org/?x=шеллы"
} catch (e) {
  // Catches a malformed URI
  console.error(e);
}
```

Syntax

js

```
encodeURI(uri)
```

Parameters

`uri`

A string to be encoded as a URI.

Return value

A new string representing the provided string encoded as a URI.

Exceptions

`URIError`

Thrown if `uri` contains a lone surrogate.

Description

`encodeURI()` is a function property of the global object.

The `encodeURI()` function escapes characters by UTF-8 code units, with each octet encoded in the format `%XX`, left-padded with 0 if necessary. Because lone surrogates in UTF-16 do not encode any valid Unicode character, they cause `encodeURI()` to throw a `URIError`.

`encodeURI()` escapes all characters **except**:

```
A–Z a–z 0–9 - _ . ! ~ * ' ( )

; / ? : @ & = + $ , #
```

The characters on the second line are characters that may be part of the URI syntax, and are only escaped by `encodeURIComponent()`. Both `encodeURI()` and `encodeURIComponent()` do not encode the characters `-.!~*'()`, known as "unreserved marks", which do not have a reserved purpose but are allowed in a URI "as is". (See RFC2396)

The `encodeURI()` function does not encode characters that have special meaning (reserved characters) for a URI. The following example shows all the parts that a URI can possibly contain. Note how certain characters are used to signify special meaning:

url

```
http://username:password@www.example.com:80/path/to/file.php?
foo=316&bar=this+has+spaces#anchor
```

`encodeURI`, as the name implies, is used to encode a URL as a whole, assuming it is already well-formed. If you want to dynamically assemble string values into a URL, you probably want to use `encodeURIComponent()` on each dynamic segment instead, to avoid URL syntax characters in unwanted places.

js

```
const name = "Ben & Jerry's";

// This is bad:
const badLink = encodeURI(`https://example.com/?choice=${name}`); // "https://example.com/?
choice=Ben%20&%20Jerry's"
console.log([...new URL(badLink).searchParams]); // [['choice', 'Ben '], [" Jerry's", '']]

// Instead:
const goodLink = `https://example.com/?choice=${encodeURIComponent(name)}`;
// "https://example.com/?choice=Ben%20%26%20Jerry's"
console.log([...new URL(goodLink).searchParams]); // [['choice', "Ben & Jerry's"]]
```

Examples

encodeURI() vs. encodeURIComponent()

`encodeURI()` differs from `encodeURIComponent()` as follows:

js

```
const set1 = ";/?:@&=+$,#"; // Reserved Characters
const set2 = "-.!~*'()"; // Unreserved Marks
const set3 = "ABC abc 123"; // Alphanumeric Characters + Space

console.log(encodeURI(set1)); // ;/?:@&=+$,#
console.log(encodeURI(set2)); // -.!~*'()
console.log(encodeURI(set3)); // ABC%20abc%20123 (the space gets encoded as %20)

console.log(encodeURIComponent(set1)); // %3B%2C%2F%3F%3A%40%26%3D%2B%24%23
console.log(encodeURIComponent(set2)); // -.!~*'()
console.log(encodeURIComponent(set3)); // ABC%20abc%20123 (the space gets encoded as %20)
```

Encoding a lone surrogate throws

A URIError will be thrown if one attempts to encode a surrogate which is not part of a high-low pair. For example:

```js
// High-low pair OK
encodeURI("\uD800\uDFFF"); // "%F0%90%8F%BF"

// Lone high-surrogate code unit throws "URIError: malformed URI sequence"
encodeURI("\uD800");

// Lone low-surrogate code unit throws "URIError: malformed URI sequence"
encodeURI("\uDFFF");
```

You can use String.prototype.toWellFormed(), which replaces lone surrogates with the Unicode replacement character (U+FFFD), to avoid this error. You can also use String.prototype.isWellFormed() to check if a string contains lone surrogates before passing it to encodeURI().

Encoding for RFC3986

The more recent RFC3986 makes square brackets reserved (for IPv6) and thus not encoded when forming something which could be part of a URL (such as a host). It also reserves !, ', (,), and *, even though these characters have no formalized URI delimiting uses. The following function encodes a string for RFC3986-compliant URL format.

```js
function encodeRFC3986URI(str) {
  return encodeURI(str)
    .replace(/%5B/g, "[")
    .replace(/%5D/g, "]")
    .replace(
      /[!'()*]/g,
      (c) => `%${c.charCodeAt(0).toString(16).toUpperCase()}`,
    );
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/global-object.html#sec-encodeuri-uri

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
encodeURI	1	12	1	7	1.1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.encodeURI`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `decodeURI()`
- `encodeURIComponent()`
- `decodeURIComponent()`

JavaScript › Reference › Standard built-in objects

encodeURIComponent()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/encodeURIComponent · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since July 2015.

The **`encodeURIComponent()`** function encodes a URI by replacing each instance of certain characters by one, two, three, or four escape sequences representing the UTF-8 encoding of the character (will only be four escape sequences for characters composed of two surrogate characters). Compared to `encodeURI()`, this function encodes more characters, including those that are part of the URI syntax.

Try it

▷ Interactive example on MDN: "JavaScript Demo: encodeURIComponent()". It runs live in the browser there; its source code follows.

js · interactive example source

```
// Encodes characters such as ?,=,/,&,:
console.log(`?x=${encodeURIComponent("test?")}`);
// Expected output: "?x=test%3F"

console.log(`?x=${encodeURIComponent("шеллы")}`);
// Expected output: "?x=%D1%88%D0%B5%D0%BB%D0%BB%D1%8B"
```

Syntax

js

```
encodeURIComponent(uriComponent)
```

Parameters

`uriComponent`

A string to be encoded as a URI component (a path, query string, fragment, etc.). Other values are converted to strings.

Return value

A new string representing the provided `uriComponent` encoded as a URI component.

Exceptions

`URIError`

Thrown if `uriComponent` contains a lone surrogate.

Description

`encodeURIComponent()` is a function property of the global object.

`encodeURIComponent()` uses the same encoding algorithm as described in `encodeURI()`. It escapes all characters **except**:

```
A–Z a–z 0–9 - _ . ! ~ * ' ( )
```

Compared to `encodeURI()`, `encodeURIComponent()` escapes a larger set of characters. Use `encodeURIComponent()` on user-entered fields from forms sent to the server — this will encode &

symbols that may inadvertently be generated during data entry for character references or other characters that require encoding/decoding. For example, if a user writes `Jack & Jill`, without `encodeURIComponent()`, the ampersand could be interpreted on the server as the start of a new field and jeopardize the integrity of the data.

For `application/x-www-form-urlencoded`, spaces are to be replaced by `+`, so one may wish to follow an `encodeURIComponent()` replacement with an additional replacement of `%20` with `+`.

Examples

Encoding for Content-Disposition and Link headers

The following example provides the special encoding required within UTF-8 `Content-Disposition` and `Link` server response header parameters (e.g., UTF-8 filenames):

```js
const fileName = "my file(2).txt";
const header = `Content-Disposition: attachment; filename*=UTF-8''${encodeRFC5987ValueChars(
  fileName,
)}`;

console.log(header);
// "Content-Disposition: attachment; filename*=UTF-8''my%20file%282%29.txt"

function encodeRFC5987ValueChars(str) {
  return (
    encodeURIComponent(str)
      // The following creates the sequences %27 %28 %29 %2A (Note that
      // the valid encoding of "*" is %2A, which necessitates calling
      // toUpperCase() to properly encode). Although RFC3986 reserves "!",
      // RFC5987 does not, so we do not need to escape it.
      .replace(
        /['()*]/g,
        (c) => `%${c.charCodeAt(0).toString(16).toUpperCase()}`,
      )
      // The following are not required for percent-encoding per RFC5987,
      // so we can allow for a little better readability over the wire: |`^
      .replace(/%(7C|60|5E)/g, (str, hex) =>
        String.fromCharCode(parseInt(hex, 16)),
      )
  );
}
```

Encoding for RFC3986

The more recent RFC3986 reserves `!`, `'`, `(`, `)`, and `*`, even though these characters have no formalized URI delimiting uses. The following function encodes a string for RFC3986-compliant URL component format. It also encodes `[` and `]`, which are part of the IPv6 URI syntax. An RFC3986-compliant `encodeURI` implementation should not escape them, which is demonstrated in the `encodeURI()` example.

```js
function encodeRFC3986URIComponent(str) {
  return encodeURIComponent(str).replace(
    /[!'()*]/g,
    (c) => `%${c.charCodeAt(0).toString(16).toUpperCase()}`,
  );
}
```

Encoding a lone surrogate throws

A `URIError` will be thrown if one attempts to encode a surrogate which is not part of a high-low pair. For example:

```js
// High-low pair OK
encodeURIComponent("\uD800\uDFFF"); // "%F0%90%8F%BF"

// Lone high-surrogate code unit throws "URIError: malformed URI sequence"
encodeURIComponent("\uD800");

// Lone high-surrogate code unit throws "URIError: malformed URI sequence"
encodeURIComponent("\uDFFF");
```

You can use `String.prototype.toWellFormed()`, which replaces lone surrogates with the Unicode replacement character (U+FFFD), to avoid this error. You can also use `String.prototype.isWellFormed()` to check if a string contains lone surrogates before passing it to `encodeURIComponent()`.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/global-object.html#sec-encodeuricomponent-uricomponent

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`encodeURIComponent`	1	12	1	7	1.1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.encodeURIComponent`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `decodeURI()`
- `encodeURI()`
- `decodeURIComponent()`

JavaScript › Reference › Standard built-in objects

Error

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

`Error` objects are thrown when runtime errors occur. The `Error` object can also be used as a base object for user-defined exceptions. See below for standard built-in error types.

Description

Runtime errors result in new `Error` objects being created and thrown.

`Error` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

Error types

Besides the generic `Error` constructor, there are other core error constructors in JavaScript. For client-side exceptions, see Exception handling statements.

`EvalError`

Creates an instance representing an error that occurs regarding the global function `eval()`.

`RangeError`

Creates an instance representing an error that occurs when a numeric variable or parameter is outside its valid range.

`ReferenceError`

Creates an instance representing an error that occurs when de-referencing an invalid reference.

`SyntaxError`

Creates an instance representing a syntax error.

`TypeError`

Creates an instance representing an error that occurs when a variable or parameter is not of a valid type.

`URIError`

Creates an instance representing an error that occurs when `encodeURI()` or `decodeURI()` are passed invalid parameters.

`AggregateError`

Creates an instance representing several errors wrapped in a single error when multiple errors need to be reported by an operation, for example by `Promise.any()`.

`InternalError` Non-standard

Creates an instance representing an error that occurs when an internal error in the JavaScript engine is thrown. E.g. "too much recursion".

Constructor

`Error()`

Creates a new `Error` object.

Static properties

`Error.stackTraceLimit` Non-standard

A non-standard numerical property that limits how many stack frames to include in an error stack trace.

Static methods

`Error.captureStackTrace()`

A non-standard function that creates the `stack` property on the provided object.

`Error.isError()`

Returns `true` if the argument is an error, or `false` otherwise.

`Error.prepareStackTrace()` Non-standard Optional

A non-standard function that, if provided by user code, is called by the JavaScript engine for thrown exceptions, allowing the user to provide custom formatting for stack traces. See the V8 Stack Trace API docs.

Instance properties

These properties are defined on `Error.prototype` and shared by all `Error` instances.

`Error.prototype.constructor`

The constructor function that created the instance object. For `Error` instances, the initial value is the `Error` constructor.

`Error.prototype.name`

Represents the name for the type of error. For `Error.prototype.name`, the initial value is `"Error"`. Subclasses like `TypeError` and `SyntaxError` provide their own `name` properties.

`Error.prototype.stack` Non-standard

A non-standard property for a stack trace.

These properties are own properties of each `Error` instance.

`cause`

Error cause indicating the reason why the current error is thrown — usually another caught error. For user-created `Error` objects, this is the value provided as the `cause` property of the constructor's second argument.

`columnNumber` Non-standard

A non-standard Mozilla property for the column number in the line that raised this error.

`fileName` Non-standard

A non-standard Mozilla property for the path to the file that raised this error.

`lineNumber` Non-standard

A non-standard Mozilla property for the line number in the file that raised this error.

`message`

Error message. For user-created `Error` objects, this is the string provided as the constructor's first argument.

Instance methods

`Error.prototype.toString()`

Returns a string representing the specified object. Overrides the `Object.prototype.toString()` method.

Examples

Throwing a generic error

Usually you create an `Error` object with the intention of raising it using the `throw` keyword. You can handle the error using the `try...catch` construct:

```js
try {
  throw new Error("Whoops!");
} catch (e) {
  console.error(`${e.name}: ${e.message}`);
}
```

Handling a specific error type

You can choose to handle only specific error types by testing the error type with the `instanceof` keyword:

```js
try {
  foo.bar();
} catch (e) {
  if (e instanceof EvalError) {
    console.error(`${e.name}: ${e.message}`);
  } else if (e instanceof RangeError) {
    console.error(`${e.name}: ${e.message}`);
  }
  // etc.
  else {
    // If none of our cases matched leave the Error unhandled
    throw e;
  }
}
```

Differentiate between similar errors

Sometimes a block of code can fail for reasons that require different handling, but which throw very similar errors (i.e., with the same type and message).

If you don't have control over the original errors that are thrown, one option is to catch them and throw new `Error` objects that have more specific messages. The original error should be passed to the new `Error` in the constructor's `options` parameter as its `cause` property. This ensures that the original error and stack trace are available to higher-level try/catch blocks.

The example below shows this for two methods that would otherwise fail with similar errors (`doFailSomeWay()` and `doFailAnotherWay()`):

```js
function doWork() {
  try {
    doFailSomeWay();
  } catch (err) {
    throw new Error("Failed in some way", { cause: err });
  }
  try {
    doFailAnotherWay();
  } catch (err) {
    throw new Error("Failed in another way", { cause: err });
  }
}

try {
  doWork();
} catch (err) {
  switch (err.message) {
    case "Failed in some way":
      handleFailSomeWay(err.cause);
      break;
    case "Failed in another way":
      handleFailAnotherWay(err.cause);
      break;
  }
}
```

> **Note:** If you are making a library, you should prefer to use error cause to discriminate between different errors emitted — rather than asking your consumers to parse the error message. See the error cause page for an example.

Custom error types can also use the `cause` property, provided the subclasses' constructor passes the `options` parameter when calling `super()`. The `Error()` base class constructor will read `options.cause` and define the `cause` property on the new error instance.

```js
class MyError extends Error {
  constructor(message, options) {
    // Need to pass `options` as the second parameter to install the "cause" property.
    super(message, options);
  }
}

console.log(new MyError("test", { cause: new Error("cause") }).cause);
// Error: cause
```

Custom error types

You might want to define your own error types deriving from `Error` to be able to `throw new MyError()` and use `instanceof MyError` to check the kind of error in the exception handler. This results in cleaner and more consistent error handling code.

See "What's a good way to extend Error in JavaScript?" on Stack Overflow for an in-depth discussion.

> **Warning:** Builtin subclassing cannot be reliably transpiled to pre-ES6 code, because there's no way to construct the base class with a particular `new.target` without `Reflect.construct()`. You need additional configuration or manually call `Object.setPrototypeOf(this, CustomError.prototype)` at the end of the constructor; otherwise, the constructed instance will not be a `CustomError` instance. See the TypeScript FAQ for more information.

> **Note:** Some browsers include the `CustomError` constructor in the stack trace when using ES2015 classes.

js

```
class CustomError extends Error {
  constructor(foo = "bar", ...params) {
    // Pass remaining arguments (including vendor specific ones) to parent constructor
    super(...params);

    // Maintains proper stack trace for where our error was thrown (non-standard)
    if (Error.captureStackTrace) {
      Error.captureStackTrace(this, CustomError);
    }

    this.name = "CustomError";
    // Custom debugging information
    this.foo = foo;
    this.date = new Date();
  }
}

try {
  throw new CustomError("baz", "bazMessage");
} catch (e) {
  console.error(e.name); // CustomError
  console.error(e.foo); // baz
  console.error(e.message); // bazMessage
  console.error(e.stack); // stack trace
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-error-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Error`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`Error()` constructor	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`fileName` parameter Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**
`lineNumber` parameter Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**
`options.cause` parameter	93	93	91	79	15	93	91	66	15	17.0	93	15	1.0.0	1.13	16.9.0
`captureStackTrace`	3	79	138	15	17.2	18	138	14	17.2	1.0	4.4	17.2	1.0.0	1.0	0.10.0
`cause`	93[1]	93[1]	91	79[2]	15[3]	93[1]	91	66[4]	15[3]	17.0[5]	93[1]	15[3]	1.0.0	1.13	16.9.0
`columnNumber` Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**
`fileName` Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**
`isError`	134	134	138	119	Prev.[6,7]	134	138	88	18.4[8,9]	29.0	134	18.4[8,9]	1.1.39[8,10]	2.2	24.3.0[11,12]
`lineNumber` Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**
`message`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`name`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`Error` is serializable	77	79	103[13,14,15]	64	**No**	77	103[13,14,15]	55	**No**	12.0	77	**No**	?	**No**	**No**
`stack` Non-standard	3	12	1	10.5	6	18	4	11	6	1.0	4.4	6	1.0.0	1.0	0.10.0
`stackTraceLimit` Non-standard	3	79	153	15	11.1	18	153	14	11.3	1.0	4.4	11.3	1.0.0	**No**	0.10.0
`toString`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

1. Before version 125, default console logging for `Error` objects does not print the cause.
2. Before version 111, default console logging for `Error` objects does not print the cause.
3. Default console logging for `Error` objects does not print the cause.
4. Before version 83, default console logging for `Error` objects does not print the cause.
5. Before version 27.0, default console logging for `Error` objects does not print the cause.
6. Also from version 18.4: partial support.
7. Also from version 18.4: Returns `false` for `DOMException` instances. See bug 292727.
8. Partial support.
9. Returns `false` for `DOMException` instances. See bug 292727.
10. Returns `false` for `DOMException` instances. See issue 15821.
11. Also from version 24.0.0–24.3.0: partial support.
12. Also from version 24.0.0–24.3.0: Returns `false` for `DOMException` instances. See issue 56497.
13. Version 103 serialized properties: `name`, `message`, `cause`, `fileName`, `lineNumber` and `columnNumber`.
14. Version 104 adds serialization of `stack` in the main thread (`window.postMessage()` and `structuredClone()`).
15. Version 110 adds serialization of `stack` in workers (`worker.postMessage()` and `structuredClone()`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Error` with `cause` support in `core-js`
- es-shims polyfill of `Error cause`
- `throw`
- `try...catch`
- Stack trace API in the V8 docs

JavaScript › Reference › Standard built-in objects › Error

Error() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/Error · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Error()`** constructor creates `Error` objects.

Syntax

```js
new Error()
new Error(message)
new Error(message, options)
new Error(message, fileName)
new Error(message, fileName, lineNumber)

Error()
Error(message)
Error(message, options)
Error(message, fileName)
Error(message, fileName, lineNumber)
```

> **Note:** `Error()` can be called with or without `new`. Both create a new `Error` instance.

Parameters

`message` Optional

A human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A value indicating the specific cause of the error, reflected in the `cause` property. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional Non-standard

The path to the file that raised this error, reflected in the `fileName` property. Defaults to the name of the file containing the code that called the `Error()` constructor.

`lineNumber` Optional Non-standard

The line number within the file on which the error was raised, reflected in the `lineNumber` property. Defaults to the line number containing the `Error()` constructor invocation.

Examples

Function call or new construction

When `Error` is used like a function, that is without `new`, it will return an `Error` object. Therefore, a mere call to `Error` will produce the same output that constructing an `Error` object via the `new` keyword would.

```js
const x = Error("I was created using a function call!");

// above has the same functionality as following
const y = new Error('I was constructed via the "new" keyword!');
```

Rethrowing an error with a cause

It is sometimes useful to catch an error and re-throw it with a new message. In this case you should pass the original error into the constructor for the new `Error`, as shown.

```js
try {
  frameworkThatCanThrow();
} catch (err) {
  throw new Error("New error message", { cause: err });
}
```

For a more detailed example see Error > Differentiate between similar errors.

Omitting options argument

JavaScript only tries to read `options.cause` if `options` is an object — this avoids ambiguity with the other non-standard `Error(message, fileName, lineNumber)` signature, which requires the second parameter to be a string. If you omit `options`, pass a primitive value as `options`, or pass an object without the `cause` property, then the created `Error` object will have no `cause` property.

```js
// Omitting options
const error1 = new Error("Error message");
console.log("cause" in error1); // false

// Passing a primitive value
const error2 = new Error("Error message", "");
console.log("cause" in error2); // false

// Passing an object without a cause property
const error3 = new Error("Error message", { details: "http error" });
console.log("cause" in error3); // false
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-error-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Error()` constructor	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fileName` parameter Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**
`lineNumber` parameter Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**
`options.cause` parameter	93	93	91	79	15	93	91	66	15	17.0	93	15	1.0.0	1.13	16.9.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.Error`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Error` with `cause` support in `core-js`
- es-shims polyfill of Error `cause`
- `throw`
- `try...catch`
- Error causes on v8.dev (2021)

JavaScript › Reference › Standard built-in objects › Error

Error.captureStackTrace()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/captureStackTrace · MDN page last modified 2025-07-10

The **`Error.captureStackTrace()`** static method installs stack trace information on a provided object as the `stack` property.

Syntax

```
Error.captureStackTrace(object)
Error.captureStackTrace(object, constructor)
```

Parameters

`object`

The object on which to add the `stack` property.

`constructor` Optional

A function, typically the constructor where the `object` was created. When collecting the stack trace, all frames above the topmost call to this function, including that call, are left out of the stack trace.

Return value

None (`undefined`).

The `object` is modified in-place with an extra own property called `stack` defined, whose string value follows the same format as `Error.prototype.stack`. This property is non-enumerable and configurable. In V8, it is a getter-setter pair. In SpiderMonkey and JavaScriptCore, it is a data property that is writable.

Examples

Using Error.captureStackTrace()

The `getStack()` utility function returns the current stack trace at the point it is called, removing itself from the stack. This serves the same debugging purpose as `console.trace()`, but allows you to output the string elsewhere. Note that it does not construct an `Error` instance for this purpose, but installs `stack` on a plain object, which would be more efficient for our purposes. Normally, you would call `Error.captureStackTrace` on objects intended to be thrown as errors, as shown in the next example.

```js
function getStack() {
  const obj = {};
  if ("captureStackTrace" in Error) {
    // Avoid getStack itself in the stack trace
    Error.captureStackTrace(obj, getStack);
  }
  return obj.stack;
}

function foo() {
  console.log(getStack());
}

foo();
// Error
//     at foo (<anonymous>:8:15)
//     at <anonymous>:11:1
```

Installing stack trace on a custom error object

The main use case for `Error.captureStackTrace()` is to install a stack trace on a custom error object. Typically, you define custom errors by extending the `Error` class, which automatically makes the `stack` property available via inheritance. However, the problem with the default stack trace is that it includes the constructor call itself, which leaks implementation details. You can avoid this by using `Error.captureStackTrace()`, which allows the stack trace to be installed even for custom errors that do not inherit from `Error`.

```js
class MyError extends Error {
  constructor(message, options) {
    super(message, options);
    if ("captureStackTrace" in Error) {
      // Avoid MyError itself in the stack trace
      Error.captureStackTrace(this, MyError);
    }
  }
}

const myError = new MyError("Something went wrong");
console.log(myError.stack);
// Error: Something went wrong
//     at <anonymous>:8:17
```

Note that even if you don't call `Error.captureStackTrace()` here, some engines are still smart enough to avoid `MyError` in the stack trace if the constructor inherits from `Error`. Calling

`Error.captureStackTrace()` is more important for custom errors that, for some reason, do not inherit from `Error`.

```js
class MyError {
  constructor(message) {
    this.message = message;
    if ("captureStackTrace" in Error) {
      // Avoid MyError itself in the stack trace
      Error.captureStackTrace(this, MyError);
    }
  }
}

const myError = new MyError("Something went wrong");
console.log(myError.stack);
// Error: Something went wrong
//     at <anonymous>:8:17
```

Specifications

Specification
Unknown specification https://github.com/tc39/proposal-error-capturestacktrace?tab=readme-ov-file#errorcapturestacktrace-1

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
captureStackTrace	3	79	138	15	17.2	18	138	14	17.2	1.0	4.4	17.2	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.captureStackTrace`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error.prototype.stack`
- `Error.stackTraceLimit`
- Stack trace API in the V8 docs

JavaScript › Reference › Standard built-in objects › Error

Error.isError()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/isError · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Error.isError()`** static method determines whether the passed value is an `Error`.

Syntax

js

```
Error.isError(value)
```

Parameters

`value`

The value to be checked.

Return value

`true` if `value` is an `Error`; otherwise, `false`.

Description

`Error.isError()` checks if the passed value is an `Error`. It does so by performing a *branded check* for a private field initialized by the `Error()` constructor. This is the same mechanism used by `Array.isArray()`, which is in turn similar to the mechanism used by the `in` operator.

It is a more robust alternative to `instanceof Error` because it avoids false positives and false negatives:

- `Error.isError()` rejects values that aren't actual `Error` instances, even if they have `Error.prototype` in their prototype chain — `instanceof Error` would accept these as it does check the prototype chain.
- `Error.isError()` accepts `Error` objects constructed in another realm — `instanceof Error` returns `false` for these because the identity of the `Error` constructor is different across realms.

`Error.isError()` returns `true` for `DOMException` instances. This is because, although `DOMException` is not specified as a real subclass of `Error` (the `Error` constructor is not the prototype of the `DOMException` constructor), `DOMException` still behaves like `Error` for all branded checking purposes.

Examples

Using Error.isError()

js

```
// all following calls return true
Error.isError(new Error());
Error.isError(new TypeError());
Error.isError(new DOMException());
try {
  1 + 1n;
} catch (e) {
  console.log(Error.isError(e)); // The operation threw a TypeError, so this returns true
}
```

(code continues on the next page)

(code continued from the previous page)

```
// all following calls return false
Error.isError();
Error.isError({});
Error.isError(null);
Error.isError(undefined);
Error.isError(17);
Error.isError("Error");
Error.isError(true);
Error.isError(false);
// This is not an error, because the object does not have the private field
// initialized by the Error constructor
Error.isError({ __proto__: Error.prototype });
```

instanceof vs. Error.isError()

When checking for `Error` instance, `Error.isError()` is preferred over `instanceof` because it works across realms.

js

```
const iframe = document.createElement("iframe");
document.body.appendChild(iframe);
const xError = window.frames[window.frames.length - 1].Error;
const error = new xError();

// Correctly checking for Error
Error.isError(error); // true
// The prototype of error is xError.prototype, which is a
// different object from Error.prototype
error instanceof Error; // false
```

Normalizing caught errors

You can use `Error.isError()` to detect if the caught value is an error and normalize it to an error object.

js

```
try {
  throw "Oops; this is not an Error object";
} catch (e) {
  if (!Error.isError(e)) {
    e = new Error(e);
  }
  console.error(e.message);
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-error.iserror

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`isError`	134	134	138	119	Prev.[1,2]	134	138	88	18.4[3,4]	29.0	134	18.4[3,4]	1.1.39[3,5]	2.2	24.3.0[6,7]

1. Also from version 18.4: partial support.
2. Also from version 18.4: Returns `false` for `DOMException` instances. See bug 292727.
3. Partial support.
4. Returns `false` for `DOMException` instances. See bug 292727.
5. Returns `false` for `DOMException` instances. See issue 15821.
6. Also from version 24.0.0–24.3.0: partial support.
7. Also from version 24.0.0–24.3.0: Returns `false` for `DOMException` instances. See issue 56497.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.isError`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Error.isError` in `core-js`
- es-shims polyfill of `Error.isError`
- `Error`

JavaScript › Reference › Standard built-in objects › Error

Error.stackTraceLimit

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/stackTraceLimit · MDN page last modified 2025-07-10

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** This feature is part of the currently non-standard V8 stack trace API. However, for compatibility reasons, it is also implemented by JavaScriptCore.

The **`Error.stackTraceLimit`** static data property indicates the maximum number of stack frames captured by the stack trace of an error. It can be set by user code to change the engine's behavior.

Generally, *reading* this property is not very useful, but you can *set* it to a new value. Setting it to a larger value can be useful for debugging, as it allows you to see more of the call stack. Setting it to a smaller value can improve performance as it reduces the amount of stack captured.

Value

An integer representing the maximum number of stack frames captured by the stack trace of an error.

Property attributes of `Error.stackTraceLimit`	
Writable	yes

Property attributes of `Error.stackTraceLimit`	
Enumerable	yes
Configurable	yes

Description

Because `stackTraceLimit` is a static property of `Error`, you always use it as `Error.stackTraceLimit`, rather than as a property of an `Error` object you created. If you want to customize the stack trace for a single error only, you may need to set the property, create the error, and then reset the property to its original value.

Examples

Setting Error.stackTraceLimit

This code is safe to run even in environments that don't support `Error.stackTraceLimit`, because it doesn't read the property, only sets it, and engines that don't support it will ignore the setting.

```js
Error.stackTraceLimit = 2;
const a = () => b();
const b = () => c();
const c = () => d();
const d = () => e();
const e = () => {
  throw new Error("My error");
};
try {
  a();
} catch (e) {
  console.log(e.stack);
}
// Only two frames in supporting engines; all frames in others
```

Specifications

▷ *MDN lists no specification for this feature.*

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`stackTraceLimit` Non-standard	3	79	153	15	11.1	18	153	14	11.3	1.0	4.4	11.3	1.0.0	**No**	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.stackTraceLimit`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error.prototype.stack`
- `Error.captureStackTrace()`
- Stack trace API in the V8 docs

JavaScript › Reference › Standard built-in objects › Error

Error.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/toString · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`toString()`** method of `Error` instances returns a string representing this error.

Syntax

```js
toString()
```

Parameters

None.

Return value

A string representing the specified `Error` object.

Description

The `Error` object overrides the `Object.prototype.toString()` method inherited by all objects. Its semantics are as follows:

```js
Error.prototype.toString = function () {
  if (
    this === null ||
    (typeof this !== "object" && typeof this !== "function")
  ) {
    throw new TypeError();
  }
  let name = this.name;
  name = name === undefined ? "Error" : `${name}`;
  let msg = this.message;
  msg = msg === undefined ? "" : `${msg}`;
  if (name === "") {
    return msg;
  }
  if (msg === "") {
    return name;
  }
  return `${name}: ${msg}`;
};
```

Examples

Using toString()

```js
const e1 = new Error("fatal error");
console.log(e1.toString()); // "Error: fatal error"

const e2 = new Error("fatal error");
e2.name = undefined;
console.log(e2.toString()); // "Error: fatal error"

const e3 = new Error("fatal error");
e3.name = "";
console.log(e3.toString()); // "fatal error"

const e4 = new Error("fatal error");
e4.name = "";
e4.message = undefined;
console.log(e4.toString()); // ""

const e5 = new Error("fatal error");
e5.name = "hello";
e5.message = undefined;
console.log(e5.toString()); // "hello"
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-error.prototype.tostring

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
toString	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Error.prototype.toString` with many bug fixes in `core-js`

JavaScript › Reference › Standard built-in objects › Error

Error.prototype.name

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/name · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **name** data property of `Error.prototype` is shared by all `Error` instances. It represents the name for the type of error. For `Error.prototype.name`, the initial value is `"Error"`. Subclasses like `TypeError` and `SyntaxError` provide their own `name` properties.

Value

A string. For `Error.prototype.name`, the initial value is `"Error"`.

Property attributes of `Error.prototype.name`	
Writable	yes
Enumerable	no
Configurable	yes

Description

By default, `Error` instances are given the name "Error". The `name` property, in addition to the `message` property, is used by the `Error.prototype.toString()` method to create a string representation of the error.

Examples

Throwing a custom error

```js
const e = new Error("Malformed input"); // e.name is 'Error'

e.name = "ParseError";
throw e;
// e.toString() would return 'ParseError: Malformed input'
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-error.prototype.name`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`name`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.name`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Error.prototype.message`
- `Error.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Error

Error.prototype.stack

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/stack · MDN page last modified 2025-07-19

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** The `stack` property is de facto implemented by all major JavaScript engines, and the JavaScript standards committee is looking to standardize it. You cannot rely on the precise content of the stack string due to implementation inconsistencies, but you can generally assume it exists and use it for debugging purposes.

The non-standard **`stack`** property of an `Error` instance offers a trace of which functions were called, in what order, from which line and file, and with what arguments. The stack string proceeds from the most recent calls to earlier ones, leading back to the original global scope call.

Value

A string.

Because the `stack` property is non-standard, implementations differ about where it's installed.

- In Firefox, it's an accessor property on `Error.prototype`.
- In Chrome and Safari, it's a data property on each `Error` instance, with the descriptor:

Property attributes of `Error.prototype.stack`	
Writable	yes
Enumerable	no
Configurable	yes

Description

Each JavaScript engine uses its own format for stack traces, but they are fairly consistent in their high-level structure. Every implementation uses a separate line in the stack to represent each function call. The call that directly caused the error is placed at the top, and the call that started the whole call chain is placed at the bottom. Below are some examples of stack traces:

js

```
function foo() {
  bar();
}

function bar() {
  baz();
}

function baz() {
  console.log(new Error().stack);
}

foo();
```

```
#### JavaScriptCore
baz@filename.js:10:24
bar@filename.js:6:6
foo@filename.js:2:6
global code@filename.js:13:4

#### SpiderMonkey
baz@filename.js:10:15
bar@filename.js:6:3
foo@filename.js:2:3
@filename.js:13:1

#### V8
Error
    at baz (filename.js:10:15)
    at bar (filename.js:6:3)
    at foo (filename.js:2:3)
    at filename.js:13:1
```

V8 provides the non-standard stack trace API for customizing the stack trace, including `Error.captureStackTrace()`, `Error.stackTraceLimit`, and `Error.prepareStackTrace()`. Other engines support this API to varying extents.

Different engines set this value at different times. Most modern engines set it when the `Error` object is created. This means you can get the full call stack information within a function using the following:

js

```
function foo() {
  console.log(new Error().stack);
}
```

Without having to throw an error and then catch it.

Stack frames can be things other than explicit function calls, too. For example, event listeners, timeout jobs, and promise handlers all begin their own call chain. Source code within `eval()` and `Function`

constructor calls also appear in the stack:

js

```
console.log(new Function("return new Error('Function failed')")().stack);
console.log("====");
console.log(eval("new Error('eval failed')").stack);
```

```
#### JavaScriptCore
anonymous@
global code@filename.js:1:65
====
eval code@
eval@[native code]
global code@filename.js:3:17

#### SpiderMonkey
anonymous@filename.js line 1 > Function:1:8
@filename.js:1:65

====
@filename.js line 3 > eval:1:1
@filename.js:3:13

#### V8
Error: Function failed
    at eval (eval at <anonymous> (filename.js:1:13), <anonymous>:1:8)
    at filename.js:1:65
====
Error: eval failed
    at eval (eval at <anonymous> (filename.js:3:13), <anonymous>:1:1)
    at filename.js:3:13
```

In Firefox, you can use the `//# sourceURL` directive to name an eval source. See the Firefox Debug eval sources docs.

Examples

Using the stack property

The following script demonstrates how to use the `stack` property to output a stack trace into your browser window. You can use this to check what your browser's stack structure looks like.

html · hidden on the MDN page (supporting code for the live sample)

```
<div id="output"></div>
```

css · hidden on the MDN page (supporting code for the live sample)

```
#output {
  white-space: pre;
  font-family: monospace;
}
```

```js
function trace() {
  throw new Error("trace() failed");
}
function b() {
  trace();
}
function a() {
  b(3, 4, "\n\n", undefined, {});
}
try {
  a("first call, first arg");
} catch (e) {
  document.getElementById("output").textContent = e.stack;
}
```

▷ Live sample output ("Using the stack property sample") is rendered in the browser on MDN and cannot be reproduced on paper. See it running at developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/stack#using_the_stack_property .

Specifications

Not part of any standard.

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`stack` Non-standard	3	12	1	10.5	6	18	4	11	6	1.0	4.4	6	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.stack`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- TraceKit on GitHub
- stacktrace.js on GitHub
- Stack trace API in the V8 docs

JavaScript › Reference › Standard built-in objects › Error

Error: cause

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/cause · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2021.

The **cause** data property of an `Error` instance indicates the specific original cause of the error.

It is used when catching and re-throwing an error with a more-specific or useful error message in order to still have access to the original error.

Value

The value that was passed to the `Error()` constructor in the `options.cause` argument. It may not be present.

Property attributes of `Error: cause`	
Writable	yes
Enumerable	no
Configurable	yes

Description

The value of `cause` can be of any type. You should not make assumptions that the error you caught has an `Error` as its `cause`, in the same way that you cannot be sure the variable bound in the `catch` statement is an `Error` either. The "Providing structured data as the error cause" example below shows a case where a non-error is deliberately provided as the cause.

The `SuppressedError` and `AggregateError` subclasses both serve the purpose of associating multiple errors. They both represent multiple failing sites: `SuppressedError` represents an error that occurred while handling another error, while `AggregateError` represents a collection of multiple, unrelated errors that occurred during the same operation. The `cause` property represents a single failing site, with the wrapper error only adding context to the cause and not representing an additional failure.

The following is typical usage of `cause`. There is a single failing site, which originates within `mainLogic()`. The `throw new Error()` statement merely wraps that original error to add context and does not constitute an additional failure.

```js
try {
  mainLogic();
} catch (err) {
  throw new Error("Main logic failed", { cause: err });
}
```

The following is typical usage of `SuppressedError`. There are two failing sites: one within `mainLogic()` and one within `cleanup()`. The `SuppressedError` instance associates the two errors.

```js
try {
  mainLogic();
} catch (primaryError) {
  try {
    cleanup();
  } catch (cleanupError) {
    throw new SuppressedError(
      cleanupError,
      primaryError,
      "Main logic failed; while handling that, cleanup also failed",
    );
  }
}
```

The following is typical usage of `AggregateError`. There are multiple failing sites within `mainLogic()`. The `AggregateError` instance associates all the errors.

```js
function mainLogic() {
  const errors = [];
  try {
    operation1();
  } catch (e1) {
    errors.push(e1);
  }
  try {
    operation2();
  } catch (e2) {
    errors.push(e2);
  }
  if (errors.length > 0) {
    throw new AggregateError(errors, "Multiple operations failed");
  }
}
```

Examples

Rethrowing an error with a cause

It is sometimes useful to catch an error and re-throw it with a new message. In this case you should pass the original error into the constructor for the new `Error`, as shown.

```js
try {
  connectToDatabase();
} catch (err) {
  throw new Error("Connecting to database failed.", { cause: err });
}
```

For a more detailed example see Error > Differentiate between similar errors.

Providing structured data as the error cause

Error messages written for human consumption may be inappropriate for machine parsing — since they're subject to rewording or punctuation changes that may break any existing parsing written to consume them. So when throwing an error from a function, as an alternative to a human-readable error message, you can instead provide the cause as structured data, for machine parsing.

```js
function makeRSA(p, q) {
  if (!Number.isInteger(p) || !Number.isInteger(q)) {
    throw new Error("RSA key generation requires integer inputs.", {
      cause: { code: "NonInteger", values: [p, q] },
    });
  }
  if (!areCoprime(p, q)) {
    throw new Error("RSA key generation requires two co-prime integers.", {
      cause: { code: "NonCoprime", values: [p, q] },
    });
  }
  // rsa algorithm…
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-installerrorcause`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`cause`	93[1]	93[1]	91	79[2]	15[3]	93[1]	91	66[4]	15[3]	17.0[5]	93[1]	15[3]	1.0.0	1.13	16.9.0

1. Before version 125, default console logging for `Error` objects does not print the cause.
2. Before version 111, default console logging for `Error` objects does not print the cause.
3. Default console logging for `Error` objects does not print the cause.
4. Before version 83, default console logging for `Error` objects does not print the cause.
5. Before version 27.0, default console logging for `Error` objects does not print the cause.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.cause`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Error` with `cause` support in `core-js`
- `Error.prototype.message`
- `Error.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Error

Error: columnNumber

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/columnNumber · MDN page last modified 2025-07-10

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`columnNumber`** data property of an `Error` instance contains the column number in the line of the file that raised this error.

Value

A positive integer.

Property attributes of `Error: columnNumber`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using columnNumber

```js
try {
  throw new Error("Could not parse input");
} catch (err) {
  console.log(err.columnNumber); // 9
}
```

Specifications

Not part of any standard.

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`columnNumber` Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.columnNumber`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error.prototype.stack`
- `Error.prototype.lineNumber`
- `Error.prototype.fileName`

JavaScript › Reference › Standard built-in objects › Error

Error: fileName

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/fileName · MDN page last modified 2025-07-10

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`fileName`** data property of an `Error` instance contains the path to the file that raised this error.

Value

A string.

Property attributes of `Error: fileName`	
Writable	yes
Enumerable	no

Property attributes of `Error: fileName`	
Configurable	yes

Description

This non-standard property contains the path to the file that raised this error. If called from a debugger context, the Firefox Developer Tools for example, "debugger eval code" is returned.

Examples

Using fileName

```js
const e = new Error("Could not parse input");
throw e;
// e.fileName could look like "file:///C:/example.html"
```

Specifications

Not part of any standard.

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`fileName` Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.fileName`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Error.prototype.stack`
- `Error.prototype.columnNumber`
- `Error.prototype.lineNumber`

JavaScript › Reference › Standard built-in objects › Error

Error: lineNumber

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/lineNumber · MDN page last modified 2025-07-10

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`lineNumber`** data property of an `Error` instance contains the line number in the file that raised this error.

Value

A positive integer.

Property attributes of `Error: lineNumber`	
Writable	yes
Enumerable	no
Configurable	yes

Examples

Using lineNumber

```js
try {
  throw new Error("Could not parse input");
} catch (err) {
  console.log(err.lineNumber); // 2
}
```

Alternative example using error event

```js
window.addEventListener("error", (e) => {
  console.log(e.lineNumber); // 5
});
const e = new Error("Could not parse input");
throw e;
```

This is not a standard feature and lacks widespread support. See the browser compatibility table below.

Specifications

Not part of any standard.

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`lineNumber` Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.lineNumber`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error.prototype.stack`
- `Error.prototype.columnNumber`
- `Error.prototype.fileName`

JavaScript › Reference › Standard built-in objects › Error

Error: message

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error/message · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`message`** data property of an `Error` instance is a human-readable description of the error.

Value

A string corresponding to the value passed to the `Error()` constructor as the first argument.

Property attributes of `Error: message`	
Writable	yes
Enumerable	no
Configurable	yes

Description

This property contains a brief description of the error if one is available or has been set. The `message` property combined with the `name` property is used by the `Error.prototype.toString()` method to create a string representation of the Error.

By default, the `message` property is an empty string, but this behavior can be overridden for an instance by specifying a message as the first argument to the `Error` constructor.

Examples

Throwing a custom error

```js
const e = new Error("Could not parse input");
// e.message is 'Could not parse input'
throw e;
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-error.prototype.message`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`message`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Error.message`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error.prototype.name`
- `Error.prototype.toString()`

JavaScript › Reference › Standard built-in objects

escape()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/escape · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Note:** `escape()` is a non-standard function implemented by browsers and was only standardized for cross-engine compatibility. It is not required to be implemented by all JavaScript engines and may not work everywhere. Use `encodeURIComponent()` or `encodeURI()` if possible.

The **`escape()`** function computes a new string in which certain characters have been replaced by hexadecimal escape sequences.

Syntax

```js
escape(str)
```

Parameters

`str`

A string to be encoded.

Return value

A new string in which certain characters have been escaped.

Description

`escape()` is a function property of the global object.

The `escape()` function replaces all characters with escape sequences, with the exception of ASCII word characters (A–Z, a–z, 0–9, _) and `@\*_+-./`. Characters are escaped by UTF-16 code units. If the code unit's value is less than 256, it is represented by a two-digit hexadecimal number in the format `%XX`, left-padded with 0 if necessary. Otherwise, it is represented by a four-digit hexadecimal number in the format `%uXXXX`, left-padded with 0 if necessary.

> **Note:** This function was used mostly for percent-encoding and is partly based on the escape format in RFC 1738. The escape format is *not* an escape sequence in string literals. You can replace `%XX` with `\xXX` and `%uXXXX` with `\uXXXX` to get a string containing actual string-literal escape sequences.

Examples

Using escape()

```js
escape("abc123"); // "abc123"
escape("äöü"); // "%E4%F6%FC"
escape("ć"); // "%u0107"

// special characters
escape("@*_+-./"); // "@*_+-./"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/additional-ecmascript-features-for-web-browsers.html#sec-escape-string`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`escape` Deprecated	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.escape`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `escape` in `core-js`
- `encodeURI`
- `encodeURIComponent`
- `unescape`

JavaScript › Reference › Standard built-in objects

eval()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/eval · MDN page last modified 2026-10-08

Baseline: Widely available — available across major browsers since July 2015.

> **Warning:** The argument passed to this function is dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations for more information.

The **`eval()`** function evaluates JavaScript code represented as a string and returns its completion value. The source is parsed as a script.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: eval()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(eval("2 + 2"));
// Expected output: 4

console.log(eval(new String("2 + 2")));
// Expected output: 2 + 2

console.log(eval("2 + 2") === eval("4"));
// Expected output: true

console.log(eval("2 + 2") === eval(new String("2 + 2")));
// Expected output: false
```

Syntax

js

```
eval(script)
```

Parameters

`script`

A `TrustedScript` instance or string representing a JavaScript expression, statement, or sequence of statements. The expression can include variables and properties of existing objects. It will be parsed as a script, so `import` declarations (which can only exist in modules) are not allowed.

Return value

The completion value of evaluating the given code. If the completion value is empty, `undefined` is returned. If `script` is not a `TrustedScript` or string primitive, `eval()` returns the argument unchanged.

Exceptions

`SyntaxError`

The `script` parameter cannot be parsed as a script.

`TypeError`

`script` is a string when Trusted Types are enforced by a CSP and no default policy is defined.

The method also throws any exception that occurs during evaluation of the code.

Description

`eval()` is a function property of the global object.

The argument of the `eval()` function is a string. It will evaluate the source string as a script body, which means both statements and expressions are allowed. It returns the completion value of the code. For expressions, it's the value the expression evaluates to. Many statements and declarations have completion values as well, but the result may be surprising (for example, the completion value of an assignment is the assigned value, but the completion value of `let` is undefined), so it's recommended to not rely on statements' completion values.

In strict mode, declaring a variable named `eval` or re-assigning `eval` is a `SyntaxError`.

js · ✗ bad example

```
"use strict";

const eval = 1; // SyntaxError: Unexpected eval or arguments in strict mode
```

If the argument of `eval()` is not a `TrustedScript` or string, `eval()` returns the argument unchanged. In the following example, passing a `String` object instead of a primitive causes `eval()` to return the `String` object rather than evaluating the string.

js

```
eval(new String("2 + 2")); // returns a String object containing "2 + 2"
eval("2 + 2"); // returns 4
```

To work around the issue in a generic fashion, you can coerce the argument to a string yourself before passing it to `eval()`.

js

```
const expression = new String("2 + 2");
eval(String(expression)); // returns 4
```

Direct and indirect eval

There are two modes of `eval()` calls: *direct* eval and *indirect* eval. Direct eval, as the name implies, refers to *directly* calling the global `eval` function with `eval(...)`. Everything else, including invoking it via an aliased variable, via a member access or other expression, or through the optional chaining `?.` operator, is indirect.

js

```
// Direct call
eval("x + y");

// Indirect call using the comma operator to return eval
(0, eval)("x + y");
```

(code continues on the next page)

(code continued from the previous page)

```
// Indirect call through optional chaining
eval?.("x + y");

// Indirect call using a variable to store and return eval
const myEval = eval;
myEval("x + y");

// Indirect call through member access
const obj = { eval };
obj.eval("x + y");
```

Indirect eval can be seen as if the code is evaluated within a separate `<script>` tag. This means:

- Indirect eval works in the global scope rather than the local scope, and the code being evaluated doesn't have access to local variables within the scope where it's being called.

  js

  ```
  function test() {
    const x = 2;
    const y = 4;
    // Direct call, uses local scope
    console.log(eval("x + y")); // Result is 6
    // Indirect call, uses global scope
    console.log(eval?.("x + y")); // Throws because x is not defined in global scope
  }
  ```

- Indirect `eval` does not inherit the strictness of the surrounding context, and is only in strict mode if the source string itself has a `"use strict"` directive.

  js

  ```
  function nonStrictContext() {
    eval?.(`with (Math) console.log(PI);`);
  }
  function strictContext() {
    "use strict";
    eval?.(`with (Math) console.log(PI);`);
  }
  function strictContextStrictEval() {
    "use strict";
    eval?.(`"use strict"; with (Math) console.log(PI);`);
  }
  nonStrictContext(); // Logs 3.141592653589793
  strictContext(); // Logs 3.141592653589793
  strictContextStrictEval(); // Uncaught SyntaxError: Strict mode code may not include a
  with statement
  ```

  On the other hand, direct eval inherits the strictness of the invoking context.

```js
function nonStrictContext() {
  eval(`with (Math) console.log(PI);`);
}
function strictContext() {
  "use strict";
  eval(`with (Math) console.log(PI);`);
}
function strictContextStrictEval() {
  "use strict";
  eval(`"use strict"; with (Math) console.log(PI);`);
}
nonStrictContext(); // Logs 3.141592653589793
strictContext(); // Uncaught SyntaxError: Strict mode code may not include a with
statement
strictContextStrictEval(); // Uncaught SyntaxError: Strict mode code may not include a
with statement
```

- `var`-declared variables and function declarations would go into the surrounding scope if the source string is not interpreted in strict mode — for indirect eval, they become global variables. If it's a direct eval in a strict mode context, or if the `eval` source string itself is in strict mode, then `var` and function declarations do not "leak" into the surrounding scope.

```js
// Neither context nor source string is strict,
// so var creates a variable in the surrounding scope
eval("var a = 1;");
console.log(a); // 1
// Context is not strict, but eval source is strict,
// so b is scoped to the evaluated script
eval("'use strict'; var b = 1;");
console.log(b); // ReferenceError: b is not defined

function strictContext() {
  "use strict";
  // Context is strict, but this is indirect and the source
  // string is not strict, so c is still global
  eval?.("var c = 1;");
  // Direct eval in a strict context, so d is scoped
  eval("var d = 1;");
}
strictContext();
console.log(c); // 1
console.log(d); // ReferenceError: d is not defined
```

  `let` and `const` declarations within the evaluated string are always scoped to that script.

- Direct eval may have access to additional contextual expressions. For example, in a function's body, one can use `new.target`:

```js
function Ctor() {
  eval("console.log(new.target)");
}
new Ctor(); // [Function: Ctor]
```

Never use direct eval()!

Using direct `eval()` suffers from multiple problems:

- `eval()` executes the code it's passed with the privileges of the caller. If you run `eval()` with a string that could be affected by a malicious party, you may end up running malicious code on the user's machine with the permissions of your webpage / extension. More importantly, allowing third-party code to access the scope in which `eval()` was invoked (if it's a direct eval) can lead to possible attacks that reads or changes local variables. See Security considerations for approaches that mitigate these risks.
- `eval()` is slower than the alternatives, since it has to invoke the JavaScript interpreter, while many other constructs are optimized by modern JS engines.
- Modern JavaScript interpreters convert JavaScript to machine code. This means that any concept of variable naming gets obliterated. Thus, any use of `eval()` will force the browser to do long expensive variable name lookups to figure out where the variable exists in the machine code and set its value. Additionally, new things can be introduced to that variable through `eval()`, such as changing the type of that variable, forcing the browser to re-evaluate all of the generated machine code to compensate.
- Minifiers give up on any minification if the scope is transitively depended on by `eval()`, because otherwise `eval()` cannot read the correct variable at runtime.

There are many cases where the use of `eval()` or related methods can be optimized or avoided altogether.

Using indirect eval()

Consider this code:

```js
function looseJsonParse(obj) {
  return eval(`(${obj})`);
}
console.log(looseJsonParse("{ a: 4 - 1, b: function () {}, c: new Map() }"));
```

Simply using indirect eval and forcing strict mode can make the code much better:

```js
function looseJsonParse(obj) {
  return eval?.(`"use strict";(${obj})`);
}
console.log(looseJsonParse("{ a: 4 - 1, b: function () {}, c: new Map() }"));
```

The two code snippets above may seem to work the same way, but they do not; the first one using direct eval suffers from multiple problems.

- It is a great deal slower, due to more scope inspections. Notice `c: new Map()` in the evaluated string. In the indirect eval version, the object is being evaluated in the global scope, so it is safe for the interpreter to assume that `Map` refers to the global `Map()` constructor instead of a local variable called `Map`. However, in the code using direct eval, the interpreter cannot assume this. For example, in the following code, `Map` in the evaluated string doesn't refer to `window.Map()`.

```js
function looseJsonParse(obj) {
  class Map {}
  return eval(`(${obj})`);
}
console.log(looseJsonParse(`{ a: 4 - 1, b: function () {}, c: new Map() }`));
```

Thus, in the `eval()` version of the code, the browser is forced to make the expensive lookup call to check to see if there are any local variables called `Map()`.

- If not using strict mode, `var` declarations within the `eval()` source becomes variables in the surrounding scope. This leads to hard-to-debug issues if the string is acquired from external input, especially if there's an existing variable with the same name.
- Direct eval can read and mutate bindings in the surrounding scope, which may lead to external input corrupting local data.
- When using direct `eval`, especially when the eval source cannot be proven to be in strict mode, the engine — and build tools — have to disable all optimizations related to inlining, because the `eval()` source can depend on any variable name in its surrounding scope.

However, using indirect `eval()` does not allow passing extra bindings other than existing global variables for the evaluated source to read. If you need to specify additional variables that the evaluated source should have access to, consider using the `Function()` constructor.

Using the Function() constructor

The `Function()` constructor is very similar to the indirect eval example above: it also evaluates the JavaScript source passed to it in the global scope without reading or mutating any local bindings, and therefore allows engines to do more optimizations than direct `eval()`.

The difference between `eval()` and `Function()` is that the source string passed to `Function()` is parsed as a function body, not as a script. There are a few nuances — for example, you can use `return` statements at the top level of a function body, but not in a script.

The `Function()` constructor is useful if you wish to create local bindings within your eval source, by passing the variables as parameter bindings.

```js
function add(a, b) {
  return a + b;
}
function runCodeWithAddFunction(obj) {
  return Function("add", `"use strict";return (${obj});`)(add);
}
console.log(runCodeWithAddFunction("add(5, 7)")); // 12
```

Both `eval()` and `Function()` implicitly evaluate arbitrary code, and are forbidden in strict CSP settings. There are also additional safer (and faster!) alternatives to `eval()` or `Function()` for common use-cases.

Using bracket accessors

You should not use `eval()` to access properties dynamically. Consider the following example where the property of the object to be accessed is not known until the code is executed. This can be done with `eval()`:

```js
const obj = { a: 20, b: 30 };
const propName = getPropName(); // returns "a" or "b"

const result = eval(`obj.${propName}`);
```

However, `eval()` is not necessary here — in fact, it's more error-prone, because if `propName` is not a valid identifier, it leads to a syntax error. Moreover, if `getPropName` is not a function you control, this may lead to execution of arbitrary code. Instead, use the property accessors, which are much faster and safer:

```js
const obj = { a: 20, b: 30 };
const propName = getPropName(); // returns "a" or "b"
const result = obj[propName]; // obj["a"] is the same as obj.a
```

You can even use this method to access descendant properties. Using `eval()`, this would look like:

```js
const obj = { a: { b: { c: 0 } } };
const propPath = getPropPath(); // suppose it returns "a.b.c"

const result = eval(`obj.${propPath}`); // 0
```

Avoiding `eval()` here could be done by splitting the property path and looping through the different properties:

```js
function getDescendantProp(obj, desc) {
  const arr = desc.split(".");
  while (arr.length) {
    obj = obj[arr.shift()];
  }
  return obj;
}

const obj = { a: { b: { c: 0 } } };
const propPath = getPropPath(); // suppose it returns "a.b.c"
const result = getDescendantProp(obj, propPath); // 0
```

Setting a property that way works similarly:

```js
function setDescendantProp(obj, desc, value) {
  const arr = desc.split(".");
  while (arr.length > 1) {
    obj = obj[arr.shift()];
  }
  return (obj[arr[0]] = value);
}

const obj = { a: { b: { c: 0 } } };
const propPath = getPropPath(); // suppose it returns "a.b.c"
const result = setDescendantProp(obj, propPath, 1); // obj.a.b.c is now 1
```

However, beware that using bracket accessors with unconstrained input is not safe either — it may lead to object injection attacks.

Using callbacks

JavaScript has first-class functions, which means you can pass functions as arguments to other APIs, store them in variables and objects' properties, and so on. Many DOM APIs are designed with this in mind, so you can (and should) write:

```js
// Instead of setTimeout("…", 1000) use:
setTimeout(() => {
  // …
}, 1000);

// Instead of elt.setAttribute("onclick", "…") use:
elt.addEventListener("click", () => {
  // …
});
```

Closures are also helpful as a way to create parameterized functions without concatenating strings.

Using JSON

If the string you're calling `eval()` on contains data (for example, an array: `"[1, 2, 3]"`), as opposed to code, you should consider switching to JSON, which allows the string to use a subset of JavaScript syntax to represent data.

Note that since JSON syntax is limited compared to JavaScript syntax, many valid JavaScript literals will not parse as JSON. For example, trailing commas are not allowed in JSON, and property names (keys) in object literals must be enclosed in quotes. Be sure to use a JSON serializer to generate strings that will be later parsed as JSON.

Passing carefully constrained data instead of arbitrary code is a good idea in general. For example, an extension designed to scrape contents of web-pages could have the scraping rules defined in XPath instead of JavaScript code.

Security considerations

The method can be used to execute arbitrary input with the privileges of the caller. If the input is a potentially unsafe string provided by a user, this is a possible vector for Cross-site-scripting (XSS) attacks.

For example, the following code shows how `eval()` might execute `untrustedCode` provided by a user:

js · ✗ bad example

```
const untrustedCode = "alert('Potentially evil code!');";
const adder = eval(untrustedCode);
```

Websites with a Content Security Policy (CSP) that specifies `script-src` or `default-src` will prevent such code running by default. If you must allow the scripts to run via `eval()`, you can mitigate the risks by always assigning a `TrustedScript` instance instead of a string, and enforcing trusted types using the `require-trusted-types-for` CSP directive. This ensures that the input is passed through a transformation function.

To allow `eval()` to run, you additionally need to specify the `trusted-types-eval` keyword in your CSP `script-src` directive. The `unsafe-eval` keyword also allows `eval()`, but is much less safe than `trusted-types-eval` because it would allow execution even on browsers that do not support trusted types.

For example, the required CSP for your site might look like this:

```http
Content-Security-Policy: require-trusted-types-for 'script'; script-src '<your_allowlist>'
'trusted-types-eval'
```

The behavior of the transformation function implemented in your trusted types policy depends on the specific use case that requires a user-provided script. If possible, you should lock the allowed scripts to exactly the code that you trust to run. If that is not possible, you might allow or block the use of certain functions within the provided input.

Examples

Note that the first example shows how to use the method with trusted types. The other examples omit this step for brevity.

Using TrustedScript

To mitigate the risk of XSS, we should always assign `TrustedScript` instances to the `script` parameter. We also need to do this if we're enforcing trusted types for other reasons and we want to allow some script sources that have been permitted (by `CSP: script-src`).

Trusted types are not yet supported on all browsers, so first we define the trusted types tinyfill. This acts as a transparent replacement for the Trusted Types JavaScript API:

```js
if (typeof trustedTypes === "undefined")
  trustedTypes = { createPolicy: (n, rules) => rules };
```

Next we create a `TrustedTypePolicy` that defines a `createScript()` method for transforming input strings into `TrustedScript` instances.

For the purpose of this example, we'll assume that we have a function `transformedScript()` that defines our transformation/filtering logic.

```js
const policy = trustedTypes.createPolicy("script-policy", {
  createScript(input) {
    const transformed = transformedScript(input); // Our filter method
    return transformed;
  },
});
```

Then we use the `policy` object to create a `TrustedScript` object from a potentially unsafe input string:

```js
// The potentially malicious string
const untrustedScript = "alert('Potentially evil code!');";

// Create a TrustedScriptURL instance using the policy
const trustedScript = policy.createScript(untrustedScript);
```

The `TrustedScript` object can now be passed to `eval()`:

```js
eval(trustedScript);
```

Using eval()

In the following code, both of the statements containing `eval()` return 42. The first evaluates the string `"x + y + 1"`; the second evaluates the string `"42"`.

```js
const x = 2;
const y = 39;
const z = "42";
eval("x + y + 1"); // 42
eval(z); // 42
```

eval() returns the completion value of statements

`eval()` returns the completion value of statements. For `if`, it would be the last expression or statement evaluated.

```js
const str = "if (a) { 1 + 1 } else { 1 + 2 }";
let a = true;
let b = eval(str);

console.log(`b is: ${b}`); // b is: 2

a = false;
b = eval(str);

console.log(`b is: ${b}`); // b is: 3
```

The following example uses `eval()` to evaluate the string `str`. This string consists of JavaScript statements that assign z a value of 42 if x is five, and assign 0 to z otherwise. When the second statement is executed, `eval()` will cause these statements to be performed, and it will also evaluate the set of statements and return the value that is assigned to z, because the completion value of an assignment is the assigned value.

```js
const x = 5;
const str = `if (x === 5) {
  console.log("z is 42");
  z = 42;
} else {
  z = 0;
}`;

console.log("z is ", eval(str)); // z is 42  z is 42
```

If you assign multiple values then the last value is returned.

```js
let x = 5;
const str = `if (x === 5) {
  console.log("z is 42");
  z = 42;
  x = 420;
} else {
  z = 0;
}`;

console.log("x is", eval(str)); // z is 42  x is 420
```

eval() as a string defining function requires "(" and ")" as prefix and suffix

```js
// This is a function declaration
const fctStr1 = "function a() {}";
// This is a function expression
const fctStr2 = "(function b() {})";
const fct1 = eval(fctStr1); // return undefined, but `a` is available as a global function
now
const fct2 = eval(fctStr2); // return the function `b`
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/global-object.html#sec-eval-x

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
eval	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.eval`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Property accessors
- WebExtensions: Using eval in content scripts

JavaScript › Reference › Standard built-in objects

EvalError

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/EvalError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`EvalError`** object indicates an error regarding the global `eval()` function. This exception is not thrown by JavaScript anymore, however the `EvalError` object remains for compatibility.

`EvalError` is a serializable object, so it can be cloned with `structuredClone()` or copied between Workers using `postMessage()`.

`EvalError` is a subclass of `Error`.

Constructor

`EvalError()`

Creates a new `EvalError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `EvalError.prototype` and shared by all `EvalError` instances.

`EvalError.prototype.constructor`

The constructor function that created the instance object. For `EvalError` instances, the initial value is the `EvalError` constructor.

`EvalError.prototype.name`

Represents the name for the type of error. For `EvalError.prototype.name`, the initial value is `"EvalError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Creating an EvalError

```js
try {
  throw new EvalError("Hello");
} catch (e) {
  console.log(e instanceof EvalError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "EvalError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-native-error-types-used-in-this-standard-evalerror`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`EvalError`	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`EvalError()` constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`EvalError` is serializable	77	79	103[1,2,3]	64	**No**	77	103[1,2,3]	55	**No**	12.0	77	**No**	?	**No**	**No**

1. Version 103 serializable properties: `name`, `message`, `cause`, `fileName`, `lineNumber` and `columnNumber`.
2. Version 104 also serializes `stack` in the main thread (`window.postMessage()` and `structuredClone()`).
3. Version 110 also serializes `stack` in workers (`worker.postMessage()` and `structuredClone()`).

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.EvalError`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Error`
- `eval()`

JavaScript › Reference › Standard built-in objects › EvalError

EvalError() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/EvalError/EvalError · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`EvalError()`** constructor creates `EvalError` objects.

Syntax

```js
new EvalError()
new EvalError(message)
new EvalError(message, options)
new EvalError(message, fileName)
new EvalError(message, fileName, lineNumber)

EvalError()
EvalError(message)
EvalError(message, options)
EvalError(message, fileName)
EvalError(message, fileName, lineNumber)
```

> **Note:** `EvalError()` can be called with or without `new`. Both create a new `EvalError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional Non-standard

The name of the file containing the code that caused the exception

`lineNumber` Optional Non-standard

The line number of the code that caused the exception

Examples

`EvalError` is not used in the current ECMAScript specification and will thus not be thrown by the runtime. However, the object itself remains for backwards compatibility with earlier versions of the specification.

Creating an EvalError

```js
try {
  throw new EvalError("Hello");
} catch (e) {
  console.log(e instanceof EvalError); // true
  console.log(e.message); // "Hello"
  console.log(e.name); // "EvalError"
  console.log(e.stack); // Stack of the error
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-nativeerror-constructors`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`EvalError()` constructor	1	12	1	5	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.EvalError.EvalError`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error`
- `eval()`

JavaScript › Reference › Standard built-in objects

FinalizationRegistry

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/FinalizationRegistry · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

A **`FinalizationRegistry`** object lets you request a callback when a value is garbage-collected.

Description

`FinalizationRegistry` provides a way to request that a *cleanup callback* get called at some point when a value registered with the registry has been *reclaimed* (garbage-collected). (Cleanup callbacks are sometimes called *finalizers*.)

> **Note:** Cleanup callbacks should not be used for essential program logic. See Notes on cleanup callbacks for details.

You create the registry passing in the callback:

```js
const registry = new FinalizationRegistry((heldValue) => {
  // …
});
```

Then you register any value you want a cleanup callback for by calling the `register` method, passing in the value and a *held value* for it:

```js
registry.register(target, "some value");
```

The registry does not keep a strong reference to the value, as that would defeat the purpose (if the registry held it strongly, the value would never be reclaimed). In JavaScript, objects and non-registered symbols are garbage collectable, so they can be registered in a `FinalizationRegistry` object as the target or the token.

If `target` is reclaimed, your cleanup callback may be called at some point with the *held value* you provided for it (`"some value"` in the above). The held value can be any value you like: a primitive or an object, even `undefined`. If the held value is an object, the registry keeps a *strong* reference to it (so it can pass it to your cleanup callback later).

If you might want to unregister a registered target value later, you pass a third value, which is the *unregistration token* you'll use later when calling the registry's `unregister` function to unregister the value. The registry only keeps a weak reference to the unregister token.

It's common to use the target value itself as the unregister token, which is just fine:

```js
registry.register(target, "some value", target);
// …

// some time later, if you don't care about `target` anymore, unregister it
registry.unregister(target);
```

It doesn't have to be the same value, though; it can be a different one:

```js
registry.register(target, "some value", token);
// …

// some time later
registry.unregister(token);
```

Avoid where possible

Correct use of `FinalizationRegistry` takes careful thought, and it's best avoided if possible. It's also important to avoid relying on any specific behaviors not guaranteed by the specification. When, how, and whether garbage collection occurs is down to the implementation of any given JavaScript engine. Any behavior you observe in one engine may be different in another engine, in another version of the same engine, or even in a slightly different situation with the same version of the same engine. Garbage collection is a hard problem that JavaScript engine implementers are constantly refining and improving their solutions to.

Here are some specific points included by the authors in the proposal that introduced `FinalizationRegistry`:

> Garbage collectors are complicated. If an application or library depends on GC cleaning up a WeakRef or calling a finalizer [cleanup callback] in a timely, predictable manner, it's likely to be disappointed: the cleanup may happen much later than expected, or not at all. Sources of variability include:
>
> - One object might be garbage-collected much sooner than another object, even if they become unreachable at the same time, e.g., due to generational collection.
> - Garbage collection work can be split up over time using incremental and concurrent techniques.
> - Various runtime heuristics can be used to balance memory usage, responsiveness.
> - The JavaScript engine may hold references to things which look like they are unreachable (e.g., in closures, or inline caches).
> - Different JavaScript engines may do these things differently, or the same engine may change its algorithms across versions.
> - Complex factors may lead to objects being held alive for unexpected amounts of time, such as use with certain APIs.

Notes on cleanup callbacks

- Developers shouldn't rely on cleanup callbacks for essential program logic. Cleanup callbacks may be useful for reducing memory usage across the course of a program, but are unlikely to be useful otherwise.
- If your code has just registered a value to the registry, that target will not be reclaimed until the end of the current JavaScript job. See notes on WeakRefs for details.
- A conforming JavaScript implementation, even one that does garbage collection, is not required to call cleanup callbacks. When and whether it does so is entirely down to the implementation of the JavaScript engine. When a registered object is reclaimed, any cleanup callbacks for it may be called then, or some time later, or not at all.
- It's likely that major implementations will call cleanup callbacks at some point during execution, but those calls may be substantially after the related object was reclaimed. Furthermore, if there is an object registered in two registries, there is no guarantee that the two callbacks are called next to each other — one may be called and the other never called, or the other may be called much later.

- There are also situations where even implementations that normally call cleanup callbacks are unlikely to call them:
  - When the JavaScript program shuts down entirely (for instance, closing a tab in a browser).
  - When the `FinalizationRegistry` instance itself is no longer reachable by JavaScript code.
- If the target of a `WeakRef` is also in a `FinalizationRegistry`, the `WeakRef`'s target is cleared at the same time or before any cleanup callback associated with the registry is called; if your cleanup callback calls `deref` on a `WeakRef` for the object, it will receive `undefined`.

Constructor

`FinalizationRegistry()`

Creates a new `FinalizationRegistry` object.

Instance properties

These properties are defined on `FinalizationRegistry.prototype` and shared by all `FinalizationRegistry` instances.

`FinalizationRegistry.prototype.constructor`

The constructor function that created the instance object. For `FinalizationRegistry` instances, the initial value is the `FinalizationRegistry` constructor.

`FinalizationRegistry.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"FinalizationRegistry"`. This property is used in `Object.prototype.toString()`.

Instance methods

`FinalizationRegistry.prototype.register()`

Registers an object with the registry in order to get a cleanup callback when/if the object is garbage-collected.

`FinalizationRegistry.prototype.unregister()`

Unregisters an object from the registry.

Examples

Creating a new registry

You create the registry passing in the callback:

```js
const registry = new FinalizationRegistry((heldValue) => {
  // …
});
```

Registering objects for cleanup

Then you register any objects you want a cleanup callback for by calling the `register` method, passing in the object and a *held value* for it:

```js
registry.register(theObject, "some value");
```

Callbacks never called synchronously

No matter how much pressure you put on the garbage collector, the cleanup callback will never be called synchronously. The object may be reclaimed synchronously, but the callback will always be called sometime after the current job finishes:

```js
let counter = 0;
const registry = new FinalizationRegistry(() => {
  console.log(`Array gets garbage collected at ${counter}`);
});

registry.register(["foo"]);

(function allocateMemory() {
  // Allocate 50000 functions — a lot of memory!
  Array.from({ length: 50000 }, () => () => {});
  if (counter > 5000) return;
  counter++;
  allocateMemory();
})();

console.log("Main job ends");
// Logs:
// Main job ends
// Array gets garbage collected at 5001
```

However, if you allow a little break between each allocation, the callback may be called sooner:

```js
let arrayCollected = false;
let counter = 0;
const registry = new FinalizationRegistry(() => {
  console.log(`Array gets garbage collected at ${counter}`);
  arrayCollected = true;
});

registry.register(["foo"]);

(function allocateMemory() {
  // Allocate 50000 functions — a lot of memory!
  Array.from({ length: 50000 }, () => () => {});
  if (counter > 5000 || arrayCollected) return;
  counter++;
  // Use setTimeout to make each allocateMemory a different job
  setTimeout(allocateMemory);
})();

console.log("Main job ends");
```

There's no guarantee that the callback will be called sooner or if it will be called at all, but there's a possibility that the logged message has a counter value smaller than 5000.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/managing-memory.html#sec-finalization-registry-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`FinalizationRegistry`	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0
`FinalizationRegistry()` constructor	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0
`register`	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0
Non-registered symbol as target	109	109	146	95	16.4	109	146	74	16.4	21.0	109	16.4	1.0.0	1.28	20.0.0 [1]
`unregister`	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0
Non-registered symbol as target	109	109	146	95	16.4	109	146	74	16.4	21.0	109	16.4	1.0.0	1.28	20.0.0

1. Also from version 18.20.0–19.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.FinalizationRegistry`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `WeakRef`
- `WeakSet`
- `WeakMap`

JavaScript › Reference › Standard built-in objects › FinalizationRegistry

FinalizationRegistry() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/FinalizationRegistry/FinalizationRegistry · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

The **`FinalizationRegistry()`** constructor creates `FinalizationRegistry` objects.

Syntax

```
new FinalizationRegistry(callbackFn)
```

> **Note:** `FinalizationRegistry()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`callback`

A function to be invoked each time a registered target value is garbage collected. Its return value is ignored. The function is called with the following arguments:

`heldValue`

The value that was passed to the second parameter of the `register()` method when the `target` object was registered.

Examples

Creating a new registry

You create the registry passing in the callback:

```js
const registry = new FinalizationRegistry((heldValue) => {
  // …
});
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/managing-memory.html#sec-finalization-registry-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`FinalizationRegistry()` constructor	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.FinalizationRegistry.FinalizationRegistry`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `FinalizationRegistry`

JavaScript › Reference › Standard built-in objects › FinalizationRegistry

FinalizationRegistry.prototype.register()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/FinalizationRegistry/register · MDN page last modified 2025-08-15

Baseline: Widely available — available across major browsers since April 2021.

The **`register()`** method of `FinalizationRegistry` instances registers a value with this `FinalizationRegistry` so that if the value is garbage-collected, the registry's callback may get called.

Syntax

```js
register(target, heldValue)
register(target, heldValue, unregisterToken)
```

Parameters

`target`

The target value to register.

`heldValue`

The value to pass to the finalizer for this `target`. This cannot be the `target` itself but can be anything else, including functions and primitives.

`unregisterToken` Optional

A token that may be used with the `unregister` method later to unregister the target value. If provided (and not `undefined`), this must be an object or a non-registered symbol. If not provided, the target cannot be unregistered.

Return value

None (`undefined`).

Exceptions

`TypeError`

Thrown in one of the following cases:

- `target` is not an object or a non-registered symbol (object as opposed to primitives; functions are objects as well)
- `target` is the same as `heldValue` (`target === heldValue`)
- `unregisterToken` is not an object or a non-registered symbol

Description

See the Avoid where possible and Notes on cleanup callbacks sections of the `FinalizationRegistry` page for important caveats.

Examples

Using register

The following registers the value referenced by `target`, passing in the held value `"some value"` and passing the target itself as the unregistration token:

```js
registry.register(target, "some value", target);
```

The following registers the value referenced by `target`, passing in another object as the held value, and not passing in any unregistration token (which means `target` can't be unregistered):

```js
registry.register(target, { useful: "info about target" });
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/managing-memory.html#sec-finalization-registry.prototype.register

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
register	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0
Non-registered symbol as target	109	109	146	95	16.4	109	146	74	16.4	21.0	109	16.4	1.0.0	1.28	20.0.0[1]

1. Also from version 18.20.0–19.0.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.FinalizationRegistry.register`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `FinalizationRegistry`

JavaScript › Reference › Standard built-in objects › FinalizationRegistry

FinalizationRegistry.prototype.unregister()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/FinalizationRegistry/unregister · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

The **`unregister()`** method of `FinalizationRegistry` instances unregisters a target value from this `FinalizationRegistry`.

Syntax

```js
unregister(unregisterToken)
```

Parameters

`unregisterToken`

The token used with the `register()` method when registering the target value. Multiple cells registered with the same `unregisterToken` will be unregistered together.

Return value

A boolean value that is `true` if at least one cell was unregistered and `false` if no cell was unregistered.

Exceptions

`TypeError`

Thrown if `unregisterToken` is not an object or a non-registered symbol.

Description

When a target value has been reclaimed, it is no longer registered in the registry. There is no need to call `unregister` in your cleanup callback. Only call `unregister` if you haven't received a cleanup callback and no longer need to receive one.

Examples

Using unregister

This example shows registering a target object using that same object as the unregister token, then later unregistering it via `unregister`:

```js
class Thingy {
  static #cleanup = (label) => {
    //               ^^^^^----- held value
    console.error(
      `The "release" method was never called for the object with the label "${label}"`,
    );
  };
  #registry = new FinalizationRegistry(Thingy.#cleanup);

  /**
   * Constructs a `Thingy` instance.
   * Be sure to call `release` when you're done with it.
   *
   * @param label A label for the `Thingy`.
   */
  constructor(label) {
    //                            vvvvv----- held value
    this.#registry.register(this, label, this);
    //          target -----^^^^         ^^^^----- unregister token
  }

  /**
   * Releases resources held by this `Thingy` instance.
   */
  release() {
    this.#registry.unregister(this);
    //                        ^^^^----- unregister token
  }
}
```

This example shows registering a target object using a different object as its unregister token:

```js
class Thingy {
  static #cleanup = (file) => {
    //               ^^^^----- held value
    console.error(
      `The "release" method was never called for the "Thingy" for the file "${file.name}"`,
    );
  };
  #registry = new FinalizationRegistry(Thingy.#cleanup);
  #file;

  /**
   * Constructs a `Thingy` instance for the given file.
   * Be sure to call `release` when you're done with it.
   *
   * @param filename The name of the file.
   */
  constructor(filename) {
    this.#file = File.open(filename);
    //                           vvvvv----- held value
    this.#registry.register(this, label, this.#file);
    //          target -----^^^^         ^^^^^^^^^^----- unregister token
  }

  /**
   * Releases resources held by this `Thingy` instance.
   */
  release() {
    if (this.#file) {
      this.#registry.unregister(this.#file);
      //                        ^^^^^^^^^^----- unregister token
      File.close(this.#file);
      this.#file = null;
    }
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/managing-memory.html#sec-finalization-registry.prototype.unregister

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
unregister	84	84	79	70	14.1	84	79	60	14.5	14.0	84	14.5	1.0.0	1.0	14.6.0
Non-registered symbol as target	109	109	146	95	16.4	109	146	74	16.4	21.0	109	16.4	1.0.0	1.28	20.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.FinalizationRegistry.unregister`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `FinalizationRegistry`

JavaScript › Reference › Standard built-in objects

Float16Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Float16Array · MDN page last modified 2025-07-10

Baseline 2025: Newly available — works across the latest browser versions since April 2025; may not work in older ones.

The **`Float16Array`** typed array represents an array of 16-bit floating point numbers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Float16Array` is a subclass of the hidden `TypedArray` class.

> **Note:** Float16 support is not universal, both in the JavaScript API and the underlying CPU architecture. Using it may result in slower performance on some platforms. It is intended for interacting with highly optimized and performance-sensitive systems such as float-backed canvases, WebGPU, WebGL, and deep learning models including stable diffusion.

Constructor

`Float16Array()`

Creates a new `Float16Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Float16Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of `Float16Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Float16Array.prototype` and shared by all `Float16Array` instances.

`Float16Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of a `Float16Array`.

`Float16Array.prototype.constructor`

The constructor function that created the instance object. For `Float16Array` instances, the initial value is the `Float16Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Float16Array

js

```
// From a length
const float16 = new Float16Array(2);
float16[0] = 42;
console.log(float16[0]); // 42
console.log(float16.length); // 2
console.log(float16.BYTES_PER_ELEMENT); // 2

// From an array
const x = new Float16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Float16Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float16FromIterable = new Float16Array(iterable);
console.log(float16FromIterable);
// Float16Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/global-object.html#sec-float16array

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Float16Array	135	135	129	120	18.2	135	129	89	18.2	29.0	135	18.2	1.1.23	1.43	24.0.0
Float16Array() constructor	135	135	129	120	18.2	135	129	89	18.2	29.0	135	18.2	1.1.23	1.43	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Float16Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Float16Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Float16Array/Float16Array · MDN page last modified 2025-07-10

Baseline 2025: Newly available — works across the latest browser versions since April 2025; may not work in older ones.

The **`Float16Array()`** constructor creates `Float16Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Float16Array()
new Float16Array(length)
new Float16Array(typedArray)
new Float16Array(object)

new Float16Array(buffer)
new Float16Array(buffer, byteOffset)
new Float16Array(buffer, byteOffset, length)
```

> **Note:** `Float16Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Float16Array

```js
// From a length
const float16 = new Float16Array(2);
float16[0] = 42;
console.log(float16[0]); // 42
console.log(float16.length); // 2
console.log(float16.BYTES_PER_ELEMENT); // 2
```

(code continues on the next page)

(code continued from the previous page)

```
// From an array
const x = new Float16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Float16Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float16FromIterable = new Float16Array(iterable);
console.log(float16FromIterable);
// Float16Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Float16Array()` constructor	135	135	129	120	18.2	135	129	89	18.2	29.0	135	18.2	1.1.23	1.43	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Float16Array.Float16Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

Float32Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Float32Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Float32Array`** typed array represents an array of 32-bit floating point numbers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Float32Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Float32Array()`

Creates a new `Float32Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Float32Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of `Float32Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Float32Array.prototype` and shared by all `Float32Array` instances.

`Float32Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of a `Float32Array`.

`Float32Array.prototype.constructor`

The constructor function that created the instance object. For `Float32Array` instances, the initial value is the `Float32Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Float32Array

```js
// From a length
const float32 = new Float32Array(2);
float32[0] = 42;
console.log(float32[0]); // 42
console.log(float32.length); // 2
console.log(float32.BYTES_PER_ELEMENT); // 4

// From an array
const x = new Float32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Float32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float32FromIterable = new Float32Array(iterable);
console.log(float32FromIterable);
// Float32Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Float32Array`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`Float32Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Float32Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Float32Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Float32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Float32Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Float32Array/Float32Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Float32Array()`** constructor creates `Float32Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Float32Array()
new Float32Array(length)
new Float32Array(typedArray)
new Float32Array(object)

new Float32Array(buffer)
new Float32Array(buffer, byteOffset)
new Float32Array(buffer, byteOffset, length)
```

> **Note:** `Float32Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Float32Array

```js
// From a length
const float32 = new Float32Array(2);
float32[0] = 42;
console.log(float32[0]); // 42
console.log(float32.length); // 2
console.log(float32.BYTES_PER_ELEMENT); // 4
```

(code continues on the next page)

(code continued from the previous page)

```
// From an array
const x = new Float32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Float32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float32FromIterable = new Float32Array(iterable);
console.log(float32FromIterable);
// Float32Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Float32Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Float32Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Float32Array.Float32Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Float32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

Float64Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Float64Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Float64Array`** typed array represents an array of 64-bit floating point numbers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Float64Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Float64Array()`

Creates a new `Float64Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Float64Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of `Float64Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Float64Array.prototype` and shared by all `Float64Array` instances.

`Float64Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `8` in the case of a `Float64Array`.

`Float64Array.prototype.constructor`

The constructor function that created the instance object. For `Float64Array` instances, the initial value is the `Float64Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create a Float64Array

```js
// From a length
const float64 = new Float64Array(2);
float64[0] = 42;
console.log(float64[0]); // 42
console.log(float64.length); // 2
console.log(float64.BYTES_PER_ELEMENT); // 8

// From an array
const x = new Float64Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float64Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new Float64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float64FromIterable = new Float64Array(iterable);
console.log(float64FromIterable);
// Float64Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Float64Array`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`Float64Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Float64Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Float64Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Float64Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Float64Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Float64Array/Float64Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Float64Array()`** constructor creates `Float64Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Float64Array()
new Float64Array(length)
new Float64Array(typedArray)
new Float64Array(object)

new Float64Array(buffer)
new Float64Array(buffer, byteOffset)
new Float64Array(buffer, byteOffset, length)
```

> **Note:** `Float64Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create a Float64Array

```js
// From a length
const float64 = new Float64Array(2);
float64[0] = 42;
console.log(float64[0]); // 42
console.log(float64.length); // 2
console.log(float64.BYTES_PER_ELEMENT); // 8
```

(code continues on the next page)

(code continued from the previous page)

```
// From an array
const x = new Float64Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Float64Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(64);
const z = new Float64Array(buffer, 8, 4);
console.log(z.byteOffset); // 8

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const float64FromIterable = new Float64Array(iterable);
console.log(float64FromIterable);
// Float64Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Float64Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Float64Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Float64Array.Float64Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Float64Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

Function

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`Function`** object provides methods for functions. In JavaScript, every function is actually a `Function` object.

Constructor

`Function()`

Creates a new `Function` object. Calling the constructor directly can create functions dynamically but suffers from security and similar (but far less significant) performance issues to `eval()`. However, unlike `eval()`, the `Function` constructor creates functions that execute in the global scope only.

Instance properties

These properties are defined on `Function.prototype` and shared by all `Function` instances.

`Function.prototype.arguments` Deprecated Non-standard

Represents the arguments passed to this function. For strict, arrow, async, and generator functions, accessing the `arguments` property throws a `TypeError`. Use the `arguments` object inside function closures instead.

`Function.prototype.caller` Deprecated Non-standard

Represents the function that invoked this function. For strict, arrow, async, and generator functions, accessing the `caller` property throws a `TypeError`.

`Function.prototype.constructor`

The constructor function that created the instance object. For `Function` instances, the initial value is the `Function` constructor.

These properties are own properties of each `Function` instance.

`displayName` Non-standard Optional

The display name of the function.

`length`

Specifies the number of arguments expected by the function.

`name`

The name of the function.

`prototype`

Used when the function is used as a constructor with the `new` operator. It will become the new object's prototype.

Instance methods

`Function.prototype.apply()`

Calls a function with a given `this` value and optional arguments provided as an array (or an array-like object).

`Function.prototype.bind()`

Creates a new function that, when called, has its `this` keyword set to a provided value, optionally with a given sequence of arguments preceding any provided when the new function is called.

`Function.prototype.call()`

Calls a function with a given `this` value and optional arguments.

`Function.prototype.toString()`

Returns a string representing the source code of the function. Overrides the `Object.prototype.toString` method.

`Function.prototype[Symbol.hasInstance]()`

Specifies the default procedure for determining if a constructor function recognizes an object as one of the constructor's instances. Called by the `instanceof` operator.

Examples

Difference between Function constructor and function declaration

Functions created with the `Function` constructor do not create closures to their creation contexts; they always are created in the global scope. When running them, they will only be able to access their own local variables and global ones, not the ones from the scope in which the `Function` constructor was created. This is different from using `eval()` with code for a function expression.

```js
// Create a global property with `var`
var x = 10;

function createFunction1() {
  const x = 20;
  return new Function("return x;"); // this `x` refers to global `x`
}

function createFunction2() {
  const x = 20;
  function f() {
    return x; // this `x` refers to the local `x` above
  }
  return f;
}

const f1 = createFunction1();
console.log(f1()); // 10
const f2 = createFunction2();
console.log(f2()); // 20
```

While this code works in web browsers, `f1()` will produce a `ReferenceError` in Node.js, as `x` will not be found. This is because the top-level scope in Node is not the global scope, and `x` will be local to the module.

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Function`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
[Symbol.hasInstance]	50	15	50	37	10	50	50	37	10	5.0	50	10	1.0.0	1.0	6.5.0
`Function()` constructor	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Requires arguments to be `TrustedScript` instance when trusted types are enforced	**No**	**No**	Prev.	**No**	26	**No**	**No**	**No**	26	**No**	**No**	26	?	?	?
`apply`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
ES 5.1: generic array-like object as `arguments`	17	12	4	5	6	18	4	10.1	6	1.0	4.4	6	1.0.0	1.0	0.10.0
`arguments` Deprecated Non-standard	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`bind`	7	12	4	11.6	5.1	18	4	12	6	1.0	4.4	6	1.0.0	1.0	0.10.0
`call`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0 [1]
`caller` Deprecated Non-standard	1	12	1	9.6	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
`displayName` Non-standard	**No**	**No**	13	**No**	**No**	**No**	14	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**
`length`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Configurable: true	43	12	37	30	10	43	37	30	10	4.0	43	10	1.0.0	1.0	4.0.0
`name`	15	14	1	10.5	6	18	4	11	6	1.0	4.4	6	1.0.0	1.0	0.10.0
Configurable: true	43	14	38	30	10	43	38	30	10	4.0	43	10	1.0.0	1.0	4.0.0
Inferred names on anonymous functions	51	79[2,3]	53	38	10	51	53	41	10	5.0	51	10	1.0.0	1.0	6.5.0
`toString`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Implements `Function.prototype.toString` revision	66	79	54	53	17	66	54	47	17	9.0	66	17	1.0.0	1.0	**No**

1. When calling this method, `thisArg` does not default to the global object.
2. Also from version 14–79: partial support.
3. Also from version 14–79: Names for functions defined in a dictionary are properly assigned; however, anonymous functions defined on a var/let variable assignment have blank names.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `function`
- `function expression`
- `AsyncFunction`
- `AsyncGeneratorFunction`
- `GeneratorFunction`
- Functions

JavaScript › Reference › Standard built-in objects › Function

Function() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/Function · MDN page last modified 2026-10-08

Baseline: Widely available — available across major browsers since July 2015.

> **Warning:** The arguments passed to this constructor are dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations for more information.

The **`Function()`** constructor creates `Function` objects. Calling the constructor directly can create functions dynamically, but suffers from security and similar (but far less significant) performance issues as `eval()`. However, unlike `eval` (which may have access to the local scope), the `Function` constructor creates functions which execute in the global scope only.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Function() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const sum = new Function("a", "b", "return a + b");

console.log(sum(2, 6));
// Expected output: 8
```

Syntax

js

```
new Function(functionBody)
new Function(arg1, functionBody)
new Function(arg1, arg2, functionBody)
new Function(arg1, arg2, /* …, */ argN, functionBody)

Function(functionBody)
Function(arg1, functionBody)
Function(arg1, arg2, functionBody)
Function(arg1, arg2, /* …, */ argN, functionBody)
```

> **Note:** `Function()` can be called with or without `new`. Both create a new `Function` instance.

Parameters

`arg1, ..., argN` Optional

`TrustedScript` instances or strings specifying names to be used by the function as formal argument names. The value must correspond to a valid JavaScript parameter (any of plain identifier, rest parameter, or destructured parameter, optionally with a default), or a list of such strings separated with commas.

As the parameters are parsed in the same way as function expressions, whitespace and comments are accepted. For example: `"x", "theValue = 42", "[a, b] /* numbers */"` — or `"x, theValue = 42, [a, b] /* numbers */"`. (`"x, theValue = 42", "[a, b]"` is also correct, though very confusing to read.)

`functionBody`

A `TrustedScript` or a string containing the JavaScript statements comprising the function definition.

Exceptions

`SyntaxError`

Function parameter arguments can't be parsed as a valid parameter list, or the `functionBody` can't be parsed as valid JavaScript statements.

`TypeError`

Any parameter is a string when Trusted Types are enforced by a CSP and no default policy is defined.

Description

`Function` objects created with the `Function` constructor are parsed when the function is created. This is less efficient than creating a function with a function expression or function declaration and calling it within your code, because such functions are parsed with the rest of the code.

All arguments passed to the function, except the last, are treated as the names of the identifiers of the parameters in the function to be created, in the order in which they are passed. The function will be dynamically compiled as a function expression, with the source assembled in the following fashion:

```js
`function anonymous(${args.join(",")}
) {
${functionBody}
}`;
```

This is observable by calling the function's `toString()` method.

However, unlike normal function expressions, the name `anonymous` is not added to the `functionBody`'s scope, since `functionBody` only has access the global scope. If `functionBody` is not in strict mode (the body itself needs to have the `"use strict"` directive since it doesn't inherit the strictness from the context), you may use `arguments.callee` to refer to the function itself. Alternatively, you can define the recursive part as an inner function:

```js
const recursiveFn = new Function(
  "count",
  `
(function recursiveFn(count) {
  if (count < 0) {
    return;
  }
  console.log(count);
  recursiveFn(count - 1);
})(count);
`,
);
```

Note that the two dynamic parts of the assembled source — the parameters list `args.join(",")` and `functionBody` — will first be parsed separately to ensure they are each syntactically valid. This prevents injection-like attempts.

```js
new Function("/*", "*/) {");
// SyntaxError: Unexpected end of arg string
// Doesn't become "function anonymous(/*) {*/) {}"
```

Security considerations

The method can be used to execute arbitrary input passed to any parameter. If the input is a potentially unsafe string provided by a user, this is a possible vector for Cross-site-scripting (XSS) attacks. For example, the following example assumes the `untrustedCode` was provided by a user:

```
js · ✗ bad example
const untrustedCode = "alert('Potentially evil code!');";
const adder = new Function("a", "b", untrustedCode);
```

Websites with a Content Security Policy (CSP) that specifies `script-src` or `default-src` will prevent such code running by default. If you must allow the scripts to run via `Function()`, you can mitigate these issues by always assigning `TrustedScript` objects instead of strings, and enforcing trusted types using the `require-trusted-types-for` CSP directive. This ensures that the input is passed through a transformation function.

To allow `Function()` to run, you additionally need to specify the `trusted-types-eval` keyword in your CSP `script-src` directive. The `unsafe-eval` keyword also allows `Function()`, but is much less safe than `trusted-types-eval` because it would allow execution even on browsers that do not support trusted types.

For example, the required CSP for your site might look like this:

```http
Content-Security-Policy: require-trusted-types-for 'script'; script-src '<your_allowlist>'
'trusted-types-eval'
```

The behavior of the transformation function depends on the specific use case that requires a user-provided script. If possible, you should lock the allowed scripts to exactly the code that you trust to run. If that is not possible, you might allow or block the use of certain functions within the provided string.

Examples

Note that these examples omit the use of trusted types for brevity. For code showing the recommended approach, see Using `TrustedScript` in `eval()`.

Specifying arguments with the Function constructor

The following code creates a `Function` object that takes two arguments.

```js
// Example can be run directly in your JavaScript console

// Create a function that takes two arguments, and returns the sum of those arguments
const adder = new Function("a", "b", "return a + b");

// Call the function
adder(2, 6);
// 8
```

The arguments `a` and `b` are formal argument names that are used in the function body, `return a + b`.

Creating a function object from a function declaration or function expression

```js
// The function constructor can take in multiple statements separated by a semicolon.
Function expressions require a return statement with the function's name

// Observe that new Function is called. This is so we can call the function we created
directly afterwards
const sumOfArray = new Function(
  "const sumArray = (arr) => arr.reduce((previousValue, currentValue) => previousValue +
currentValue); return sumArray",
)();

// call the function
sumOfArray([1, 2, 3, 4]);
// 10

// If you don't call new Function at the point of creation, you can use the Function.call()
method to call it
const findLargestNumber = new Function(
  "function findLargestNumber (arr) { return Math.max(...arr) }; return findLargestNumber",
);

// call the function
findLargestNumber.call({}).call({}, [2, 4, 1, 8, 5]);
// 8

// Function declarations do not require a return statement
const sayHello = new Function(
  "return function (name) { return `Hello, ${name}` }",
)();

// call the function
sayHello("world");
// Hello, world
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Function()` constructor	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Requires arguments to be `TrustedScript` instance when trusted types are enforced	**No**	**No**	Prev.	**No**	26	**No**	**No**	**No**	26	**No**	**No**	26	?	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.Function`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Using the function constructor in `eval()`
- `function`
- `function` expression
- Functions

JavaScript › Reference › Standard built-in objects › Function

Function.prototype.apply()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/apply · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`apply()`** method of `Function` instances calls this function with a given `this` value, and `arguments` provided as an array (or an array-like object).

Try it

▷ Interactive example on MDN: "JavaScript Demo: Function.prototype.apply()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const numbers = [5, 6, 2, 3, 7];

const max = Math.max.apply(null, numbers);

console.log(max);
// Expected output: 7

const min = Math.min.apply(null, numbers);

console.log(min);
// Expected output: 2
```

Syntax

```js
apply(thisArg)
apply(thisArg, argsArray)
```

Parameters

`thisArg`

The value of `this` provided for the call to `func`. If the function is not in strict mode, `null` and `undefined` will be replaced with the global object, and primitive values will be converted to objects.

`argsArray` Optional

An array-like object, specifying the arguments with which `func` should be called, or `null` or `undefined` if no arguments should be provided to the function.

Return value

The result of calling the function with the specified `this` value and arguments.

Description

> **Note:** This function is almost identical to `call()`, except that the function arguments are passed to `call()` individually as a list, while for `apply()` they are combined in one object, typically an array — for example, `func.call(this, "eat", "bananas")` vs. `func.apply(this, ["eat", "bananas"])`.

Normally, when calling a function, the value of `this` inside the function is the object that the function was accessed on. With `apply()`, you can assign an arbitrary value as `this` when calling an existing function, without first attaching the function to the object as a property. This allows you to use methods of one object as generic utility functions.

You can also use any kind of object which is array-like as the second parameter. In practice, this means that it needs to have a `length` property, and integer ("index") properties in the range `(0..length - 1)`. For example, you could use a `NodeList`, or a custom object like `{ 'length': 2, '0': 'eat', '1': 'bananas' }`. You can also use `arguments`, for example:

```js
function wrapper() {
  return anotherFn.apply(null, arguments);
}
```

With the rest parameters and parameter spread syntax, this can be rewritten as:

```js
function wrapper(...args) {
  return anotherFn(...args);
}
```

In general, `fn.apply(null, args)` is equivalent to `fn(...args)` with the parameter spread syntax, except `args` is expected to be an array-like object in the former case with `apply()`, and an iterable object in the latter case with spread syntax.

> **Warning:** Do not use `apply()` to chain constructors (for example, to implement inheritance). This invokes the constructor function as a plain function, which means `new.target` is `undefined`, and classes throw an error because they can't be called without `new`. Use `Reflect.construct()` or `extends` instead.

Examples

Using apply() to append an array to another

You can use `Array.prototype.push()` to append an element to an array. Because `push()` accepts a variable number of arguments, you can also push multiple elements at once. But if you pass an array to `push()`, it will actually add that array as a single element, instead of adding the elements individually, ending up with an array inside an array. On the other hand, `Array.prototype.concat()` does have the desired behavior in this case, but it does not append to the *existing* array — it creates and returns a new array.

In this case, you can use `apply` to implicitly "spread" an array as a series of arguments.

```js
const array = ["a", "b"];
const elements = [0, 1, 2];
array.push.apply(array, elements);
console.info(array); // ["a", "b", 0, 1, 2]
```

The same effect can be achieved with the spread syntax.

```js
const array = ["a", "b"];
const elements = [0, 1, 2];
array.push(...elements);
console.info(array); // ["a", "b", 0, 1, 2]
```

Using apply() and built-in functions

Clever usage of `apply()` allows you to use built-in functions for some tasks that would probably otherwise require manually looping over a collection (or using the spread syntax).

For example, we can use `Math.max()` and `Math.min()` to find out the maximum and minimum value in an array.

```js
// min/max number in an array
const numbers = [5, 6, 2, 3, 7];

// using Math.min/Math.max apply
let max = Math.max.apply(null, numbers);
// This about equal to Math.max(numbers[0], …)
// or Math.max(5, 6, …)

let min = Math.min.apply(null, numbers);

// vs. loop based algorithm
max = -Infinity;
min = Infinity;
```

(code continues on the next page)

(code continued from the previous page)

```
for (const n of numbers) {
  if (n > max) {
    max = n;
  }
  if (n < min) {
    min = n;
  }
}
```

But beware: by using `apply()` (or the spread syntax) with an arbitrarily long arguments list, you run the risk of exceeding the JavaScript engine's argument length limit.

The consequences of calling a function with too many arguments (that is, more than tens of thousands of arguments) is unspecified and varies across engines. (The JavaScriptCore engine has a hard-coded argument limit of 65536.) Most engines throw an exception; but there's no normative specification preventing other behaviors, such as arbitrarily limiting the number of arguments actually passed to the applied function. To illustrate this latter case: if such an engine had a limit of four arguments (actual limits are of course significantly higher), it would be as if the arguments `5, 6, 2, 3` had been passed to `apply` in the examples above, rather than the full array.

If your value array might grow into the tens of thousands, use a hybrid strategy: apply your function to chunks of the array at a time:

js

```
function minOfArray(arr) {
  let min = Infinity;
  const QUANTUM = 32768;

  for (let i = 0; i < arr.length; i += QUANTUM) {
    const subMin = Math.min.apply(
      null,
      arr.slice(i, Math.min(i + QUANTUM, arr.length)),
    );
    min = Math.min(subMin, min);
  }

  return min;
}

const min = minOfArray([5, 6, 2, 3, 7]);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function.prototype.apply

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`apply`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
ES 5.1: generic array-like object as `arguments`	17	12	4	5	6	18	4	10.1	6	1.0	4.4	6	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.apply`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `arguments`
- `Function.prototype.bind()`
- `Function.prototype.call()`
- `Reflect.apply()`
- Functions
- Spread syntax (`...`)

JavaScript › Reference › Standard built-in objects › Function

Function.prototype.bind()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/bind · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`bind()`** method of `Function` instances creates a new function that, when called, calls this function with its `this` keyword set to the provided value, and a given sequence of arguments preceding any provided when the new function is called.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Function.prototype.bind()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const module = {
  x: 42,
  getX() {
    return this.x;
  },
};

const unboundGetX = module.getX;
console.log(unboundGetX()); // The function gets invoked at the global scope
// Expected output: undefined

const boundGetX = unboundGetX.bind(module);
console.log(boundGetX());
// Expected output: 42
```

Syntax

```js
bind(thisArg)
bind(thisArg, arg1)
bind(thisArg, arg1, arg2)
bind(thisArg, arg1, arg2, /* …, */ argN)
```

Parameters

`thisArg`

The value to be passed as the `this` parameter to the target function `func` when the bound function is called. If the function is not in strict mode, `null` and `undefined` will be replaced with the global object, and primitive values will be converted to objects. The value is ignored if the bound function is constructed using the `new` operator.

`arg1, ..., argN` Optional

Arguments to prepend to arguments provided to the bound function when invoking `func`.

Return value

A copy of the given function with the specified `this` value, and initial arguments (if provided).

Description

The `bind()` function creates a new *bound function*. Calling the bound function generally results in the execution of the function it wraps, which is also called the *target function*. The bound function will store the parameters passed — which include the value of `this` and the first few arguments — as its internal state. These values are stored in advance, instead of being passed at call time. You can generally see `const boundFn = fn.bind(thisArg, arg1, arg2)` as being equivalent to `const boundFn = (...restArgs) => fn.call(thisArg, arg1, arg2, ...restArgs)` for the effect when it's called (but not when `boundFn` is constructed).

A bound function can be further bound by calling `boundFn.bind(thisArg, /* more args */)`, which creates another bound function `boundFn2`. The newly bound `thisArg` value is ignored, because the target function of `boundFn2`, which is `boundFn`, already has a bound `this`. When `boundFn2` is called, it would call `boundFn`, which in turn calls `fn`. The arguments that `fn` ultimately receives are, in order: the arguments bound by `boundFn`, arguments bound by `boundFn2`, and the arguments received by `boundFn2`.

```js
"use strict"; // prevent `this` from being boxed into the wrapper object

function log(...args) {
  console.log(this, ...args);
}
const boundLog = log.bind("this value", 1, 2);
const boundLog2 = boundLog.bind("new this value", 3, 4);
boundLog2(5, 6); // "this value", 1, 2, 3, 4, 5, 6
```

A bound function may also be constructed using the `new` operator if its target function is constructable. Doing so acts as though the target function had instead been constructed. The prepended arguments are provided to the target function as usual, while the provided `this` value is ignored (because construction prepares its own `this`, as seen by the parameters of `Reflect.construct`). If the bound

function is directly constructed, `new.target` will be the target function instead. (That is, the bound function is transparent to `new.target`.)

```js
class Base {
  constructor(...args) {
    console.log(new.target === Base);
    console.log(args);
  }
}

const BoundBase = Base.bind(null, 1, 2);

new BoundBase(3, 4); // true, [1, 2, 3, 4]
```

However, because a bound function does not have the `prototype` property, it cannot be used as a base class for `extends`.

```js · ✗ bad example
class Derived extends class {}.bind(null) {}
// TypeError: Class extends value does not have valid prototype property undefined
```

When using a bound function as the right-hand side of `instanceof`, `instanceof` would reach for the target function (which is stored internally in the bound function) and read its `prototype` instead.

```js
class Base {}
const BoundBase = Base.bind(null, 1, 2);
console.log(new Base() instanceof BoundBase); // true
```

The bound function has the following properties:

`length`
: The `length` of the target function minus the number of arguments being bound (not counting the `thisArg` parameter), with 0 being the minimum value.

`name`
: The `name` of the target function plus a `"bound "` prefix.

The bound function also inherits the prototype chain of the target function. However, it doesn't have other own properties of the target function (such as static properties if the target function is a class).

Examples

Creating a bound function

The most common use of `bind()` is to make a function that, no matter how it is called, is called with a particular `this` value.

A common mistake for new JavaScript programmers is to extract a method from an object, then to later call that function and expect it to use the original object as its `this` (e.g., by using the method in callback-based code).

Without special care, however, the original object is usually lost. Creating a bound function from the function, using the original object, neatly solves this problem:

```js
// Top-level 'this' is bound to 'globalThis' in scripts.
this.x = 9;
const module = {
  x: 81,
  getX() {
    return this.x;
  },
};

// The 'this' parameter of 'getX' is bound to 'module'.
console.log(module.getX()); // 81

const retrieveX = module.getX;
// The 'this' parameter of 'retrieveX' is bound to 'globalThis' in non-strict mode.
console.log(retrieveX()); // 9

// Create a new function 'boundGetX' with the 'this' parameter bound to 'module'.
const boundGetX = retrieveX.bind(module);
console.log(boundGetX()); // 81
```

> **Note:** If you run this example in strict mode, the `this` parameter of `retrieveX` will be bound to `undefined` instead of `globalThis`, causing the `retrieveX()` call to fail.
>
> If you run this example in an ECMAScript module, top-level `this` will be bound to `undefined` instead of `globalThis`, causing the `this.x = 9` assignment to fail.
>
> If you run this example in a Node CommonJS module, top-level `this` will be bound to `module.exports` instead of `globalThis`. However, the `this` parameter of `retrieveX` will still be bound to `globalThis` in non-strict mode and to `undefined` in strict mode. Therefore, in non-strict mode (the default), the `retrieveX()` call will return `undefined` because `this.x = 9` is writing to a different object (`module.exports`) from what `getX` is reading from (`globalThis`).

In fact, some built-in "methods" are also getters that return bound functions — one notable example being `Intl.NumberFormat.prototype.format()`, which, when accessed, returns a bound function that you can directly pass as a callback.

Partially applied functions

Another use of `bind()` is to make a function with pre-specified initial arguments.

These arguments (if any) follow the provided `this` value and are then inserted at the start of the arguments passed to the target function, followed by whatever arguments are passed to the bound function at the time it is called.

```js
function list(...args) {
  return args;
}

function addArguments(arg1, arg2) {
  return arg1 + arg2;
}
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(list(1, 2, 3)); // [1, 2, 3]

console.log(addArguments(1, 2)); // 3

// Create a function with a preset leading argument
const leadingThirtySevenList = list.bind(null, 37);

// Create a function with a preset first argument.
const addThirtySeven = addArguments.bind(null, 37);

console.log(leadingThirtySevenList()); // [37]
console.log(leadingThirtySevenList(1, 2, 3)); // [37, 1, 2, 3]
console.log(addThirtySeven(5)); // 42
console.log(addThirtySeven(5, 10)); // 42
// (the last argument 10 is ignored)
```

With setTimeout()

By default, within `setTimeout()`, the `this` keyword will be set to `globalThis`, which is `window` in browsers. When working with class methods that require `this` to refer to class instances, you may explicitly bind `this` to the callback function, in order to maintain the instance.

js

```
class LateBloomer {
  constructor() {
    this.petalCount = Math.floor(Math.random() * 12) + 1;
  }
  bloom() {
    // Declare bloom after a delay of 1 second
    setTimeout(this.declare.bind(this), 1000);
  }
  declare() {
    console.log(`I am a beautiful flower with ${this.petalCount} petals!`);
  }
}

const flower = new LateBloomer();
flower.bloom();
// After 1 second, calls 'flower.declare()'
```

You can also use arrow functions for this purpose.

js

```
class LateBloomer {
  bloom() {
    // Declare bloom after a delay of 1 second
    setTimeout(() => this.declare(), 1000);
  }
}
```

Bound functions used as constructors

Bound functions are automatically suitable for use with the `new` operator to construct new instances created by the target function. When a bound function is used to construct a value, the provided `this` is ignored. However, provided arguments are still prepended to the constructor call.

```js
function Point(x, y) {
  this.x = x;
  this.y = y;
}

Point.prototype.toString = function () {
  return `${this.x},${this.y}`;
};

const p = new Point(1, 2);
p.toString();
// '1,2'

// The thisArg's value doesn't matter because it's ignored
const YAxisPoint = Point.bind(null, 0 /* x */);

const axisPoint = new YAxisPoint(5);
axisPoint.toString(); // '0,5'

axisPoint instanceof Point; // true
axisPoint instanceof YAxisPoint; // true
new YAxisPoint(17, 42) instanceof Point; // true
```

Note that you need not do anything special to create a bound function for use with `new`. `new.target`, `instanceof`, `this` etc. all work as expected, as if the constructor was never bound. The only difference is that it can no longer be used for `extends`.

The corollary is that you need not do anything special to create a bound function to be called plainly, even if you would rather require the bound function to only be called using `new`. If you call it without `new`, the bound `this` is suddenly not ignored.

```js
const emptyObj = {};
const YAxisPoint = Point.bind(emptyObj, 0 /* x */);

// Can still be called as a normal function
// (although usually this is undesirable)
YAxisPoint(13);

// The modifications to `this` is now observable from the outside
console.log(emptyObj); // { x: 0, y: 13 }
```

If you wish to restrict a bound function to only be callable with `new`, or only be callable without `new`, the target function must enforce that restriction, such as by checking `new.target !== undefined` or using a `class` instead.

Binding classes

Using `bind()` on classes preserves most of the class's semantics, except that all static own properties of the current class are lost. However, because the prototype chain is preserved, you can still access static properties inherited from the parent class.

```js
class Base {
  static baseProp = "base";
}

class Derived extends Base {
  static derivedProp = "derived";
}

const BoundDerived = Derived.bind(null);
console.log(BoundDerived.baseProp); // "base"
console.log(BoundDerived.derivedProp); // undefined
console.log(new BoundDerived() instanceof Derived); // true
```

Transforming methods to utility functions

`bind()` is also helpful in cases where you want to transform a method which requires a specific `this` value to a plain utility function that accepts the previous `this` parameter as a normal parameter. This is similar to how general-purpose utility functions work: instead of calling `array.map(callback)`, you use `map(array, callback)`, which allows you to use `map` with array-like objects that are not arrays (for example, `arguments`) without mutating `Object.prototype`.

Take `Array.prototype.slice()`, for example, which you want to use for converting an array-like object to a real array. You could create a shortcut like this:

```js
const slice = Array.prototype.slice;

// …

slice.call(arguments);
```

Note that you can't save `slice.call` and call it as a plain function, because the `call()` method also reads its `this` value, which is the function it should call. In this case, you can use `bind()` to bind the value of `this` for `call()`. In the following piece of code, `slice()` is a bound version of `Function.prototype.call()`, with the `this` value bound to `Array.prototype.slice()`. This means that additional `call()` calls can be eliminated:

```js
// Same as "slice" in the previous example
const unboundSlice = Array.prototype.slice;
const slice = Function.prototype.call.bind(unboundSlice);

// …

slice(arguments);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function.prototype.bind

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`bind`	7	12	4	11.6	5.1	18	4	12	6	1.0	4.4	6	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.bind`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Function.prototype.bind` in `core-js`
- `Function.prototype.apply()`
- `Function.prototype.call()`
- Functions

JavaScript › Reference › Standard built-in objects › Function

Function.prototype.call()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/call · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`call()`** method of `Function` instances calls this function with a given `this` value and arguments provided individually.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Function.prototype.call()". It runs live in the browser there; its source code follows.

js · interactive example source

```
function Product(name, price) {
  this.name = name;
  this.price = price;
}

function Food(name, price) {
  Product.call(this, name, price);
  this.category = "food";
}

console.log(new Food("cheese", 5).name);
// Expected output: "cheese"
```

Syntax

```js
call(thisArg)
call(thisArg, arg1)
call(thisArg, arg1, arg2)
call(thisArg, arg1, arg2, /* …, */ argN)
```

Parameters

`thisArg`

The value to use as `this` when calling `func`. If the function is not in strict mode, `null` and `undefined` will be replaced with the global object, and primitive values will be converted to objects.

`arg1, ..., argN` Optional

Arguments for the function.

Return value

The result of calling the function with the specified `this` value and arguments.

Description

> **Note:** This function is almost identical to `apply()`, except that the function arguments are passed to `call()` individually as a list, while for `apply()` they are combined in one object, typically an array — for example, `func.call(this, "eat", "bananas")` vs. `func.apply(this, ["eat", "bananas"])`.

Normally, when calling a function, the value of `this` inside the function is the object that the function was accessed on. With `call()`, you can assign an arbitrary value as `this` when calling an existing function, without first attaching the function to the object as a property. This allows you to use methods of one object as generic utility functions.

> **Warning:** Do not use `call()` to chain constructors (for example, to implement inheritance). This invokes the constructor function as a plain function, which means `new.target` is `undefined`, and classes throw an error because they can't be called without `new`. Use `Reflect.construct()` or `extends` instead.

Examples

Using call() to invoke a function and specifying the this value

In the example below, when we call `greet`, the value of `this` will be bound to object `obj`, even when `greet` is not a method of `obj`.

```js
function greet() {
  console.log(this.animal, "typically sleep between", this.sleepDuration);
}

const obj = {
  animal: "cats",
  sleepDuration: "12 and 16 hours",
};

greet.call(obj); // cats typically sleep between 12 and 16 hours
```

Using call() to invoke a function without specifying the first argument

If the first `thisArg` parameter is omitted, it defaults to `undefined`. In non-strict mode, the `this` value is then substituted with `globalThis` (which is akin to the global object).

```js
globalThis.globProp = "foo";

function display() {
  console.log(`globProp value is ${this.globProp}`);
}

display.call(); // Logs "globProp value is foo"
```

In strict mode, the value of `this` is not substituted, so it stays as `undefined`.

```js
"use strict";

globalThis.globProp = "foo";

function display() {
  console.log(`globProp value is ${this.globProp}`);
}

display.call(); // throws TypeError: Cannot read the property of 'globProp' of undefined
```

Transforming methods to utility functions

`call()` is almost equivalent to a normal function call, except that `this` is passed as a normal parameter instead of as the value that the function was accessed on. This is similar to how general-purpose utility functions work: instead of calling `array.map(callback)`, you use `map(array, callback)`, which allows you to use `map` with array-like objects that are not arrays (for example, `arguments`) without mutating `Object.prototype`.

Take `Array.prototype.slice()`, for example, which you want to use for converting an array-like object to a real array. You could create a shortcut like this:

```js
const slice = Array.prototype.slice;

// …

slice.call(arguments);
```

Note that you can't save `slice.call` and call it as a plain function, because the `call()` method also reads its `this` value, which is the function it should call. In this case, you can use `bind()` to bind the value of `this` for `call()`. In the following piece of code, `slice()` is a bound version of `Function.prototype.call()`, with the `this` value bound to `Array.prototype.slice()`. This means that additional `call()` calls can be eliminated:

```js
// Same as "slice" in the previous example
const unboundSlice = Array.prototype.slice;
const slice = Function.prototype.call.bind(unboundSlice);

// …

slice(arguments);
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function.prototype.call

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
call	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0 [1]

1. When calling this method, `thisArg` does not default to the global object.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.call`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Function.prototype.bind()`
- `Function.prototype.apply()`
- `Reflect.apply()`
- Spread syntax (`...`)
- Introduction to Object-Oriented JavaScript

JavaScript › Reference › Standard built-in objects › Function

Function.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/toString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`toString()`** method of `Function` instances returns a string representing the source code of this function.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Function.prototype.toString()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function sum(a, b) {
  return a + b;
}

console.log(sum.toString());
// Expected output: "function sum(a, b) {
//                     return a + b;
//                   }"

console.log(Math.abs.toString());
// Expected output: "function abs() { [native code] }"
```

Syntax

js

```
toString()
```

Parameters

None.

Return value

A string representing the source code of the function.

Description

The `Function` object overrides the `toString()` method inherited from `Object`; it does not inherit `Object.prototype.toString`. For user-defined `Function` objects, the `toString` method returns a string containing the source text segment which was used to define the function.

JavaScript calls the `toString` method automatically when a `Function` is to be represented as a text value, e.g., when a function is concatenated with a string.

The `toString()` method will throw a `TypeError` exception ("Function.prototype.toString called on incompatible object"), if its `this` value object is not a `Function` object.

js · ✗ bad example

```
Function.prototype.toString.call("foo"); // throws TypeError
```

If the `toString()` method is called on built-in function objects, a function created by `Function.prototype.bind()`, or other non-JavaScript functions, then `toString()` returns a *native function string* which looks like

```
function someName() { [native code] }
```

For intrinsic object methods and functions, `someName` is the initial name of the function; otherwise its content may be implementation-defined, but will always be in property name syntax, like `[1 + 1]`, `someName`, or `1`.

> **Note:** This means using `eval()` on native function strings is a guaranteed syntax error.

If the `toString()` method is called on a function created by the `Function` constructor, `toString()` returns the source code of a synthesized function declaration named "anonymous" using the provided parameters and function body. For example, `Function("a", "b", "return a + b").toString()` will return:

```
function anonymous(a,b
) {
return a + b
}
```

Since ES2018, the spec requires the return value of `toString()` to be the exact same source code as it was declared, including any whitespace and/or comments — or, if the host doesn't have the source code available for some reason, requires returning a native function string. Support for this revised behavior can be found in the compatibility table.

Examples

Comparing actual source code and toString results

js

```
function test(fn) {
  console.log(fn.toString());
}

function f() {}
class A {
  a() {}
}
function* g() {}

test(f); // "function f() {}"
test(A); // "class A { a() {} }"
test(g); // "function* g() {}"
test((a) => a); // "(a) => a"
test({ a() {} }.a); // "a() {}"
test({ *a() {} }.a); // "*a() {}"
test({ [0]() {} }[0]); // "[0]() {}"
test(Object.getOwnPropertyDescriptor({ get a() {} }, "a").get); // "get a() {}"
test(Object.getOwnPropertyDescriptor({ set a(x) {} }, "a").set); // "set a(x) {}"
test(Function.prototype.toString); // "function toString() { [native code] }"
test(function f() {}.bind(0)); // "function () { [native code] }"
test(Function("a", "b")); // function anonymous(a\n) {\nb\n}
```

Note that after the `Function.prototype.toString()` revision, when `toString()` is called, implementations are never allowed to synthesize a function's source that is not a native function string. The method always returns the exact source code used to create the function — including the getter and setter examples above. The `Function` constructor itself has the capability of synthesizing the source code for the function (and is therefore a form of implicit `eval()`).

Getting source text of a function

It is possible to get the source text of a function by coercing it to a string — for example, by wrapping it in a template literal:

```js
function foo() {
  return "bar";
}
console.log(`${foo}`);
// function foo() {
//   return "bar";
// }
```

This source text is *exact*, including any interspersed comments (which won't be stored by the engine's internal representation otherwise).

```js
function foo /* a comment */() {
  return "bar";
}
console.log(foo.toString());
// function foo /* a comment */() {
//   return "bar";
// }
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function.prototype.tostring`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0
Implements `Function.prototype.toString` revision	66	79	54	53	17	66	54	47	17	9.0	66	17	1.0.0	1.0	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Object.prototype.toString()`

JavaScript › Reference › Standard built-in objects › Function

Function.prototype[Symbol.hasInstance]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/Symbol.hasInstance · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since April 2017.

The **`[Symbol.hasInstance]()`** method of `Function` instances specifies the default procedure for determining if a constructor function recognizes an object as one of the constructor's instances. It is called by the `instanceof` operator.

Syntax

```
func[Symbol.hasInstance](value)
```

Parameters

`value`

The object to test. Primitive values always return `false`.

Return value

`true` if `func.prototype` is in the prototype chain of `value`; otherwise, `false`. Always returns `false` if `value` is not an object or `this` is not a function. If `this` is a bound function, returns the result of an `instanceof` test on `value` and the underlying target function.

Exceptions

`TypeError`

Thrown if `this` is not a bound function and `this.prototype` is not an object.

Description

The `instanceof` operator calls the `[Symbol.hasInstance]()` method of the right-hand side whenever such a method exists. Because all functions inherit from `Function.prototype` by default, they would all have the `[Symbol.hasInstance]()` method, so most of the time, the `Function.prototype[Symbol.hasInstance]()` method specifies the behavior of `instanceof` when the right-hand side is a function. This method implements the default behavior of the `instanceof` operator (the same algorithm when `constructor` has no `[Symbol.hasInstance]()` method).

Unlike most methods, the `Function.prototype[Symbol.hasInstance]()` property is non-configurable and non-writable. This is a security feature to prevent the underlying target function of a bound function from being obtainable. See this Stack Overflow answer for an example.

Examples

Reverting to default instanceof behavior

You would rarely need to call this method directly. Instead, this method is called by the `instanceof` operator. You should expect the two results to usually be equivalent.

```js
class Foo {}
const foo = new Foo();
console.log(foo instanceof Foo === Foo[Symbol.hasInstance](foo)); // true
```

You may want to use this method if you want to invoke the default `instanceof` behavior, but you don't know if a constructor has an overridden `[Symbol.hasInstance]()` method.

```js
class Foo {
  static [Symbol.hasInstance](value) {
    // A custom implementation
    return false;
  }
}

const foo = new Foo();
console.log(foo instanceof Foo); // false
console.log(Function.prototype[Symbol.hasInstance].call(Foo, foo)); // true
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function.prototype-%symbol.hasinstance%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.hasInstance]	50	15	50	37	10	50	50	37	10	5.0	50	10	1.0.0	1.0	6.5.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.@@hasInstance`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `instanceof`
- `Symbol.hasInstance`

JavaScript › Reference › Standard built-in objects › Function

Function.prototype.arguments

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/arguments · MDN page last modified 2026-08-21

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** The `arguments` property of `Function` objects is deprecated. The recommended way to access the `arguments` object is to refer to the variable `arguments` available within functions.

The **`arguments`** accessor property of `Function` instances returns the arguments passed to this function. For strict, arrow, async, and generator functions, accessing the `arguments` property throws a `TypeError`.

Description

The value of `arguments` is an array-like object corresponding to the arguments passed to a function.

In the case of recursion, i.e., if function `f` appears several times on the call stack, the value of `f.arguments` represents the arguments corresponding to the most recent invocation of the function.

The value of the `arguments` property is normally `null` if there is no outstanding invocation of the function in progress (that is, the function has been called but has not yet returned).

Note that the only behavior specified by the ECMAScript specification is that `Function.prototype` has an initial `arguments` accessor that unconditionally throws a `TypeError` for any `get` or `set` request (known as a "poison pill accessor"), and that implementations are not allowed to change this semantic for any function except non-strict plain functions. The actual behavior of the `arguments` property, if it's anything other than throwing an error, is implementation-defined. For example, Chrome defines it as an own data property, while Firefox and Safari extend the initial poison-pill `Function.prototype.arguments` accessor to specially handle `this` values that are non-strict functions.

```js
(function f() {
  if (Object.hasOwn(f, "arguments")) {
    console.log(
      "arguments is an own property with descriptor",
      Object.getOwnPropertyDescriptor(f, "arguments"),
    );
  } else {
    console.log(
      "f doesn't have an own property named arguments. Trying to get f.
[[Prototype]].arguments",
    );
    console.log(
      Object.getOwnPropertyDescriptor(
        Object.getPrototypeOf(f),
        "arguments",
      ).get.call(f),
    );
```

(code continues on the next page)

(code continued from the previous page)

```
  }
})();

// In Chrome:
// arguments is an own property with descriptor {value: Arguments(0), writable: false,
enumerable: false, configurable: false}

// In Firefox:
// f doesn't have an own property named arguments. Trying to get f.[[Prototype]].arguments
// Arguments { … }
```

Examples

Using the arguments property

js

```
function f(n) {
  g(n - 1);
}

function g(n) {
  console.log(`before: ${g.arguments[0]}`);
  if (n > 0) {
    f(n);
  }
  console.log(`after: ${g.arguments[0]}`);
}

f(2);

console.log(`returned: ${g.arguments}`);

// Logs:
// before: 1
// before: 0
// after: 0
// after: 1
// returned: null
```

Specifications

Not part of any standard.

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`arguments` Deprecated Non-standard	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.arguments`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `arguments`
- Functions

JavaScript › Reference › Standard built-in objects › Function

Function.prototype.caller

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/caller · MDN page last modified 2026-08-27

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

> **Note:** In strict mode, accessing `caller` of a function throws an error — the API is removed with no replacement. This is to prevent code from being able to "walk the stack", which both poses security risks and severely limits the possibility of optimizations like inlining and tail-call optimization. For more explanation, you can read the rationale for the deprecation of `arguments.callee`.

The **`caller`** accessor property of `Function` instances returns the function that invoked this function. For strict, arrow, async, and generator functions, accessing the `caller` property throws a `TypeError`.

Description

If the function `f` was invoked by the top-level code, the value of `f.caller` is `null`; otherwise it's the function that called `f`. If the function that called `f` is a strict mode function, the value of `f.caller` is also `null`.

Note that the only behavior specified by the ECMAScript specification is that `Function.prototype` has an initial `caller` accessor that unconditionally throws a `TypeError` for any `get` or `set` request (known as a "poison pill accessor"), and that implementations are not allowed to change this semantic for any function except non-strict plain functions, in which case it must not have the value of a strict mode function. The actual behavior of the `caller` property, if it's anything other than throwing an error, is implementation-defined. For example, Chrome defines it as an own data property, while Firefox and Safari extend the initial poison-pill `Function.prototype.caller` accessor to specially handle `this` values that are non-strict functions.

js

```
(function f() {
  if (Object.hasOwn(f, "caller")) {
    console.log(
      "caller is an own property with descriptor",
      Object.getOwnPropertyDescriptor(f, "caller"),
    );
  } else {
    console.log(
      "f doesn't have an own property named caller. Trying to get f.[[Prototype]].caller",
    );
    console.log(
      Object.getOwnPropertyDescriptor(
        Object.getPrototypeOf(f),
```

(code continues on the next page)

(code continued from the previous page)

```
        "caller",
      ).get.call(f),
    );
  }
})();

// In Chrome:
// caller is an own property with descriptor {value: null, writable: false, enumerable:
false, configurable: false}

// In Firefox:
// f doesn't have an own property named caller. Trying to get f.[[Prototype]].caller
// null
```

This property replaces the obsolete `arguments.caller` property of the `arguments` object.

The special property `__caller__`, which returned the activation object of the caller thus allowing you to reconstruct the stack, was removed for security reasons.

Examples

Checking the value of a function's caller property

The following code checks the value a function's `caller` property.

js

```
function myFunc() {
  if (myFunc.caller === null) {
    return "The function was called from the top!";
  }
  return `This function's caller was ${myFunc.caller}`;
}
```

Reconstructing the stack and recursion

Note that in case of recursion, you can't reconstruct the call stack using this property. Consider:

js

```
function f(n) {
  g(n - 1);
}
function g(n) {
  if (n > 0) {
    f(n);
  } else {
    stop();
  }
}
f(2);
```

At the moment `stop()` is called the call stack will be:

```
f(2) -> g(1) -> f(1) -> g(0) -> stop()
```

The following is true:

js

```
stop.caller === g && f.caller === g && g.caller === f;
```

so if you tried to get the stack trace in the `stop()` function like this:

```js
let f = stop;
let stack = "Stack trace:";
while (f) {
  stack += `\n${f.name}`;
  f = f.caller;
}
```

the loop would never stop.

Strict mode caller

If the caller is a strict mode function, the value of `caller` is `null`.

```js
function callerFunc() {
  calleeFunc();
}

function strictCallerFunc() {
  "use strict";
  calleeFunc();
}

function calleeFunc() {
  console.log(calleeFunc.caller);
}

(function () {
  callerFunc();
})();
// Logs [Function: callerFunc]

(function () {
  strictCallerFunc();
})();
// Logs null
```

Specifications

Not part of any standard.

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`caller` Deprecated Non-standard	1	12	1	9.6	3	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.caller`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Function.prototype.name`
- `arguments`

JavaScript › Reference › Standard built-in objects › Function

Function: displayName

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/displayName · MDN page last modified 2025-07-10

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The optional **`displayName`** property of a `Function` instance specifies the display name of the function.

Value

The `displayName` property is not initially present on any function — it's added by the code authors. For the purpose of display, it should be a string.

Description

The `displayName` property, if present, may be preferred by consoles and profilers over the `name` property to be displayed as the name of a function.

Among browsers, only the Firefox console utilizes this property. React devtools also use the `displayName` property when displaying the component tree.

Firefox does some basic attempts to decode the `displayName` that's possibly generated by the anonymous JavaScript functions naming convention algorithm. The following patterns are detected:

- If `displayName` ends with a sequence of alphanumeric characters, `_`, and `$`, the longest such suffix is displayed.
- If `displayName` ends with a sequence of `[]`-enclosed characters, that sequence is displayed without the square brackets.
- If `displayName` ends with a sequence of alphanumeric characters and `_` followed by some `/`, `.`, or `<`, the sequence is returned without the trailing `/`, `.`, or `<` characters.
- If `displayName` ends with a sequence of alphanumeric characters and `_` followed by `(^)`, the sequence is displayed without the `(^)`.

If none of the above patterns match, the entire `displayName` is displayed.

Examples

Setting a displayName

By entering the following in a Firefox console, it should display as something like `function MyFunction()`:

```js
function a() {}
a.displayName = "MyFunction";

a; // function MyFunction()
```

Changing displayName dynamically

You can dynamically change the `displayName` of a function:

```js
const object = {
  // anonymous
  someMethod: function someMethod(value) {
    someMethod.displayName = `someMethod (${value})`;
  },
};

console.log(object.someMethod.displayName); // undefined

object.someMethod("123");
console.log(object.someMethod.displayName); // "someMethod (123)"
```

Cleaning of displayName

Firefox devtools would clean up a few common patterns in the `displayName` property before displaying it.

```js
function foo() {}

function testName(name) {
  foo.displayName = name;
  console.log(foo);
}

testName("$foo$"); // function $foo$()
testName("foo bar"); // function bar()
testName("Foo.prototype.add"); // function add()
testName("foo ."); // function foo .()
testName("foo <"); // function foo <()
testName("foo?"); // function foo?()
testName("foo()"); // function foo()()

testName("[...]"); // function ...()
testName("foo<"); // function foo()
testName("foo..."); // function foo()
testName("foo(^)"); // function foo()
```

Specifications

Not part of any standard.

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`displayName` Non-standard	**No**	**No**	13	**No**	**No**	**No**	14	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.displayName`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Function.prototype.name`

JavaScript › Reference › Standard built-in objects › Function

Function: length

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/length · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since July 2015.

The **`length`** data property of a `Function` instance indicates the number of parameters expected by the function.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Function: length". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function func1() {}

function func2(a, b) {}

console.log(func1.length);
// Expected output: 0

console.log(func2.length);
// Expected output: 2
```

Value

A number.

Property attributes of `Function: length`	
Writable	no
Enumerable	no
Configurable	yes

Description

A `Function` object's `length` property indicates how many arguments the function expects, i.e., the number of formal parameters:

- Only parameters before the first one with a default value are counted.
- A destructuring pattern counts as a single parameter.
- The rest parameter is excluded.

By contrast, `arguments.length` is local to a function and provides the number of arguments actually passed to the function.

The `Function` constructor is itself a `Function` object. Its `length` data property has a value of `1`.

Due to historical reasons, `Function.prototype` is a callable itself. The `length` property of `Function.prototype` has a value of `0`.

Examples

Using function length

```js
console.log(Function.length); // 1

console.log((() => {}).length); // 0
console.log(((a) => {}).length); // 1
console.log(((a, b) => {}).length); // 2 etc.

console.log(((...args) => {}).length);
// 0, rest parameter is not counted

console.log(((a, b = 1, c) => {}).length);
// 1, only parameters before the first one with
// a default value are counted

console.log((({ a, b }, [c, d]) => {}).length);
// 2, destructuring patterns each count as
// a single parameter
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function-instances-length

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`length`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
Configurable: true	43	12	37	30	10	43	37	30	10	4.0	43	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.length`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Function`

JavaScript › Reference › Standard built-in objects › Function

Function: name

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/name · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since August 2016.

The **`name`** data property of a `Function` instance indicates the function's name as specified when it was created, or it may be either `anonymous` or `''` (an empty string) for functions created anonymously.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Function: name". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const func1 = function () {};

const object = {
  func2: function () {},
};

console.log(func1.name);
// Expected output: "func1"

console.log(object.func2.name);
// Expected output: "func2"
```

Value

A string.

Property attributes of `Function: name`	
Writable	no
Enumerable	no
Configurable	yes

> **Note:** In non-standard, pre-ES2015 implementations the `configurable` attribute was `false` as well.

Description

The function's `name` property can be used to identify the function in debugging tools or error messages. It has no semantic significance to the language itself.

The `name` property is read-only and cannot be changed by the assignment operator:

```js
function someFunction() {}

someFunction.name = "otherFunction";
console.log(someFunction.name); // someFunction
```

To change it, use `Object.defineProperty()`.

The `name` property is typically inferred from how the function is defined. In the following sections, we will describe the various ways in which it can be inferred.

Function declaration

The `name` property returns the name of a function declaration.

```js
function doSomething() {}
doSomething.name; // "doSomething"
```

Default-exported function declaration

An `export default` declaration exports the function as a declaration instead of an expression. If the declaration is anonymous, the name is `"default"`.

```js
// -- someModule.js --
export default function () {}

// -- main.js --
import someModule from "./someModule.js";

someModule.name; // "default"
```

Function constructor

Functions created with the `Function()` constructor have name "anonymous".

```js
new Function().name; // "anonymous"
```

Function expression

If the function expression is named, that name is used as the `name` property.

```js
const someFunction = function someFunctionName() {};
someFunction.name; // "someFunctionName"
```

Anonymous function expressions, created using either the `function` keyword or the arrow function syntax, have `""` (an empty string) as their name by default.

```js
(function () {}).name; // ""
(() => {}).name; // ""
```

However, such cases are rare — usually, in order to call the function elsewhere, the function expression is associated with an identifier. The name of an anonymous function expression can be inferred within certain syntactic contexts, including: variable declaration, method, initializer, and default value.

One practical case where the name cannot be inferred is a function returned from another function:

```js
function getFoo() {
  return () => {};
}
getFoo().name; // ""
```

Variable declaration and method

Variables and methods can infer the name of an anonymous function from its syntactic position.

```js
const f = function () {};
const object = {
  someMethod: function () {},
};

console.log(f.name); // "f"
console.log(object.someMethod.name); // "someMethod"
```

The same applies to assignment:

```js
let f;
f = () => {};
f.name; // "f"
```

Initializer and default value

Functions in initializers (default values) of destructuring, default parameters, class fields, etc., will inherit the name of the bound identifier as their `name`.

```js
const [f = () => {}] = [];
f.name; // "f"

const { someMethod: m = () => {} } = {};
m.name; // "m"

function foo(f = () => {}) {
  console.log(f.name);
}
foo(); // "f"

class Foo {
  static someMethod = () => {};
}
Foo.someMethod.name; // someMethod
```

Shorthand method

```js
const o = {
  foo() {},
};
o.foo.name; // "foo";
```

Bound function

`Function.prototype.bind()` produces a function whose name is "bound " plus the function name.

```js
function foo() {}
foo.bind({}).name; // "bound foo"
```

Getter and setter

When using `get` and `set` accessor properties, "get" or "set" will appear in the function name.

```js
const o = {
  get foo() {
    return 1;
  },
  set foo(x) {},
};

const descriptor = Object.getOwnPropertyDescriptor(o, "foo");
descriptor.get.name; // "get foo"
descriptor.set.name; // "set foo";
```

Class

A class's name follows the same algorithm as function declarations and expressions.

```js
class Foo {}
Foo.name; // "Foo"
```

> **Warning:** JavaScript will set the function's `name` property only if a function does not have an own property called `name`. However, classes' static members will be set as own properties of the class constructor function, and thus prevent the built-in `name` from being applied. See an example below.

Symbol as function name

If a `Symbol` is used a function name and the symbol has a description, the method's name is the description in square brackets.

```js
const sym1 = Symbol("foo");
const sym2 = Symbol();

const o = {
  [sym1]() {},
  [sym2]() {},
};

o[sym1].name; // "[foo]"
o[sym2].name; // "[]"
```

Private fields and methods

Private fields and private methods have the hash (#) as part of their names.

```js
class Foo {
  #field = () => {};
  #method() {}
  getNames() {
    console.log(this.#field.name);
    console.log(this.#method.name);
  }
}

new Foo().getNames();
// "#field"
// "#method"
```

Examples

Telling the constructor name of an object

You can use `obj.constructor.name` to check the "class" of an object.

```js
function Foo() {} // Or: class Foo {}

const fooInstance = new Foo();
console.log(fooInstance.constructor.name); // "Foo"
```

However, because static members will become own properties of the class, we can't obtain the class name for virtually any class with a static method property `name()`:

```js
class Foo {
  constructor() {}
  static name() {}
}
```

With a `static name()` method `Foo.name` no longer holds the actual class name but a reference to the `name()` function object. Trying to obtain the class of `fooInstance` via `fooInstance.constructor.name` won't give us the class name at all, but instead a reference to the static class method. Example:

```js
const fooInstance = new Foo();
console.log(fooInstance.constructor.name); // ƒ name() {}
```

Due to the existence of static fields, `name` may not be a function either.

```js
class Foo {
  static name = 123;
}
console.log(new Foo().constructor.name); // 123
```

If a class has a static property called `name`, it will also become *writable*. The built-in definition in the absence of a custom static definition is *read-only*:

```js
Foo.name = "Hello";
console.log(Foo.name); // "Hello" if class Foo has a static "name" property, but "Foo" if
not.
```

Therefore you may not rely on the built-in `name` property to always hold a class's name.

JavaScript compressors and minifiers

> **Warning:** Be careful when using the `name` property with source-code transformations, such as those carried out by JavaScript compressors (minifiers) or obfuscators. These tools are often used as part of a JavaScript build pipeline to reduce the size of a program prior to deploying it to production. Such transformations often change a function's name at build time.

Source code such as:

```js
function Foo() {}
const foo = new Foo();

if (foo.constructor.name === "Foo") {
  console.log("'foo' is an instance of 'Foo'");
} else {
  console.log("Oops!");
}
```

may be compressed to:

```js
function a() {}
const b = new a();
if (b.constructor.name === "Foo") {
  console.log("'foo' is an instance of 'Foo'");
} else {
  console.log("Oops!");
}
```

In the uncompressed version, the program runs into the truthy branch and logs "'foo' is an instance of 'Foo'" — whereas, in the compressed version it behaves differently, and runs into the else branch. If you rely on the `name` property, like in the example above, make sure your build pipeline doesn't change function names, or don't assume a function has a particular name.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function-instances-name`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`name`	15	14	1	10.5	6	18	4	11	6	1.0	4.4	6	1.0.0	1.0	0.10.0
Configurable: true	43	14	38	30	10	43	38	30	10	4.0	43	10	1.0.0	1.0	4.0.0
Inferred names on anonymous functions	51	79[1,2]	53	38	10	51	53	41	10	5.0	51	10	1.0.0	1.0	6.5.0

1. Also from version 14–79: partial support.
2. Also from version 14–79: Names for functions defined in a dictionary are properly assigned; however, anonymous functions defined on a var/let variable assignment have blank names.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Function.name`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill for `Function: name` in `core-js`
- es-shims polyfill of `Function.prototype.name`
- `Function`

JavaScript › Reference › Standard built-in objects › Function

Function: prototype

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Function/prototype · MDN page last modified 2025-07-10

The **`prototype`** data property of a `Function` instance is used when the function is used as a constructor with the `new` operator. It will become the new object's prototype.

> **Note:** Not all `Function` objects have the `prototype` property — see description.

Value

An object.

Property attributes of `Function: prototype`	
Writable	yes
Enumerable	no
Configurable	no

> **Note:** Classes are a type of function, so most of the description here applies to the `prototype` property of classes too. The only salient difference is that the `prototype` property of a class is not writable.

Description

When a function is called with `new`, the constructor's `prototype` property will become the resulting object's prototype.

```js
function Ctor() {}
const inst = new Ctor();
console.log(Object.getPrototypeOf(inst) === Ctor.prototype); // true
```

You can read Inheritance and the prototype chain for more information about the interactions between a constructor function's `prototype` property and the resulting object's prototype.

A function having a `prototype` property is not sufficient for it to be eligible as a constructor. Generator functions have a `prototype` property, but cannot be called with `new`:

```js
async function* asyncGeneratorFunction() {}
function* generatorFunction() {}
```

Instead, generator functions' `prototype` property is used when they are called *without* `new`. The `prototype` property will become the returned `Generator` object's prototype.

In addition, some functions may have a `prototype` but throw unconditionally when called with `new`. For example, the `Symbol()` and `BigInt()` functions throw when called with `new`, because `Symbol.prototype` and `BigInt.prototype` are only intended to provide methods for the primitive values, but the wrapper objects should not be directly constructed.

The following functions do not have `prototype`, and are therefore ineligible as constructors, even if a `prototype` property is later manually assigned:

```js
const method = { foo() {} }.foo;
const arrowFunction = () => {};
async function asyncFunction() {}
```

The following are valid constructors that have `prototype`:

```js
class Class {}
function fn() {}
```

A bound function does not have a `prototype` property, but may be constructable. When it's constructed, the target function is constructed instead, and if the target function is constructable, it would return a normal instance.

```js
const boundFunction = function () {}.bind(null);
```

A function's `prototype` property, by default, is a plain object with one property: `constructor`, which is a reference to the function itself. The `constructor` property is writable, non-enumerable, and configurable.

If the `prototype` of a function is reassigned with something other than an `Object`, when the function is called with `new`, the returned object's prototype would be `Object.prototype` instead. (In other words, `new` ignores the `prototype` property and constructs a plain object.)

```js
function Ctor() {}
Ctor.prototype = 3;
console.log(Object.getPrototypeOf(new Ctor()) === Object.prototype); // true
```

Examples

Changing the prototype of all instances by mutating the prototype property

```js
function Ctor() {}
const p1 = new Ctor();
const p2 = new Ctor();
Ctor.prototype.prop = 1;
console.log(p1.prop); // 1
console.log(p2.prop); // 1
```

Adding a non-method property to a class's prototype property

Class fields add properties to each instance. Class methods declare *function* properties on the prototype. However, there's no way to add a non-function property to the prototype. In case you want to share static data between all instances (for example, `Error.prototype.name` is the same between all error instances), you can manually assign it on the `prototype` of a class.

```js
class Dog {
  constructor(name) {
    this.name = name;
  }
}

Dog.prototype.species = "dog";

console.log(new Dog("Jack").species); // "dog"
```

This can be made more ergonomic using static initialization blocks, which are called when the class is initialized.

```js
class Dog {
  static {
    Dog.prototype.species = "dog";
  }
  constructor(name) {
    this.name = name;
  }
}

console.log(new Dog("Jack").species); // "dog"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/fundamental-objects.html#sec-function-instances-prototype`

See also

- `Function`
- Inheritance and the prototype chain

JavaScript › Reference › Standard built-in objects

Generator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator · MDN page last modified 2026-01-24

Baseline: Widely available — available across major browsers since September 2016.

The **`Generator`** object is returned by a generator function and it conforms to both the iterable protocol and the iterator protocol.

`Generator` is a subclass of the `Iterator` class.

Constructor

There's no JavaScript entity that corresponds to the `Generator` constructor. Instances of `Generator` must be returned from generator functions:

```js
function* generator() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = generator(); // "Generator { }"

console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3
```

There's only a hidden object which is the prototype object shared by all objects created by generator functions. This object is often stylized as `Generator.prototype` to make it look like a class, but it should be more appropriately called `GeneratorFunction.prototype.prototype`, because `GeneratorFunction` is an actual JavaScript entity. To understand the prototype chain of `Generator` instances, see `GeneratorFunction.prototype.prototype`.

Instance properties

These properties are defined on `Generator.prototype` and shared by all `Generator` instances.

`Generator.prototype.constructor`

The constructor function that created the instance object. For `Generator` instances, the initial value is `GeneratorFunction.prototype`.

> **Note:** `Generator` objects do not store a reference to the generator function that created them.

`Generator.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Generator"`. This property is used in `Object.prototype.toString()`.

Instance methods

Also inherits instance methods from its parent `Iterator`.

`Generator.prototype.next()`

Returns a value yielded by the `yield` expression.

`Generator.prototype.return()`

Acts as if a `return` statement is inserted in the generator's body at the current suspended position, which finishes the generator and allows the generator to perform any cleanup tasks when combined with a `try...finally` block.

`Generator.prototype.throw()`

Acts as if a `throw` statement is inserted in the generator's body at the current suspended position, which informs the generator of an error condition and allows it to handle the error, or perform cleanup and close itself.

Examples

An infinite iterator

With a generator function, values are not evaluated until they are needed. Therefore a generator allows us to define a potentially infinite data structure.

```js
function* infinite() {
  let index = 0;

  while (true) {
    yield index++;
  }
}

const generator = infinite(); // "Generator { }"

console.log(generator.next().value); // 0
console.log(generator.next().value); // 1
console.log(generator.next().value); // 2
// …
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-generator-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Generator`	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0
`next`	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0
`return`	50	13	38	37	10	50	38	37	10	5.0	50	10	1.0.0	1.0	6.0.0
`throw`	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Generator`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `function*`
- `function*` expression
- `GeneratorFunction`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › Generator

Generator.prototype.next()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator/next · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`next()`** method of `Generator` instances returns an object with two properties `done` and `value`. You can also provide a parameter to the `next` method to send a value to the generator.

Syntax

```js
next()
next(value)
```

Parameters

`value` Optional

The value to send to the generator.

The value will be assigned as a result of a `yield` expression. For example, in `variable = yield expression`, the value passed to the `.next()` function will be assigned to `variable`.

Return value

An `Object` with two properties:

`done`

A boolean value:

- `true` if the generator is past the end of its control flow. In this case `value` specifies the *return value* of the generator (which may be undefined).
- `false` if the generator is able to produce more values.

`value`

Any JavaScript value yielded or returned by the generator.

Exceptions

`TypeError`

Thrown if the generator is already running.

Examples

Using next()

The following example shows a generator and the object that the `next` method returns:

```js
function* gen() {
  yield 1;
  yield 2;
  yield 3;
}

const g = gen(); // Generator { }
g.next(); // { value: 1, done: false }
g.next(); // { value: 2, done: false }
g.next(); // { value: 3, done: false }
g.next(); // { value: undefined, done: true }
```

Using next() with a list

In this example, `getPage` takes a list and "paginates" it into chunks of size `pageSize`. Each call to `next` will yield one such chunk.

```js
function* getPage(list, pageSize = 1) {
  for (let index = 0; index < list.length; index += pageSize) {
    yield list.slice(index, index + pageSize);
  }
}

const list = [1, 2, 3, 4, 5, 6, 7, 8];
const page = getPage(list, 3); // Generator { }

page.next(); // { value: [1, 2, 3], done: false }
page.next(); // { value: [4, 5, 6], done: false }
page.next(); // { value: [7, 8], done: false }
page.next(); // { value: undefined, done: true }
```

Sending values to the generator

In this example, `next` is called with a value.

> **Note:** The first call does not log anything, because the generator was not yielding anything initially.

```js
function* gen() {
  while (true) {
    const value = yield;
    console.log(value);
  }
}

const g = gen();
g.next(1); // Returns { value: undefined, done: false }
// No log at this step: the first value sent through `next` is lost
g.next(2); // Returns { value: undefined, done: false }
// Logs 2
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-generator.prototype.next`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`next`	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Generator.next`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `function*`
- Iterators and generators guide

JavaScript › Reference › Standard built-in objects › Generator

Generator.prototype.return()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator/return · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since September 2016.

The **`return()`** method of `Generator` instances acts as if a `return` statement is inserted in the generator's body at the current suspended position, which finishes the generator and allows the generator to perform any cleanup tasks when combined with a `try...finally` block.

Syntax

```js
generatorInstance.return()
generatorInstance.return(value)
```

Parameters

`value` Optional

The value to return.

Return value

An `Object` with two properties:

done

A boolean value:

- `true` if the generator function's control flow has reached the end.

- `false` if the generator function's control flow hasn't reached the end and can produce more values. This can only happen if the `return` is captured in a `try...finally` and there are more `yield` expressions in the `finally` block.

`value`

The value that is given as an argument, or, if the `yield` expression is wrapped in a `try...finally`, the value yielded/returned from the `finally` block.

Exceptions

`TypeError`

Thrown if the generator is already running.

Description

The `return()` method, when called, can be seen as if a `return value;` statement is inserted in the generator's body at the current suspended position, where `value` is the value passed to the `return()` method. Therefore, in a typical flow, calling `return(value)` will return `{ done: true, value: value }`. However, if the `yield` expression is wrapped in a `try...finally` block, the control flow doesn't exit the function body, but proceeds to the `finally` block instead. In this case, the value returned may be different, and `done` may even be `false`, if there are more `yield` expressions within the `finally` block.

Examples

Using return()

The following example shows a generator and the `return` method.

```js
function* gen() {
  yield 1;
  yield 2;
  yield 3;
}

const g = gen();

g.next(); // { value: 1, done: false }
g.return("foo"); // { value: "foo", done: true }
g.next(); // { value: undefined, done: true }
```

If `return(value)` is called on a generator that is already in "completed" state, the generator will remain in "completed" state.

If no argument is provided, the `value` property of the returned object will be `undefined`. If an argument is provided, it will become the value of the `value` property of the returned object, unless the `yield` expression is wrapped in a `try...finally`.

```js
function* gen() {
  yield 1;
  yield 2;
  yield 3;
}

const g = gen();
g.next(); // { value: 1, done: false }
g.next(); // { value: 2, done: false }
g.next(); // { value: 3, done: false }
g.next(); // { value: undefined, done: true }
g.return(); // { value: undefined, done: true }
g.return(1); // { value: 1, done: true }
```

Using return() with try...finally

The fact that the `return` method has been called can only be made known to the generator itself if the `yield` expression is wrapped in a `try...finally` block.

When the `return` method is called on a generator that is suspended within a `try` block, execution in the generator proceeds to the `finally` block — since the `finally` block of `try...finally` statements always executes.

```js
function* gen() {
  yield 1;
  try {
    yield 2;
    yield 3;
  } finally {
    yield "cleanup";
  }
}

const g1 = gen();
g1.next(); // { value: 1, done: false }

// Execution is suspended before the try...finally.
g1.return("early return"); // { value: 'early return', done: true }

const g2 = gen();
g2.next(); // { value: 1, done: false }
g2.next(); // { value: 2, done: false }

// Execution is suspended within the try...finally.
g2.return("early return"); // { value: 'cleanup', done: false }

// The completion value is preserved
g2.next(); // { value: 'early return', done: true }

// Generator is in the completed state
g2.return("not so early return"); // { value: 'not so early return', done: true }
```

The return value of the finally block can also become the `value` of the result returned from the `return` call.

```js
function* gen() {
  try {
    yield 1;
  } finally {
    return "cleanup";
  }
}

const generator = gen();
generator.next(); // { value: 1, done: false }
generator.return("early return"); // { value: 'cleanup', done: true }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-generator.prototype.return

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
return	50	13	38	37	10	50	38	37	10	5.0	50	10	1.0.0	1.0	6.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Generator.return`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `function*`

JavaScript › Reference › Standard built-in objects › Generator

Generator.prototype.throw()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Generator/throw · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`throw()`** method of `Generator` instances acts as if a `throw` statement is inserted in the generator's body at the current suspended position, which informs the generator of an error condition and allows it to handle the error, or perform cleanup and close itself.

Syntax

```js
generatorInstance.throw(exception)
```

Parameters

`exception`

The exception to throw. For debugging purposes, it is useful to make it an `instanceof` `Error`.

Return value

If the thrown exception is caught by a `try...catch` and the generator resumes to yield more values, it will return an `Object` with two properties:

`done`

A boolean value:

- `true` if the generator function's control flow has reached the end.
- `false` if the generator function is able to produce more values.

`value`

The value yielded from the next `yield` expression.

Exceptions

`TypeError`

Thrown if the generator is already running.

If the `exception` is not caught by a `try...catch` within the generator function, it is also thrown to the caller of `throw()`.

Description

The `throw()` method, when called, can be seen as if a `throw exception;` statement is inserted in the generator's body at the current suspended position, where `exception` is the exception passed to the `throw()` method. Therefore, in a typical flow, calling `throw(exception)` will cause the generator to throw. However, if the `yield` expression is wrapped in a `try...catch` block, the error may be caught and control flow can either resume after error handling, or exit gracefully.

Examples

Using throw()

The following example shows a generator and an error that is thrown using the `throw` method. An error can be caught by a `try...catch` block as usual.

```js
function* gen() {
  while (true) {
    try {
      yield 42;
    } catch (e) {
      console.log("Error caught!");
    }
  }
}
```

(code continues on the next page)

(code continued from the previous page)

```
const g = gen();
g.next();
// { value: 42, done: false }
g.throw(new Error("Something went wrong"));
// "Error caught!"
// { value: 42, done: false }
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-generator.prototype.throw`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`throw`	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Generator.throw`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `function*`

JavaScript › Reference › Standard built-in objects

GeneratorFunction

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/GeneratorFunction · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`GeneratorFunction`** object provides methods for generator functions. In JavaScript, every generator function is actually a `GeneratorFunction` object.

Note that `GeneratorFunction` is *not* a global object. It can be obtained with the following code:

```
js
const GeneratorFunction = function* () {}.constructor;
```

`GeneratorFunction` is a subclass of `Function`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: GeneratorFunction". It runs live in the browser there; its source code follows.*

```
js · interactive example source
const GeneratorFunction = function* () {}.constructor;

const foo = new GeneratorFunction(`
  yield 'a';
  yield 'b';
  yield 'c';
`);

let str = "";
for (const val of foo()) {
  str += val;
}

console.log(str);
// Expected output: "abc"
```

Constructor

`GeneratorFunction()`

Creates a new `GeneratorFunction` object.

Instance properties

Also inherits instance properties from its parent `Function`.

These properties are defined on `GeneratorFunction.prototype` and shared by all `GeneratorFunction` instances.

`GeneratorFunction.prototype.constructor`

The constructor function that created the instance object. For `GeneratorFunction` instances, the initial value is the `GeneratorFunction` constructor.

`GeneratorFunction.prototype.prototype`

All generator functions share the same `prototype` property, which is `Generator.prototype`. Each generator function created with the `function*` syntax or the `GeneratorFunction()` constructor also

has its own `prototype` property, whose prototype is `GeneratorFunction.prototype.prototype`. When the generator function is called, its `prototype` property becomes the prototype of the returned generator object.

`GeneratorFunction.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"GeneratorFunction"`. This property is used in `Object.prototype.toString()`.

These properties are own properties of each `GeneratorFunction` instance.

`prototype`

Used when the function is used as a constructor with the `new` operator. It will become the new object's prototype.

Instance methods

Inherits instance methods from its parent `Function`.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-generatorfunction-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`GeneratorFunction`	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0
`GeneratorFunction()` constructor	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0
Requires arguments to be `TrustedScript` instance when trusted types are enforced	**No**	**No**	Prev.	**No**	26	**No**	**No**	**No**	26	**No**	**No**	26	?	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.GeneratorFunction`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `function*`
- `function*` expression
- `Function`
- `AsyncFunction`
- `AsyncGeneratorFunction`
- Functions

JavaScript › Reference › Standard built-in objects › GeneratorFunction

GeneratorFunction() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/GeneratorFunction/GeneratorFunction · MDN page last modified 2026-01-21

Baseline: Widely available — available across major browsers since September 2016.

> **Warning:** The arguments passed to this constructor are dynamically parsed and executed as JavaScript. APIs like this are known as injection sinks, and are potentially a vector for cross-site-scripting (XSS) attacks.
>
> You can mitigate this risk by always passing `TrustedScript` objects instead of strings and enforcing trusted types.
>
> See Security considerations in the `Function()` constructor reference for more information.

The **`GeneratorFunction()`** constructor creates `GeneratorFunction` objects.

Note that `GeneratorFunction` is *not* a global object. It can be obtained with the following code:

```js
const GeneratorFunction = function* () {}.constructor;
```

The `GeneratorFunction()` constructor is not intended to be used directly, and all caveats mentioned in the `Function()` description apply to `GeneratorFunction()`.

Syntax

```js
new GeneratorFunction(functionBody)
new GeneratorFunction(arg1, functionBody)
new GeneratorFunction(arg1, arg2, functionBody)
new GeneratorFunction(arg1, arg2, /* …, */ argN, functionBody)

GeneratorFunction(functionBody)
GeneratorFunction(arg1, functionBody)
GeneratorFunction(arg1, arg2, functionBody)
GeneratorFunction(arg1, arg2, /* …, */ argN, functionBody)
```

> **Note:** `GeneratorFunction()` can be called with or without `new`. Both create a new `GeneratorFunction` instance.

Parameters

See `Function()`.

Examples

Note that these examples omit the use of trusted types for brevity. For code showing the recommended approach, see Using `TrustedScript` in `eval()`.

Creating and using a GeneratorFunction() constructor

```js
const GeneratorFunction = function* () {}.constructor;
const g = new GeneratorFunction("a", "yield a * 2");
const iterator = g(10);
console.log(iterator.next().value); // 20
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-generatorfunction-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`GeneratorFunction()` constructor	39	13	26	26	10	39	26	26	10	4.0	39	10	1.0.0	1.0	4.0.0
Requires arguments to be `TrustedScript` instance when trusted types are enforced	**No**	**No**	Prev.	**No**	26	**No**	**No**	**No**	26	**No**	**No**	26	?	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.GeneratorFunction.GeneratorFunction`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `function*`
- `function*` expression
- `Function()` constructor
- Iterators and generators guide
- Functions

JavaScript › Reference › Standard built-in objects › GeneratorFunction

GeneratorFunction.prototype.prototype

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/GeneratorFunction/prototype · MDN page last modified 2025-07-10

The **`prototype`** property of `GeneratorFunction.prototype` is shared by all generator functions. Its value is `Generator.prototype`. Each generator function created with the `function*` syntax or the `GeneratorFunction()` constructor also has its own `prototype` property, whose prototype is `GeneratorFunction.prototype.prototype`. When the generator function is called, its `prototype` property becomes the prototype of the returned generator object.

Value

The same object as `Generator.prototype`. `GeneratorFunction.prototype.prototype` is the technically more accurate name, but `Generator.prototype` appeals to the intuition that it's the prototype of generator objects.

Property attributes of `GeneratorFunction.prototype.prototype`	
Writable	yes
Enumerable	no
Configurable	no

The `prototype` property of each `GeneratorFunction` instance is an empty object with no properties, whose prototype is `GeneratorFunction.prototype.prototype`. It has the following property attributes:

Property attributes of `GeneratorFunction.prototype.prototype`	
Writable	no
Enumerable	no
Configurable	yes

Description

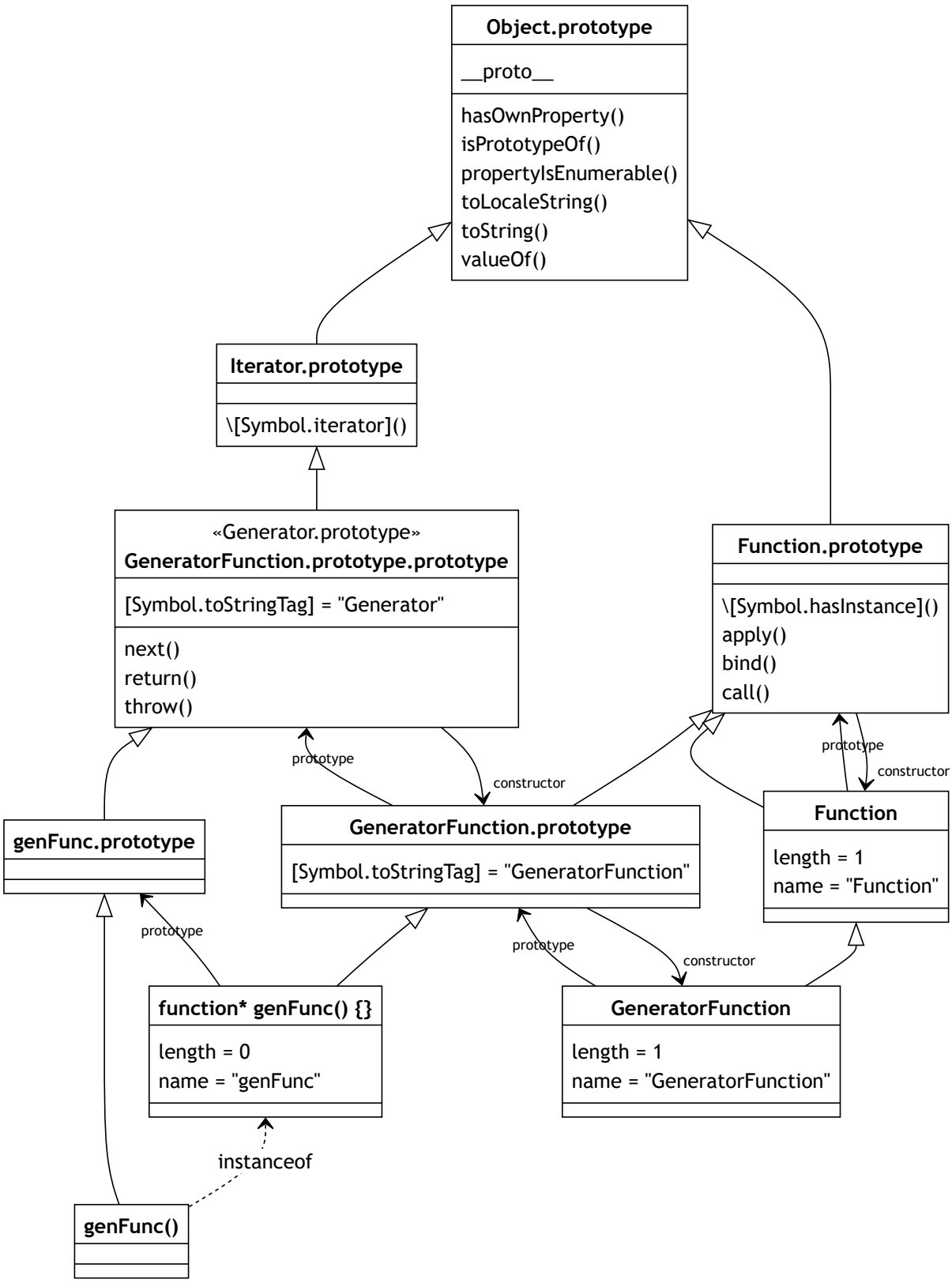
A generator function instance has two `prototype` properties. The first one is its own `prototype` property. The second one is the `prototype` property on its prototype, which is `GeneratorFunction.prototype`. (Remember that every generator function is an instance of `GeneratorFunction`, so it has `GeneratorFunction.prototype` as its prototype.)

```js
function* genFunc() {}
const GeneratorFunctionPrototype = Object.getPrototypeOf(genFunc);
console.log(Object.hasOwn(genFunc, "prototype")); // true
console.log(Object.hasOwn(GeneratorFunctionPrototype, "prototype")); // true
```

When a generator function is called, the generator function's `prototype` property becomes the prototype of the returned generator object.

```js
const gen = genFunc();
const proto = Object.getPrototypeOf;
console.log(proto(gen) === genFunc.prototype); // true
console.log(proto(proto(gen)) === GeneratorFunctionPrototype.prototype); // true
```

The following diagram illustrates the prototype chain of a generator function and its instances. Each hollow arrow indicates an inheritance relationship (i.e., a prototype link), and each solid arrow indicates a property relationship. Note that there's no way to access `genFunc` from `gen` — they only have an `instanceof` relationship.



Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-generatorfunction.prototype.prototype
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-generatorfunction-instances-prototype

See also

- `function*`
- `function*` expression
- `AsyncGeneratorFunction`
- `GeneratorFunction`
- Inheritance and the prototype chain
- Iterators and generators

JavaScript › Reference › Standard built-in objects

globalThis

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/globalThis · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since January 2020.

The **`globalThis`** global property contains the global `this` value, which is usually akin to the global object.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: globalThis". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function canMakeHTTPRequest() {
  return typeof globalThis.XMLHttpRequest === "function";
}

console.log(canMakeHTTPRequest());
// Expected output (in a browser): true
```

Value

The global `this` object.

Property attributes of `globalThis`	
Writable	yes
Enumerable	no
Configurable	yes

> **Note:** The `globalThis` property is configurable and writable so that code authors can hide it when executing untrusted code and prevent exposing the global object.

Description

Historically, accessing the global object has required different syntax in different JavaScript environments. On the web you can use `window`, `self`, or `frames` - but in Web Workers only `self` will work. In Node.js none of these work, and you must instead use `global`. The `this` keyword could be used inside functions running in non–strict mode, but `this` will be `undefined` in modules and inside functions running in strict mode. You can also use `Function('return this')()`, but environments that disable `eval()`, like CSP in browsers, prevent use of `Function` in this way.

The `globalThis` property provides a standard way of accessing the global `this` value (and hence the global object itself) across environments. Unlike similar properties such as `window` and `self`, it's guaranteed to work in window and non-window contexts. In this way, you can access the global object in a consistent manner without having to know which environment the code is being run in. To help you remember the name, just remember that in global scope the `this` value is `globalThis`.

> **Note:** `globalThis` is generally the same concept as the global object (i.e., adding properties to `globalThis` makes them global variables) — this is the case for browsers and Node — but hosts are allowed to provide a different value for `globalThis` that's unrelated to the global object.

HTML and the WindowProxy

In many engines `globalThis` will be a reference to the actual global object, but in web browsers, due to iframe and cross-window security considerations, it references a `Proxy` around the actual global object (which you can't directly access). This distinction is rarely relevant in common usage, but important to be aware of.

Naming

Several other popular name choices such as `self` and `global` were removed from consideration because of their potential to break compatibility with existing code. See the language proposal's "naming" document for more details.

`globalThis` is, quite literally, the global `this` value. It's the same value as the `this` value in a non-strict function called without an object. It's also the value of `this` in the global scope of a script.

Examples

Search for the global across environments

Usually, the global object does not need to be explicitly specified — its properties are automatically accessible as global variables.

```js
console.log(window.Math === Math); // true
```

However, one case where one needs to explicitly access the global object is when *writing* to it, usually for the purpose of polyfills.

Prior to `globalThis`, the only reliable cross-platform way to get the global object for an environment was `Function('return this')()`. However, this causes CSP violations in some settings, so authors would use a piecewise definition like this (slightly adapted from the original core-js source):

```js
function check(it) {
  // Math is known to exist as a global in every environment.
  return it && it.Math === Math && it;
}

const globalObject =
  check(typeof window === "object" && window) ||
  check(typeof self === "object" && self) ||
  check(typeof global === "object" && global) ||
  // This returns undefined when running in strict mode
  (function () {
    return this;
  })() ||
  Function("return this")();
```

After obtaining the global object, we can define new globals on it. For example, adding an implementation for `Intl`:

```js
if (typeof globalObject.Intl === "undefined") {
  // No Intl in this environment; define our own on the global scope
  Object.defineProperty(globalObject, "Intl", {
    value: {
      // Our Intl implementation
    },
    enumerable: false,
    configurable: true,
    writable: true,
  });
}
```

With `globalThis` available, the additional search for the global across environments is not necessary anymore:

```js
if (typeof globalThis.Intl === "undefined") {
  Object.defineProperty(globalThis, "Intl", {
    value: {
      // Our Intl implementation
    },
    enumerable: false,
    configurable: true,
    writable: true,
  });
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/global-object.html#sec-globalthis`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`globalThis`	71	79	65	58	12.1	71	65	50	12.2	10.0	71	12.2	1.0.0	1.0	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.globalThis`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `globalThis` in `core-js`
- es-shims polyfill of `globalThis`
- `this`

JavaScript › Reference › Standard built-in objects

Infinity

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Infinity · MDN page last modified 2025-07-08

Baseline: Widely available — available across major browsers since July 2015.

The **`Infinity`** global property is a numeric value representing infinity.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Infinity". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const maxNumber = 10 ** 1000; // Max positive number

if (maxNumber === Infinity) {
  console.log("Let's call it Infinity!");
  // Expected output: "Let's call it Infinity!"
}

console.log(1 / maxNumber);
// Expected output: 0
```

Value

The same number value as `Number.POSITIVE_INFINITY`.

Property attributes of `Infinity`	
Writable	no
Enumerable	no
Configurable	no

Description

`Infinity` is a property of the *global object*. In other words, it is a variable in global scope.

The value `Infinity` (positive infinity) is greater than any other number.

This value behaves slightly differently than mathematical infinity; see `Number.POSITIVE_INFINITY` for details.

Examples

Using Infinity

js

```
console.log(Infinity); /* Infinity */
console.log(Infinity + 1); /* Infinity */
console.log(10 ** 1000); /* Infinity */
console.log(Math.log(0)); /* -Infinity */
console.log(1 / Infinity); /* 0 */
console.log(1 / 0); /* Infinity */
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/global-object.html#sec-value-properties-of-the-global-object-infinity`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Infinity`	1	12	1	4	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Infinity`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Number.NEGATIVE_INFINITY`
- `Number.POSITIVE_INFINITY`
- `Number.isFinite`

JavaScript › Reference › Standard built-in objects

Int8Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Int8Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Int8Array`** typed array represents an array of 8-bit signed integers. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Int8Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Int8Array()`

Creates a new `Int8Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Int8Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of `Int8Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Int8Array.prototype` and shared by all `Int8Array` instances.

`Int8Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `1` in the case of an `Int8Array`.

`Int8Array.prototype.constructor`

The constructor function that created the instance object. For `Int8Array` instances, the initial value is the `Int8Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create an Int8Array

```js
// From a length
const int8 = new Int8Array(2);
int8[0] = 42;
console.log(int8[0]); // 42
console.log(int8.length); // 2
console.log(int8.BYTES_PER_ELEMENT); // 1

// From an array
const x = new Int8Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int8Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Int8Array(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int8FromIterable = new Int8Array(iterable);
console.log(int8FromIterable);
// Int8Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Int8Array`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`Int8Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Int8Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Int8Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Int8Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

Int8Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Int8Array/Int8Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Int8Array()`** constructor creates `Int8Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Int8Array()
new Int8Array(length)
new Int8Array(typedArray)
new Int8Array(object)

new Int8Array(buffer)
new Int8Array(buffer, byteOffset)
new Int8Array(buffer, byteOffset, length)
```

> **Note:** `Int8Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create an Int8Array

```js
// From a length
const int8 = new Int8Array(2);
int8[0] = 42;
console.log(int8[0]); // 42
console.log(int8.length); // 2
console.log(int8.BYTES_PER_ELEMENT); // 1
```

(code continues on the next page)

(code continued from the previous page)

```
// From an array
const x = new Int8Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int8Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(8);
const z = new Int8Array(buffer, 1, 4);
console.log(z.byteOffset); // 1

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int8FromIterable = new Int8Array(iterable);
console.log(int8FromIterable);
// Int8Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Int8Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Int8Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Int8Array.Int8Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Int8Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

Int16Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Int16Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Int16Array`** typed array represents an array of 16-bit signed integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Int16Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Int16Array()`

Creates a new `Int16Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Int16Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of `Int16Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Int16Array.prototype` and shared by all `Int16Array` instances.

`Int16Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `2` in the case of an `Int16Array`.

`Int16Array.prototype.constructor`

The constructor function that created the instance object. For `Int16Array` instances, the initial value is the `Int16Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create an Int16Array

```js
// From a length
const int16 = new Int16Array(2);
int16[0] = 42;
console.log(int16[0]); // 42
console.log(int16.length); // 2
console.log(int16.BYTES_PER_ELEMENT); // 2

// From an array
const x = new Int16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(16);
const z = new Int16Array(buffer, 2, 4);
console.log(z.byteOffset); // 2

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int16FromIterable = new Int16Array(iterable);
console.log(int16FromIterable);
// Int16Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Int16Array`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`Int16Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Int16Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Int16Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Int16Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects › Int16Array

Int16Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Int16Array/Int16Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Int16Array()`** constructor creates `Int16Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Int16Array()
new Int16Array(length)
new Int16Array(typedArray)
new Int16Array(object)

new Int16Array(buffer)
new Int16Array(buffer, byteOffset)
new Int16Array(buffer, byteOffset, length)
```

> **Note:** `Int16Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create an Int16Array

```js
// From a length
const int16 = new Int16Array(2);
int16[0] = 42;
console.log(int16[0]); // 42
console.log(int16.length); // 2
console.log(int16.BYTES_PER_ELEMENT); // 2
```

(code continues on the next page)

(code continued from the previous page)

```
// From an array
const x = new Int16Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int16Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(16);
const z = new Int16Array(buffer, 2, 4);
console.log(z.byteOffset); // 2

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int16FromIterable = new Int16Array(iterable);
console.log(int16FromIterable);
// Int16Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Int16Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Int16Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Int16Array.Int16Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Int16Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects

Int32Array

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Int32Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Int32Array`** typed array represents an array of 32-bit signed integers in the platform byte order. If control over byte order is needed, use `DataView` instead. The contents are initialized to `0` unless initialization data is explicitly provided. Once established, you can reference elements in the array using the object's methods, or using standard array index syntax (that is, using bracket notation).

`Int32Array` is a subclass of the hidden `TypedArray` class.

Constructor

`Int32Array()`

Creates a new `Int32Array` object.

Static properties

Also inherits static properties from its parent `TypedArray`.

`Int32Array.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of `Int32Array`.

Static methods

Inherits static methods from its parent `TypedArray`.

Instance properties

Also inherits instance properties from its parent `TypedArray`.

These properties are defined on `Int32Array.prototype` and shared by all `Int32Array` instances.

`Int32Array.prototype.BYTES_PER_ELEMENT`

Returns a number value of the element size. `4` in the case of an `Int32Array`.

`Int32Array.prototype.constructor`

The constructor function that created the instance object. For `Int32Array` instances, the initial value is the `Int32Array` constructor.

Instance methods

Inherits instance methods from its parent `TypedArray`.

Examples

Different ways to create an Int32Array

```js
// From a length
const int32 = new Int32Array(2);
int32[0] = 42;
console.log(int32[0]); // 42
console.log(int32.length); // 2
console.log(int32.BYTES_PER_ELEMENT); // 4

// From an array
const x = new Int32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Int32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int32FromIterable = new Int32Array(iterable);
console.log(int32FromIterable);
// Int32Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Int32Array`	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
`Int32Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Int32Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Int32Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Int32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

JavaScript › Reference › Standard built-in objects › Int32Array

Int32Array() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Int32Array/Int32Array · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since July 2015.

The **`Int32Array()`** constructor creates `Int32Array` objects. The contents are initialized to `0` unless initialization data is explicitly provided.

Syntax

```js
new Int32Array()
new Int32Array(length)
new Int32Array(typedArray)
new Int32Array(object)

new Int32Array(buffer)
new Int32Array(buffer, byteOffset)
new Int32Array(buffer, byteOffset, length)
```

> **Note:** `Int32Array()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

See `TypedArray`.

Exceptions

See `TypedArray`.

Examples

Different ways to create an Int32Array

```js
// From a length
const int32 = new Int32Array(2);
int32[0] = 42;
console.log(int32[0]); // 42
console.log(int32.length); // 2
console.log(int32.BYTES_PER_ELEMENT); // 4
```

(code continues on the next page)

(code continued from the previous page)

```
// From an array
const x = new Int32Array([21, 31]);
console.log(x[1]); // 31

// From another TypedArray
const y = new Int32Array(x);
console.log(y[0]); // 21

// From an ArrayBuffer
const buffer = new ArrayBuffer(32);
const z = new Int32Array(buffer, 4, 4);
console.log(z.byteOffset); // 4

// From an iterable
const iterable = (function* () {
  yield* [1, 2, 3];
})();
const int32FromIterable = new Int32Array(iterable);
console.log(int32FromIterable);
// Int32Array [1, 2, 3]
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/indexed-collections.html#sec-typedarray-constructors`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Int32Array()` constructor	7	12	4	11.6	5.1	18	4	12	4.2	1.0	4	4.2	1.0.0	1.0	0.10.0
Constructor without parameters	7	12	55	11.6	5.1	18	55	12	5	1.0	4.4	5	1.0.0	1.0	0.10.0
`new Int32Array(iterable)`	39	14	52	26	10	39	52	26	10	4.0	39	10	1.0.0	1.0	4.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Int32Array.Int32Array`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Int32Array` in `core-js`
- JavaScript typed arrays guide
- `TypedArray`
- `ArrayBuffer`
- `DataView`

InternalError

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/InternalError · MDN page last modified 2025-07-10

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`InternalError`** object indicates an error that occurred internally in the JavaScript engine.

Example cases are mostly when something is too large, e.g.:

- "too many switch cases",
- "too many parentheses in regular expression",
- "array initializer too large",
- "too much recursion".

`InternalError` is a subclass of `Error`.

Constructor

`InternalError()` Non-standard

Creates a new `InternalError` object.

Instance properties

Also inherits instance properties from its parent `Error`.

These properties are defined on `InternalError.prototype` and shared by all `InternalError` instances.

`InternalError.prototype.constructor`

The constructor function that created the instance object. For `InternalError` instances, the initial value is the `InternalError` constructor.

`InternalError.prototype.name`

Represents the name for the type of error. For `InternalError.prototype.name`, the initial value is `"InternalError"`.

Instance methods

Inherits instance methods from its parent `Error`.

Examples

Too much recursion

This recursive function runs 10 times, as per the exit condition.

```js
function loop(x) {
  // "x >= 10" is the exit condition
  if (x >= 10) return;

  // do stuff
  loop(x + 1); // the recursive call
}
loop(0);
```

Setting this condition to an extremely high value, may not work:

```js · ✗ bad example
function loop(x) {
  if (x >= 1000000000000) return;

  // do stuff
  loop(x + 1);
}
loop(0);

// InternalError: too much recursion
```

For more information, see InternalError: too much recursion.

Specifications

Not part of any standard.

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`InternalError` Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**
`InternalError()` constructor Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.InternalError`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error`
- InternalError: too much recursion

JavaScript › Reference › Standard built-in objects › InternalError

InternalError() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/InternalError/InternalError · MDN page last modified 2025-07-10

> **Non-standard:** This feature is not standardized. We do not recommend using non-standard features in production, as they have limited browser support, and may change or be removed. However, they can be a suitable alternative in specific cases where no standard option exists.

The **`InternalError()`** constructor creates `InternalError` objects.

Syntax

```js
new InternalError()
new InternalError(message)
new InternalError(message, options)
new InternalError(message, fileName)
new InternalError(message, fileName, lineNumber)

InternalError()
InternalError(message)
InternalError(message, options)
InternalError(message, fileName)
InternalError(message, fileName, lineNumber)
```

> **Note:** `InternalError()` can be called with or without `new`. Both create a new `InternalError` instance.

Parameters

`message` Optional

Human-readable description of the error.

`options` Optional

An object that has the following properties:

`cause` Optional

A property indicating the specific cause of the error. When catching and re-throwing an error with a more-specific or useful error message, this property can be used to pass the original error.

`fileName` Optional Non-standard

The name of the file containing the code that caused the exception

`lineNumber` Optional Non-standard

The line number of the code that caused the exception

Examples

Creating a new InternalError

```js
new InternalError("Engine failure");
```

Specifications

Not part of any standard.

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`InternalError()` constructor Non-standard	**No**	**No**	1	**No**	**No**	**No**	4	**No**	**No**	**No**	**No**	**No**	?	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.InternalError.InternalError`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Error`

JavaScript › Reference › Standard built-in objects

Intl

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl`** namespace object contains several constructors as well as functionality common to the internationalization constructors and other language sensitive functions. Collectively, they comprise the ECMAScript Internationalization API, which provides language sensitive string comparison, number formatting, date and time formatting, and more.

Description

Unlike most global objects, `Intl` is not a constructor. You cannot use it with the new operator or invoke the `Intl` object as a function. All properties and methods of `Intl` are static (just like the `Math` object).

The internationalization constructors as well as several language sensitive methods of other constructors (listed under See also) use a common pattern for identifying locales and determining the one they will actually use: they all accept `locales` and `options` arguments, and negotiate the requested locale(s) against the locales they support using an algorithm specified in the `options.localeMatcher` property.

locales argument

The `locales` argument is used to determine the locale used in a given operation. The JavaScript implementation examines `locales`, and then computes a locale it understands that comes closest to satisfying the expressed preference. `locales` may be:

- `undefined` (or omitted): The implementation's default locale will be used.
- A locale: A locale identifier or an `Intl.Locale` object that wraps a locale identifier.
- A list of locales: Any other value, that will be converted into an object and then treated as an array of locales.

In the latter two cases, the actual locale used is the best-supported locale determined by locale negotiation. If a locale identifier is not a string or an object, a `TypeError` is thrown. If a locale identifier is a string that's syntactically invalid, a `RangeError` is thrown. If a locale identifier is well-formed but the implementation doesn't recognize it, it is ignored and the next locale in the list is considered, eventually falling back to the system's locale. However, you shouldn't rely on a particular locale name being ignored, because the implementation may add data for any locale in the future. For example, `new Intl.DateTimeFormat("default")` uses the implementation's default locale only because `"default"` is syntactically valid but not recognized as any locale.

A locale identifier is a string that consists of:

1. A language subtag with 2–3 or 5–8 letters
2. A script subtag with 4 letters Optional
3. A region subtag with either 2 letters or 3 digits Optional
4. One or more variant subtags (all of which must be unique), each with either 5–8 alphanumerals or a digit followed by 3 alphanumerals Optional
5. One or more BCP 47 extension sequences Optional
6. A private-use extension sequence Optional

Each subtag and sequence are separated by hyphens. Locale identifiers are case-insensitive ASCII. However, it's conventional to use title case (the first letter is capitalized, successive letters are lower case) for script subtags, upper case for region subtags, and lower case for everything else. For example:

- `"hi"`: Hindi (language)
- `"de-AT"`: German (language) as used in Austria (region)
- `"zh-Hans-CN"`: Chinese (language) written in simplified characters (script) as used in China (region)
- `"en-emodeng"`: English (language) in the "Early modern English" dialect (variant)

Subtags identifying languages, scripts, regions (including countries), and (rarely used) variants are registered in the IANA Language Subtag Registry. This registry is periodically updated over time, and implementations may not always be up to date, so don't rely too much on subtags being universally supported.

BCP 47 extension sequences consist of a single digit or letter (other than `"x"`) and one or more two- to eight-letter or digit subtags separated by hyphens. Only one sequence is permitted for each digit or letter: `"de-a-foo-a-foo"` is invalid. BCP 47 extension subtags are defined in the Unicode CLDR Project. Currently only two extensions have defined semantics:

- The `"u"` (Unicode) extension can be used to request additional customization of `Intl` API objects. Examples:
  - `"de-DE-u-co-phonebk"`: Use the phonebook variant of the German sort order, which interprets umlauted vowels as corresponding character pairs: ä → ae, ö → oe, ü → ue.
  - `"th-TH-u-nu-thai"`: Use Thai digits (๐, ๑, ๒, ๓, ๔, ๕, ๖, ๗, ๘, ๙) in number formatting.
  - `"ja-JP-u-ca-japanese"`: Use the Japanese calendar in date and time formatting, so that 2013 is expressed as the year 25 of the Heisei period, or 平成 25.
  - `"en-GB-u-ca-islamic-umalqura"`: use British English with the Umm al-Qura (Hijri) calendar, where the Gregorian date 14 October, 2017 is the Hijri date 24 Muharram, 1439.
- The `"t"` (transformed) extension indicates transformed content: for example, text that was translated from another locale. No `Intl` functionality currently considers the `"t"` extension. However, this extension sometimes contains a nested locale (with no extensions): for example, the transformed extension in `"de-t-en"` contains the locale identifier for English. If a nested locale is present, it must be a valid locale identifier. For example, because `"en-emodeng-emodeng"` is invalid (because it contains a duplicate `emodeng` variant subtag), `"de-t-en-emodeng-emodeng"` is also invalid.

Finally, a private-use extension sequence using the letter `"x"` may appear, followed by one or more one- to eight-letter or digit subtags separated by hyphens. This allows applications to encode information for their own private use, that will be ignored by all `Intl` operations.

options argument

The `options` argument must be an object with properties that vary between constructors and functions. If the `options` argument is not provided or is undefined, default values are used for all properties.

One property is supported by all language sensitive constructors and functions: The `localeMatcher` property, whose value must be a string `"lookup"` or `"best fit"` and which selects one of the locale matching algorithms described below.

Locale identification and negotiation

The list of locales specified by the `locales` argument, after Unicode extensions have been removed from them, is interpreted as a prioritized request from the application. The runtime compares it against the locales it has available and picks the best one available. Two matching algorithms exist: the `"lookup"` matcher follows the Lookup algorithm specified in BCP 47; the `"best fit"` matcher lets the runtime provide a locale that's at least, but possibly more, suited for the request than the result of the Lookup algorithm. If the application doesn't provide a `locales` argument, or the runtime doesn't have a locale that matches the request, then the runtime's default locale is used. The matcher can be selected using a property of the `options` argument (see below).

If the selected locale identifier had a Unicode extension sequence, that extension is now used to customize the constructed object or the behavior of the function. Each constructor or function supports only a subset of the keys defined for the Unicode extension, and the supported values often depend on the locale identifier. For example, the `"co"` key (collation) is only supported by `Intl.Collator`, and its `"phonebk"` value is only supported for German.

Static properties

`Intl.Collator`
: Constructor for collators, which are objects that enable language-sensitive string comparison.

`Intl.DateTimeFormat`
: Constructor for objects that enable language-sensitive date and time formatting.

`Intl.DisplayNames`
: Constructor for objects that enable the consistent translation of language, region and script display names.

`Intl.DurationFormat`
: Constructor for objects that enable locale-sensitive duration formatting.

`Intl.ListFormat`
: Constructor for objects that enable language-sensitive list formatting.

`Intl.Locale`
: Constructor for objects that represents a Unicode locale identifier.

`Intl.NumberFormat`
: Constructor for objects that enable language-sensitive number formatting.

`Intl.PluralRules`
: Constructor for objects that enable plural-sensitive formatting and language-specific rules for plurals.

`Intl.RelativeTimeFormat`
: Constructor for objects that enable language-sensitive relative time formatting.

`Intl.Segmenter`
: Constructor for objects that enable locale-sensitive text segmentation.

`Intl[Symbol.toStringTag]`
: The initial value of the `[Symbol.toStringTag]` property is the string `"Intl"`. This property is used in `Object.prototype.toString()`.

Static methods

`Intl.getCanonicalLocales()`

Returns canonical locale names.

`Intl.supportedValuesOf()`

Returns a sorted array containing the supported unique calendar, collation, currency, numbering systems, or unit values supported by the implementation.

Examples

Formatting dates and numbers

You can use `Intl` to format dates and numbers in a form that's conventional for a specific language and region:

```js
const count = 26254.39;
const date = new Date("2012-05-24");

function log(locale) {
  console.log(
    `${new Intl.DateTimeFormat(locale).format(date)} ${new Intl.NumberFormat(
      locale,
    ).format(count)}`,
  );
}

log("en-US"); // 5/24/2012 26,254.39

log("de-DE"); // 24.5.2012 26.254,39
```

Using the browser's preferred language

Instead of passing a hardcoded locale name to the `Intl` methods, you can use the user's preferred language provided by `navigator.language`:

```js
const date = new Date("2012-05-24");

const formattedDate = new Intl.DateTimeFormat(navigator.language).format(date);
```

Alternatively, the `navigator.languages` property provides a sorted list of the user's preferred languages. This list can be passed directly to the `Intl` constructors to implement preference-based fallback selection of locales. The locale negotiation process is used to pick the most appropriate locale available:

```js
const count = 26254.39;

const formattedCount = new Intl.NumberFormat(navigator.languages).format(count);
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#intl-object`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Intl`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`Collator`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]
`Collator()` constructor	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [2,3]
`options.caseFirst` parameter	24	18	55	15	11	25	56	14	11	1.5	4.4	11	1.0.0	1.8	0.12.0
`options.collation` parameter	86	86	85	72	14.1	86	85	61	14.5	14.0	86	14.5	1.0.0	1.8	15.0.0
`options.ignorePunctuation` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.localeMatcher` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.numeric` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.sensitivity` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.usage` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`compare`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]
`resolvedOptions`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]
`supportedLocalesOf`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [2,4]
`DateTimeFormat`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [5]
`DateTimeFormat()` constructor	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [2,6]
Supports normative optional `ChainDateTimeFormat` behavior	91 [7,8]	91 [9,10]	54	77 [11,12]	14.1	91 [7,8]	56	64 [13,14]	14.5	16.0 [15,16]	91 [7,8]	14.5	1.0.0	1.8	16.0.0 [17,18]
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [2,6]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.calendar` parameter	80	80	76	67	14.1	80	79	57	14.5	13.0	80	14.5	1.0.0	1.8	14.0.0
`options.dateStyle` parameter	76	79	79	63	14.1	76	79	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0
`options.dayPeriod` parameter	92	92	90	78	14.1	92	90	65	14.5	16.0	92	14.5	1.0.0	1.12	16.6.0
`options.fractionalSecondDigits` parameter	84	84	84	70	14.1	84	84	60	14.5	14.0	84	14.5	1.0.0	1.8	14.6.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`options.hourCycle` parameter	73	18	58	60	13	73	58	52	13	11.0	73	13	1.0.0	1.8	12.0.0
`options.numberingSystem` parameter	80	80	76	67	14.1	80	79	57	14.5	13.0	80	14.5	1.0.0	1.8	14.0.0
`options.timeStyle` parameter	76	79	79	63	14.1	76	79	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0
`options.timeZoneName` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`shortGeneric`/`longGeneric`/`shortOffset`/`longOffset` as `options.timeZoneName` option	95	95	91	81	15.4	95	91	67	15.4	17.0	95	15.4	1.0.0	**No**	17.0.0
`options.timeZone` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
IANA time zone names in `options.timeZone` option	24	14	52	15	10	25	56	14	10	1.5	37	10	1.0.0	1.8	4.0.0
UTC offset in `options.timeZone` option	120	120	121	106	17.4	120	121	80	17.4	25.0	120	17.4	1.0.19	1.38	22.0.0
`format`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[5]
`formatRange`	76	79	91	63	14.1	76	91	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0[5]
`formatRangeToParts`	76	79	91	63	14.1	76	91	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0[5]
`formatToParts`	57[19]	18	51	44[20]	11	57[19]	56	43[21]	11	7.0[19]	57[19]	11	1.0.0	1.8	8.0.0[2,5]
`resolvedOptions`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[5]
computed `timeZone`	35	14	53	30	10	35	56	22	10	3.0	37	10	1.0.0	1.8	8.10.0
`supportedLocalesOf`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0[2,4]
`DisplayNames`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0
`DisplayNames()` constructor	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0
`of`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0
`resolvedOptions`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0
`supportedLocalesOf`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0
`DurationFormat`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`DurationFormat()` constructor	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`format`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`formatToParts`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`resolvedOptions`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`supportedLocalesOf`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`ListFormat`	72	79	78	60	14.1[23]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[24]
`ListFormat()` constructor	72	79	78	60	14.1[23]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	13.0.0[25,26]
`format`	72	79	78	60	14.1[23]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[24]

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`formatToParts`	72	79	78	60	14.1[23]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[24]
`resolvedOptions`	72	79	78	60	14.1[23]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[24]
`supportedLocalesOf`	72	79	78	60	14.1[23]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	13.0.0[25,27]
`Locale`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`Locale()` constructor	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`baseName`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`calendar`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`caseFirst`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`collation`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`getCalendars`	130[28,29]	130[28,29]	153	115[30,31]	17[32,33]	130[28,29]	153	86[34,35]	17[36,37]	28.0[38,39]	130[28,29]	17[36,37]	1.0.0	2.1[40,41]	24.0.0[42,43]
`getCollations`	130[44,29]	130[44,29]	153	115[45,31]	17[46,33]	130[44,29]	153	86[47,35]	17[48,37]	28.0[49,39]	130[44,29]	17[48,37]	1.0.0	2.1[50,41]	24.0.0[51,43]
`getHourCycles`	130[52,29]	130[52,29]	153	115[53,31]	17[54,33]	130[52,29]	153	86[55,35]	17[56,37]	28.0[57,39]	130[52,29]	17[56,37]	1.0.0	2.1[58,41]	24.0.0[59,43]
`getNumberingSystems`	130[60,29]	130[60,29]	153	115[61,31]	17[62,33]	130[60,29]	153	86[63,35]	17[64,37]	28.0[65,39]	130[60,29]	17[64,37]	1.0.0	2.1[66,41]	24.0.0[67,43]
`getTextInfo`	130[68,29]	130[68,29]	153	115[69,31]	17[70,33]	130[68,29]	153	86[71,35]	17[72,37]	28.0[73,39]	130[68,29]	17[72,37]	1.0.0	2.1[74,41]	24.0.0[75,43]
`getTimeZones`	130[76,29]	130[76,29]	153	115[77,31]	17[78,33]	130[76,29]	153	86[79,35]	17[80,37]	28.0[81,39]	130[76,29]	17[80,37]	1.0.0	2.1[82,41]	24.0.0[83,43]
`getWeekInfo()` method	130[84,29]	130[84,29]	153	115[85,31]	17[86,33]	130[84,29]	153	86[87,35]	17[88,37]	28.0[89,39]	130[84,29]	17[88,37]	1.0.0	2.1[90,41]	24.0.0[91,43]
`minimalDays` in returned object Deprecated Non-standard	**99–136**	**99–136**	**No**	**85–121**	15.4	**99–136**	**No**	**68–90**	15.4	**18.0–29.0**	**99–136**	15.4	1.0.0	1.19	**18.0.0–24.0.0**
`hourCycle`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`language`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`maximize`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`minimize`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`numberingSystem`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`numeric`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`region`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`script`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`toString`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`variants`	149	149	141	133	26	149	141	100	26	**No**	149	26	1.2.18	2.8	**No**
`NumberFormat`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[92]
`NumberFormat()` constructor	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
Supports normative optional `ChainNumberFormat` behavior	91[7,8]	91[9,10]	54	77[11,12]	14.1	91[7,8]	56	64[13,14]	14.5	16.0[15,16]	91[7,8]	14.5	1.0.0	1.8	16.0.0[17,18]
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0[2,93]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`options.compactDisplay` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.currencyDisplay` parameter	77	79	42	64	14.1[94,95]	77	56	55	14.5[96,97]	12.0	77	14.5[96,97]	1.0.0	1.8	12.11.0
`options.currencySign` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.currency` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.localeMatcher` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.maximumFractionDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.maximumSignificantDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.minimumFractionDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.minimumIntegerDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.minimumSignificantDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.notation` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.numberingSystem` parameter	24	12	76	15	14.1	25	56	14	14.5	1.5	4.4	14.5	1.0.0	1.8	0.12.0
`options.roundingIncrement` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.roundingMode` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.roundingPriority` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.signDisplay` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`negative` value	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.style` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.trailingZeroDisplay` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.unitDisplay` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.unit` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.useGrouping` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0[2,93]
`options.useGrouping` parameter accepts: `'always'`, `'auto'`, `'min2'` (in addition to: `true` and `false`)	106	106	116	92	16.4	106	116	72	16.4	20.0	106	16.4	1.0.0	**No**	19.0.0
`format`	24	12[98]	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[92]
`number` param string value is decimal (not `Number`)	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	1.25	19.0.0
`formatRange`	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	19.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
formatRangeToParts	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	19.0.0
formatToParts	64	12	58	51	13	64	58	47	13	9.0	64	13	1.0.0	1.8	10.0.0 [92]
resolvedOptions	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [92]
supportedLocalesOf	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [2,4]
PluralRules	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [99]
PluralRules() constructor	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	13.0.0 [100,101]
options parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
options.roundingIncrement parameter	**No**	**No**	116	**No**	17.2	**No**	116	**No**	17.2	**No**	**No**	17.2	1.0.3	**No**	**No**
options.roundingMode parameter	117	117	116	103	17.2	117	116	78	17.2	24.0	117	17.2	1.0.3	**No**	21.0.0
options.roundingPriority parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
options.trailingZeroDisplay parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	**No**
resolvedOptions	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [99]
select	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [99]
selectRange	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	19.0.0
supportedLocalesOf	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [99]
RelativeTimeFormat	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [102]
RelativeTimeFormat() constructor	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0
locales parameter	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	13.0.0 [25,103]
options.localeMatcher parameter	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0
options.numberingSystem parameter	71	79	76	58	14	71	79	50	14	10.0	71	14	1.0.0	1.8	12.0.0
options.numeric parameter	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0
format	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [102]
formatToParts	71	79	70	58	14	71	79	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [102]
resolvedOptions	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [102]
supportedLocalesOf	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	13.0.0 [25,27]
Segmenter	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
Segmenter() constructor	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
resolvedOptions	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
segment	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedLocalesOf`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
`Segments`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
[Symbol.iterator]	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
`containing`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
`getCanonicalLocales`	54	16	48	41	10.1	54	56	41	10.3	6.0	54	10.3	1.0.0	1.8	7.0.0
`supportedValuesOf()`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"calendar"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"collation"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"currency"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"numberingSystem"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"timeZone"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`Includes "UTC"` Non-standard	**No**	**No**	93	**No**	15.4	**No**	93	**No**	15.4	**No**	**No**	15.4	1.0.0	**No**	**No**
`"unit"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `Collator()` constructor for more details.
2. Also from version 0.12.0–13.0.0: partial support.
3. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `Collator` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
5. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `DateTimeFormat()` constructor for more details.
6. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `DateTimeFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
7. Also from version 57–91: partial support.
8. Also from version 57–91: The fallback symbol property has description `IntlFallback`.
9. Also from version 79–91: partial support.
10. Also from version 79–91: The fallback symbol property has description `IntlFallback`.
11. Also from version 44–77: partial support.
12. Also from version 44–77: The fallback symbol property has description `IntlFallback`.
13. Also from version 43–64: partial support.
14. Also from version 43–64: The fallback symbol property has description `IntlFallback`.
15. Also from version 7.0–16.0: partial support.
16. Also from version 7.0–16.0: The fallback symbol property has description `IntlFallback`.
17. Also from version 8.0.0–16.0.0: partial support.
18. Also from version 8.0.0–16.0.0: The fallback symbol property has description `IntlFallback`.
19. Before version 71, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 71 and later use the specification defined `dayPeriod`. See bug 41402623.
20. Before version 58, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 58 and later use the specification defined `dayPeriod`. See bug 41402623.
21. Before version 50, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 50 and later use the specification defined `dayPeriod`. See bug 41402623.
22. Before version 12.0.0, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 12.0.0 and later use the specification defined `dayPeriod`. See bug 41402623.
23. Only available on macOS Big Sur (11) and above.
24. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `ListFormat()` constructor for more details.
25. Also from version 12.0.0–13.0.0: partial support.
26. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `ListFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
27. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

28. Also from version 99: uses the non-standard name `calendars`.
29. Also from version 99: Implemented as an accessor property.
30. Also from version 85: uses the non-standard name `calendars`.
31. Also from version 85: Implemented as an accessor property.
32. Also from version 15.4–Prev.: uses the non-standard name `calendars`.
33. Also from version 15.4–Prev.: Implemented as an accessor property.
34. Also from version 68: uses the non-standard name `calendars`.
35. Also from version 68: Implemented as an accessor property.
36. Also from version 15.4: uses the non-standard name `calendars`.
37. Also from version 15.4: Implemented as an accessor property.
38. Also from version 18.0: uses the non-standard name `calendars`.
39. Also from version 18.0: Implemented as an accessor property.
40. Also from version 1.19–2.6.7: uses the non-standard name `calendars`.
41. Also from version 1.19–2.6.7: Implemented as an accessor property.
42. Also from version 18.0.0: uses the non-standard name `calendars`.
43. Also from version 18.0.0: Implemented as an accessor property.
44. Also from version 99: uses the non-standard name `collations`.
45. Also from version 85: uses the non-standard name `collations`.
46. Also from version 15.4–Prev.: uses the non-standard name `collations`.
47. Also from version 68: uses the non-standard name `collations`.
48. Also from version 15.4: uses the non-standard name `collations`.
49. Also from version 18.0: uses the non-standard name `collations`.
50. Also from version 1.19–2.6.7: uses the non-standard name `collations`.
51. Also from version 18.0.0: uses the non-standard name `collations`.
52. Also from version 99: uses the non-standard name `hourCycles`.
53. Also from version 85: uses the non-standard name `hourCycles`.
54. Also from version 15.4–Prev.: uses the non-standard name `hourCycles`.
55. Also from version 68: uses the non-standard name `hourCycles`.
56. Also from version 15.4: uses the non-standard name `hourCycles`.
57. Also from version 18.0: uses the non-standard name `hourCycles`.
58. Also from version 1.19–2.6.7: uses the non-standard name `hourCycles`.
59. Also from version 18.0.0: uses the non-standard name `hourCycles`.
60. Also from version 99: uses the non-standard name `numberingSystems`.
61. Also from version 85: uses the non-standard name `numberingSystems`.
62. Also from version 15.4–Prev.: uses the non-standard name `numberingSystems`.
63. Also from version 68: uses the non-standard name `numberingSystems`.
64. Also from version 15.4: uses the non-standard name `numberingSystems`.
65. Also from version 18.0: uses the non-standard name `numberingSystems`.
66. Also from version 1.19–2.6.7: uses the non-standard name `numberingSystems`.
67. Also from version 18.0.0: uses the non-standard name `numberingSystems`.
68. Also from version 99: uses the non-standard name `textInfo`.
69. Also from version 85: uses the non-standard name `textInfo`.
70. Also from version 15.4–Prev.: uses the non-standard name `textInfo`.
71. Also from version 68: uses the non-standard name `textInfo`.
72. Also from version 15.4: uses the non-standard name `textInfo`.
73. Also from version 18.0: uses the non-standard name `textInfo`.
74. Also from version 1.19–2.6.7: uses the non-standard name `textInfo`.
75. Also from version 18.0.0: uses the non-standard name `textInfo`.
76. Also from version 99: uses the non-standard name `timeZones`.
77. Also from version 85: uses the non-standard name `timeZones`.
78. Also from version 15.4–Prev.: uses the non-standard name `timeZones`.
79. Also from version 68: uses the non-standard name `timeZones`.
80. Also from version 15.4: uses the non-standard name `timeZones`.
81. Also from version 18.0: uses the non-standard name `timeZones`.
82. Also from version 1.19–2.6.7: uses the non-standard name `timeZones`.
83. Also from version 18.0.0: uses the non-standard name `timeZones`.
84. Also from version 99: uses the non-standard name `weekInfo`.
85. Also from version 85: uses the non-standard name `weekInfo`.
86. Also from version 15.4–Prev.: uses the non-standard name `weekInfo`.
87. Also from version 68: uses the non-standard name `weekInfo`.
88. Also from version 15.4: uses the non-standard name `weekInfo`.
89. Also from version 18.0: uses the non-standard name `weekInfo`.
90. Also from version 1.19–2.6.7: uses the non-standard name `weekInfo`.
91. Also from version 18.0.0: uses the non-standard name `weekInfo`.

92. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `NumberFormat()` constructor for more details.
93. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `NumberFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
94. Also from version 10–14.1: partial support.
95. Also from version 10–14.1: Doesn't support `currencyDisplay: 'narrowSymbol'`.
96. Also from version 10–14.5: partial support.
97. Also from version 10–14.5: Doesn't support `currencyDisplay: 'narrowSymbol'`.
98. Before Edge 18, numbers are rounded to 15 decimal digits. For example, `new Intl.NumberFormat('en-US').format(1000000000000005)` returns `"1,000,000,000,000,010"`.
99. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `PluralRules()` constructor for more details.
100. Also from version 10.0.0–13.0.0: partial support.
101. Also from version 10.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `PluralRules` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
102. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `RelativeTimeFormat()` constructor for more details.
103. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `RelativeTimeFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Keyboard.getLayoutMap()`
- `navigator.language`
- `navigator.languages`
- The ECMAScript Internationalization API by Norbert Lindenberg (2012)

JavaScript › Reference › Standard built-in objects › Intl

Intl.getCanonicalLocales()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/getCanonicalLocales · MDN page last modified 2025-09-08

Baseline: Widely available — available across major browsers since October 2017.

The **`Intl.getCanonicalLocales()`** static method returns an array containing the canonical locale names. Duplicates will be omitted and elements will be validated as structurally valid language tags.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.getCanonicalLocales()". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(Intl.getCanonicalLocales("EN-US"));
// Expected output: Array ["en-US"]

console.log(Intl.getCanonicalLocales(["EN-US", "Fr"]));
// Expected output: Array ["en-US", "fr"]

try {
  Intl.getCanonicalLocales("EN_US");
} catch (err) {
  console.log(err.toString());
  // Expected output: RangeError: invalid language tag: "EN_US"
}
```

Syntax

```js
Intl.getCanonicalLocales(locales)
```

Parameters

`locales`

A list of `String` values for which to get the canonical locale names.

Return value

An array containing the canonical locale names.

Examples

Using getCanonicalLocales

```js
Intl.getCanonicalLocales("EN-US"); // ["en-US"]
Intl.getCanonicalLocales(["EN-US", "Fr"]); // ["en-US", "fr"]

Intl.getCanonicalLocales("EN_US");
// RangeError: invalid language tag: "EN_US"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.getcanonicallocales`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getCanonicalLocales`	54	16	48	41	10.1	54	56	41	10.3	6.0	54	10.3	1.0.0	1.8	7.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.getCanonicalLocales`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.getCanonicalLocales` in FormatJS
- `Intl.NumberFormat.supportedLocalesOf()`
- `Intl.DateTimeFormat.supportedLocalesOf()`
- `Intl.Collator.supportedLocalesOf()`

JavaScript › Reference › Standard built-in objects › Intl

Intl.supportedValuesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/supportedValuesOf · MDN page last modified 2026-02-27

Baseline: Widely available — available across major browsers since March 2022.

The **`Intl.supportedValuesOf()`** static method returns an array containing the supported calendar, collation, currency, numbering systems, or unit values supported by the implementation.

Duplicates are omitted and the array is sorted in ascending lexicographical order (or more precisely, using `Array.prototype.sort()` with an `undefined` compare function).

The method can be used to feature-test whether values are supported in a particular implementation and download a polyfill only if necessary. It can also be used to build UIs that allow users to select their preferred localized values, for example when the UI is created from WebGL or server-side.

This method is locale-unaware: it is possible that certain identifiers are only supported or preferred in certain locales. If you want to determine the preferred values for a specific locale, you should use the `Intl.Locale` object, such as `Intl.Locale.prototype.getCalendars()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.supportedValuesOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(Intl.supportedValuesOf("calendar"));
console.log(Intl.supportedValuesOf("collation"));
console.log(Intl.supportedValuesOf("currency"));
console.log(Intl.supportedValuesOf("numberingSystem"));
console.log(Intl.supportedValuesOf("timeZone"));
console.log(Intl.supportedValuesOf("unit"));
// Expected output: Array ['key'] (for each key)

try {
  Intl.supportedValuesOf("someInvalidKey");
} catch (err) {
  console.log(err.toString());
  // Expected output: RangeError: invalid key: "someInvalidKey"
}
```

Syntax

js

```
Intl.supportedValuesOf(key)
```

Parameters

key

A key string indicating the category of values to be returned. This is one of:

- `"calendar"`: see supported calendar types
- `"collation"`: see supported collation types
- `"currency"`: see supported currency identifiers

- `"numberingSystem"`: see supported numbering system types
- `"timeZone"`: see supported time zone identifiers
- `"unit"`: see supported unit identifiers

Return value

A sorted array of unique string values indicating the values supported by the implementation for the given key. The values that could be returned are listed below.

Supported calendar types

Below are all values that are commonly supported by browsers for the `calendar` key. These values can be used for the `calendar` option or the `ca` Unicode extension key when creating objects such as `Intl.DateTimeFormat`, as well as for creating `Temporal` date objects. This list is explicitly sanctioned by the ECMA-402 specification, so all implementations should be consistent.

Value	Description
`buddhist`	Thai Buddhist calendar, proleptic. Month numbers, month codes, and days are the same as in the ISO 8601 calendar, but the epoch year is different. There is one era.
`chinese`	Traditional Chinese calendar, proleptic. Lunisolar calendar used in China based on data published by the Purple Mountain Observatory between 1900 and 2100 (which compiles with GB/T 33661-2017 between 1912 and 2100), falling back to an implementation-defined approximation outside that range. The arithmetic year is identical to `gregory`, and there are no eras.
`coptic`	Coptic calendar, proleptic. Similar solar algorithm to `ethioaa` and `ethiopic`, with one era and a different epoch year.
`dangi`	Traditional Korean calendar, proleptic. Lunisolar calendar using months published by the Korea Astronomy and Space Science Institute (KASI) between 1900 and 2050, falling back to an implementation-defined approximation outside that range. The arithmetic year is identical to `gregory`, and there are no eras.
`ethioaa`	Ethiopic calendar, Amete Alem, proleptic. Similar solar algorithm to `coptic` and `ethiopic`, with one era and a different epoch year.
`ethiopic`	Ethiopic calendar, Amete Mihret, proleptic. Similar solar algorithm to `coptic` and `ethioaa`, with two eras and a different epoch year.
`gregory`	Gregorian calendar, proleptic. Solar calendar almost identical to the ISO 8601 calendar, except that it does not define week numbering and it contains two eras, one before the epoch year.
`hebrew`	Hebrew calendar, proleptic. Civil calendar with Tishrei as the first month of the year. Lunisolar calendar with one leap month inserted after month 5. There is one era.
`indian`	Indian national (or Śaka) calendar, proleptic. Solar calendar with one era.
`islamic-civil`	Hijri calendar, proleptic, tabular/rule-based with leap year rule II (leap years 2,5,7,10,13,16,18,21,24,26,29 in the 30-year cycle (1-based numbering)) and civil epoch (Friday July 16, 622 Julian / 0622-07-19 ISO)
`islamic-tbla`	Hijri calendar, proleptic, tabular/rule-based with leap year rule II (leap years 2,5,7,10,13,16,18,21,24,26,29 in the 30-year cycle (1-based numbering)) and astronomical epoch (Thursday July 15, 622 Julian / 0622-07-18 ISO)
`islamic-umalqura`	Hijri calendar, proleptic, Umm al-Qura. Lunar calendar using KACST-calculated months from the start of 1300 AH (1882-11-12 ISO) to the end of 1600 AH (2174-11-25 ISO), falling back to `islamic-civil` outside that range.
`iso8601`	ISO calendar (variant of the Gregorian calendar with week rules and formatting parameters made region-independent)

Value	Description
`japanese`	Japanese Imperial calendar (this calendar adds an era for each new emperor, so the output year and era for a future date may not match the input year and era when your code runs on a future engine version. **Note:** See the remarks below this table about dates prior to 1868-10-23 ISO.)
`persian`	Persian (or Solar Hijri) calendar, proleptic. There is one era.
`roc`	Republic of China (or Minguo) calendar, proleptic. Month numbers, month codes, and days are the same as in the ISO 8601 calendar, but the epoch year is different. There are two eras, one before the epoch year and one after.

As of October 2025, in the `japanese` calendar, dates prior to 1868-10-23 ISO (the start date of the year 1 Meiji) don't work as expected in browsers in two ways. First, CLDR had the wrong start date for the Meiji era, which causes calendar implementations to extend the Meiji era further to the past than it actually did. Second, the upcoming Intl era and monthCode Proposal specifies that dates prior to 1873-01-01 ISO should use Gregorian eras, but browsers have traditionally used approximations of prior Japanese eras instead. The `japanese` calendar was taken into use on January 1, 6 Meiji / 1873-01-01 ISO, so these problems only affect proleptic dates.

The types below are specified in CLDR but do not have implementations distinct from the above calendars in browsers.

Value	Description	Notes
`ethiopic-amete-alem`	Ethiopic calendar, Amete Alem, proleptic.	This is an alias for `ethioaa` and therefore is not returned by `supportedValuesOf()`. Use `ethioaa` instead.
`islamic`	Hijri calendar, unspecified algorithm.	As of April 2025, this is an astronomical simulation whose parameters are undocumented and that is not known to match a specific Hijri calendar variant from non-software contexts. It is specified to be canonicalized to a different calendar, usually one of `islamic-umalqura`, `islamic-tbla`, or `islamic-civil`, and raise a warning.
`islamicc` Deprecated	Civil (algorithmic) Arabic calendar.	This is an alias for `islamic-civil` and therefore is not returned by `supportedValuesOf()`. Use `islamic-civil` instead.

The `Temporal.PlainDate.prototype.era` and `Temporal.PlainDate.prototype.monthCode` docs provide more information about different calendars.

References:

- CLDR Calendar type keys
- UTS 35, Dates
- Islamic calendar types (CLDR design proposal)

Supported collation types

Below are all values that are commonly supported by browsers for the `collation` key. These values can be used for the `collation` option or the `co` Unicode extension key when creating objects such as `Intl.Collator`.

Value	Description
`compat`	A previous version of the ordering, for compatibility (for Arabic)
`dict`	Dictionary style ordering (for Sinhala). Also recognized as `dictionary`.

Value	Description
`emoji`	Recommended ordering for emoji characters. The requested language has no effect, so combining this with a language that requires language-specific rules breaks sorting for text in that language!
`eor`	European ordering rules (ENV 13710). This order is designed not to be language-specific. While the API requires attaching this to a language, the requested language has no effect on the order.
`phonebk`	Phonebook style ordering (for German). Also recognized as `phonebook`.
`phonetic`	Phonetic ordering (sorting based on pronunciation; for Lingala)
`pinyin`	Pinyin ordering for Latin and for Han characters (for Chinese)
`searchjl`	Special collation type for Korean initial consonant search. **Warning:** This collation is not for sorting, even though you can only use it with `Intl.Collator` of `usage: "sort"`.
`stroke`	Pinyin ordering for Latin, stroke order for Han characters (for Chinese)
`trad`	Traditional style ordering (such as in Spanish). Also recognized as `traditional`.
`unihan`	Unihan radical-stroke ordering for Han characters (for Chinese, Japanese, and Korean), Pinyin ordering for Latin for Chinese and Japanese standard kana ordering for Japanese
`zhuyin`	Pinyin ordering for Latin, zhuyin order for Bopomofo and Han characters (for Chinese)

The types below are specified in CLDR data, but are deprecated, are discouraged from explicit usage, and/or may not be indicated by browsers as supported for various reasons. Avoid using them:

Value	Description	Notes
`big5han` Deprecated	Pinyin ordering for Latin, big5 charset ordering for CJK characters (used in Chinese)	Deprecated.
`direct` Deprecated	Binary code point order (used in Hindi)	Deprecated.
`ducet`	The default Unicode collation element table order	The `ducet` collation type is not available to the Web.
`gb2312` Deprecated	Pinyin ordering for Latin, gb2312han charset ordering for CJK characters (for Chinese). Also recognized as `gb2312han`.	Deprecated.
`reformed` Deprecated	Reformed ordering (such as Swedish)	Deprecated. This is the old name for the default ordering for Swedish whose collation naming used to differ from other languages. Since this was the default, request `sv` instead of requesting `sv-u-co-reformed`.
`search`	Special collation type for string search	Do not use as a collation type, since in `Intl.Collator`, this collation is activated via the `usage: "search"` option. There is currently no API for substring search, so this is currently only good for filtering a list of strings by trying a full-string match of the key against each list item.
`standard`	Default ordering for each language, except Chinese (and, previously, Swedish)	Do not use explicitly. In general, it's unnecessary to specify this explicitly and specifying this for Swedish is problematic due to the different meaning for Swedish in the past.

References:

- CLDR Collation type keys
- UTS 35, Collation

Supported currency identifiers

Currency identifiers are three-letter uppercase codes defined in ISO 4217. These values can be used for the `currency` option when creating objects such as `Intl.NumberFormat`, as well as for `Intl.DisplayNames.prototype.of()`. There are over 300 identifiers in common use so we won't list them. For an exhaustive list of possible identifiers, see the Wikipedia article.

References:

- CLDR Currency type keys
- ISO 4217 Currency codes
- UTS 35, Currencies

Supported numbering system types

Below are all values that are commonly supported by browsers for the `numberingSystem` key. These values can be used for the `numberingSystem` option or the `nu` Unicode extension key when creating objects such as `Intl.NumberFormat`. For the rows with "digit characters", the runtime translates the digits one-by-one without extra actions. The others marked as "algorithmic" need additional algorithms to translate the digits. The higher the Unicode code point is, the newer the numbering system is and the more likely it is unsupported by all browsers.

Value	Description	Digit characters
`adlm`	Adlam digits	𞥐𞥑𞥒𞥓𞥔𞥕𞥖𞥗𞥘𞥙 (U+1E950 to U+1E959)
`ahom`	Ahom digits	𑜰𑜱𑜲𑜳𑜴𑜵𑜶𑜷𑜸𑜹 (U+11730 to U+11739)
`arab`	Arabic-Indic digits	٠١٢٣٤٥٦٧٨٩ (U+0660 to U+0669)
`arabext`	Extended Arabic-Indic digits	۰۱۲۳۴۵۶۷۸۹ (U+06F0 to U+06F9)
`armn`	Armenian upper case numerals	algorithmic
`armnlow`	Armenian lower case numerals	algorithmic
`bali`	Balinese digits	᭐᭑᭒᭓᭔᭕᭖᭗᭘᭙ (U+1B50 to U+1B59)
`beng`	Bengali digits	০১২৩৪৫৬৭৮৯ (U+09E6 to U+09EF)
`bhks`	Bhaiksuki digits	𑱐𑱑𑱒𑱓𑱔𑱕𑱖𑱗𑱘𑱙 (U+11C50 to U+11C59)
`brah`	Brahmi digits	𑁦𑁧𑁨𑁩𑁪𑁫𑁬𑁭𑁮𑁯 (U+11066 to U+1106F)
`cakm`	Chakma digits	𑄶𑄷𑄸𑄹𑄺𑄻𑄼𑄽𑄾𑄿 (U+11136 to U+1113F)
`cham`	Cham digits	꩐꩑꩒꩓꩔꩕꩖꩗꩘꩙ (U+AA50 to U+AA59)
`cyrl`	Cyrillic numerals	algorithmic
`deva`	Devanagari digits	०१२३४५६७८९ (U+0966 to U+096F)
`diak`	Dives Akuru digits	𑥐𑥑𑥒𑥓𑥔𑥕𑥖𑥗𑥘𑥙 (U+11950 to U+11959)
`ethi`	Ethiopic numerals	algorithmic
`fullwide`	Full width digits	０１２３４５６７８９ (U+FF10 to U+FF19)
`gara`	Garay digits	𐵀𐵁𐵂𐵃𐵄𐵅𐵆𐵇𐵈𐵉 (U+10D40 to U+10D49)
`geor`	Georgian numerals	algorithmic
`gong`	Gunjala Gondi digits	𑶠𑶡𑶢𑶣𑶤𑶥𑶦𑶧𑶨𑶩 (U+11DA0 to U+11DA9)
`gonm`	Masaram Gondi digits	𑵐𑵑𑵒𑵓𑵔𑵕𑵖𑵗𑵘𑵙 (U+11D50 to U+11D59)
`grek`	Greek upper case numerals	algorithmic
`greklow`	Greek lower case numerals	algorithmic

Value	Description	Digit characters
gujr	Gujarati digits	૦૧૨૩૪૫૬૭૮૯ (U+0AE6 to U+0AEF)
gukh	Gurung Khema digits	𖵰𖵱𖵲𖵳𖵴𖵵𖵶𖵷𖵸𖵹 (U+16130 to U+16139)
guru	Gurmukhi digits	੦੧੨੩੪੫੬੭੮੯ (U+0A66 to U+0A6F)
hanidays	Han-character day-of-month numbering for lunar/other traditional calendars	
hanidec	Positional decimal system using Chinese number ideographs as digits	〇一二三四五六七八九 (U+3007, U+4E00, U+4E8C, U+4E09, U+56DB, U+4E94, U+516D, U+4E03, U+516B, U+4E5D)
hans	Simplified Chinese numerals	algorithmic
hansfin	Simplified Chinese financial numerals	algorithmic
hant	Traditional Chinese numerals	algorithmic
hantfin	Traditional Chinese financial numerals	algorithmic
hebr	Hebrew numerals	algorithmic
hmng	Pahawh Hmong digits	𖭐𖭑𖭒𖭓𖭔𖭕𖭖𖭗𖭘𖭙 (U+16B50 to U+16B59)
hmnp	Nyiakeng Puachue Hmong digits	𞅀𞅁𞅂𞅃𞅄𞅅𞅆𞅇𞅈𞅉 (U+1E140 to U+1E149)
java	Javanese digits	ꦐꦑꦒꦓꦔꦕꦖꦗꦘꦙ (U+A9D0 to U+A9D9)
jpan	Japanese numerals	algorithmic
jpanfin	Japanese financial numerals	algorithmic
jpanyear	Japanese first-year Gannen numbering for Japanese calendar	algorithmic
kali	Kayah Li digits	꤀꤁꤂꤃꤄꤅꤆꤇꤈꤉ (U+A900 to U+A909)
kawi	Kawi digits	𑽐𑽑𑽒𑽓𑽔𑽕𑽖𑽗𑽘𑽙 (U+11F50 to U+11F59)
khmr	Khmer digits	០១២៣៤៥៦៧៨៩ (U+17E0 to U+17E9)
knda	Kannada digits	೦೧೨೩೪೫೬೭೮೯ (U+0CE6 to U+0CEF)
krai	Kirat Rai digits	𖵀𖵁𖵂𖵃𖵄𖵅𖵆𖵇𖵈𖵉 (U+16D70 to U+16D79)
lana	Tai Tham Hora (secular) digits	᪀᪁᪂᪃᪄᪅᪆᪇᪈᪉ (U+1A80 to U+1A89)
lanatham	Tai Tham (ecclesiastical) digits	᪐᪑᪒᪓᪔᪕᪖᪗᪘᪙ (U+1A90 to U+1A99)
laoo	Lao digits	๐๑๒๓๔๕๖๗๘๙ (U+0ED0 to U+0ED9)
latn	Latin digits	0123456789 (U+0030 to U+0039)
lepc	Lepcha digits	᱀᱁᱂᱃᱄᱅᱆᱇᱈᱉ (U+1C40 to U+1C49)
limb	Limbu digits	᥆᥇᥈᥉᥊᥋᥌᥍᥎᥏ (U+1946 to U+194F)
mathbold	Mathematical bold digits	**0123456789** (U+1D7CE to U+1D7D7)
mathdbl	Mathematical double-struck digits	𝟘𝟙𝟚𝟛𝟜𝟝𝟞𝟟𝟠𝟡 (U+1D7D8 to U+1D7E1)
mathmono	Mathematical monospace digits	0123456789 (U+1D7F6 to U+1D7FF)
mathsanb	Mathematical sans-serif bold digits	**0123456789** (U+1D7EC to U+1D7F5)
mathsans	Mathematical sans-serif digits	0123456789 (U+1D7E2 to U+1D7EB)
mlym	Malayalam digits	൦൧൨൩൪൫൬൭൮൯ (U+0D66 to U+0D6F)
modi	Modi digits	𑙐𑙑𑙒𑙓𑙔𑙕𑙖𑙗𑙘𑙙 (U+11650 to U+11659)
mong	Mongolian digits	᠐᠑᠒᠓᠔᠕᠖᠗᠘᠙ (U+1810 to U+1819)
mroo	Mro digits	𖩠𖩡𖩢𖩣𖩤𖩥𖩦𖩧𖩨𖩩 (U+16A60 to U+16A69)
mtei	Meetei Mayek digits	꯰꯱꯲꯳꯴꯵꯶꯷꯸꯹ (U+ABF0 to U+ABF9)

Value	Description	Digit characters
mymr	Myanmar digits	၀၁၂၃၄၅၆၇၈၉ (U+1040 to U+1049)
mymrepka	Myanmar Eastern Pwo Karen digits	𑋰𑋱𑋲𑋳𑋴𑋵𑋶𑋷𑋸𑋹 (U+116DA to U+116E3)
mymrpao	Myanmar Pao digits	𑑐𑑑𑑒𑑓𑑔𑑕𑑖𑑗𑑘𑑙 (U+116D0 to U+116D9)
mymrshan	Myanmar Shan digits	၀၁၂၃၄၅၆၇၈၉ (U+1090 to U+1099)
mymrtlng	Myanmar Tai Laing digits	꧰꧱꧲꧳꧴꧵꧶꧷꧸꧹ (U+A9F0 to U+A9F9)
nagm	Nag Mundari digits	𞓰𞓱𞓲𞓳𞓴𞓵𞓶𞓷𞓸𞓹 (U+1E4F0 to U+1E4F9)
newa	Newa digits	𑐐𑐑𑐒𑐓𑐔𑐕𑐖𑐗𑐘𑐙 (U+11450 to U+11459)
nkoo	N'Ko digits	߀߁߂߃߄߅߆߇߈߉ (U+07C0 to U+07C9)
olck	Ol Chiki digits	᱐᱑᱒᱓᱔᱕᱖᱗᱘᱙ (U+1C50 to U+1C59)
onao	Ol Onal digits	𞗱𞗲𞗳𞗴𞗵𞗶𞗷𞗸𞗹𞗺 (U+1E5F1 to U+1E5FA)
orya	Oriya digits	୦୧୨୩୪୫୬୭୮୯ (U+0B66 to U+0B6F)
osma	Osmanya digits	𐒠𐒡𐒢𐒣𐒤𐒥𐒦𐒧𐒨𐒩 (U+104A0 to U+104A9)
outlined	Legacy computing outlined digits	𜳰𜳱𜳲𜳳𜳴𜳵𜳶𜳷𜳸𜳹 (U+1CCF0 to U+1CCF9)
rohg	Hanifi Rohingya digits	𐴰𐴱𐴲𐴳𐴴𐴵𐴶𐴷𐴸𐴹 (U+10D30 to U+10D39)
roman	Roman upper case numerals	algorithmic
romanlow	Roman lowercase numerals	algorithmic
saur	Saurashtra digits	꣐꣑꣒꣓꣔꣕꣖꣗꣘꣙ (U+A8D0 to U+A8D9)
segment	Legacy computing segmented digits	🯰🯱🯲🯳🯴🯵🯶🯷🯸🯹 (U+1FBF0 to U+1FBF9)
shrd	Sharada digits	𑇐𑇑𑇒𑇓𑇔𑇕𑇖𑇗𑇘𑇙 (U+111D0 to U+111D9)
sind	Khudawadi digits	𑋰𑋱𑋲𑋳𑋴𑋵𑋶𑋷𑋸𑋹 (U+112F0 to U+112F9)
sinh	Sinhala Lith digits	෦෧෨෩෪෫෬෭෮෯ (U+0DE6 to U+0DEF)
sora	Sora_Sompeng digits	𑃰𑃱𑃲𑃳𑃴𑃵𑃶𑃷𑃸𑃹 (U+110F0 to U+110F9)
sund	Sundanese digits	᮰᮱᮲᮳᮴᮵᮶᮷᮸᮹ (U+1BB0 to U+1BB9)
sunu	Sunuwar digits	𑯰𑯱𑯲𑯳𑯴𑯵𑯶𑯷𑯸𑯹 (U+11BF0 to U+11BF9)
takr	Takri digits	𑛀𑛁𑛂𑛃𑛄𑛅𑛆𑛇𑛈𑛉 (U+116C0 to U+116C9)
talu	New Tai Lue digits	᧐᧑᧒᧓᧔᧕᧖᧗᧘᧙ (U+19D0 to U+19D9)
taml	Tamil numerals	algorithmic
tamldec	Modern Tamil decimal digits	௦௧௨௩௪௫௬௭௮௯ (U+0BE6 to U+0BEF)
telu	Telugu digits	౦౧౨౩౪౫౬౭౮౯ (U+0C66 to U+0C6F)
thai	Thai digits	๐๑๒๓๔๕๖๗๘๙ (U+0E50 to U+0E59)
tibt	Tibetan digits	༠༡༢༣༤༥༦༧༨༩ (U+0F20 to U+0F29)
tirh	Tirhuta digits	𑓐𑓑𑓒𑓓𑓔𑓕𑓖𑓗𑓘𑓙 (U+114D0 to U+114D9)
tnsa	Tangsa digits	𖩠𖩡𖩢𖩣𖩤𖩥𖩦𖩧𖩨𖩩 (U+16AC0 to U+16AC9)
vaii	Vai digits	꘠꘡꘢꘣꘤꘥꘦꘧꘨꘩ (U+A620 to U+A629)
wara	Warang Citi digits	𑣠𑣡𑣢𑣣𑣤𑣥𑣦𑣧𑣨𑣩 (U+118E0 to U+118E9)
wcho	Wancho digits	𞋰𞋱𞋲𞋳𞋴𞋵𞋶𞋷𞋸𞋹 (U+1E2F0 to U+1E2F9)

There are three special values: `native`, `traditio`, and `finance`, whose meanings are locale-dependent, and will be resolved to the right system depending on the locale. Therefore, the `resolvedOptions()`

methods will never return these values, but `Intl.Locale.prototype.numberingSystem` will (if provided as input).

References:

- CLDR Numbering system type keys
- CLDR Numbering system definitions
- UTS 35, Numbering systems

Supported time zone identifiers

Supported time zone identifiers can be used for the `timeZone` option when creating objects such as `Intl.DateTimeFormat`, as well as for creating `Temporal` date objects. There are over 400 identifiers in common use so we won't list them. For an exhaustive list of possible identifiers, see the Wikipedia article or the IANA time zone database.

As you browse the list, note that the standardization of `Temporal` requires browsers to always return the primary identifier in the IANA database, which may change over time. See time zones and offsets for more information. For example, the returned array should contain `"Asia/Kolkata"` instead of `"Asia/Calcutta"` because the latter is an alias of the former and they both correspond to India; however, it should contain both `"Africa/Abidjan"` and `"Atlantic/Reykjavik"` because they are in different countries, despite the latter also being an alias of the former.

References:

- IANA Time Zone Database
- UTS 35, Time Zone Identifiers

Supported unit identifiers

Below are all values that are commonly supported by browsers for the `unit` key. These values can be used for the `unit` option when creating objects such as `Intl.NumberFormat`. This list is a subset of the CLDR explicitly sanctioned by the ECMA-402 specification, so all implementations should be consistent.

- `acre`
- `bit`
- `byte`
- `celsius`
- `centimeter`
- `day`
- `degree`
- `fahrenheit`
- `fluid-ounce`
- `foot`
- `gallon`
- `gigabit`
- `gigabyte`
- `gram`
- `hectare`
- `hour`
- `inch`

- `kilobit`
- `kilobyte`
- `kilogram`
- `kilometer`
- `liter`
- `megabit`
- `megabyte`
- `meter`
- `microsecond`
- `mile`
- `mile-scandinavian`
- `milliliter`
- `millimeter`
- `millisecond`
- `minute`
- `month`
- `nanosecond`
- `ounce`
- `percent`
- `petabyte`
- `pound`
- `second`
- `stone`
- `terabit`
- `terabyte`
- `week`
- `yard`
- `year`

When specifying units, you can also combine two units with the "-per-" separator. For example, `meter-per-second` or `liter-per-megabyte`.

References:

- ECMA-402 sanctioned single units
- CLDR Unit validity data
- UTS 35, Unit identifiers

Exceptions

`RangeError`

Thrown if an unsupported key was passed as a parameter.

Examples

Feature testing

You can check that the method is supported by comparing to `undefined`:

```js
if (typeof Intl.supportedValuesOf !== "undefined") {
  // method is supported
}
```

Get all values for key

To get the supported values for calendar you call the method with the key `"calendar"`. You can then iterate through the returned array as shown below:

```js
Intl.supportedValuesOf("calendar").forEach((calendar) => {
  // "buddhist", "chinese", "coptic", "dangi", etc.
});
```

The other values are all obtained in the same way:

```js
Intl.supportedValuesOf("collation").forEach((collation) => {
  // "compat", "dict", "emoji", etc.
});

Intl.supportedValuesOf("currency").forEach((currency) => {
  // "ADP", "AED", "AFA", "AFN", "ALK", "ALL", "AMD", etc.
});

Intl.supportedValuesOf("numberingSystem").forEach((numberingSystem) => {
  // "adlm", "ahom", "arab", "arabext", "bali", etc.
});

Intl.supportedValuesOf("timeZone").forEach((timeZone) => {
  // "Africa/Abidjan", "Africa/Accra", "Africa/Addis_Ababa", "Africa/Algiers", etc.
});

Intl.supportedValuesOf("unit").forEach((unit) => {
  // "acre", "bit", "byte", "celsius", "centimeter", etc.
});
```

Invalid key throws RangeError

```js
try {
  Intl.supportedValuesOf("someInvalidKey");
} catch (err) {
  // RangeError: invalid key: "someInvalidKey"
}
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.supportedvaluesof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedValuesOf()`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"calendar"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"collation"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"currency"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"numberingSystem"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`"timeZone"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0
`Includes "UTC"` Non-standard	**No**	**No**	93	**No**	15.4	**No**	93	**No**	15.4	**No**	**No**	15.4	1.0.0	**No**	**No**
`"unit"`	99	99	93	85	15.4	99	93	68	15.4	18.0	99	15.4	1.0.0	1.19	18.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.supportedValuesOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.supportedValuesOf` in FormatJS
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl

Intl.Collator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl.Collator`** object enables language-sensitive string comparison.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.Collator". It runs live in the browser there; its source code follows.*

js · interactive example source

```
console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("de").compare));
// Expected output: Array ["a", "ä", "z", "Z"]

console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("sv").compare));
// Expected output: Array ["a", "z", "Z", "ä"]

console.log(
  ["Z", "a", "z", "ä"].sort(
    new Intl.Collator("de", { caseFirst: "upper" }).compare,
  ),
);
// Expected output: Array ["a", "ä", "Z", "z"]
```

Constructor

`Intl.Collator()`

Creates a new `Collator` object.

Static methods

`Intl.Collator.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.Collator.prototype` and shared by all `Intl.Collator` instances.

`Intl.Collator.prototype.constructor`

The constructor function that created the instance object. For `Intl.Collator` instances, the initial value is the `Intl.Collator` constructor.

`Intl.Collator.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.Collator"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.Collator.prototype.compare()`

Getter function that compares two strings according to the sort order of this `Intl.Collator` object.

`Intl.Collator.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and collation options computed during initialization of the object.

Examples

Using Collator

The following example demonstrates the different potential results for a string occurring before, after, or at the same level as another:

```js
console.log(new Intl.Collator().compare("a", "c")); // -1, or some other negative value
console.log(new Intl.Collator().compare("c", "a")); // 1, or some other positive value
console.log(new Intl.Collator().compare("a", "a")); // 0
```

Note that the results shown in the code above can vary between browsers and browser versions. This is because the values are implementation-specific. That is, the specification requires only that the before and after values are negative and positive.

Using locales

The results provided by `Intl.Collator.prototype.compare()` vary between languages. In order to get the sort order of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```js
// in German, ä sorts with a
console.log(new Intl.Collator("de").compare("ä", "z"));
// -1, or some other negative value

// in Swedish, ä sorts after z
console.log(new Intl.Collator("sv").compare("ä", "z"));
// 1, or some other positive value
```

Using options

The results provided by `Intl.Collator.prototype.compare()` can be customized using the `options` argument:

```js
// in German, ä has a as the base letter
console.log(new Intl.Collator("de", { sensitivity: "base" }).compare("ä", "a"));
// 0

// in Swedish, ä and a are separate base letters
console.log(new Intl.Collator("sv", { sensitivity: "base" }).compare("ä", "a"));
// 1, or some other positive value
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#collator-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Collator`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]
`Collator()` constructor	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [2,3]
`options.caseFirst` parameter	24	18	55	15	11	25	56	14	11	1.5	4.4	11	1.0.0	1.8	0.12.0
`options.collation` parameter	86	86	85	72	14.1	86	85	61	14.5	14.0	86	14.5	1.0.0	1.8	15.0.0
`options.ignorePunctuation` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.localeMatcher` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.numeric` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.sensitivity` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.usage` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`compare`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]
`resolvedOptions`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]
`supportedLocalesOf`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [2,4]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `Collator()` constructor for more details.
2. Also from version 0.12.0–13.0.0: partial support.
3. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `Collator` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
4. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Collator`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl`
- `String.prototype.localeCompare()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Collator

Intl.Collator() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator/Collator · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl.Collator()`** constructor creates `Intl.Collator` objects.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.Collator() constructor". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("de").compare));
// Expected output: Array ["a", "ä", "z", "Z"]

console.log(["Z", "a", "z", "ä"].sort(new Intl.Collator("sv").compare));
// Expected output: Array ["a", "z", "Z", "ä"]

console.log(
  ["Z", "a", "z", "ä"].sort(
    new Intl.Collator("de", { caseFirst: "upper" }).compare,
  ),
);
// Expected output: Array ["a", "ä", "Z", "z"]
```

Syntax

js

```
new Intl.Collator()
new Intl.Collator(locales)
new Intl.Collator(locales, options)

Intl.Collator()
Intl.Collator(locales)
Intl.Collator(locales, options)
```

> **Note:** `Intl.Collator()` can be called with or without `new`. Both create a new `Intl.Collator` instance.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

The following Unicode extension keys are allowed:

`co`

See `collation`.

`kn`

See `numeric`.

`kf`

See `caseFirst`.

These keys can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`usage`

Whether the comparison is for sorting a list of strings or fuzzy (for the Latin script diacritic-insensitive and case-insensitive) filtering a list of strings by key. Possible values are:

`"sort"` (default)

For sorting a list of strings.

`"search"`

For filtering a list of strings by testing each list item for a full-string match against a key. With `"search"`, the caller should only pay attention to whether `compare()` returns zero or non-zero and should not distinguish the non-zero return values from each other. That is, it is inappropriate to use `"search"` for sorting/ordering.

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`collation`

Variant collations for certain locales, such as `"emoji"`, `"pinyin"`, `"stroke"`, and so on. Only has an effect when `usage` is `"sort"` (because `"search"` is underlyingly its own collation type). For a list of supported collation types, see `Intl.supportedValuesOf()`; the default is `"default"`. This option can also be set through the `co` Unicode extension key; if both are provided, this `options` property takes precedence.

`numeric`

Whether numeric collation should be used, such that "1" < "2" < "10". Possible values are `true` and `false`; the default is `false`. This option can also be set through the `kn` Unicode extension key; if both are provided, this `options` property takes precedence.

`caseFirst`

Whether upper case or lower case should sort first. Possible values are `"upper"`, `"lower"`, and `"false"` (use the locale's default); the default is `"false"`. This option can also be set through the `kf` Unicode extension key; if both are provided, this `options` property takes precedence.

`sensitivity`

Which differences in the strings should lead to non-zero result values. Possible values are:

`"base"`

Only strings that differ in base letters compare as unequal. Examples: a ≠ b, a = á, a = A. In the Unicode collation algorithm, this is equivalent to the primary strength level.

`"accent"`
: Only strings that differ in base letters or accents and other diacritic marks compare as unequal. Examples: a ≠ b, a ≠ á, a = A. In the Unicode collation algorithm, this is equivalent to the secondary strength level.

`"case"`
: Only strings that differ in base letters or case compare as unequal. Examples: a ≠ b, a = á, a ≠ A. In the Unicode collation algorithm, this is equivalent to the primary strength level with case level handling.

`"variant"`
: Strings that differ in base letters, accents and other diacritic marks, or case compare as unequal. Other differences may also be taken into consideration. Examples: a ≠ b, a ≠ á, a ≠ A. In the Unicode collation algorithm, this is equivalent to the tertiary strength level.

The default is `"variant"` for usage `"sort"`; it's locale dependent for usage `"search"` per spec, but is usually also `"variant"`. Because the core functionality of `"search"` is accent-insensitive and case-insensitive filtering, setting it to `"base"` makes the most sense (and perhaps `"case"`).

`ignorePunctuation`
: Whether punctuation should be ignored. Possible values are `true` and `false`. The default is `true` for Thai (`th`) and `false` for all other languages.

Exceptions

`RangeError`
: Thrown if `locales` or `options` contain invalid values.

Examples

Using Collator

The following example demonstrates the different potential results for a string occurring before, after, or at the same level as another:

```js
console.log(new Intl.Collator().compare("a", "c")); // -1, or some other negative value
console.log(new Intl.Collator().compare("c", "a")); // 1, or some other positive value
console.log(new Intl.Collator().compare("a", "a")); // 0
```

Note that the results shown in the code above can vary between browsers and browser versions. This is because the values are implementation-specific. That is, the specification requires only that the before and after values are negative and positive.

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl-collator-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Collator()` constructor	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [1,2]
`options.caseFirst` parameter	24	18	55	15	11	25	56	14	11	1.5	4.4	11	1.0.0	1.8	0.12.0
`options.collation` parameter	86	86	85	72	14.1	86	85	61	14.5	14.0	86	14.5	1.0.0	1.8	15.0.0
`options.ignorePunctuation` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.localeMatcher` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.numeric` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.sensitivity` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.usage` parameter	24	79	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0

1. Also from version 0.12.0–13.0.0: partial support.
2. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `Collator` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Collator.Collator`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Collator`
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Collator

Intl.Collator.supportedLocalesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator/supportedLocalesOf · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl.Collator.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in collation without having to fall back to the runtime's default locale.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.Collator.supportedLocalesOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };

console.log(Intl.Collator.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

js

```
Intl.Collator.supportedLocalesOf(locales)
Intl.Collator.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in collation without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in collation, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is not used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

js

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.Collator.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.collator.supportedlocalesof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedLocalesOf`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [1,2]

1. Also from version 0.12.0–13.0.0: partial support.
2. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Collator.supportedLocalesOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Collator`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Collator

Intl.Collator.prototype.compare()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator/compare · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2017.

The **`compare()`** method of `Intl.Collator` instances compares two strings according to the sort order of this collator object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.Collator.prototype.compare()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const enCollator = new Intl.Collator("en");
const deCollator = new Intl.Collator("de");
const svCollator = new Intl.Collator("sv");

console.log(enCollator.compare("z", "a") > 0);
// Expected output: true

console.log(deCollator.compare("z", "ä") > 0);
// Expected output: true

console.log(svCollator.compare("z", "ä") > 0);
// Expected output: false
```

Syntax

```js
compare(string1, string2)
```

Parameters

`string1`, `string2`

The strings to compare against each other.

Return value

A number indicating how `string1` and `string2` compare to each other according to the sort order of this `Intl.Collator` object:

- A negative value if `string1` comes before `string2`;
- A positive value if `string1` comes after `string2`;
- 0 if they are considered equal.

Examples

Using compare for array sort

Use the `compare` function for sorting arrays. Note that the function is bound to the collator from which it was obtained, so it can be passed directly to `Array.prototype.sort()`.

```js
const a = ["Offenbach", "Österreich", "Odenwald"];
const collator = new Intl.Collator("de-u-co-phonebk");
a.sort(collator.compare);
console.log(a.join(", ")); // "Odenwald, Österreich, Offenbach"
```

Using compare for array search

Use the `compare` function for finding matching strings in arrays:

```js
const a = ["Congrès", "congres", "Assemblée", "poisson"];
const collator = new Intl.Collator("fr", {
  usage: "search",
  sensitivity: "base",
});
const s = "congres";
const matches = a.filter((v) => collator.compare(v, s) === 0);
console.log(matches.join(", ")); // "Congrès, congres"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.collator.prototype.compare`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`compare`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `Collator()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Collator.compare`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Collator`
- `String.prototype.localeCompare()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Collator

Intl.Collator.prototype.resolvedOptions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator/resolvedOptions · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2017.

The **`resolvedOptions()`** method of `Intl.Collator` instances returns a new object with properties reflecting the options computed during initialization of this `Collator` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.Collator.prototype.resolvedOptions()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const numberDe = new Intl.NumberFormat("de-DE");
const numberAr = new Intl.NumberFormat("ar");

console.log(numberDe.resolvedOptions().numberingSystem);
// Expected output: "latn"

console.log(numberAr.resolvedOptions().numberingSystem);
// Expected output: "arab"
```

Syntax

js

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `Collator` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `co`, `kn`, and `kf` Unicode extension keys, if requested and supported, may be included in the output.

`usage`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"sort"` or `"search"`. The default is `"sort"`.

`sensitivity`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"base"`, `"accent"`, `"case"`, or `"variant"`. The default is `"variant"` for usage `"sort"`; it's locale dependent for usage `"search"`.

`ignorePunctuation`

The value provided for this property in the `options` argument, with default filled in as needed. It is a boolean. The default is `true` for Thai (`th`) and `false` for all other languages.

`collation`

The value provided for this property in the `options` argument, or using the Unicode extension key `"co"`, with default filled in as needed. It is a supported collation type for this locale. The default is `"default"`.

`numeric`

The value provided for this property in the `options` argument, or using the Unicode extension key `"kn"`, with default filled in as needed. It is a boolean. The default is `false`. If the implementation does not support this Unicode extension key, this property is omitted.

`caseFirst`

The value provided for this property in the `options` argument, or using the Unicode extension key `"kf"`, with default filled in as needed. It is either `"upper"`, `"lower"`, or `"false"`. The default is `"false"`. If the implementation does not support this Unicode extension key, this property is omitted.

Examples

Using the resolvedOptions method

```js
const de = new Intl.Collator("de", { sensitivity: "base" });
const usedOptions = de.resolvedOptions();

usedOptions.locale; // "de"
usedOptions.usage; // "sort"
usedOptions.sensitivity; // "base"
usedOptions.ignorePunctuation; // false
usedOptions.collation; // "default"
usedOptions.numeric; // false
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.collator.prototype.resolvedoptions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resolvedOptions`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `Collator()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Collator.resolvedOptions`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Collator`

JavaScript › Reference › Standard built-in objects › Intl

Intl.DateTimeFormat

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DateTimeFormat · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl.DateTimeFormat`** object enables language-sensitive date and time formatting.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.DateTimeFormat". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const date = new Date(Date.UTC(2020, 11, 20, 3, 23, 16, 738));
// Results below assume UTC timezone - your results may vary

// Specify default date formatting for language (locale)
console.log(new Intl.DateTimeFormat("en-US").format(date));
// Expected output: "12/20/2020"

// Specify default date formatting for language with a fallback language (in this case
Indonesian)
console.log(new Intl.DateTimeFormat(["ban", "id"]).format(date));
// Expected output: "20/12/2020"

// Specify date and time format using "style" options (i.e. full, long, medium, short)
console.log(
  new Intl.DateTimeFormat("en-GB", {
    dateStyle: "full",
    timeStyle: "long",
    timeZone: "Australia/Sydney",
  }).format(date),
);
// Expected output: "Sunday, 20 December 2020 at 14:23:16 GMT+11"
```

Constructor

`Intl.DateTimeFormat()`

Creates a new `Intl.DateTimeFormat` object.

Static methods

`Intl.DateTimeFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.DateTimeFormat.prototype` and shared by all `Intl.DateTimeFormat` instances.

`Intl.DateTimeFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.DateTimeFormat` instances, the initial value is the `Intl.DateTimeFormat` constructor.

`Intl.DateTimeFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.DateTimeFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.DateTimeFormat.prototype.format()`

Getter function that formats a date according to the locale and formatting options of this `DateTimeFormat` object.

`Intl.DateTimeFormat.prototype.formatRange()`

This method receives two Dates and formats the date range in the most concise way based on the locale and options provided when instantiating `DateTimeFormat`.

`Intl.DateTimeFormat.prototype.formatRangeToParts()`

This method receives two Dates and returns an Array of objects containing the locale-specific tokens representing each part of the formatted date range.

`Intl.DateTimeFormat.prototype.formatToParts()`

Returns an `Array` of objects representing the date string in parts that can be used for custom locale-aware formatting.

`Intl.DateTimeFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and formatting options computed during initialization of the object.

Examples

Using DateTimeFormat

In basic use without specifying a locale, `DateTimeFormat` uses the default locale and default options.

```js
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// toLocaleString without arguments depends on the implementation,
// the default locale, and the default time zone
console.log(new Intl.DateTimeFormat().format(date));
// "12/19/2012" if run with en-US locale (language) and time zone America/Los_Angeles (UTC-0800)
```

Using locales

This example shows some of the variations in localized date and time formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```js
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// Results below use the time zone of America/Los_Angeles (UTC-0800, Pacific Standard Time)

// US English uses month-day-year order
console.log(new Intl.DateTimeFormat("en-US").format(date));
// "12/19/2012"
```

(code continues on the next page)

(code continued from the previous page)

```
// British English uses day-month-year order
console.log(new Intl.DateTimeFormat("en-GB").format(date));
// "19/12/2012"

// Korean uses year-month-day order
console.log(new Intl.DateTimeFormat("ko-KR").format(date));
// "2012. 12. 19."

// Arabic in most Arabic speaking countries uses real Arabic digits
console.log(new Intl.DateTimeFormat("ar-EG").format(date));
// "١٩/١٢/٢٠١٢"

// for Japanese, applications may want to use the Japanese calendar,
// where 2012 was the year 24 of the Heisei era
console.log(new Intl.DateTimeFormat("ja-JP-u-ca-japanese").format(date));
// "24/12/19"

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(new Intl.DateTimeFormat(["ban", "id"]).format(date));
// "19/12/2012"
```

Using options

The date and time formats can be customized using the `options` argument:

js

```
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0, 200));

// request a weekday along with a long date
let options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
console.log(new Intl.DateTimeFormat("de-DE", options).format(date));
// "Donnerstag, 20. Dezember 2012"

// an application may want to use UTC and make that visible
options.timeZone = "UTC";
options.timeZoneName = "short";
console.log(new Intl.DateTimeFormat("en-US", options).format(date));
// "Thursday, December 20, 2012, GMT"

// sometimes you want to be more precise
options = {
  hour: "numeric",
  minute: "numeric",
  second: "numeric",
  timeZone: "Australia/Sydney",
  timeZoneName: "short",
};
console.log(new Intl.DateTimeFormat("en-AU", options).format(date));
// "2:00:00 pm AEDT"
```

(code continues on the next page)

(code continued from the previous page)

```
// sometimes you want to be very precise
options.fractionalSecondDigits = 3; // number digits for fraction-of-seconds
console.log(new Intl.DateTimeFormat("en-AU", options).format(date));
// "2:00:00.200 pm AEDT"

// sometimes even the US needs 24-hour time
options = {
  year: "numeric",
  month: "numeric",
  day: "numeric",
  hour: "numeric",
  minute: "numeric",
  second: "numeric",
  hour12: false,
  timeZone: "America/Los_Angeles",
};
console.log(new Intl.DateTimeFormat("en-US", options).format(date));
// "12/19/2012, 19:00:00"

// to specify options but use the browser's default locale, use undefined
console.log(new Intl.DateTimeFormat(undefined, options).format(date));
// "12/19/2012, 19:00:00"

// sometimes it's helpful to include the period of the day
options = { hour: "numeric", dayPeriod: "short" };
console.log(new Intl.DateTimeFormat("en-US", options).format(date));
// 10 at night
```

The used calendar and numbering formats can also be set independently via `options` arguments:

js

```
const options = { calendar: "chinese", numberingSystem: "arab" };
const dateFormat = new Intl.DateTimeFormat(undefined, options);
const usedOptions = dateFormat.resolvedOptions();

console.log(usedOptions.calendar);
// "chinese"

console.log(usedOptions.numberingSystem);
// "arab"

console.log(usedOptions.timeZone);
// "America/New_York" (the users default timezone)
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#datetimeformat-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`DateTimeFormat`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[1]
`DateTimeFormat()` constructor	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0[2,3]
Supports normative optional `ChainDateTimeFormat` behavior	91[4,5]	91[6,7]	54	77[8,9]	14.1	91[4,5]	56	64[10,1]1	14.5	16.0[12],13	91[4,5]	14.5	1.0.0	1.8	16.0.0[14,15]
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0[2,3]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.calendar` parameter	80	80	76	67	14.1	80	79	57	14.5	13.0	80	14.5	1.0.0	1.8	14.0.0
`options.dateStyle` parameter	76	79	79	63	14.1	76	79	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0
`options.dayPeriod` parameter	92	92	90	78	14.1	92	90	65	14.5	16.0	92	14.5	1.0.0	1.12	16.6.0
`options.fractionalSecondDigits` parameter	84	84	84	70	14.1	84	84	60	14.5	14.0	84	14.5	1.0.0	1.8	14.6.0
`options.hourCycle` parameter	73	18	58	60	13	73	58	52	13	11.0	73	13	1.0.0	1.8	12.0.0
`options.numberingSystem` parameter	80	80	76	67	14.1	80	79	57	14.5	13.0	80	14.5	1.0.0	1.8	14.0.0
`options.timeStyle` parameter	76	79	79	63	14.1	76	79	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0
`options.timeZoneName` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`shortGeneric`/`longGeneric`/`shortOffset`/`longOffset` as `options.timeZoneName` option	95	95	91	81	15.4	95	91	67	15.4	17.0	95	15.4	1.0.0	**No**	17.0.0
`options.timeZone` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
IANA time zone names in `options.timeZone` option	24	14	52	15	10	25	56	14	10	1.5	37	10	1.0.0	1.8	4.0.0
UTC offset in `options.timeZone` option	120	120	121	106	17.4	120	121	80	17.4	25.0	120	17.4	1.0.19	1.38	22.0.0
`format`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[1]
`formatRange`	76	79	91	63	14.1	76	91	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0[1]
`formatRangeToParts`	76	79	91	63	14.1	76	91	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0[1]
`formatToParts`	57[16]	18	51	44[17]	11	57[16]	56	43[18]	11	7.0[16]	57[16]	11	1.0.0	1.8	8.0.0[1]9,1
`resolvedOptions`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[1]
computed `timeZone`	35	14	53	30	10	35	56	22	10	3.0	37	10	1.0.0	1.8	8.10.0
`supportedLocalesOf`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0[2,20]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `DateTimeFormat()` constructor for more details.

2. Also from version 0.12.0–13.0.0: partial support.
3. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `DateTimeFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
4. Also from version 57–91: partial support.
5. Also from version 57–91: The fallback symbol property has description `IntlFallback`.
6. Also from version 79–91: partial support.
7. Also from version 79–91: The fallback symbol property has description `IntlFallback`.
8. Also from version 44–77: partial support.
9. Also from version 44–77: The fallback symbol property has description `IntlFallback`.
10. Also from version 43–64: partial support.
11. Also from version 43–64: The fallback symbol property has description `IntlFallback`.
12. Also from version 7.0–16.0: partial support.
13. Also from version 7.0–16.0: The fallback symbol property has description `IntlFallback`.
14. Also from version 8.0.0–16.0.0: partial support.
15. Also from version 8.0.0–16.0.0: The fallback symbol property has description `IntlFallback`.
16. Before version 71, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 71 and later use the specification defined `dayPeriod`. See bug 41402623.
17. Before version 58, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 58 and later use the specification defined `dayPeriod`. See bug 41402623.
18. Before version 50, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 50 and later use the specification defined `dayPeriod`. See bug 41402623.
19. Before version 12.0.0, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 12.0.0 and later use the specification defined `dayPeriod`. See bug 41402623.
20. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DateTimeFormat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.DateTimeFormat` in FormatJS
- `Intl`
- `Date.prototype.toLocaleString()`
- `Date.prototype.toLocaleDateString()`
- `Date.prototype.toLocaleTimeString()`
- `Temporal.Instant.prototype.toLocaleString()`
- `Temporal.PlainDate.prototype.toLocaleString()`
- `Temporal.PlainDateTime.prototype.toLocaleString()`
- `Temporal.PlainTime.prototype.toLocaleString()`
- `Temporal.PlainYearMonth.prototype.toLocaleString()`
- `Temporal.PlainMonthDay.prototype.toLocaleString()`
- `Temporal.ZonedDateTime.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DateTimeFormat

Intl.DateTimeFormat() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DateTimeFormat/DateTimeFormat · MDN page last modified 2026-01-21

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl.DateTimeFormat()`** constructor creates `Intl.DateTimeFormat` objects.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.DateTimeFormat() constructor". It runs live in the browser there; its source code follows.

js · interactive example source

```
const date = new Date(Date.UTC(2020, 11, 20, 3, 23, 16, 738));
// Results below assume UTC timezone - your results may vary

// Specify default date formatting for language (locale)
console.log(new Intl.DateTimeFormat("en-US").format(date));
// Expected output: "12/20/2020"

// Specify default date formatting for language with a fallback language (in this case
Indonesian)
console.log(new Intl.DateTimeFormat(["ban", "id"]).format(date));
// Expected output: "20/12/2020"

// Specify date and time format using "style" options (i.e. full, long, medium, short)
console.log(
  new Intl.DateTimeFormat("en-GB", {
    dateStyle: "full",
    timeStyle: "long",
    timeZone: "Australia/Sydney",
  }).format(date),
);
// Expected output: "Sunday, 20 December 2020 at 14:23:16 GMT+11"
```

Syntax

js

```
new Intl.DateTimeFormat()
new Intl.DateTimeFormat(locales)
new Intl.DateTimeFormat(locales, options)

Intl.DateTimeFormat()
Intl.DateTimeFormat(locales)
Intl.DateTimeFormat(locales, options)
```

Note: `Intl.DateTimeFormat()` can be called with or without new. Both create a new `Intl.DateTimeFormat` instance. However, there's a special behavior when it's called without `new` and the `this` value is another `Intl.DateTimeFormat` instance; see Return value.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

The following Unicode extension keys are allowed:

`nu`

See `numberingSystem`.

`ca`

See `calendar`.

`hc`

See `hourCycle`.

These keys can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object. For ease of reading, the property list is broken into sections based on their purposes, including locale options, date-time component options, and style shortcuts.

Locale options

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`calendar`

The calendar to use, such as `"chinese"`, `"gregory"`, `"persian"`, and so on. For a list of supported calendar types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `ca` Unicode extension key; if both are provided, this `options` property takes precedence.

`numberingSystem`

The numbering system to use for number formatting, such as `"arab"`, `"hans"`, `"mathsans"`, and so on. For a list of supported numbering system types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `nu` Unicode extension key; if both are provided, this `options` property takes precedence.

`hour12`

Whether to use 12-hour time (as opposed to 24-hour time). Possible values are `true` and `false`; the default is locale dependent. When `true`, this option sets `hourCycle` to either `"h11"` or `"h12"`, depending on the locale. When `false`, it sets `hourCycle` to `"h23"`. `hour12` overrides both the `hc` locale extension tag and the `hourCycle` option, should either or both of those be present.

`hourCycle`

The hour cycle to use. Possible values are `"h11"`, `"h12"`, `"h23"`, and `"h24"`; the default is inferred from `hour12` and locale. This option can also be set through the `hc` Unicode extension key; if both are provided, this `options` property takes precedence.

`timeZone`

The time zone to use. Can be any IANA time zone name, including named identifiers such as `"UTC"`, `"America/New_York"`, and `"Etc/GMT+8"`, and offset identifiers such as `"+01:00"`, `"-2359"`, and `"+23"`. The default is the runtime's time zone, the same time zone used by `Date.prototype.toString()`.

Date-time component options

`weekday`

The representation of the weekday. Possible values are:

`"long"`

E.g., `Thursday`

`"short"`

E.g., `Thu`

`"narrow"`

E.g., `T`. Two weekdays may have the same narrow style for some locales (e.g., both `Tuesday`'s and `Thursday`'s narrow styles are `T` in the `en-US` locale).

`era`

The representation of the era. Possible values are:

`"long"`

E.g., `Anno Domini`

`"short"`

E.g., `AD`

`"narrow"`

E.g., `A`

`year`

The representation of the year. Possible values are `"numeric"` and `"2-digit"`.

`month`

The representation of the month. Possible values are:

`"numeric"`

E.g., `3`

`"2-digit"`

E.g., `03`

`"long"`

E.g., `March`

`"short"`

E.g., `Mar`

`"narrow"`

E.g., `M`. Two months may have the same narrow style for some locales (e.g., both `March`'s and `May`'s narrow styles are `M` in the `en-US` locale).

`day`

The representation of the day. Possible values are `"numeric"` and `"2-digit"`.

`dayPeriod`

The formatting style used for day periods like "in the morning", "am", "noon", "n" etc. Possible values are `"narrow"`, `"short"`, and `"long"`.

> **Note:** This option only has an effect if a 12-hour clock (`hourCycle: "h12"` or `hourCycle: "h11"`) is used. Many locales use the same string irrespective of the width specified.

`hour`

The representation of the hour. Possible values are `"numeric"` and `"2-digit"`.

`minute`

The representation of the minute. Possible values are `"numeric"` and `"2-digit"`.

`second`

The representation of the second. Possible values are `"numeric"` and `"2-digit"`.

`fractionalSecondDigits`

The number of digits used to represent fractions of a second (any additional digits are truncated). Possible values are from `1` to `3`.

`timeZoneName`

The localized representation of the time zone name. Possible values are:

`"long"`

Long localized form (e.g., `Pacific Standard Time`, `Nordamerikanische Westküsten-Normalzeit`)

`"short"`

Short localized form (e.g.: `PST`, `GMT-8`)

`"shortOffset"`

Short localized GMT format (e.g., `GMT-8`)

`"longOffset"`

Long localized GMT format (e.g., `GMT-08:00`)

`"shortGeneric"`

Short generic non-location format (e.g.: `PT`, `Los Angeles Zeit`).

`"longGeneric"`

Long generic non-location format (e.g.: `Pacific Time`, `Nordamerikanische Westküstenzeit`)

> **Note:** Timezone display may fall back to another format if a required string is unavailable. For example, the non-location formats should display the timezone without a specific country/city location like "Pacific Time", but may fall back to a timezone like "Los Angeles Time".

Date-time component default values

If any of the date-time component options are specified, then `dateStyle` and `timeStyle` must be `undefined`. If all date-time component options and `dateStyle`/`timeStyle` are `undefined`, some default options for date-time components are set, which depend on the object that the formatting method was called with:

- When formatting `Temporal.PlainDate` and `Date`, `year`, `month`, and `day` default to `"numeric"`.
- When formatting `Temporal.PlainTime`, `hour`, `minute`, and `second` default to `"numeric"`.
- When formatting `Temporal.PlainYearMonth`, `year` and `month` default to `"numeric"`.
- When formatting `Temporal.PlainMonthDay`, `month` and `day` default to `"numeric"`.
- When formatting `Temporal.PlainDateTime` and `Temporal.Instant`, `year`, `month`, `day`, `hour`, `minute`, and `second` default to `"numeric"`.

Format matching

Implementations are required to support displaying at least the following subsets of date-time components:

- `weekday`, `year`, `month`, `day`, `hour`, `minute`, `second`
- `weekday`, `year`, `month`, `day`
- `year`, `month`, `day`
- `year`, `month`

- `month`, `day`
- `hour`, `minute`, `second`
- `hour`, `minute`

The date-time component styles requested might not directly correspond to a valid format supported by the locale, so the format matcher allows you to specify how to match the requested styles to the closest supported format.

`formatMatcher`

The format matching algorithm to use. Possible values are `"basic"` and `"best fit"`; the default is `"best fit"`. The algorithm for `"best fit"` is implementation-defined, and `"basic"` is defined by the spec. This option is only used when both `dateStyle` and `timeStyle` are `undefined` (so that each date-time component's format is individually customizable).

Style shortcuts

`dateStyle`

The date formatting style to use. Possible values are `"full"`, `"long"`, `"medium"`, and `"short"`. It expands to styles for `weekday`, `day`, `month`, `year`, and `era`, with the exact combination of values depending on the locale. When formatting objects such as `Temporal.PlainDate`, `Temporal.PlainYearMonth`, and `Temporal.PlainMonthDay`, `dateStyle` will resolve to only those fields relevant to the object.

`timeStyle`

The time formatting style to use. Possible values are `"full"`, `"long"`, `"medium"`, and `"short"`. It expands to styles for `hour`, `minute`, `second`, and `timeZoneName`, with the exact combination of values depending on the locale.

> **Note:** `dateStyle` and `timeStyle` can be used with each other, but not with other date-time component options (e.g., `weekday`, `hour`, `month`, etc.).

You can format different object types depending on which of the style shortcut options you include:

- If the `dateStyle` is specified, then you can format `Temporal.PlainDate`, `Temporal.PlainYearMonth`, and `Temporal.PlainMonthDay` objects.
- If the `timeStyle` is specified, then you can format `Temporal.PlainTime` objects.
- If either `dateStyle` or `timeStyle` is specified, then you can format `Temporal.PlainDateTime` and `Date` objects.

Return value

A new `Intl.DateTimeFormat` object.

> **Note:** The text below describes behavior that is marked by the specification as "optional". It may not work in all environments. Check the browser compatibility table.

Normally, `Intl.DateTimeFormat()` can be called with or without `new`, and a new `Intl.DateTimeFormat` instance is returned in both cases. However, if the `this` value is an object that is `instanceof Intl.DateTimeFormat` (doesn't necessarily mean it's created via `new Intl.DateTimeFormat`; just that it has `Intl.DateTimeFormat.prototype` in its prototype chain), then the value of `this` is returned instead, with the newly created `Intl.DateTimeFormat` object hidden in a

`[Symbol(IntlLegacyConstructedSymbol)]` property (a unique symbol that's reused between instances).

```js
const formatter = Intl.DateTimeFormat.call(
  { __proto__: Intl.DateTimeFormat.prototype },
  "en-US",
  { dateStyle: "full" },
);
console.log(Object.getOwnPropertyDescriptors(formatter));
// {
//   [Symbol(IntlLegacyConstructedSymbol)]: {
//     value: DateTimeFormat [Intl.DateTimeFormat] {},
//     writable: false,
//     enumerable: false,
//     configurable: false
//   }
// }
```

Note that there's only one actual `Intl.DateTimeFormat` instance here: the one hidden in `[Symbol(IntlLegacyConstructedSymbol)]`. Calling the `format()` and `resolvedOptions()` methods on `formatter` would correctly use the options stored in that instance, but calling all other methods (e.g., `formatRange()`) would fail: "TypeError: formatRange method called on incompatible Object", because those methods don't consult the hidden instance's options.

This behavior, called `ChainDateTimeFormat`, does not happen when `Intl.DateTimeFormat()` is called without `new` but with `this` set to anything else that's not an `instanceof Intl.DateTimeFormat`. If you call it directly as `Intl.DateTimeFormat()`, the `this` value is `Intl`, and a new `Intl.DateTimeFormat` instance is created normally.

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Using DateTimeFormat

In basic use without specifying a locale, `DateTimeFormat` uses the default locale and default options.

```js
const date = new Date(Date.UTC(2012, 11, 20, 3, 0, 0));

// toLocaleString without arguments depends on the implementation,
// the default locale, and the default time zone
console.log(new Intl.DateTimeFormat().format(date));
// "12/19/2012" if run with en-US locale (language) and time zone America/Los_Angeles (UTC-0800)
```

Using timeStyle and dateStyle

`dateStyle` and `timeStyle` provide a shortcut for setting multiple date-time component options at once. For example, for `en-US`, `dateStyle: "short"` is equivalent to setting `year: "2-digit", month: "numeric", day: "numeric"`, and `timeStyle: "short"` is equivalent to setting `hour: "numeric", minute: "numeric"`.

```js
const shortTime = new Intl.DateTimeFormat("en-US", {
  timeStyle: "short",
});
console.log(shortTime.format(Date.now())); // "1:31 PM"

const shortDate = new Intl.DateTimeFormat("en-US", {
  dateStyle: "short",
});
console.log(shortDate.format(Date.now())); // "7/7/20"

const mediumTime = new Intl.DateTimeFormat("en-US", {
  timeStyle: "medium",
  dateStyle: "short",
});
console.log(mediumTime.format(Date.now())); // "7/7/20, 1:31:55 PM"
```

However, the exact (locale dependent) component styles they resolve to are not included in the resolved options. This ensures the result of `resolvedOptions()` can be passed directly to the `Intl.DateTimeFormat()` constructor (because an `options` object with both `dateStyle` or `timeStyle` and individual date or time component styles is not valid).

```js
console.log(shortDate.resolvedOptions().year); // undefined
```

Using dayPeriod

Use the `dayPeriod` option to output a string for the times of day ("in the morning", "at night", "noon", etc.). Note, that this only works when formatting for a 12 hour clock (`hourCycle: 'h12'` or `hourCycle: 'h11'`) and that for many locales the strings are the same irrespective of the value passed for the `dayPeriod`.

```js
const date = Date.UTC(2012, 11, 17, 4, 0, 42);

console.log(
  new Intl.DateTimeFormat("en-GB", {
    hour: "numeric",
    hourCycle: "h12",
    dayPeriod: "short",
    timeZone: "UTC",
  }).format(date),
);
// 4 at night"  (same formatting in en-GB for all dayPeriod values)

console.log(
  new Intl.DateTimeFormat("fr", {
    hour: "numeric",
    hourCycle: "h12",
    dayPeriod: "narrow",
    timeZone: "UTC",
  }).format(date),
);
// "4 mat."  (same output in French for both narrow/short dayPeriod)
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(
  new Intl.DateTimeFormat("fr", {
    hour: "numeric",
    hourCycle: "h12",
    dayPeriod: "long",
    timeZone: "UTC",
  }).format(date),
);
// "4 du matin"
```

Using timeZoneName

Use the `timeZoneName` option to output a string for the timezone ("GMT", "Pacific Time", etc.).

js

```
const date = Date.UTC(2021, 11, 17, 3, 0, 42);
const timezoneNames = [
  "short",
  "long",
  "shortOffset",
  "longOffset",
  "shortGeneric",
  "longGeneric",
];

for (const zoneName of timezoneNames) {
  // Do something with currentValue
  const formatter = new Intl.DateTimeFormat("en-US", {
    timeZone: "America/Los_Angeles",
    timeZoneName: zoneName,
  });
  console.log(`${zoneName}: ${formatter.format(date)}`);
}

// Logs:
// short: 12/16/2021, PST
// long: 12/16/2021, Pacific Standard Time
// shortOffset: 12/16/2021, GMT-8
// longOffset: 12/16/2021, GMT-08:00
// shortGeneric: 12/16/2021, PT
// longGeneric: 12/16/2021, Pacific Time
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl-datetimeformat-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`DateTimeFormat()` constructor	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [1,2]
Supports normative optional `ChainDateTimeFormat` behavior	91 [3,4]	91 [5,6]	54	77 [7,8]	14.1	91 [3,4]	56	64 [9,10]	14.5	16.0 [11,12]	91 [3,4]	14.5	1.0.0	1.8	16.0.0 [13,14]
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [1,2]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.calendar` parameter	80	80	76	67	14.1	80	79	57	14.5	13.0	80	14.5	1.0.0	1.8	14.0.0
`options.dateStyle` parameter	76	79	79	63	14.1	76	79	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0
`options.dayPeriod` parameter	92	92	90	78	14.1	92	90	65	14.5	16.0	92	14.5	1.0.0	1.12	16.6.0
`options.fractionalSecondDigits` parameter	84	84	84	70	14.1	84	84	60	14.5	14.0	84	14.5	1.0.0	1.8	14.6.0
`options.hourCycle` parameter	73	18	58	60	13	73	58	52	13	11.0	73	13	1.0.0	1.8	12.0.0
`options.numberingSystem` parameter	80	80	76	67	14.1	80	79	57	14.5	13.0	80	14.5	1.0.0	1.8	14.0.0
`options.timeStyle` parameter	76	79	79	63	14.1	76	79	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0
`options.timeZoneName` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`shortGeneric`/`longGeneric`/`shortOffset`/`longOffset` as `options.timeZoneName` option	95	95	91	81	15.4	95	91	67	15.4	17.0	95	15.4	1.0.0	**No**	17.0.0
`options.timeZone` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
IANA time zone names in `options.timeZone` option	24	14	52	15	10	25	56	14	10	1.5	37	10	1.0.0	1.8	4.0.0
UTC offset in `options.timeZone` option	120	120	121	106	17.4	120	121	80	17.4	25.0	120	17.4	1.0.19	1.38	22.0.0

1. Also from version 0.12.0–13.0.0: partial support.
2. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `DateTimeFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
3. Also from version 57–91: partial support.
4. Also from version 57–91: The fallback symbol property has description `IntlFallback`.
5. Also from version 79–91: partial support.
6. Also from version 79–91: The fallback symbol property has description `IntlFallback`.
7. Also from version 44–77: partial support.
8. Also from version 44–77: The fallback symbol property has description `IntlFallback`.
9. Also from version 43–64: partial support.
10. Also from version 43–64: The fallback symbol property has description `IntlFallback`.
11. Also from version 7.0–16.0: partial support.
12. Also from version 7.0–16.0: The fallback symbol property has description `IntlFallback`.
13. Also from version 8.0.0–16.0.0: partial support.
14. Also from version 8.0.0–16.0.0: The fallback symbol property has description `IntlFallback`.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DateTimeFormat.DateTimeFormat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DateTimeFormat`
- `Intl.supportedValuesOf()`
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DateTimeFormat

Intl.DateTimeFormat.supportedLocalesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DateTimeFormat/supportedLocalesOf · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl.DateTimeFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in date and time formatting without having to fall back to the runtime's default locale.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.DateTimeFormat.supportedLocalesOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };

console.log(Intl.DateTimeFormat.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

js

```
Intl.DateTimeFormat.supportedLocalesOf(locales)
Intl.DateTimeFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in date and time formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in date and time formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to date and time formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```js
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.DateTimeFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.datetimeformat.supportedlocalesof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedLocalesOf`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [1,2]

1. Also from version 0.12.0–13.0.0: partial support.
2. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DateTimeFormat.supportedLocalesOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DateTimeFormat`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DateTimeFormat

Intl.DateTimeFormat.prototype.format()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DateTimeFormat/format · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since September 2017.

The **`format()`** method of `Intl.DateTimeFormat` instances formats a date according to the locale and formatting options of this `Intl.DateTimeFormat` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.DateTimeFormat.prototype.format()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
const date = new Date(2012, 5);

const dateTimeFormat1 = new Intl.DateTimeFormat("sr-RS", options);
console.log(dateTimeFormat1.format(date));
// Expected output: "петак, 1. јун 2012."

const dateTimeFormat2 = new Intl.DateTimeFormat("en-GB", options);
console.log(dateTimeFormat2.format(date));
// Expected output: "Friday, 1 June 2012"

const dateTimeFormat3 = new Intl.DateTimeFormat("en-US", options);
console.log(dateTimeFormat3.format(date));
// Expected output: "Friday, June 1, 2012"
```

Syntax

js

```
format(date)
```

Parameters

`date`

The date to format. Can be a `Date` or `Temporal.PlainDateTime` object. Additionally can be a `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay` object if the `DateTimeFormat` object was configured to print at least one relevant part of the date.

> **Note:** A `Temporal.ZonedDateTime` object will always throw a `TypeError`; use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object instead.

Omitting it results in formatting the current date (as returned by `Date.now()`), which could be slightly confusing, so it is advisable to always explicitly pass a date.

Return value

A string representing the given `date` formatted according to the locale and formatting options of this `Intl.DateTimeFormat` object.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Examples

Using format

Use the `format` getter function for formatting a single date, here for Serbia:

js

```
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
const dateTimeFormat = new Intl.DateTimeFormat("sr-RS", options);
console.log(dateTimeFormat.format(new Date()));
// "недеља, 7. април 2013."
```

Using format with map

Use the `format` getter function for formatting all dates in an array. Note that the function is bound to the `Intl.DateTimeFormat` from which it was obtained, so it can be passed directly to `Array.prototype.map()`.

js

```
const a = [new Date(2012, 8), new Date(2012, 11), new Date(2012, 3)];
const options = { year: "numeric", month: "long" };
const dateTimeFormat = new Intl.DateTimeFormat("pt-BR", options);
const formatted = a.map(dateTimeFormat.format);
console.log(formatted.join("; "));
// "setembro de 2012; dezembro de 2012; abril de 2012"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.datetimeformat.prototype.format`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`format`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `DateTimeFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DateTimeFormat.format`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DateTimeFormat`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DateTimeFormat

Intl.DateTimeFormat.prototype.formatRange()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DateTimeFormat/formatRange · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since August 2021.

The **`formatRange()`** method of `Intl.DateTimeFormat` instances formats a date range in the most concise way based on the locales and options provided when instantiating this `Intl.DateTimeFormat` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.DateTimeFormat.prototype.formatRange()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const options1 = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
const options2 = { year: "2-digit", month: "numeric", day: "numeric" };

const startDate = new Date(Date.UTC(2007, 0, 10, 10, 0, 0));
const endDate = new Date(Date.UTC(2008, 0, 10, 11, 0, 0));

const dateTimeFormat = new Intl.DateTimeFormat("en", options1);
console.log(dateTimeFormat.formatRange(startDate, endDate));
// Expected output: "Wednesday, January 10, 2007 – Thursday, January 10, 2008"

const dateTimeFormat2 = new Intl.DateTimeFormat("en", options2);
console.log(dateTimeFormat2.formatRange(startDate, endDate));
// Expected output: "1/10/07 – 1/10/08"
```

Syntax

```js
formatRange(startDate, endDate)
```

Parameters

`startDate`

The start of the date range. Can be a `Date` or `Temporal.PlainDateTime` object. Additionally can be a `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay` object if the `DateTimeFormat` object was configured to print at least one relevant part of the date.

> **Note:** A `Temporal.ZonedDateTime` object will always throw a `TypeError`; use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object instead.

`endDate`

The end of the date range. Must have the same type as `startDate`.

Return value

A string representing the given date range formatted according to the locale and formatting options of this `Intl.DateTimeFormat` object. If the start and end dates are equivalent at the precision of the output, the output will only contain a single date.

Examples

Basic formatRange usage

This method receives two `Date`s and formats the date range in the most concise way based on the `locale` and `options` provided when instantiating `Intl.DateTimeFormat`.

```js
const date1 = new Date(Date.UTC(1906, 0, 10, 10, 0, 0)); // Wed, 10 Jan 1906 10:00:00 GMT
const date2 = new Date(Date.UTC(1906, 0, 10, 11, 0, 0)); // Wed, 10 Jan 1906 11:00:00 GMT
const date3 = new Date(Date.UTC(1906, 0, 20, 10, 0, 0)); // Sat, 20 Jan 1906 10:00:00 GMT

const fmt1 = new Intl.DateTimeFormat("en", {
  year: "2-digit",
  month: "numeric",
  day: "numeric",
  hour: "numeric",
  minute: "numeric",
});
console.log(fmt1.format(date1)); // '1/10/06, 10:00 AM'
console.log(fmt1.formatRange(date1, date2)); // '1/10/06, 10:00 – 11:00 AM'
console.log(fmt1.formatRange(date1, date3)); // '1/10/06, 10:00 AM – 1/20/07, 10:00 AM'

const fmt2 = new Intl.DateTimeFormat("en", {
  year: "numeric",
  month: "short",
  day: "numeric",
});
console.log(fmt2.format(date1)); // 'Jan 10, 1906'
console.log(fmt2.formatRange(date1, date2)); // 'Jan 10, 1906'
console.log(fmt2.formatRange(date1, date3)); // 'Jan 10 – 20, 1906'
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.datetimeformat.prototype.formatRange`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`formatRange`	76	79	91	63	14.1	76	91	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0[1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `DateTimeFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DateTimeFormat.formatRange`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.DateTimeFormat`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DateTimeFormat

Intl.DateTimeFormat.prototype.formatRangeToParts()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DateTimeFormat/formatRangeToParts · MDN page last modified 2026-05-21

Baseline: Widely available — available across major browsers since August 2021.

The **`formatRangeToParts()`** method of `Intl.DateTimeFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `formatRange()`. It is useful for building custom strings from the locale-specific tokens.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.DateTimeFormat.prototype.formatRangeToParts()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const startDate = new Date(Date.UTC(2007, 0, 10, 10, 0, 0)); // > 'Wed, 10 Jan 2007 10:00:00
GMT'
const endDate = new Date(Date.UTC(2007, 0, 10, 11, 0, 0)); // > 'Wed, 10 Jan 2007 11:00:00
GMT'

const dateTimeFormat = new Intl.DateTimeFormat("en", {
  hour: "numeric",
  minute: "numeric",
});

const parts = dateTimeFormat.formatRangeToParts(startDate, endDate);
for (const part of parts) {
  console.log(part);
}
// Expected output (in GMT timezone):
// Object { type: "hour", value: "2", source: "startRange" }
// Object { type: "literal", value: ":", source: "startRange" }
// Object { type: "minute", value: "00", source: "startRange" }
// Object { type: "literal", value: " – ", source: "shared" }
// Object { type: "hour", value: "3", source: "endRange" }
// Object { type: "literal", value: ":", source: "endRange" }
// Object { type: "minute", value: "00", source: "endRange" }
// Object { type: "literal", value: " ", source: "shared" }
// Object { type: "dayPeriod", value: "AM", source: "shared" }
```

Syntax

js

```
formatRangeToParts(startDate, endDate)
```

Parameters

`startDate`

The start of the date range. Can be a `Date` or `Temporal.PlainDateTime` object. Additionally can be a `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay` object if the `DateTimeFormat` object was configured to print at least one relevant part of the date.

> **Note:** A `Temporal.ZonedDateTime` object will always throw a `TypeError`; use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object instead.

`endDate`

The end of the date range. Must have the same type as `startDate`.

Return value

An `Array` of objects containing the formatted date range in parts. Each object has three properties, `type`, `value`, and `source`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `formatRange()`. The `type` may have the same values as `formatToParts()`. The `source` can be one of the following:

`startRange`

The token is a part of the start date.

`endRange`

The token is a part of the end date.

`shared`

The token is shared between the start and end; for example, if the start and end dates share the same day period, that token may get reused. All literals that are part of the range pattern itself, such as the `" – "` separator, are also marked as `shared`.

If the start and end dates are equivalent at the precision of the output, then the output has the same list of tokens as calling `formatToParts()` on the start date, with all tokens marked as `source: "shared"`.

Examples

Using formatRangeToParts()

The `formatRange()` method outputs localized, opaque strings that cannot be manipulated directly:

```js
const date1 = new Date(Date.UTC(1906, 0, 10, 10, 0, 0)); // Wed, 10 Jan 1906 10:00:00 GMT
const date2 = new Date(Date.UTC(1906, 0, 10, 11, 0, 0)); // Wed, 10 Jan 1906 11:00:00 GMT

const fmt = new Intl.DateTimeFormat("en", {
  hour: "numeric",
  minute: "numeric",
});

console.log(fmt.formatRange(date1, date2)); // '10:00 – 11:00 AM'
```

However, in many user interfaces you may want to customize the formatting of this string, or interleave it with other texts. The `formatRangeToParts()` method produces the same information in parts:

```js
console.log(fmt.formatRangeToParts(date1, date2));

// return value:
[
  { type: "hour", value: "10", source: "startRange" },
  { type: "literal", value: ":", source: "startRange" },
  { type: "minute", value: "00", source: "startRange" },
  { type: "literal", value: " – ", source: "shared" },
  { type: "hour", value: "11", source: "endRange" },
  { type: "literal", value: ":", source: "endRange" },
  { type: "minute", value: "00", source: "endRange" },
  { type: "literal", value: " ", source: "shared" },
  { type: "dayPeriod", value: "AM", source: "shared" },
];
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.DateTimeFormat.prototype.formatRangeToParts`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`formatRangeToParts`	76	79	91	63	14.1	76	91	54	14.5	12.0	76	14.5	1.0.0	1.8	12.9.0 [1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `DateTimeFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DateTimeFormat.formatRangeToParts`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DateTimeFormat`
- `Intl.DateTimeFormat.prototype.formatRange()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DateTimeFormat

Intl.DateTimeFormat.prototype.formatToParts()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DateTimeFormat/formatToParts · MDN page last modified 2026-05-21

Baseline: Widely available — available across major browsers since October 2018.

The **`formatToParts()`** method of `Intl.DateTimeFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.DateTimeFormat.prototype.formatToParts()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const date = new Date(2012, 5);
const options = {
  weekday: "long",
  year: "numeric",
  month: "long",
  day: "numeric",
};
const dateTimeFormat = new Intl.DateTimeFormat("en-US", options);

const parts = dateTimeFormat.formatToParts(date);
const partValues = parts.map((p) => p.value);

console.log(partValues);
// Expected output: "["Friday", ", ", "June", " ", "1", ", ", "2012"]"
```

Syntax

js

```
formatToParts(date)
```

Parameters

`date` Optional

The date to format. Can be a `Date` or `Temporal.PlainDateTime` object. Additionally can be a `Temporal.PlainTime`, `Temporal.PlainDate`, `Temporal.PlainYearMonth`, or `Temporal.PlainMonthDay` object if the `DateTimeFormat` object was configured to print at least one relevant part of the date.

> **Note:** A `Temporal.ZonedDateTime` object will always throw a `TypeError`; use `Temporal.ZonedDateTime.prototype.toLocaleString()` or convert it to a `Temporal.PlainDateTime` object instead.

Omitting it results in formatting the current date (as returned by `Date.now()`), which could be slightly confusing, so it is advisable to always explicitly pass a date.

Return value

An `Array` of objects containing the formatted date in parts. Each object has two properties, `type` and `value`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The `type` may be one of the date-time components:

`weekday`

For example `"M"`, `"Monday"`, or `"Montag"`.

`era`

For example `"BC"` or `"AD"`.

`year`

For example `"2012"` or `"96"`.

`month`
For example `"12"` or `"January"`.

`day`
For example `"17"`.

`dayPeriod`
For example `"AM"`, `"PM"`, `"in the morning"`, or `"noon"`.

`hour`
For example `"3"` or `"03"`.

`minute`
For example `"00"`.

`second`
For example `"07"` or `"42"`.

`fractionalSecond`
For example `"0"`, `"00"`, or `"000"`.

`timeZoneName`
For example `"UTC"`, `"CET"`, or `"Central European Time"`.

The `type` may also be one of the following:

`literal`
Any string that's a part of the format pattern and not influenced by the `date`; for example `"/"`, `", "`, `"o'clock"`, `"de"`, `" "`, etc.

`relatedYear`
A 4-digit Gregorian year, in the event that the calendar's representation would be a `yearName` instead of a year; for example `"2019"`. See named years for more details.

`yearName`
The name given to the year, usually in calendars without the concept of continuous years; for example `"geng-zi"`.

`unknown`
Reserved for any token that's not recognized as any of the above; should be rarely encountered.

Examples

Using formatToParts()

The `format()` method outputs localized, opaque strings that cannot be manipulated directly:

```js
const date = Date.UTC(2012, 11, 17, 3, 0, 42);

const formatter = new Intl.DateTimeFormat("en-us", {
  weekday: "long",
  year: "numeric",
  month: "numeric",
  day: "numeric",
  hour: "numeric",
  minute: "numeric",
  second: "numeric",
  fractionalSecondDigits: 3,
```

(code continues on the next page)

(code continued from the previous page)

```
  hour12: true,
  timeZone: "UTC",
});

formatter.format(date);
// "Monday, 12/17/2012, 3:00:42.000 AM"
```

However, in many user interfaces you may want to customize the formatting of this string, or interleave it with other texts. The `formatToParts()` method produces the same information in parts:

js

```
formatter.formatToParts(date);

// return value:
[
  { type: "weekday", value: "Monday" },
  { type: "literal", value: ", " },
  { type: "month", value: "12" },
  { type: "literal", value: "/" },
  { type: "day", value: "17" },
  { type: "literal", value: "/" },
  { type: "year", value: "2012" },
  { type: "literal", value: ", " },
  { type: "hour", value: "3" },
  { type: "literal", value: ":" },
  { type: "minute", value: "00" },
  { type: "literal", value: ":" },
  { type: "second", value: "42" },
  { type: "fractionalSecond", value: "000" },
  { type: "literal", value: " " },
  { type: "dayPeriod", value: "AM" },
];
```

Now the information is available separately and it can be formatted and concatenated again in a customized way. For example by using `Array.prototype.map()`, arrow functions, a switch statement, template literals, and `Array.prototype.join()`, to insert additional markup for certain components.

js

```
const dateString = formatter
  .formatToParts(date)
  .map(({ type, value }) => {
    switch (type) {
      case "dayPeriod":
        return `<em>${value}</em>`;
      default:
        return value;
    }
  })
  .join("");

console.log(dateString);
// "Monday, 12/17/2012, 3:00:42.000 <em>AM</em>"
```

Named years

Some calendars use named years; for example, the Chinese and Tibetan calendars use a 60-year sexagenary cycle of named years. These calendars do not have a universal way to unambiguously number each year, so years are disambiguated by relationship to corresponding years on the Gregorian

calendar. In this case, when the `DateTimeFormat` is configured to output the year component, a token for `relatedYear` is output instead of `year`.

```js
const df = new Intl.DateTimeFormat("zh-u-ca-chinese");
df.formatToParts(Date.UTC(2012, 11, 17, 3, 0, 42));

// return value:
[
  { type: "relatedYear", value: "2012" },
  { type: "literal", value: "年" },
  { type: "month", value: "十一月" },
  { type: "day", value: "4" },
];
```

Sometimes, the combination of date-time component options maps to a format that also includes a `yearName`. There isn't a separate option that controls whether `yearName` is displayed or not. For example, the options below sets `month` to `"long"` and results in a `yearName` token, despite `year` still being `"numeric"`:

```js
const opts = { year: "numeric", month: "long", day: "numeric" };
const df = new Intl.DateTimeFormat("zh-u-ca-chinese", opts);
df.formatToParts(Date.UTC(2012, 11, 17, 3, 0, 42));

// return value:
[
  { type: "relatedYear", value: "2012" },
  { type: "yearName", value: "壬辰" },
  { type: "literal", value: "年" },
  { type: "month", value: "十一月" },
  { type: "day", value: "4" },
];
```

Because `format()` just concatenates all the `value` strings together, you will see the Gregorian year and the year name together in the output in this case.

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.DateTimeFormat.prototype.formatToParts`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`formatToParts`	57[1]	18	51	44[2]	11	57[1]	56	43[3]	11	7.0[1]	57[1]	11	1.0.0	1.8	8.0.0[4,5]

1. Before version 71, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 71 and later use the specification defined `dayPeriod`. See bug 41402623.

2. Before version 58, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 58 and later use the specification defined `dayPeriod`. See bug 41402623.
3. Before version 50, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 50 and later use the specification defined `dayPeriod`. See bug 41402623.
4. Before version 12.0.0, `formatToParts()` returned an object with an incorrectly cased type key of `dayperiod`. Version 12.0.0 and later use the specification defined `dayPeriod`. See bug 41402623.
5. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `DateTimeFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DateTimeFormat.formatToParts`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.DateTimeFormat`
- `Intl.DateTimeFormat.prototype.format()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DateTimeFormat

Intl.DateTimeFormat.prototype.resolvedOptions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DateTimeFormat/resolvedOptions · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2017.

The **`resolvedOptions()`** method of `Intl.DateTimeFormat` instances returns a new object with properties reflecting the options computed during initialization of this `DateTimeFormat` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.DateTimeFormat.prototype.resolvedOptions()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const region = new Intl.DateTimeFormat("zh-CN", { timeZone: "UTC" });
const options = region.resolvedOptions();

console.log(options.locale);
// Expected output: "zh-CN"

console.log(options.calendar);
// Expected output: "gregory"

console.log(options.numberingSystem);
// Expected output: "latn"
```

Syntax

js

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `DateTimeFormat` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `ca`, `hc`, and `nu` Unicode extension keys, if requested, may be included in the output.

`calendar`

The value provided for this property in the `options` argument, or using the Unicode extension key `"ca"`, with default filled in as needed. It is a supported calendar type for this locale. The default is locale dependent.

`numberingSystem`

The value provided for this property in the `options` argument, or using the Unicode extension key `"nu"`, with default filled in as needed. It is a supported numbering system for this locale. The default is locale dependent.

`timeZone`

The value provided for this property in the `options` argument, with default filled in as needed. It is an IANA time zone name. The default is the runtime's default time zone.

> **Note:** The standardization of `Temporal` requires browsers to use the same identifier as originally specified, without canonicalization to a different alias. See time zones and offsets for more information.

`hourCycle` Optional

The value provided for this property in the `options` argument, or using the Unicode extension key `"hc"`, with default filled in as needed. If `hour12` was provided in the `options`, then that overrides other `hourCycle` settings. It is only present if the resolved options also include `hour` or `timeStyle`. It is either `"h11"`, `"h12"`, `"h23"`, or `"h24"`. The default is locale dependent, although `"h24"` is never a default.

`hour12` Optional

Calculated from `hourCycle`. It is only present if the resolved options also include `hour` or `timeStyle`. It is `true` if `hourCycle` is `"h11"` or `"h12"`, and `false` if `hourCycle` is `"h23"` or `"h24"`.

`weekday`, `era`, `year`, `month`, `day`, `dayPeriod`, `hour`, `minute`, `second`, `fractionalSecondDigits`, `timeZoneName` Optional

The values resulting from format matching between the corresponding properties in the `options` argument and the available combinations and representations for date-time formatting in the selected locale. Some of these properties may not be present, indicating that the corresponding components will not be represented in formatted output. `weekday`, `era`, and `dayPeriod` are either `"narrow"`, `"short"`, or `"long"`; `year`, `day`, `hour`, `minute`, and `second` are either `"numeric"`, `"2-digit"`, or `"narrow"`; `month` is either `"numeric"`, `"2-digit"`, `"narrow"`, `"short"`, or `"long"`; `fractionalSecondDigits` is either `1`, `2`, or `3`; `timeZoneName` is either `"short"`, `"long"`, `"shortOffset"`, `"longOffset"`, `"shortGeneric"`, or `"longGeneric"`.

If these properties were requested in `options`, the constructor prevents `dateStyle` and `timeStyle` from being specified, so the below group will never be present.

`dateStyle`, `timeStyle` Optional

The values provided for these properties in the `options` argument. They are either `"full"`, `"long"`, `"medium"`, `"short"`, or `"none"`. Some of these properties may not be present, indicating that the corresponding components will not be represented in formatted output.

If these properties were requested in `options`, the constructor prevents individual date time component options from being specified, so the above group will never be present.

> **Note:** Although `dateStyle` and `timeStyle` are shortcuts for individual date and time component styles, the exact (locale dependent) component styles they resolve to are not included in the resolved options. This ensures the result of `resolvedOptions()` can be passed directly to the `Intl.DateTimeFormat()` constructor (because an `options` object with both `dateStyle` or `timeStyle` and individual date or time component styles is not valid).

Examples

Using the resolvedOptions method

```js
const germanFakeRegion = new Intl.DateTimeFormat("de-XX", { timeZone: "UTC" });
const usedOptions = germanFakeRegion.resolvedOptions();

usedOptions.locale; // "de" (because "de-XX" does not exist)
usedOptions.calendar; // "gregory"
usedOptions.numberingSystem; // "latn"
usedOptions.timeZone; // "UTC"
usedOptions.month; // "numeric"
```

Getting the user's time zone and locale preferences

The `Intl.DateTimeFormat` constructor without any options uses the current system settings. You can use `resolvedOptions()` to get the user's current time zone and locale's preferred calendar and numbering system:

```js
const systemOptions = new Intl.DateTimeFormat().resolvedOptions();
systemOptions.timeZone; // e.g., "Europe/Brussels" or "Asia/Riyadh"
systemOptions.calendar; // e.g., "gregory" or "islamic-umalqura"
systemOptions.numberingSystem; // e.g., "latn" or "arab"
systemOptions.locale; // e.g., "nl-BE" or "ar-SA"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.datetimeformat.prototype.resolvedoptions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resolvedOptions`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[1]
computed `timeZone`	35	14	53	30	10	35	56	22	10	3.0	37	10	1.0.0	1.8	8.10.0

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `DateTimeFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DateTimeFormat.resolvedOptions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DateTimeFormat`

JavaScript › Reference › Standard built-in objects › Intl

Intl.DisplayNames

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DisplayNames · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

The **`Intl.DisplayNames`** object enables the consistent translation of language, region and script display names.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.DisplayNames". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regionNamesInEnglish = new Intl.DisplayNames(["en"], { type: "region" });
const regionNamesInTraditionalChinese = new Intl.DisplayNames(["zh-Hant"], {
  type: "region",
});

console.log(regionNamesInEnglish.of("US"));
// Expected output: "United States"

console.log(regionNamesInTraditionalChinese.of("US"));
// Expected output: "美國"
```

Constructor

`Intl.DisplayNames()`

Creates a new `Intl.DisplayNames` object.

Static methods

`Intl.DisplayNames.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.DisplayNames.prototype` and shared by all `Intl.DisplayNames` instances.

`Intl.DisplayNames.prototype.constructor`

The constructor function that created the instance object. For `Intl.DisplayNames` instances, the initial value is the `Intl.DisplayNames` constructor.

`Intl.DisplayNames.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.DisplayNames"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.DisplayNames.prototype.of()`

This method receives a `code` and returns a string based on the locale and options provided when instantiating `Intl.DisplayNames`.

`Intl.DisplayNames.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and formatting options computed during initialization of the object.

Examples

Region Code Display Names

To create an `Intl.DisplayNames` for a locale and get the display name for a region code.

```js
// Get display names of region in English
let regionNames = new Intl.DisplayNames(["en"], { type: "region" });
regionNames.of("419"); // "Latin America"
regionNames.of("BZ"); // "Belize"
regionNames.of("US"); // "United States"
regionNames.of("BA"); // "Bosnia & Herzegovina"
regionNames.of("MM"); // "Myanmar (Burma)"

// Get display names of region in Traditional Chinese
regionNames = new Intl.DisplayNames(["zh-Hant"], { type: "region" });
regionNames.of("419"); // "拉丁美洲"
regionNames.of("BZ"); // "貝里斯"
regionNames.of("US"); // "美國"
regionNames.of("BA"); // "波士尼亞與赫塞哥維納"
regionNames.of("MM"); // "緬甸"
```

Language Display Names

To create an `Intl.DisplayNames` for a locale and get the display name for a language-script-region sequence.

```js
// Get display names of language in English
let languageNames = new Intl.DisplayNames(["en"], { type: "language" });
languageNames.of("fr"); // "French"
languageNames.of("de"); // "German"
languageNames.of("fr-CA"); // "Canadian French"
languageNames.of("zh-Hant"); // "Traditional Chinese"
languageNames.of("en-US"); // "American English"
languageNames.of("zh-TW"); // "Chinese (Taiwan)"]

// Get display names of language in Traditional Chinese
languageNames = new Intl.DisplayNames(["zh-Hant"], { type: "language" });
languageNames.of("fr"); // "法文"
languageNames.of("zh"); // "中文"
languageNames.of("de"); // "德文"
```

Script Code Display Names

To create an `Intl.DisplayNames` for a locale and get the display name for a script code.

```js
// Get display names of script in English
let scriptNames = new Intl.DisplayNames(["en"], { type: "script" });
// Get script names
scriptNames.of("Latn"); // "Latin"
scriptNames.of("Arab"); // "Arabic"
scriptNames.of("Kana"); // "Katakana"

// Get display names of script in Traditional Chinese
scriptNames = new Intl.DisplayNames(["zh-Hant"], { type: "script" });
scriptNames.of("Latn"); // "拉丁文"
scriptNames.of("Arab"); // "阿拉伯文"
scriptNames.of("Kana"); // "片假名"
```

Currency Code Display Names

To create an `Intl.DisplayNames` for a locale and get the display name for currency code.

```js
// Get display names of currency code in English
let currencyNames = new Intl.DisplayNames(["en"], { type: "currency" });
// Get currency names
currencyNames.of("USD"); // "US Dollar"
currencyNames.of("EUR"); // "Euro"
currencyNames.of("TWD"); // "New Taiwan Dollar"
currencyNames.of("CNY"); // "Chinese Yuan"

// Get display names of currency code in Traditional Chinese
currencyNames = new Intl.DisplayNames(["zh-Hant"], { type: "currency" });
currencyNames.of("USD"); // "美元"
currencyNames.of("EUR"); // "歐元"
currencyNames.of("TWD"); // "新台幣"
currencyNames.of("CNY"); // "人民幣"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#intl-displaynames-objects`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`DisplayNames`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0
`DisplayNames()` constructor	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0
`of`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0
`resolvedOptions`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0
`supportedLocalesOf`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DisplayNames`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.DisplayNames` in FormatJS
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DisplayNames

Intl.DisplayNames() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DisplayNames/DisplayNames · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since April 2021.

The **`Intl.DisplayNames()`** constructor creates `Intl.DisplayNames` objects.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.DisplayNames() constructor". It runs live in the browser there; its source code follows.

js · interactive example source

```
const regionNamesInEnglish = new Intl.DisplayNames(["en"], { type: "region" });
const regionNamesInTraditionalChinese = new Intl.DisplayNames(["zh-Hant"], {
  type: "region",
});

console.log(regionNamesInEnglish.of("US"));
// Expected output: "United States"

console.log(regionNamesInTraditionalChinese.of("US"));
// Expected output: "美國"
```

Syntax

js

```
new Intl.DisplayNames(locales, options)
```

> **Note:** `Intl.DisplayNames()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales`

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options`

An object containing the following properties, in the order they are retrieved:

`localeMatcher` Optional

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`style` Optional

The formatting style to use. Possible values are `"narrow"`, `"short"`, and `"long"`; the default is `"long"`.

`type`

The type of display names to return from `of()`. Possible values are `"language"`, `"region"`, `"script"`, `"currency"`, `"calendar"`, and `"dateTimeField"`.

`fallback` Optional

What to return from `of()` if the input is structurally valid but there's no matching display name. Possible values are:

`"code"` (default)

Return the input code itself.

`"none"`

Return `undefined`.

`languageDisplay` Optional

How language names should be displayed. Only usable along with `type: "language"`. Possible values are:

`"dialect"` (default)

Display special regional dialects using their own name. E.g. `"nl-BE"` will be displayed as `"Flemish"`.

`"standard"`

Display all languages using standard format. E.g. `"nl-BE"` will be displayed as `"Dutch (Belgium)"`.

Exceptions

`TypeError`

Thrown if `options.type` is not provided.

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Basic usage

In basic use without specifying a locale, a formatted string in the default locale and with default options is returned.

```js
console.log(new Intl.DisplayNames([], { type: "language" }).of("US"));
// 'us'
```

Using type `dateTimeField`

Example using `dateTimeField` as a type option, will return the localized date time names strings.

```js
const dn = new Intl.DisplayNames("pt", { type: "dateTimeField" });
console.log(dn.of("era")); // 'era'
console.log(dn.of("year")); // 'ano'
console.log(dn.of("month")); // 'mês'
console.log(dn.of("quarter")); // 'trimestre'
console.log(dn.of("weekOfYear")); // 'semana'
console.log(dn.of("weekday")); // 'dia da semana'
console.log(dn.of("dayPeriod")); // 'AM/PM'
console.log(dn.of("day")); // 'dia'
console.log(dn.of("hour")); // 'hora'
console.log(dn.of("minute")); // 'minuto'
console.log(dn.of("second")); // 'segundo'
```

Using type `calendar`

Example using `calendar` as a type option, will return the localized calendar names strings.

```js
const dn = new Intl.DisplayNames("en", { type: "calendar" });
console.log(dn.of("roc")); // 'Minguo Calendar'
console.log(dn.of("gregory")); // 'Gregorian Calendar'
console.log(dn.of("chinese")); // 'Chinese Calendar'
```

Using type `language` with `languageDisplay`

Example using `language` as a type with `languageDisplay` options.

```js
// Using `dialect` option
const dnDialect = new Intl.DisplayNames("en", {
  type: "language",
  languageDisplay: "dialect",
});
console.log(dnDialect.of("en-GB")); // 'British English'

// Using `standard` option
const dnStd = new Intl.DisplayNames("en", {
  type: "language",
  languageDisplay: "standard",
});
console.log(dnStd.of("en-GB")); // 'English (United Kingdom)'
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl-displaynames-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`DisplayNames()` constructor	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DisplayNames.DisplayNames`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DisplayNames`
- `Intl.supportedValuesOf()`
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DisplayNames

Intl.DisplayNames.supportedLocalesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DisplayNames/supportedLocalesOf · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since April 2021.

The **`Intl.DisplayNames.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in display names without having to fall back to the runtime's default locale.

Syntax

```
Intl.DisplayNames.supportedLocalesOf(locales)
Intl.DisplayNames.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in display names without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in display names, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to display names nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```js
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.DisplayNames.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.DisplayNames.supportedLocalesOf`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedLocalesOf`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DisplayNames.supportedLocalesOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DisplayNames`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DisplayNames

Intl.DisplayNames.prototype.of()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DisplayNames/of · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since April 2021.

The **`of()`** method of `Intl.DisplayNames` instances receives a code and returns a string based on the locale and options provided when instantiating this `Intl.DisplayNames` object.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.DisplayNames.prototype.of()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const regionNamesInEnglish = new Intl.DisplayNames(["en"], { type: "region" });
const regionNamesInTraditionalChinese = new Intl.DisplayNames(["zh-Hant"], {
  type: "region",
});

console.log(regionNamesInEnglish.of("US"));
// Expected output: "United States"

console.log(regionNamesInTraditionalChinese.of("US"));
// Expected output: "美國"
```

Syntax

js

```
of(code)
```

Parameters

`code`

The `code` to provide depends on the `type`:

- If the type is "region", `code` should be either a two-letter ISO 3166 region code, or a three-digit UN M49 geographic region. It is required to follow the `unicode_region_subtag` grammar. Use uppercase codes (e.g., `"US"`), because lowercase ones (e.g., `"us"`) may not work reliably everywhere.
- If the type is "script", `code` should be a four-letter ISO 15924 script code. It is required to follow the `unicode_script_subtag` grammar.
- If the type is "language", `code` should be matched by the `unicode_language_id` nonterminal.
- If the type is "currency", `code` should be a three-letter ISO 4217 currency code. It is required to have exactly three alphabetic characters.
- If the type is "dateTimeField", `code` should be one of: `"era"`, `"year"`, `"quarter"`, `"month"`, `"weekOfYear"`, `"weekday"`, `"day"`, `"dayPeriod"`, `"hour"`, `"minute"`, `"second"`, `"timeZoneName"`.
- If the type is "calendar", `code` should be a calendar key. It is required to follow the `type` grammar of a Unicode locale identifier.

Return value

A language-specific formatted string, or `undefined` if there's no data for the input and `fallback` is `"none"`.

> **Note:** `fallback` is only used if `code` is structurally valid. See using fallback.

Exceptions

`RangeError`

Thrown if `code` is not structurally valid for the given `type`.

Examples

Using the of method

```js
const regionNames = new Intl.DisplayNames("en", { type: "region" });
regionNames.of("419"); // "Latin America"

const languageNames = new Intl.DisplayNames("en", { type: "language" });
languageNames.of("fr"); // "French"

const currencyNames = new Intl.DisplayNames("en", { type: "currency" });
currencyNames.of("EUR"); // "Euro"

const languageNamesStandard = new Intl.DisplayNames("fr", {
  type: "language",
  languageDisplay: "standard",
});
languageNamesStandard.of("fr-CA"); // "français (Canada)"

const languageNamesDialect = new Intl.DisplayNames("fr", {
  type: "language",
  languageDisplay: "dialect",
});
languageNamesDialect.of("fr-CA"); // "français canadien"
```

Using fallback

When the `Intl.DisplayNames` is constructed with `fallback: "code"`, the `of()` method will return the `code` if the input looks structurally valid but there's no data for the input. If `fallback` is `"none"`, `undefined` is returned.

```js
console.log(
  new Intl.DisplayNames("en", { type: "region", fallback: "code" }).of("ZL"),
); // "ZL"

console.log(
  new Intl.DisplayNames("en", { type: "region", fallback: "none" }).of("ZL"),
); // undefined
```

However, this only applies if the `code` is structurally valid. For example, if `type` is `"region"` but `code` does not follow the `unicode_region_subtag` grammar (2 alphabetic characters or 3 numeric characters), a `RangeError` is directly thrown instead of using the fallback.

```js
console.log(
  new Intl.DisplayNames("en", { type: "region", fallback: "code" }).of("ZLC"),
); // throws RangeError: invalid value "ZLC" for option region
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-Intl.DisplayNames.prototype.of

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`of`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DisplayNames.of`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DisplayNames`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DisplayNames

Intl.DisplayNames.prototype.resolvedOptions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DisplayNames/resolvedOptions · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since April 2021.

The **`resolvedOptions()`** method of `Intl.DisplayNames` instances returns a new object with properties reflecting the options computed during initialization of this `DisplayNames` object.

Syntax

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `DisplayNames` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. No Unicode extension key will be included in the output.

`style`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"narrow"`, `"short"`, or `"long"`. The default is `"long"`.

`type`

The value provided for this property in the `options` argument. It is either `"language"`, `"region"`, `"script"`, `"currency"`, `"calendar"`, or `"dateTimeField"`. It is required so there is no default.

`fallback`

The value provided for this property in the `options` argument. It is either `"code"` or `"none"`. The default is `"code"`.

`languageDisplay`

The value provided for this property in the `options` argument. It is either `"dialect"` or `"standard"`. The default is `"dialect"`.

Examples

Using resolvedOptions

```js
const displayNames = new Intl.DisplayNames(["de-DE"], { type: "region" });

const usedOptions = displayNames.resolvedOptions();
console.log(usedOptions.locale); // "de-DE"
console.log(usedOptions.style); // "long"
console.log(usedOptions.type); // "region"
console.log(usedOptions.fallback); // "code"
```

```js
const displayNames = new Intl.DisplayNames("en", {
  type: "language",
  languageDisplay: "standard",
});

const usedOptions = displayNames.resolvedOptions();
console.log(usedOptions.type); // "language"
console.log(usedOptions.languageDisplay); // "standard"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.DisplayNames.prototype.resolvedOptions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resolvedOptions`	81	81	86	68	14.1	81	86	58	14.5	13.0	81	14.5	1.0.0	1.8	14.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DisplayNames.resolvedOptions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DisplayNames`

JavaScript › Reference › Standard built-in objects › Intl

Intl.DurationFormat

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DurationFormat · MDN page last modified 2025-10-10

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`Intl.DurationFormat`** object enables language-sensitive duration formatting.

Constructor

`Intl.DurationFormat()`

Creates a new `Intl.DurationFormat` object.

Static methods

`Intl.DurationFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.DurationFormat.prototype` and shared by all `Intl.DurationFormat` instances.

`Intl.DurationFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.DurationFormat` instances, the initial value is the `Intl.DurationFormat` constructor.

`Intl.DurationFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.DurationFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.DurationFormat.prototype.format()`

Getter function that formats a duration according to the locale and formatting options of this `DurationFormat` object.

`Intl.DurationFormat.prototype.formatToParts()`

Returns an `Array` of objects representing the formatted duration in parts.

`Intl.DurationFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and formatting options computed during initialization of the object.

Examples

Using Intl.DurationFormat

The examples below show how to use the `Intl.DurationFormat` object to format a duration object with various locales and styles.

```js
const duration = {
  hours: 1,
  minutes: 46,
  seconds: 40,
};

// With style set to "long" and locale "fr-FR"
new Intl.DurationFormat("fr-FR", { style: "long" }).format(duration);
// "1 heure, 46 minutes et 40 secondes"

// With style set to "short" and locale "en"
new Intl.DurationFormat("en", { style: "short" }).format(duration);
// "1 hr, 46 min and 40 sec"

// With style set to "narrow" and locale "pt"
new Intl.DurationFormat("pt", { style: "narrow" }).format(duration);
// "1 h 46 min 40 s"
```

Specifications

Specification
Intl.DurationFormat `https://tc39.es/proposal-intl-duration-format/#durationformat-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`DurationFormat`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`DurationFormat()` constructor	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`format`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`formatToParts`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`resolvedOptions`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0
`supportedLocalesOf`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DurationFormat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.DurationFormat` in FormatJS
- `Intl`
- `Temporal.Duration.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DurationFormat

Intl.DurationFormat() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DurationFormat/DurationFormat · MDN page last modified 2025-09-24

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`Intl.DurationFormat()`** constructor creates `Intl.DurationFormat` objects.

Syntax

```js
new Intl.DurationFormat()
new Intl.DurationFormat(locales)
new Intl.DurationFormat(locales, options)
```

> **Note:** `Intl.DurationFormat()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

The following Unicode extension key is allowed:

`nu`

See `numberingSystem`.

This key can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`numberingSystem`

The numbering system to use for number formatting, such as `"arab"`, `"hans"`, `"mathsans"`, and so on. For a list of supported numbering system types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `nu` Unicode extension key; if both are provided, this `options` property takes precedence.

`style`

The style of the formatted duration. This value is used as the default for all other unit options, and also corresponds to the `style` option of `Intl.ListFormat()` when concatenating the list of duration units. Possible values are:

`"long"`
: E.g., 1 hour and 50 minutes

`"short"` **(default)**
: E.g., 1 hr, 50 min

`"narrow"`
: E.g., 1h 50m

`"digital"`
: E.g., 1:50:00

`years`
: The style of the formatted years. Possible values are `"long"`, `"short"`, and `"narrow"`; the default is `options.style` if it's not `"digital"`, and `"short"` otherwise.

`yearsDisplay`
: Whether to always display years, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `years` is unspecified, and `"always"` otherwise.

`months`
: The style of the formatted months. Possible values are `"long"`, `"short"`, and `"narrow"`; the default is `options.style` if it's not `"digital"`, and `"short"` otherwise.

`monthsDisplay`
: Whether to always display months, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `months` is unspecified, and `"always"` otherwise.

`weeks`
: The style of the formatted weeks. Possible values are `"long"`, `"short"`, and `"narrow"`; the default is `options.style` if it's not `"digital"`, and `"short"` otherwise.

`weeksDisplay`
: Whether to always display weeks, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `weeks` is unspecified, and `"always"` otherwise.

`days`
: The style of the formatted days. Possible values are `"long"`, `"short"`, and `"narrow"`; the default is `options.style` if it's not `"digital"`, and `"short"` otherwise.

`daysDisplay`
: Whether to always display days, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `days` is unspecified, and `"always"` otherwise.

`hours`
: The style of the formatted hours. Possible values are `"long"`, `"short"`, `"narrow"`, `"numeric"`, and `"2-digit"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`hoursDisplay`
: Whether to always display hours, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `hours` is unspecified and `options.style` is not `"digital"`, and `"always"` otherwise.

`minutes`
: The style of the formatted minutes.

- If `hours` is `"numeric"` or `"2-digit"`, possible values are `"numeric"` and `"2-digit"`, and `"numeric"` is normalized to `"2-digit"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, `"numeric"`, and `"2-digit"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`minutesDisplay`

Whether to always display minutes, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `minutes` is unspecified and `options.style` is not `"digital"`, and `"always"` otherwise.

`seconds`

The style of the formatted seconds.

- If `minutes` is `"numeric"` or `"2-digit"`, possible values are `"numeric"` and `"2-digit"`, and `"numeric"` is normalized to `"2-digit"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, `"numeric"`, and `"2-digit"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`secondsDisplay`

Whether to always display seconds, or only if nonzero. Possible values are `"always"` and `"auto"`; the default is `"auto"` if `seconds` is unspecified and `options.style` is not `"digital"`, and `"always"` otherwise.

`milliseconds`

The style of the formatted milliseconds.

- If `seconds` is `"numeric"` or `"2-digit"`, the only possible value is `"numeric"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, and `"numeric"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`millisecondsDisplay`

Whether to always display milliseconds, or only if nonzero.

- If `seconds` is `"numeric"` or `"2-digit"`, the only possible value is `"auto"`; the default is only `"auto"` when `milliseconds` is unspecified.
- Otherwise, possible values are `"always"` and `"auto"`; the default is `"auto"` if `milliseconds` is unspecified, and `"always"` otherwise.

`microseconds`

The style of the formatted microseconds.

- If `milliseconds` is `"numeric"`, the only possible value is `"numeric"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, and `"numeric"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`microsecondsDisplay`

Whether to always display microseconds, or only if nonzero.

- If `milliseconds` is `"numeric"`, the only possible value is `"auto"`; the default is only `"auto"` when `microseconds` is unspecified.
- Otherwise, possible values are `"always"` and `"auto"`; the default is `"auto"` if `microseconds` is unspecified, and `"always"` otherwise.

`nanoseconds`

The style of the formatted nanoseconds.

- If `microseconds` is `"numeric"`, the only possible value is `"numeric"`; the default is `"numeric"`.
- Otherwise, possible values are `"long"`, `"short"`, `"narrow"`, and `"numeric"`; the default is `options.style` if it's not `"digital"`, and `"numeric"` otherwise.

`nanosecondsDisplay`

Whether to always display nanoseconds, or only if nonzero.

- If `microseconds` is `"numeric"`, the only possible value is `"auto"`; the default is only `"auto"` when `nanoseconds` is unspecified.
- Otherwise, possible values are `"always"` and `"auto"`; the default is `"auto"` if `nanoseconds` is unspecified, and `"always"` otherwise.

`fractionalDigits`

Number of how many fractional second digits to display in the output. Possible values are from `0` to `9`; the default is `undefined` (include as many fractional digits as necessary).

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Description

For each time segment, an `Intl.NumberFormat` object is constructed under the hood. It uses the following options (see `Intl.NumberFormat()` for details):

- `numberingSystem`: the value of `options.numberingSystem`

When `milliseconds`, `microseconds`, or `nanoseconds` uses the `"numeric"` style, the following options are also used:

- `minimumFractionDigits`: `0` when `options.fractionalDigits` is `undefined`, `options.fractionalDigits` otherwise
- `maximumFractionDigits`: `9` when `options.fractionalDigits` is `undefined`, `options.fractionalDigits` otherwise
- `roundingMode`: `"trunc"`

When the time segment uses the `"2-digit"` style, the following options are also used:

- `minimumIntegerDigits`: `2`

When the time segment uses the `"long"`, `"short"`, or `"narrow"` style, the following options are also used:

- `style`: `"unit"` when `"long"`, `"short"`, or `"narrow"` is specified, `undefined` otherwise
- `unit`: the currently formatted unit (`"years"`, `"days"`, `"nanoseconds"`, etc.)
- `unitDisplay`: the value of the time segment style (`"long"`, `"short"`, or `"narrow"`)

Examples

Using the Intl.DurationFormat() constructor

```js
const duration = {
  hours: 2,
  minutes: 20,
  seconds: 35,
};

console.log(new Intl.DurationFormat("pt", { style: "long" }).format(duration));
// "2 horas, 20 minutos e 35 segundos"
```

Specifications

Specification
Intl.DurationFormat https://tc39.es/proposal-intl-duration-format/#sec-intl-durationformat-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`DurationFormat()` constructor	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DurationFormat.DurationFormat`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DurationFormat`
- `Intl.supportedValuesOf()`
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DurationFormat

Intl.DurationFormat.supportedLocalesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DurationFormat/supportedLocalesOf · MDN page last modified 2025-09-24

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`Intl.DurationFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in duration formatting without having to fall back to the runtime's default locale.

Syntax

```js
Intl.DurationFormat.supportedLocalesOf(locales)
Intl.DurationFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in duration formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in duration formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to duration formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```js
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.DurationFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

Specifications

Specification
Intl.DurationFormat https://tc39.es/proposal-intl-duration-format/#sec-Intl.DurationFormat.supportedLocalesOf

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedLocalesOf`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DurationFormat.supportedLocalesOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DurationFormat`

Intl.DurationFormat.prototype.format()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DurationFormat/format · MDN page last modified 2025-10-10

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`format()`** method of `Intl.DurationFormat` instances formats a duration according to the locale and formatting options of this `Intl.DurationFormat` object.

Syntax

```js
format(duration)
```

Parameters

`duration`

The duration object to be formatted. It should include some or all of the following properties: `years`, `months`, `weeks`, `days`, `hours`, `minutes`, `seconds`, `milliseconds`, `microseconds`, `nanoseconds`. Each property's value should be an integer, and their signs should be consistent. This can be a `Temporal.Duration` object; see the `Temporal.Duration` documentation for more information about these properties.

Return value

A string representing the given `duration` formatted according to the locale and formatting options of this `Intl.DurationFormat` object.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Examples

Using format()

The following example shows how to create a Duration formatter using the English language.

```js
const duration = {
  years: 1,
  months: 2,
  weeks: 3,
  days: 3,
  hours: 4,
  minutes: 5,
  seconds: 6,
  milliseconds: 7,
  microseconds: 8,
  nanoseconds: 9,
};

// Without options, style defaults to "short"
new Intl.DurationFormat("en").format(duration);
// "1 yr, 2 mths, 3 wks, 3 days, 4 hr, 5 min, 6 sec, 7 ms, 8 µs, 9 ns"

// With style set to "long"
new Intl.DurationFormat("en", { style: "long" }).format(duration);
// "1 year, 2 months, 3 weeks, 3 days, 4 hours, 5 minutes, 6 seconds, 7 milliseconds, 8
microseconds, 9 nanoseconds"

// With style set to "narrow"
new Intl.DurationFormat("en", { style: "narrow" }).format(duration);
// "1y 2mo 3w 3d 4h 5m 6s 7ms 8µs 9ns"
```

Using format() with different locales and styles

```js
const duration = {
  hours: 1,
  minutes: 46,
  seconds: 40,
};

// With style set to "long" and locale "fr-FR"
new Intl.DurationFormat("fr-FR", { style: "long" }).format(duration);
// "1 heure, 46 minutes et 40 secondes"

// With style set to "short" and locale set to "en"
new Intl.DurationFormat("en", { style: "short" }).format(duration);
// "1 hr, 46 min and 40 sec"

// With style set to "short" and locale set to "pt"
new Intl.DurationFormat("pt", { style: "narrow" }).format(duration);
// "1 h 46 min 40 s"

// With style set to "digital" and locale set to "en"
new Intl.DurationFormat("en", { style: "digital" }).format(duration);
// "1:46:40"
```

(code continues on the next page)

(code continued from the previous page)

```
// With style set to "digital", locale set to "en", and hours set to "long"
new Intl.DurationFormat("en", { style: "digital", hours: "long" }).format(
  duration,
);
// "1 hour, 46:40"
```

Using format() with the fractionalDigits option

```
const duration = {
  hours: 11,
  minutes: 30,
  seconds: 12,
  milliseconds: 345,
  microseconds: 600,
};

new Intl.DurationFormat("en", { style: "digital" }).format(duration);
// "11:30:12.3456"

new Intl.DurationFormat("en", { style: "digital", fractionalDigits: 5 }).format(
  duration,
);
// "11:30:12.34560"

new Intl.DurationFormat("en", { style: "digital", fractionalDigits: 3 }).format(
  duration,
);
// "11:30:12.346"
```

Specifications

Specification
Intl.DurationFormat https://tc39.es/proposal-intl-duration-format/#sec-Intl.DurationFormat.prototype.format

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
format	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DurationFormat.format`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DurationFormat`
- `Temporal.Duration`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DurationFormat

Intl.DurationFormat.prototype.formatToParts()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DurationFormat/formatToParts · MDN page last modified 2025-07-10

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`formatToParts()`** method of `Intl.DurationFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Syntax

```
formatToParts(duration)
```

Parameters

`duration` Optional

The duration object to be formatted. It should include some or all of the following properties: `years`, `months`, `weeks`, `days`, `hours`, `minutes`, `seconds`, `milliseconds`, `microseconds`, `nanoseconds`. Each property's value should be an integer, and their signs should be consistent. This can be a `Temporal.Duration` object; see the `Temporal.Duration` documentation for more information about these properties.

Return value

An `Array` of objects containing the formatted duration in parts. Each object has two or three properties, `type`, `value`, and optionally `unit`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The parts can be thought of as directly obtained from calling `Intl.NumberFormat.prototype.formatToParts()` with the numeric value and their respective units. All tokens that are produced by the `NumberFormat` have an additional `unit` property, which is the singular form of the input `unit`; this is for programmatic usage and is not localized. The localized unit is output as a separate `unit` token as part of the `NumberFormat` result. The parts from each duration unit are concatenated together in the same fashion as calling `Intl.ListFormat.prototype.formatToParts()` with `{ type: "unit" }`, so additional literal tokens are inserted.

Examples

The `formatToParts` method enables locale-aware formatting of strings produced by `DurationFormat` formatters by providing you the string in parts:

```
const duration = {
  hours: 7,
  minutes: 8,
  seconds: 9,
  milliseconds: 123,
  microseconds: 456,
  nanoseconds: 789,
};
```

(code continues on the next page)

(code continued from the previous page)

```
new Intl.DurationFormat("en", { style: "long" }).formatToParts(duration);

// Returned value:
[
  { type: "integer", value: "7", unit: "hour" },
  { type: "literal", value: " ", unit: "hour" },
  { type: "unit", value: "hours", unit: "hour" },
  { type: "literal", value: ", " },
  { type: "integer", value: "8", unit: "minute" },
  { type: "literal", value: " ", unit: "minute" },
  { type: "unit", value: "minutes", unit: "minute" },
  { type: "literal", value: ", " },
  { type: "integer", value: "9", unit: "second" },
  { type: "literal", value: " ", unit: "second" },
  { type: "unit", value: "seconds", unit: "second" },
  { type: "literal", value: ", " },
  { type: "integer", value: "123", unit: "millisecond" },
  { type: "literal", value: " ", unit: "millisecond" },
  { type: "unit", value: "milliseconds", unit: "millisecond" },
  { type: "literal", value: ", " },
  { type: "integer", value: "456", unit: "microsecond" },
  { type: "literal", value: " ", unit: "microsecond" },
  { type: "unit", value: "microseconds", unit: "microsecond" },
  { type: "literal", value: ", " },
  { type: "integer", value: "789", unit: "nanosecond" },
  { type: "literal", value: " ", unit: "nanosecond" },
  { type: "unit", value: "nanoseconds", unit: "nanosecond" },
];
```

Specifications

Specification
Intl.DurationFormat `https://tc39.es/proposal-intl-duration-format/#sec-Intl.DurationFormat.prototype.formatToParts`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`formatToParts`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DurationFormat.formatToParts`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DurationFormat`
- `Intl.DurationFormat.prototype.format()`
- `Temporal.Duration`

JavaScript › Reference › Standard built-in objects › Intl › Intl.DurationFormat

Intl.DurationFormat.prototype.resolvedOptions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/DurationFormat/resolvedOptions · MDN page last modified 2025-09-24

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`resolvedOptions()`** method of `Intl.DurationFormat` instances returns a new object with properties reflecting the options computed during initialization of this `DurationFormat` object.

Syntax

```js
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `DurationFormat` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `nu` Unicode extension key, if requested, may be included in the output.

`numberingSystem`

The value provided for this property in the `options` argument, or using the Unicode extension key `"nu"`, with default filled in as needed. It is a supported numbering system for this locale. The default is locale dependent.

`style`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"long"`, `"short"`, `"narrow"`, or `"digital"`. The default is `"short"`.

`years`, `yearsDisplay`, `months`, `monthsDisplay`, `weeks`, `weeksDisplay`, `days`, `daysDisplay`, `hours`, `hoursDisplay`, `minutes`, `minutesDisplay`, `seconds`, `secondsDisplay`, `milliseconds`, `millisecondsDisplay`, `nanoseconds`, `nanosecondsDisplay`

The values provided for these properties in the `options` argument, with defaults filled in as needed. For the valid values and defaults for each, see the `options` argument of the constructor.

`fractionalDigits` Optional

The value provided for this property in the `options` argument. It is only present if specified in `options`. It is an integer from 0 to 9, inclusive.

Examples

Using the resolvedOptions method

```js
const duration = new Intl.DurationFormat("en");
const usedOptions = duration.resolvedOptions();

usedOptions.locale; // "en"
usedOptions.numberingSystem; // "latn"
usedOptions.years; // "long"
usedOptions.yearsDisplay; // "auto"
usedOptions.style; // "long"
```

Specifications

Specification
Intl.DurationFormat `https://tc39.es/proposal-intl-duration-format/#sec-Intl.DurationFormat.prototype.resolvedOptions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resolvedOptions`	129	129	136	115	16.4	129	136	86	16.4	28.0	129	16.4	1.0.3	1.46	23.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.DurationFormat.resolvedOptions`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.DurationFormat`
- `Intl.supportedValuesOf()`
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl

Intl.ListFormat

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/ListFormat · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

The **`Intl.ListFormat`** object enables language-sensitive list formatting.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.ListFormat". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatter = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});
console.log(formatter.format(vehicles));
// Expected output: "Motorcycle, Bus, and Car"

const formatter2 = new Intl.ListFormat("de", {
  style: "short",
  type: "disjunction",
});
console.log(formatter2.format(vehicles));
// Expected output: "Motorcycle, Bus oder Car"

const formatter3 = new Intl.ListFormat("en", { style: "narrow", type: "unit" });
console.log(formatter3.format(vehicles));
// Expected output: "Motorcycle Bus Car"
```

Constructor

`Intl.ListFormat()`

Creates a new `Intl.ListFormat` object.

Static methods

`Intl.ListFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.ListFormat.prototype` and shared by all `Intl.ListFormat` instances.

`Intl.ListFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.ListFormat` instances, the initial value is the `Intl.ListFormat` constructor.

`Intl.ListFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.ListFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.ListFormat.prototype.format()`

Returns a language-specific formatted string representing the elements of the list.

`Intl.ListFormat.prototype.formatToParts()`

Returns an array of objects representing the different components that can be used to format a list of values in a locale-aware fashion.

`Intl.ListFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and style formatting options computed during the construction of the current `Intl.ListFormat` object.

Examples

Using format

The following example shows how to create a List formatter using the English language.

```js
const list = ["Motorcycle", "Bus", "Car"];

console.log(
  new Intl.ListFormat("en-GB", { style: "long", type: "conjunction" }).format(
    list,
  ),
);
// Motorcycle, Bus and Car

console.log(
  new Intl.ListFormat("en-GB", { style: "short", type: "disjunction" }).format(
    list,
  ),
);
// Motorcycle, Bus or Car

console.log(
  new Intl.ListFormat("en-GB", { style: "narrow", type: "unit" }).format(list),
);
// Motorcycle Bus Car
```

Using formatToParts

The following example shows how to create a List formatter returning formatted parts

```js
const list = ["Motorcycle", "Bus", "Car"];
console.log(
  new Intl.ListFormat("en-GB", {
    style: "long",
    type: "conjunction",
  }).formatToParts(list),
);

// [ { "type": "element", "value": "Motorcycle" },
//   { "type": "literal", "value": ", " },
//   { "type": "element", "value": "Bus" },
//   { "type": "literal", "value": ", and " },
//   { "type": "element", "value": "Car" } ];
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#listformat-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`ListFormat`	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[2]
`ListFormat()` constructor	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	13.0.0[3,4]
`format`	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[2]
`formatToParts`	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[2]
`resolvedOptions`	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[2]
`supportedLocalesOf`	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	13.0.0[3,5]

1. Only available on macOS Big Sur (11) and above.
2. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `ListFormat()` constructor for more details.
3. Also from version 12.0.0–13.0.0: partial support.
4. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `ListFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
5. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.ListFormat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.ListFormat` in FormatJS
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl › Intl.ListFormat

Intl.ListFormat() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/ListFormat/ListFormat · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since April 2021.

The **`Intl.ListFormat()`** constructor creates `Intl.ListFormat` objects.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.ListFormat() constructor". It runs live in the browser there; its source code follows.

js · interactive example source

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatter = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});
console.log(formatter.format(vehicles));
// Expected output: "Motorcycle, Bus, and Car"

const formatter2 = new Intl.ListFormat("de", {
  style: "short",
  type: "disjunction",
});
console.log(formatter2.format(vehicles));
// Expected output: "Motorcycle, Bus oder Car"

const formatter3 = new Intl.ListFormat("en", { style: "narrow", type: "unit" });
console.log(formatter3.format(vehicles));
// Expected output: "Motorcycle Bus Car"
```

Syntax

js

```
new Intl.ListFormat()
new Intl.ListFormat(locales)
new Intl.ListFormat(locales, options)
```

> **Note:** `Intl.ListFormat()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the

specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`type`

Indicates the type of grouping. Possible values are:

`"conjunction"` (default)

For "and"-based grouping of the list items: "A, B, and C"

`"disjunction"`

For "or"-based grouping of the list items: "A, B, or C"

`"unit"`

For grouping the list items as a compound unit (neither "and"-based nor "or"-based): "A, B, C"

`style`

The grouping style (for example, whether list separators and conjunctions are included). Possible values are:

`"long"` (default)

The typical list format. For example, "A, B, and C"

`"short"`

The spacing, the length or presence of a conjunction, and the separators may change. Usually, you would want the input elements to be abbreviated too. For example, "A, B, & C"

`"narrow"`

Where possible, the list format is further abbreviated, so that the output is as short as possible. For example, "A, B, C"

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Using format

The following example shows how to create a List formatter using the English language.

```js
const list = ["Motorcycle", "Bus", "Car"];

console.log(new Intl.ListFormat("en-GB", { type: "conjunction" }).format(list));
// Motorcycle, Bus and Car

console.log(new Intl.ListFormat("en-GB", { type: "disjunction" }).format(list));
// Motorcycle, Bus or Car
```

Oxford comma

Oxford comma is a comma placed immediately before the coordinating conjunction (usually "and" or "or") in a list of three or more terms. Somewhat controversially, the `en-US` locale uses the Oxford comma, while the `en-GB` locale does not.

```js
const list = ["Motorcycle", "Bus", "Car"];

console.log(new Intl.ListFormat("en-GB", { type: "conjunction" }).format(list));
// Motorcycle, Bus and Car

console.log(new Intl.ListFormat("en-US", { type: "conjunction" }).format(list));
// Motorcycle, Bus, and Car
```

Unit formatting

Use `style: "unit"` to format the list items as a compound unit. In fact, `Intl.DurationFormat` uses unit-style list formatting under the hood to format durations.

```js
const marathon = [
  [42, "kilometer"],
  [195, "meter"],
];

console.log(
  new Intl.ListFormat("en-US", { type: "unit" }).format(
    marathon.map((component) =>
      component[0].toLocaleString("en-US", {
        style: "unit",
        unit: component[1],
        unitDisplay: "long",
      }),
    ),
  ),
);
// 42 kilometers, 195 meters
```

Short and narrow style

The `"short"` and `"narrow"` styles are useful for compact representations of lists.

```js
const list = ["Motorcycle", "Bus", "Car"];
console.log(new Intl.ListFormat("en-US", { style: "short" }).format(list));
// Motorcycle, Bus, & Car

console.log(new Intl.ListFormat("en-US", { style: "narrow" }).format(list));
// Motorcycle, Bus, Car

console.log(new Intl.ListFormat("en-GB", { style: "short" }).format(list));
// Motorcycle, Bus and Car

console.log(new Intl.ListFormat("en-GB", { style: "narrow" }).format(list));
// Motorcycle, Bus, Car
```

The input elements are not transformed, but you will often want to abbreviate them too.

```js
const marathon = [
  [42, "kilometer"],
  [195, "meter"],
];

function formatDistance(locale, distance, style) {
  return new Intl.ListFormat(locale, { type: "unit", style }).format(
    marathon.map((component) =>
      component[0].toLocaleString(locale, {
        style: "unit",
        unit: component[1],
        unitDisplay: style,
      }),
    ),
  );
}

console.log(formatDistance("en-US", marathon, "long"));
// 42 kilometers, 195 meters
console.log(formatDistance("en-US", marathon, "short"));
// 42 km, 195 m
console.log(formatDistance("en-US", marathon, "narrow"));
// 42km 195m
```

Selection of conjunction

The conjunction word used may depend on the list items' string values. For example, in Spanish, the conjunction is `"y"` for most words, but `"e"` for words starting with the vowel `"i"`.

```js
const words = ["fuerte", "indomable"];
const formatter = new Intl.ListFormat("es-ES", { type: "conjunction" });

console.log(formatter.format(words));
// fuerte e indomable
console.log(formatter.format(words.toReversed()));
// indomable y fuerte
```

The algorithm used to determine the conjunction is not perfect (for example, it can't always tell a word's pronunciation from its spelling), but it should work in the general case.

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl-listformat-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`ListFormat()` constructor	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	13.0.0[2,3]

1. Only available on macOS Big Sur (11) and above.
2. Also from version 12.0.0–13.0.0: partial support.
3. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `ListFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.ListFormat.ListFormat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.ListFormat`
- `Intl`

Intl.ListFormat.supportedLocalesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/ListFormat/supportedLocalesOf · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since April 2021.

The **`Intl.ListFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in list formatting without having to fall back to the runtime's default locale.

Syntax

```js
Intl.ListFormat.supportedLocalesOf(locales)
Intl.ListFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in list formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in list formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to list formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```js
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.ListFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.ListFormat.supportedLocalesOf`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedLocalesOf`	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	13.0.0[2,3]

1. Only available on macOS Big Sur (11) and above.
2. Also from version 12.0.0–13.0.0: partial support.
3. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.ListFormat.supportedLocalesOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.ListFormat`

JavaScript › Reference › Standard built-in objects › Intl › Intl.ListFormat

Intl.ListFormat.prototype.format()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/ListFormat/format · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

The **`format()`** method of `Intl.ListFormat` instances returns a string with a language-specific representation of the list.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.ListFormat.prototype.format()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatter = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});
console.log(formatter.format(vehicles));
// Expected output: "Motorcycle, Bus, and Car"

const formatter2 = new Intl.ListFormat("de", {
  style: "short",
  type: "disjunction",
});
console.log(formatter2.format(vehicles));
// Expected output: "Motorcycle, Bus oder Car"

const formatter3 = new Intl.ListFormat("en", { style: "narrow", type: "unit" });
console.log(formatter3.format(vehicles));
// Expected output: "Motorcycle Bus Car"
```

Syntax

js

```
format(list)
```

Parameters

`list`

An iterable object, such as an Array, containing strings. Omitting it results in formatting the empty array, which could be slightly confusing, so it is advisable to always explicitly pass a list.

Return value

A language-specific formatted string representing the elements of the list.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Examples

Using format

The following example shows how to create a List formatter using the English language.

```js
const list = ["Motorcycle", "Bus", "Car"];

console.log(
  new Intl.ListFormat("en-GB", { style: "long", type: "conjunction" }).format(
    list,
  ),
);
// Motorcycle, Bus and Car

console.log(
  new Intl.ListFormat("en-GB", { style: "short", type: "disjunction" }).format(
    list,
  ),
);
// Motorcycle, Bus or Car

console.log(
  new Intl.ListFormat("en-GB", { style: "narrow", type: "unit" }).format(list),
);
// Motorcycle Bus Car
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.ListFormat.prototype.format`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`format`	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[2]

1. Only available on macOS Big Sur (11) and above.
2. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `ListFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.ListFormat.format`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.ListFormat`

JavaScript › Reference › Standard built-in objects › Intl › Intl.ListFormat

Intl.ListFormat.prototype.formatToParts()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/ListFormat/formatToParts · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2021.

The **`formatToParts()`** method of `Intl.ListFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.listformat.prototype.formatToParts()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const vehicles = ["Motorcycle", "Bus", "Car"];

const formatterEn = new Intl.ListFormat("en", {
  style: "long",
  type: "conjunction",
});

const formatterFr = new Intl.ListFormat("fr", {
  style: "long",
  type: "conjunction",
});

const partValuesEn = formatterEn.formatToParts(vehicles).map((p) => p.value);
const partValuesFr = formatterFr.formatToParts(vehicles).map((p) => p.value);

console.log(partValuesEn);
// Expected output: "["Motorcycle", ", ", "Bus", ", and ", "Car"]"
console.log(partValuesFr);
// Expected output: "["Motorcycle", ", ", "Bus", " et ", "Car"]"
```

Syntax

js

```
formatToParts(list)
```

Parameters

`list`

An iterable object, such as an Array, containing strings. Omitting it results in formatting the empty array, which could be slightly confusing, so it is advisable to always explicitly pass a list.

Return value

An `Array` of objects containing the formatted list in parts. Each object has two properties, `type` and `value`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The `type` may be one of the following:

`literal`

Any string that's a part of the format pattern; for example `", "`, `", and"`, etc.

`element`

An element of the list, exactly as provided.

Examples

Using formatToParts()

```js
const fruits = ["Apple", "Orange", "Pineapple"];
const myListFormat = new Intl.ListFormat("en-GB", {
  style: "long",
  type: "conjunction",
});

console.table(myListFormat.formatToParts(fruits));
// [
//  { "type": "element", "value": "Apple" },
//  { "type": "literal", "value": ", " },
//  { "type": "element", "value": "Orange" },
//  { "type": "literal", "value": " and " },
//  { "type": "element", "value": "Pineapple" }
// ]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.ListFormat.prototype.formatToParts`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`formatToParts`	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[2]

1. Only available on macOS Big Sur (11) and above.
2. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `ListFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.ListFormat.formatToParts`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.ListFormat`
- `Intl.ListFormat.prototype.format()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.ListFormat

Intl.ListFormat.prototype.resolvedOptions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/ListFormat/resolvedOptions · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since April 2021.

The **`resolvedOptions()`** method of `Intl.ListFormat` instances returns a new object with properties reflecting the options computed during initialization of this `ListFormat` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.ListFormat.prototype.resolvedOptions()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const deListFormatter = new Intl.ListFormat("de-DE", { type: "disjunction" });
const options = deListFormatter.resolvedOptions();

console.log(options.locale);
// Expected output: "de-DE"

console.log(options.style);
// Expected output: "long"

console.log(options.type);
// Expected output: "disjunction"
```

Syntax

js

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `ListFormat` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. No Unicode extension key will be included in the output.

`type`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"conjunction"`, `"disjunction"`, or `"unit"`. The default is `"conjunction"`.

`style`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"long"`, `"short"`, or `"narrow"`. The default is `"long"`.

Examples

Using resolvedOptions

```js
const deListFormatter = new Intl.ListFormat("de-DE", { style: "short" });

const usedOptions = de.resolvedOptions();
console.log(usedOptions.locale); // "de-DE"
console.log(usedOptions.style); // "short"
console.log(usedOptions.type); // "conjunction" (the default value)
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.ListFormat.prototype.resolvedoptions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resolvedOptions`	72	79	78	60	14.1[1]	72	79	51	14.5	11.0	72	14.5	1.0.3	1.8	12.0.0[2]

1. Only available on macOS Big Sur (11) and above.
2. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `ListFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.ListFormat.resolvedOptions`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.ListFormat`
- `Intl.NumberFormat.prototype.resolvedOptions()`
- `Intl.Collator.prototype.resolvedOptions()`
- `Intl.DateTimeFormat.prototype.resolvedOptions()`
- `Intl.PluralRules.prototype.resolvedOptions()`

JavaScript › Reference › Standard built-in objects › Intl

Intl.Locale

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`Intl.Locale`** object is a standard built-in property of the Intl object that represents a Unicode locale identifier.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.Locale". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const korean = new Intl.Locale("ko", {
  script: "Kore",
  region: "KR",
  hourCycle: "h23",
  calendar: "gregory",
});

const japanese = new Intl.Locale("ja-Jpan-JP-u-ca-japanese-hc-h12");

console.log(korean.baseName, japanese.baseName);
// Expected output: "ko-Kore-KR" "ja-Jpan-JP"

console.log(korean.hourCycle, japanese.hourCycle);
// Expected output: "h23" "h12"
```

Description

The **`Intl.Locale`** object was created to allow for easier manipulation of Unicode locales. Unicode represents locales with a string, called a *locale identifier*. The locale identifier consists of a *language identifier* and *extension tags*. Language identifiers are the core of the locale, consisting of *language*, *script*, *region*, and *variants* subtags. Additional information about the locale is stored in the optional *extension tags*. Extension tags hold information about locale aspects such as calendar type, clock type, and numbering system type.

Traditionally, the Intl API used strings to represent locales, just as Unicode does. This is a simple and lightweight solution that works well. Adding a Locale class, however, adds ease of parsing and manipulating the language, script, and region, as well as extension tags. The following properties of `Intl.Locale` correspond to Unicode locale identifier subtags:

Property	Corresponding subtag
`language`	Language ID, first part
`script`	Language ID, part after `language`
`region`	Language ID, part after `script`
`variants`	Language ID, part after `region`
`calendar`	`ca` (extension)
`caseFirst`	`kf` (extension)
`collation`	`co` (extension)
`hourCycle`	`hc` (extension)

Property	Corresponding subtag
numberingSystem	nu (extension)
numeric	kn (extension)

The information above is exactly provided as-is when the `Locale` object is constructed, without consulting any external database. The `Intl.Locale` object additionally provides some methods that return information about the locale's real-world information, such as available calendars, collations, and numbering systems.

Constructor

`Intl.Locale()`
: Creates a new `Locale` object.

Instance properties

These properties are defined on `Intl.Locale.prototype` and shared by all `Intl.Locale` instances.

`Intl.Locale.prototype.baseName`
: Returns basic, core information about the `Locale` in the form of a substring of the complete data string.

`Intl.Locale.prototype.calendar`
: Returns the part of the `Locale` that indicates the Locale's calendar era.

`Intl.Locale.prototype.caseFirst`
: Returns whether case is taken into account for the locale's collation rules.

`Intl.Locale.prototype.collation`
: Returns the collation type for the `Locale`, which is used to order strings according to the locale's rules.

`Intl.Locale.prototype.constructor`
: The constructor function that created the instance object. For `Intl.Locale` instances, the initial value is the `Intl.Locale` constructor.

`Intl.Locale.prototype.hourCycle`
: Returns the time keeping format convention used by the locale.

`Intl.Locale.prototype.language`
: Returns the language associated with the locale.

`Intl.Locale.prototype.numberingSystem`
: Returns the numeral system used by the locale.

`Intl.Locale.prototype.numeric`
: Returns whether the locale has special collation handling for numeric characters.

`Intl.Locale.prototype.region`
: Returns the region of the world (usually a country) associated with the locale.

`Intl.Locale.prototype.script`
: Returns the script used for writing the particular language used in the locale.

`Intl.Locale.prototype.variants`
: Returns the variants subtags (such as different orthographies) associated with the locale.

`Intl.Locale.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.Locale"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.Locale.prototype.getCalendars()`

Returns an `Array` of available calendar identifiers, according to the locale's rules.

`Intl.Locale.prototype.getCollations()`

Returns an `Array` of the collation types for the `Locale`.

`Intl.Locale.prototype.getHourCycles()`

Returns an `Array` of hour cycle identifiers, indicating either the 12-hour clock ("h12"), the Japanese 12-hour clock ("h11"), the 24-hour clock ("h23"), or the unused format "h24".

`Intl.Locale.prototype.getNumberingSystems()`

Returns an `Array` of numbering system identifiers available according to the locale's rules.

`Intl.Locale.prototype.getTextInfo()`

Returns the part indicating the ordering of characters `ltr` (left-to-right) or `rtl` (right-to-left).

`Intl.Locale.prototype.getTimeZones()`

Returns an `Array` of time zone identifiers, associated with the `Locale`.

`Intl.Locale.prototype.getWeekInfo()`

Returns UTS 35's Week Elements according to the locale rules.

`Intl.Locale.prototype.maximize()`

Gets the most likely values for the language, script, and region of the locale based on existing values.

`Intl.Locale.prototype.minimize()`

Attempts to remove information about the locale that would be added by calling `maximize()`.

`Intl.Locale.prototype.toString()`

Returns the Locale's full locale identifier string.

Examples

Basic usage

At its very simplest, the `Intl.Locale()` constructor takes a locale identifier string as its argument:

```js
const us = new Intl.Locale("en-US");
```

Using the Locale constructor with an options object

The constructor also takes an optional configuration object argument, which can contain any of several extension types. For example, set the `hourCycle` property of the configuration object to your desired hour cycle type, and then pass it into the constructor:

```js
const us12hour = new Intl.Locale("en-US", { hourCycle: "h12" });
console.log(us12hour.hourCycle); // Prints "h12"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#locale-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Locale`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`Locale()` constructor	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`baseName`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`calendar`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`caseFirst`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`collation`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`getCalendars`	130[1,2]	130[1,2]	153	115[3,4]	17[5,6]	130[1,2]	153	86[7,8]	17[9,10]	28.0[11,12]	130[1,2]	17[9,10]	1.0.0	2.1[13,14]	24.0.0[15,16]
`getCollations`	130[17,2]	130[17,2]	153	115[18,4]	17[19,6]	130[17,2]	153	86[20,8]	17[21,10]	28.0[22,12]	130[17,2]	17[21,10]	1.0.0	2.1[23,14]	24.0.0[24,16]
`getHourCycles`	130[25,2]	130[25,2]	153	115[26,4]	17[27,6]	130[25,2]	153	86[28,8]	17[29,10]	28.0[30,12]	130[25,2]	17[29,10]	1.0.0	2.1[31,14]	24.0.0[32,16]
`getNumberingSystems`	130[33,2]	130[33,2]	153	115[34,4]	17[35,6]	130[33,2]	153	86[36,8]	17[37,10]	28.0[38,12]	130[33,2]	17[37,10]	1.0.0	2.1[39,14]	24.0.0[40,16]
`getTextInfo`	130[41,2]	130[41,2]	153	115[42,4]	17[43,6]	130[41,2]	153	86[44,8]	17[45,10]	28.0[46,12]	130[41,2]	17[45,10]	1.0.0	2.1[47,14]	24.0.0[48,16]
`getTimeZones`	130[49,2]	130[49,2]	153	115[50,4]	17[51,6]	130[49,2]	153	86[52,8]	17[53,10]	28.0[54,12]	130[49,2]	17[53,10]	1.0.0	2.1[55,14]	24.0.0[56,16]
`getWeekInfo()` method	130[57,2]	130[57,2]	153	115[58,4]	17[59,6]	130[57,2]	153	86[60,8]	17[61,10]	28.0[62,12]	130[57,2]	17[61,10]	1.0.0	2.1[63,14]	24.0.0[64,16]
`minimalDays` in returned object Deprecated Non-standard	**99–136**	**99–136**	**No**	**85–121**	15.4	**99–136**	**No**	**68–90**	15.4	**18.0–29.0**	**99–136**	15.4	1.0.0	1.19	**18.0.0–24.0.0**
`hourCycle`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`language`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`maximize`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`minimize`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`numberingSystem`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`numeric`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`region`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`script`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`toString`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0
`variants`	149	149	141	133	26	149	141	100	26	**No**	149	26	1.2.18	2.8	**No**

1. Also from version 99: uses the non-standard name `calendars`.
2. Also from version 99: Implemented as an accessor property.
3. Also from version 85: uses the non-standard name `calendars`.
4. Also from version 85: Implemented as an accessor property.
5. Also from version 15.4–Prev.: uses the non-standard name `calendars`.
6. Also from version 15.4–Prev.: Implemented as an accessor property.

7. Also from version 68: uses the non-standard name `calendars`.
8. Also from version 68: Implemented as an accessor property.
9. Also from version 15.4: uses the non-standard name `calendars`.
10. Also from version 15.4: Implemented as an accessor property.
11. Also from version 18.0: uses the non-standard name `calendars`.
12. Also from version 18.0: Implemented as an accessor property.
13. Also from version 1.19–2.6.7: uses the non-standard name `calendars`.
14. Also from version 1.19–2.6.7: Implemented as an accessor property.
15. Also from version 18.0.0: uses the non-standard name `calendars`.
16. Also from version 18.0.0: Implemented as an accessor property.
17. Also from version 99: uses the non-standard name `collations`.
18. Also from version 85: uses the non-standard name `collations`.
19. Also from version 15.4–Prev.: uses the non-standard name `collations`.
20. Also from version 68: uses the non-standard name `collations`.
21. Also from version 15.4: uses the non-standard name `collations`.
22. Also from version 18.0: uses the non-standard name `collations`.
23. Also from version 1.19–2.6.7: uses the non-standard name `collations`.
24. Also from version 18.0.0: uses the non-standard name `collations`.
25. Also from version 99: uses the non-standard name `hourCycles`.
26. Also from version 85: uses the non-standard name `hourCycles`.
27. Also from version 15.4–Prev.: uses the non-standard name `hourCycles`.
28. Also from version 68: uses the non-standard name `hourCycles`.
29. Also from version 15.4: uses the non-standard name `hourCycles`.
30. Also from version 18.0: uses the non-standard name `hourCycles`.
31. Also from version 1.19–2.6.7: uses the non-standard name `hourCycles`.
32. Also from version 18.0.0: uses the non-standard name `hourCycles`.
33. Also from version 99: uses the non-standard name `numberingSystems`.
34. Also from version 85: uses the non-standard name `numberingSystems`.
35. Also from version 15.4–Prev.: uses the non-standard name `numberingSystems`.
36. Also from version 68: uses the non-standard name `numberingSystems`.
37. Also from version 15.4: uses the non-standard name `numberingSystems`.
38. Also from version 18.0: uses the non-standard name `numberingSystems`.
39. Also from version 1.19–2.6.7: uses the non-standard name `numberingSystems`.
40. Also from version 18.0.0: uses the non-standard name `numberingSystems`.
41. Also from version 99: uses the non-standard name `textInfo`.
42. Also from version 85: uses the non-standard name `textInfo`.
43. Also from version 15.4–Prev.: uses the non-standard name `textInfo`.
44. Also from version 68: uses the non-standard name `textInfo`.
45. Also from version 15.4: uses the non-standard name `textInfo`.
46. Also from version 18.0: uses the non-standard name `textInfo`.
47. Also from version 1.19–2.6.7: uses the non-standard name `textInfo`.
48. Also from version 18.0.0: uses the non-standard name `textInfo`.
49. Also from version 99: uses the non-standard name `timeZones`.
50. Also from version 85: uses the non-standard name `timeZones`.
51. Also from version 15.4–Prev.: uses the non-standard name `timeZones`.
52. Also from version 68: uses the non-standard name `timeZones`.
53. Also from version 15.4: uses the non-standard name `timeZones`.
54. Also from version 18.0: uses the non-standard name `timeZones`.
55. Also from version 1.19–2.6.7: uses the non-standard name `timeZones`.
56. Also from version 18.0.0: uses the non-standard name `timeZones`.
57. Also from version 99: uses the non-standard name `weekInfo`.
58. Also from version 85: uses the non-standard name `weekInfo`.
59. Also from version 15.4–Prev.: uses the non-standard name `weekInfo`.
60. Also from version 68: uses the non-standard name `weekInfo`.
61. Also from version 15.4: uses the non-standard name `weekInfo`.
62. Also from version 18.0: uses the non-standard name `weekInfo`.
63. Also from version 1.19–2.6.7: uses the non-standard name `weekInfo`.
64. Also from version 18.0.0: uses the non-standard name `weekInfo`.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.Locale` in FormatJS
- `Intl`
- Canonical Unicode Locale Identifiers in the Unicode locale data markup language spec

---

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/Locale · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`Intl.Locale()`** constructor creates `Intl.Locale` objects.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.Locale() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const korean = new Intl.Locale("ko", {
  script: "Kore",
  region: "KR",
  hourCycle: "h23",
  calendar: "gregory",
});

const japanese = new Intl.Locale("ja-Jpan-JP-u-ca-japanese-hc-h12");

console.log(korean.baseName, japanese.baseName);
// Expected output: "ko-Kore-KR" "ja-Jpan-JP"

console.log(korean.hourCycle, japanese.hourCycle);
// Expected output: "h23" "h12"
```

Syntax

js

```
new Intl.Locale(tag)
new Intl.Locale(tag, options)
```

> **Note:** `Intl.Locale()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`tag`

The Unicode locale identifier string. For the syntax of locale identifier strings, see the Intl main page. Note that the `Intl.Locale` constructor, unlike most other `Intl` constructors, does not accept an array of locales or `undefined`.

`options`

An object that contains configuration for the Locale. Option values here take priority over extension keys in the locale identifier. Possible properties are:

`language`
The language. Any syntactically valid string following the `unicode_language_subtag` grammar (2–3 or 5–8 letters) is accepted, but the implementation only recognizes certain kinds.

`script`
The script. Any syntactically valid string following the `unicode_script_subtag` grammar (4 letters) is accepted, but the implementation only recognizes certain kinds.

`region`
The region. Any syntactically valid string following the `unicode_region_subtag` grammar (either 2 letters or 3 digits) is accepted, but the implementation only recognizes certain kinds.

`variants`
The variants. It should be a dash (-) separated list of *unique* variant tags, where each tag is any syntactically valid string following the `unicode_variant_subtag` grammar (either 5–8 alphanumerals or a digit followed by 3 alphanumerals), but the implementation only recognizes certain kinds.

`calendar`
The calendar. Any syntactically valid string following the `type` grammar (one or more segments of 3–8 alphanumerals, joined by hyphens) is accepted, but the implementation only recognizes certain kinds, which are listed in `Intl.supportedValuesOf()`.

`collation`
The collation. Any syntactically valid string following the `type` grammar is accepted, but the implementation only recognizes certain kinds, which are listed in `Intl.supportedValuesOf()`.

`numberingSystem`
The numbering system. Any syntactically valid string following the `type` grammar is accepted, but the implementation only recognizes certain kinds, which are listed in `Intl.supportedValuesOf()`.

`caseFirst`
The case-first sort option. Possible values are `"upper"`, `"lower"`, or `"false"`.

`hourCycle`
The hour cycle. Possible values are `"h23"`, `"h12"`, `"h11"`, or the practically unused `"h24"`, which are explained in `Intl.Locale.prototype.getHourCycles`

`numeric`
The numeric sort option. A boolean.

Examples

Basic usage

At its very simplest, the `Intl.Locale()` constructor takes a locale identifier string as its argument:

```js
const us = new Intl.Locale("en-US");
```

Using the Locale constructor with an options object

The constructor also takes an optional configuration object argument, which can contain any of several extension types. For example, set the `hourCycle` property of the configuration object to your desired

hour cycle type, and then pass it into the constructor:

```js
const locale = new Intl.Locale("en-US", { hourCycle: "h12" });
console.log(locale.hourCycle); // "h12"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl-locale-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Locale()` constructor	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.Locale`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.Collator`
- Canonical Unicode Locale Identifiers in the Unicode locale data markup language spec

---

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.getCalendars()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/getCalendars · MDN page last modified 2025-07-10

Baseline 2026: Newly available — works across the latest browser versions since July 2026; may not work in older ones.

The **`getCalendars()`** method of `Intl.Locale` instances returns a list of one or more unique calendar identifiers for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `calendars`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.calendars === locale.calendars` returning `false`. Check the browser compatibility table for details.

Syntax

```js
getCalendars()
```

Parameters

None.

Return value

An array of strings representing all calendars commonly used for the `Locale`, sorted in descending preference. If the `Locale` already has a `calendar`, then the returned array contains that single value.

For a list of supported calendar types, see `Intl.supportedValuesOf()`.

Examples

Obtaining supported calendars

If the `Locale` object doesn't have a `calendar` already, `getCalendars()` lists all commonly-used calendars for the given `Locale`. For examples of explicitly setting a `calendar`, see `calendar` examples.

```js
const arEG = new Intl.Locale("ar-EG");
console.log(arEG.getCalendars()); // ["gregory", "coptic", "islamic", "islamic-civil",
"islamic-tbla"]
```

```js
const jaJP = new Intl.Locale("ja-JP");
console.log(jaJP.getCalendars()); // ["gregory", "japanese"]
```

Specifications

Specification
Intl Locale Info Proposal `https://tc39.es/proposal-intl-locale-info/#sec-Intl.Locale.prototype.getCalendars`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getCalendars`	130[1,2]	130[1,2]	153	115[3,4]	17[5,6]	130[1,2]	153	86[7,8]	17[9,10]	28.0[11,12]	130[1,2]	17[9,10]	1.0.0	2.1[13,14]	24.0.0[15,16]

1. Also from version 99: uses the non-standard name `calendars`.
2. Also from version 99: Implemented as an accessor property.
3. Also from version 85: uses the non-standard name `calendars`.
4. Also from version 85: Implemented as an accessor property.
5. Also from version 15.4–Prev.: uses the non-standard name `calendars`.
6. Also from version 15.4–Prev.: Implemented as an accessor property.
7. Also from version 68: uses the non-standard name `calendars`.
8. Also from version 68: Implemented as an accessor property.
9. Also from version 15.4: uses the non-standard name `calendars`.
10. Also from version 15.4: Implemented as an accessor property.
11. Also from version 18.0: uses the non-standard name `calendars`.
12. Also from version 18.0: Implemented as an accessor property.
13. Also from version 1.19–2.6.7: uses the non-standard name `calendars`.

14. Also from version 1.19–2.6.7: Implemented as an accessor property.
15. Also from version 18.0.0: uses the non-standard name `calendars`.
16. Also from version 18.0.0: Implemented as an accessor property.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.getCalendars`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- `Intl.Locale.prototype.calendar`
- Unicode Calendar Identifier in the Unicode locale data markup language spec

---

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.getCollations()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/getCollations · MDN page last modified 2025-07-10

Baseline 2026: Newly available — works across the latest browser versions since July 2026; may not work in older ones.

The **`getCollations()`** method of `Intl.Locale` instances returns a list of one or more collation types for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `collations`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.collations === locale.collations` returning `false`. Check the browser compatibility table for details.

Syntax

```js
getCollations()
```

Parameters

None.

Return value

An array of strings representing all collation types commonly used for the `Locale`, sorted in alphabetical order, with the `standard` and `search` values always excluded. If the `Locale` already has a `collation`, then the returned array contains that single value.

For a list of supported collation types, see `Intl.supportedValuesOf()`.

Examples

Obtaining supported collation types

If the `Locale` object doesn't have a `collation` already, `getCollations()` lists all commonly-used collation types for the given `Locale`. For examples of explicitly setting a `collation`, see `collation` examples.

```js
const locale = new Intl.Locale("zh");
console.log(locale.getCollations()); // ["pinyin", "stroke", "zhuyin", "emoji", "eor"]
```

Specifications

Specification
Intl Locale Info Proposal `https://tc39.es/proposal-intl-locale-info/#sec-Intl.Locale.prototype.getCollations`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getCollations`	130[1,2]	130[1,2]	153	115[3,4]	17[5,6]	130[1,2]	153	86[7,8]	17[9,10]	28.0[11,12]	130[1,2]	17[9,10]	1.0.0	2.1[13,14]	24.0.0[15,16]

1. Also from version 99: uses the non-standard name `collations`.
2. Also from version 99: Implemented as an accessor property.
3. Also from version 85: uses the non-standard name `collations`.
4. Also from version 85: Implemented as an accessor property.
5. Also from version 15.4–Prev.: uses the non-standard name `collations`.
6. Also from version 15.4–Prev.: Implemented as an accessor property.
7. Also from version 68: uses the non-standard name `collations`.
8. Also from version 68: Implemented as an accessor property.
9. Also from version 15.4: uses the non-standard name `collations`.
10. Also from version 15.4: Implemented as an accessor property.
11. Also from version 18.0: uses the non-standard name `collations`.
12. Also from version 18.0: Implemented as an accessor property.
13. Also from version 1.19–2.6.7: uses the non-standard name `collations`.
14. Also from version 1.19–2.6.7: Implemented as an accessor property.
15. Also from version 18.0.0: uses the non-standard name `collations`.
16. Also from version 18.0.0: Implemented as an accessor property.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.getCollations`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.Locale`
- `Intl.Locale.prototype.collation`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.getHourCycles()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/getHourCycles · MDN page last modified 2025-07-10

Baseline 2026: Newly available — works across the latest browser versions since July 2026; may not work in older ones.

The **`getHourCycles()`** method of `Intl.Locale` instances returns a list of one or more unique hour cycle identifiers for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `hourCycles`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.hourCycles === locale.hourCycles` returning `false`. Check the browser compatibility table for details.

Syntax

```js
getHourCycles()
```

Parameters

None.

Return value

An array of strings representing all hour cycle types commonly used for the `Locale`, sorted in descending preference. If the `Locale` already has an `hourCycle`, then the returned array contains that single value.

Below is a list of supported hour cycle types.

Supported hour cycle types

`h12`

Hour system using 1–12; corresponds to 'h' in patterns. The 12 hour clock, with midnight starting at 12:00 am. As used, for example, in the United States.

`h23`

Hour system using 0–23; corresponds to 'H' in patterns. The 24 hour clock, with midnight starting at 0:00.

`h11`

Hour system using 0–11; corresponds to 'K' in patterns. The 12 hour clock, with midnight starting at 0:00 am. Mostly used in Japan.

`h24`

Hour system using 1–24; corresponds to 'k' in pattern. The 24 hour clock, with midnight starting at 24:00. Not used anywhere.

Examples

Obtaining supported hour cycles

If the `Locale` object doesn't have a `hourCycle` already, `getHourCycles()` lists all commonly-used hour cycle identifiers for the given `Locale`. For examples of explicitly setting a `hourCycle`, see hourCycle examples.

```js
const arEG = new Intl.Locale("ar-EG");
console.log(arEG.getHourCycles()); // ["h12"]
```

```js
const jaJP = new Intl.Locale("ja-JP");
console.log(jaJP.getHourCycles()); // ["h23"]
```

Specifications

Specification
Intl Locale Info Proposal https://tc39.es/proposal-intl-locale-info/#sec-Intl.Locale.prototype.getHourCycles

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
getHourCycles	130[1,2]	130[1,2]	153	115[3,4]	17[5,6]	130[1,2]	153	86[7,8]	17[9,10]	28.0[11,12]	130[1,2]	17[9,10]	1.0.0	2.1[13,14]	24.0.0[15,16]

1. Also from version 99: uses the non-standard name `hourCycles`.
2. Also from version 99: Implemented as an accessor property.
3. Also from version 85: uses the non-standard name `hourCycles`.
4. Also from version 85: Implemented as an accessor property.
5. Also from version 15.4–Prev.: uses the non-standard name `hourCycles`.
6. Also from version 15.4–Prev.: Implemented as an accessor property.
7. Also from version 68: uses the non-standard name `hourCycles`.
8. Also from version 68: Implemented as an accessor property.
9. Also from version 15.4: uses the non-standard name `hourCycles`.
10. Also from version 15.4: Implemented as an accessor property.
11. Also from version 18.0: uses the non-standard name `hourCycles`.
12. Also from version 18.0: Implemented as an accessor property.
13. Also from version 1.19–2.6.7: uses the non-standard name `hourCycles`.
14. Also from version 1.19–2.6.7: Implemented as an accessor property.
15. Also from version 18.0.0: uses the non-standard name `hourCycles`.
16. Also from version 18.0.0: Implemented as an accessor property.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.getHourCycles`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- `Intl.Locale.prototype.hourCycle`
- Unicode Hour Cycle Identifier in the Unicode locale data markup language spec

---

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.getNumberingSystems()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/getNumberingSystems · MDN page last modified 2025-07-10

Baseline 2026: Newly available — works across the latest browser versions since July 2026; may not work in older ones.

The **`getNumberingSystems()`** method of `Intl.Locale` instances returns a list of one or more unique numbering system identifiers for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `numberingSystems`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.numberingSystems === locale.numberingSystems` returning `false`. Check the browser compatibility table for details.

Syntax

```
getNumberingSystems()
```

Parameters

None.

Return value

An array of strings representing all numbering systems commonly used for the `Locale`, sorted in descending preference. If the `Locale` already has a `numberingSystem`, then the returned array contains that single value.

For a list of supported numbering system types, see `Intl.supportedValuesOf()`.

Examples

Obtaining supported numbering systems

If the `Locale` object doesn't have a `numberingSystem` already, `getNumberingSystems()` lists all commonly-used numbering systems for the given `Locale`. For examples of explicitly setting a `numberingSystem`, see `numberingSystem` examples.

```
const arEG = new Intl.Locale("ar-EG");
console.log(arEG.getNumberingSystems()); // ["arab"]
```

```
const ja = new Intl.Locale("ja");
console.log(ja.getNumberingSystems()); // ["latn"]
```

Specifications

Specification
Intl Locale Info Proposal `https://tc39.es/proposal-intl-locale-info/#sec-Intl.Locale.prototype.getNumberingSystems`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getNumberingSystems`	130[1,2]	130[1,2]	153	115[3,4]	17[5,6]	130[1,2]	153	86[7,8]	17[9,10]	28.0[11,12]	130[1,2]	17[9,10]	1.0.0	2.1[13,14]	24.0.0[15,16]

1. Also from version 99: uses the non-standard name `numberingSystems`.
2. Also from version 99: Implemented as an accessor property.
3. Also from version 85: uses the non-standard name `numberingSystems`.
4. Also from version 85: Implemented as an accessor property.
5. Also from version 15.4–Prev.: uses the non-standard name `numberingSystems`.
6. Also from version 15.4–Prev.: Implemented as an accessor property.
7. Also from version 68: uses the non-standard name `numberingSystems`.
8. Also from version 68: Implemented as an accessor property.
9. Also from version 15.4: uses the non-standard name `numberingSystems`.
10. Also from version 15.4: Implemented as an accessor property.
11. Also from version 18.0: uses the non-standard name `numberingSystems`.
12. Also from version 18.0: Implemented as an accessor property.
13. Also from version 1.19–2.6.7: uses the non-standard name `numberingSystems`.
14. Also from version 1.19–2.6.7: Implemented as an accessor property.
15. Also from version 18.0.0: uses the non-standard name `numberingSystems`.
16. Also from version 18.0.0: Implemented as an accessor property.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.getNumberingSystems`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- `Intl.Locale.prototype.numberingSystem`
- Details on the standard Unicode numeral systems

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.getTextInfo()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/getTextInfo · MDN page last modified 2025-07-10

Baseline 2026: Newly available — works across the latest browser versions since July 2026; may not work in older ones.

The **`getTextInfo()`** method of `Intl.Locale` instances returns the ordering of characters indicated by either `ltr` (left-to-right) or by `rtl` (right-to-left) for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `textInfo`. However, because it returns a new object on each access, it is now implemented as a method to prevent the situation of `locale.textInfo === locale.textInfo` returning `false`. Check the browser compatibility table for details.

Syntax

```js
getTextInfo()
```

Parameters

None.

Return value

An object representing text typesetting information associated with the Locale data specified in UTS 35's Layouts Elements. It has the following properties:

`direction`

A string indicating the direction of text for the locale. Can be either `"ltr"` (left-to-right) or `"rtl"` (right-to-left).

Examples

Obtaining text info

Return the supported text directions for a given `Locale`.

```js
const ar = new Intl.Locale("ar");
console.log(ar.getTextInfo()); // { direction: "rtl" }
console.log(ar.getTextInfo().direction); // "rtl"
```

```js
const es = new Intl.Locale("es");
console.log(es.getTextInfo()); // { direction: "ltr" }
console.log(es.getTextInfo().direction); // "ltr"
```

Specifications

Specification
Intl Locale Info Proposal `https://tc39.es/proposal-intl-locale-info/#sec-Intl.Locale.prototype.getTextInfo`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getTextInfo`	130[1,2]	130[1,2]	153	115[3,4]	17[5,6]	130[1,2]	153	86[7,8]	17[9,10]	28.0[11,12]	130[1,2]	17[9,10]	1.0.0	2.1[13,14]	24.0.0[15,16]

1. Also from version 99: uses the non-standard name `textInfo`.
2. Also from version 99: Implemented as an accessor property.
3. Also from version 85: uses the non-standard name `textInfo`.
4. Also from version 85: Implemented as an accessor property.
5. Also from version 15.4–Prev.: uses the non-standard name `textInfo`.
6. Also from version 15.4–Prev.: Implemented as an accessor property.
7. Also from version 68: uses the non-standard name `textInfo`.
8. Also from version 68: Implemented as an accessor property.
9. Also from version 15.4: uses the non-standard name `textInfo`.
10. Also from version 15.4: Implemented as an accessor property.
11. Also from version 18.0: uses the non-standard name `textInfo`.
12. Also from version 18.0: Implemented as an accessor property.
13. Also from version 1.19–2.6.7: uses the non-standard name `textInfo`.
14. Also from version 1.19–2.6.7: Implemented as an accessor property.
15. Also from version 18.0.0: uses the non-standard name `textInfo`.
16. Also from version 18.0.0: Implemented as an accessor property.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.getTextInfo`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`

Intl.Locale.prototype.getTimeZones()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/getTimeZones · MDN page last modified 2025-07-10

Baseline 2026: Newly available — works across the latest browser versions since July 2026; may not work in older ones.

The **`getTimeZones()`** method of `Intl.Locale` instances returns a list of supported time zones for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `timeZones`. However, because it returns a new array on each access, it is now implemented as a method to prevent the situation of `locale.timeZones === locale.timeZones` returning `false`. Check the browser compatibility table for details.

Syntax

```js
getTimeZones()
```

Parameters

None.

Return value

An array of strings representing supported time zones for the associated `Locale`, where each value is an IANA time zone canonical name, sorted in alphabetical order. If the locale identifier does not contain a region subtag, the returned value is `undefined`.

> **Note:** The standardization of `Temporal` requires browsers to always return the primary identifier in the IANA database, which may change over time. See time zones and offsets for more information.

Examples

Obtaining supported time zones

List supported time zones for a given `Locale`.

```js
const arEG = new Intl.Locale("ar-EG");
console.log(arEG.getTimeZones()); // ["Africa/Cairo"]
```

```js
const jaJP = new Intl.Locale("ja-JP");
console.log(jaJP.getTimeZones()); // ["Asia/Tokyo"]
```

```js
const ar = new Intl.Locale("ar");
console.log(ar.getTimeZones()); // undefined
```

Specifications

Specification
Intl Locale Info Proposal `https://tc39.es/proposal-intl-locale-info/#sec-Intl.Locale.prototype.getTimeZones`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getTimeZones`	130[1,2]	130[1,2]	153	115[3,4]	17[5,6]	130[1,2]	153	86[7,8]	17[9,10]	28.0[11,12]	130[1,2]	17[9,10]	1.0.0	2.1[13,14]	24.0.0[15,16]

1. Also from version 99: uses the non-standard name `timeZones`.
2. Also from version 99: Implemented as an accessor property.
3. Also from version 85: uses the non-standard name `timeZones`.
4. Also from version 85: Implemented as an accessor property.
5. Also from version 15.4–Prev.: uses the non-standard name `timeZones`.
6. Also from version 15.4–Prev.: Implemented as an accessor property.
7. Also from version 68: uses the non-standard name `timeZones`.

8. Also from version 68: Implemented as an accessor property.
9. Also from version 15.4: uses the non-standard name `timeZones`.
10. Also from version 15.4: Implemented as an accessor property.
11. Also from version 18.0: uses the non-standard name `timeZones`.
12. Also from version 18.0: Implemented as an accessor property.
13. Also from version 1.19–2.6.7: uses the non-standard name `timeZones`.
14. Also from version 1.19–2.6.7: Implemented as an accessor property.
15. Also from version 18.0.0: uses the non-standard name `timeZones`.
16. Also from version 18.0.0: Implemented as an accessor property.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.getTimeZones`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- IANA time zone database on Wikipedia

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.getWeekInfo()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/getWeekInfo · MDN page last modified 2025-07-10

Baseline 2026: Newly available — works across the latest browser versions since July 2026; may not work in older ones.

The **`getWeekInfo()`** method of `Intl.Locale` instances returns a `weekInfo` object with the properties `firstDay`, `weekend` and `minimalDays` for this locale.

> **Note:** In some versions of some browsers, this method was implemented as an accessor property called `weekInfo`. However, because it returns a new object on each access, it is now implemented as a method to prevent the situation of `locale.weekInfo === locale.weekInfo` returning `false`. Check the browser compatibility table for details.

Syntax

```
getWeekInfo()
```

Parameters

None.

Return value

An object representing week information associated with the Locale data specified in UTS 35's Week Elements. It has the following properties:

`firstDay`

An integer between 1 (Monday) and 7 (Sunday) indicating the first day of the week for the locale. Commonly 1, 5, 6, or 7.

`weekend`

An array of integers between 1 and 7 indicating the weekend days for the locale. This is usually continuous because UTS 35 stores `weekendStart` and `weekendEnd` instead.

`minimalDays`

An integer between 1 and 7 (commonly 1 and 4) indicating the minimal days required in the first week of a month or year, for week-of-year or week-of-month calculations (e.g., The 20th week of the year). For example, in the ISO 8601 calendar, the first week of a year must have at least 4 days in this year, so if January 1 is a Friday, Saturday, or Sunday, it will be numbered as part of the last week of the previous year.

Examples

Obtaining the Week Information

Return the week information for a given `Locale`.

```js
const he = new Intl.Locale("he"); // Hebrew (Israel)
console.log(he.getWeekInfo()); // { firstDay: 7, weekend: [5, 6], minimalDays: 1 }

const af = new Intl.Locale("af"); // Afrikaans (South Africa)
console.log(af.getWeekInfo()); // { firstDay: 7, weekend: [6, 7], minimalDays: 1 }

const enGB = new Intl.Locale("en-GB"); // English (United Kingdom)
console.log(enGB.getWeekInfo()); // { firstDay: 1, weekend: [6, 7], minimalDays: 4 }

const arAF = new Intl.Locale("ar-AF"); // Arabic (Afghanistan)
console.log(arAF.getWeekInfo()); // { firstDay: 6, weekend: [4, 5], minimalDays: 1 }

const dvMV = new Intl.Locale("dv-MV"); // Divehi (Maldives)
console.log(dvMV.getWeekInfo()); // { firstDay: 5, weekend: [6, 7], minimalDays: 1 }
```

Specifications

Specification
Intl Locale Info Proposal `https://tc39.es/proposal-intl-locale-info/#sec-Intl.Locale.prototype.getWeekInfo`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`getWeekInfo()` method	130[1,2]	130[1,2]	153	115[3,4]	17[5,6]	130[1,2]	153	86[7,8]	17[9,10]	28.0[11,12]	130[1,2]	17[9,10]	1.0.0	2.1[13,14]	24.0.0[15,16]
`minimalDays` in returned object Deprecated Non-standard	**99–136**	**99–136**	**No**	**85–121**	15.4	**99–136**	**No**	**68–90**	15.4	**18.0–29.0**	**99–136**	15.4	1.0.0	1.19	**18.0.0–24.0.0**

1. Also from version 99: uses the non-standard name `weekInfo`.
2. Also from version 99: Implemented as an accessor property.
3. Also from version 85: uses the non-standard name `weekInfo`.
4. Also from version 85: Implemented as an accessor property.
5. Also from version 15.4–Prev.: uses the non-standard name `weekInfo`.
6. Also from version 15.4–Prev.: Implemented as an accessor property.

7. Also from version 68: uses the non-standard name `weekInfo`.
8. Also from version 68: Implemented as an accessor property.
9. Also from version 15.4: uses the non-standard name `weekInfo`.
10. Also from version 15.4: Implemented as an accessor property.
11. Also from version 18.0: uses the non-standard name `weekInfo`.
12. Also from version 18.0: Implemented as an accessor property.
13. Also from version 1.19–2.6.7: uses the non-standard name `weekInfo`.
14. Also from version 1.19–2.6.7: Implemented as an accessor property.
15. Also from version 18.0.0: uses the non-standard name `weekInfo`.
16. Also from version 18.0.0: Implemented as an accessor property.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.getWeekInfo`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.maximize()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/maximize · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since September 2020.

The **`maximize()`** method of `Intl.Locale` instances gets the most likely values for the language, script, and region of this locale based on existing values.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.Locale.prototype.maximize()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const english = new Intl.Locale("en");
const korean = new Intl.Locale("ko");
const arabic = new Intl.Locale("ar");

console.log(english.maximize().baseName);
// Expected output: "en-Latn-US"

console.log(korean.maximize().baseName);
// Expected output: "ko-Kore-KR"

console.log(arabic.maximize().baseName);
// Expected output: "ar-Arab-EG"
```

Syntax

js

```
maximize()
```

Parameters

None.

Return value

An `Intl.Locale` instance whose `baseName` property returns the result of the Add Likely Subtags algorithm executed against *`Locale.baseName`*.

Description

Sometimes, it is convenient to be able to identify the most likely locale language identifier subtags based on an incomplete language ID. The Add Likely Subtags algorithm gives us this functionality. For instance, given the language ID "en", the algorithm would return "en-Latn-US", since English can only be written in the Latin script, and is most likely to be used in the United States, as it is the largest English-speaking country in the world. This functionality is provided to JavaScript programmers via the `maximize()` method. `maximize()` only affects the main subtags that comprise the language identifier: language, script, and region subtags. Other subtags after the "-u" in the locale identifier are called extension subtags and are not affected by the `maximize()` method. Examples of these subtags include `hourCycle`, `calendar`, and `numeric`.

Examples

Using maximize

```js
const myLocale = new Intl.Locale("fr", {
  hourCycle: "h12",
  calendar: "gregory",
});
console.log(myLocale.baseName); // Prints "fr"
console.log(myLocale.toString()); // Prints "fr-u-ca-gregory-hc-h12"
const myLocMaximized = myLocale.maximize();

// Prints "fr-Latn-FR". The "Latn" and "FR" tags are added,
// since French is only written in the Latin script and is most likely to be spoken in
France.
console.log(myLocMaximized.baseName);

// Prints "fr-Latn-FR-u-ca-gregory-hc-h12".
// Note that the extension tags (after "-u") remain unchanged.
console.log(myLocMaximized.toString());
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.maximize`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`maximize`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.maximize`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- `baseName`
- Likely Subtags in the Unicode locale data markup language spec

---

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.minimize()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/minimize · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since September 2020.

The **`minimize()`** method of `Intl.Locale` instances attempts to remove information about this locale that would be added by calling `maximize()`.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.Locale.prototype.minimize()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const english = new Intl.Locale("en-Latn-US");
const korean = new Intl.Locale("ko-Kore-KR");
const arabic = new Intl.Locale("ar-Arab-EG");

console.log(english.minimize().baseName);
// Expected output: "en"

console.log(korean.minimize().baseName);
// Expected output: "ko"

console.log(arabic.minimize().baseName);
// Expected output: "ar"
```

Syntax

js

```
minimize()
```

Parameters

None.

Return value

An `Intl.Locale` instance whose `baseName` property returns the result of the Remove Likely Subtags algorithm executed against *`Locale.baseName`*.

Description

This method carries out the reverse of `maximize()`, removing any language, script, or region subtags from the locale language identifier (essentially the contents of `baseName`). This is useful when there are superfluous subtags in the language identifier; for instance, "en-Latn" can be simplified to "en", since "Latn" is the only script used to write English. `minimize()` only affects the main subtags that comprise the language identifier: language, script, and region subtags. Other subtags after the "-u" in the locale identifier are called extension subtags and are not affected by the `minimize()` method. Examples of these subtags include `hourCycle`, `calendar`, and `numeric`.

Examples

Using minimize

```js
const myLocale = new Intl.Locale("fr-Latn-FR", {
  hourCycle: "h12",
  calendar: "gregory",
});
console.log(myLocale.baseName); // Prints "fr-Latn-FR"
console.log(myLocale.toString()); // Prints "fr-Latn-FR-u-ca-gregory-hc-h12"

const myLocMinimized = myLocale.minimize();

// Prints "fr", since French is only written in the Latin script
// and is most likely to be spoken in France.
console.log(myLocMinimized.baseName);

// Prints "fr-u-ca-gregory-hc-h12".
// Note that the extension tags (after "-u") remain unchanged.
console.log(myLocMinimized.toString());
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-Intl.Locale.prototype.minimize

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`minimize`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.minimize`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- `baseName`

Intl.Locale.prototype.toString()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/toString · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2020.

The **`toString()`** method of `Intl.Locale` instances returns this Locale's full locale identifier string.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.Locale.prototype.toString()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const french = new Intl.Locale("fr-Latn-FR", {
  calendar: "gregory",
  hourCycle: "h12",
});
const korean = new Intl.Locale("ko-Kore-KR", {
  numeric: true,
  caseFirst: "upper",
});

console.log(french.toString());
// Expected output: "fr-Latn-FR-u-ca-gregory-hc-h12"

console.log(korean.toString());
// Expected output: "ko-Kore-KR-u-kf-upper-kn"
```

Syntax

js

```
toString()
```

Parameters

None.

Return value

The *locale*'s Unicode locale identifier string.

Description

The `Locale` object is a JavaScript representation of a concept Unicode locale identifier. Information about a particular locale (language, script, calendar type, etc.) can be encoded in a locale identifier string. To make it easier to work with these locale identifiers, the `Locale` object was introduced to JavaScript. Calling the `toString` method on a Locale object will return the identifier string for that particular Locale. The `toString` method allows `Locale` instances to be provided as an argument to existing `Intl` constructors, serialized in JSON, or any other context where an exact string representation is useful.

Examples

Using toString

```js
const myLocale = new Intl.Locale("fr-Latn-FR", {
  hourCycle: "h12",
  calendar: "gregory",
});
console.log(myLocale.baseName); // Prints "fr-Latn-FR"
console.log(myLocale.toString()); // Prints "fr-Latn-FR-u-ca-gregory-hc-h12"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.toString`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toString`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.toString`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- `baseName`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.baseName

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/baseName · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`baseName`** accessor property of `Intl.Locale` instances returns a substring of this locale's string representation, containing core information about this locale, including the language, script, region, and variants, if available.

Description

`baseName` returns the `language ["-" script] ["-" region] *("-" variant)` subsequence of the unicode_language_id grammar. It only includes information explicitly specified in the constructor, either through the locale identifier string or the options object.

The set accessor of `baseName` is `undefined`. You cannot change this property directly.

Examples

Basic example

```js
const myLoc = new Intl.Locale("fr-Latn-CA"); // Sets locale to Canadian French
console.log(myLoc.toString()); // "fr-Latn-CA-u-ca-gregory"
console.log(myLoc.baseName); // "fr-Latn-CA"
```

Example with extension tags in the input string

```js
// Sets language to Japanese, region to Japan,
// calendar to Gregorian, hour cycle to 24 hours
const japan = new Intl.Locale("ja-JP-u-ca-gregory-hc-24");
console.log(japan.toString()); // "ja-JP-u-ca-gregory-hc-h24"
console.log(japan.baseName); // "ja-JP"
```

Example with options that override input string

```js
// Input string indicates language as Dutch and region as Belgium,
// but options object overrides the region and sets it to the Netherlands
const dutch = new Intl.Locale("nl-Latn-BE", { region: "NL" });

console.log(dutch.baseName); // "nl-Latn-NL"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.baseName`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`baseName`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.baseName`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`

Intl.Locale.prototype.calendar

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/calendar · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`calendar`** accessor property of `Intl.Locale` instances returns the calendar type for this locale.

Description

While most of the world uses the Gregorian calendar, there are several regional calendar eras used around the world. For a list of supported calendar types, see `Intl.supportedValuesOf()`.

The `calendar` property's value is set at construction time, either through the `ca` key of the locale identifier or through the `calendar` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `calendar` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the calendar type can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a calendar type via the locale string

In the Unicode locale string spec, `calendar` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the calendar type to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u`

extension key if it doesn't exist. Next, add the `-ca` extension to indicate that you are adding a calendar type. Finally, add the calendar era type.

```js
const locale = new Intl.Locale("fr-FR-u-ca-buddhist");
console.log(locale.calendar); // "buddhist"
```

Adding a calendar type via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including calendars. Set the `calendar` property of the configuration object to your desired calendar era, and then pass it into the constructor.

```js
const locale = new Intl.Locale("fr-FR", { calendar: "buddhist" });
console.log(locale.calendar); // "buddhist"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.calendar`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`calendar`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.calendar`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- `Intl.Locale.prototype.getCalendars()`
- Unicode Calendar Identifier in the Unicode locale data markup language spec

---

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.caseFirst

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/caseFirst · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since September 2020.

The **`caseFirst`** accessor property of `Intl.Locale` instances returns whether case is taken into account for this locale's collation rules.

Description

A locale's collation rules are used to determine how strings are ordered in that locale. Certain locales use a character's case (UPPERCASE or lowercase) in the collation process. This additional rule can be expressed in an `Intl.Locale` object's `caseFirst` property. There are 3 values that the `caseFirst` property can have, outlined in the table below.

Value	Description
`upper`	Upper case to be sorted before lower case.
`lower`	Lower case to be sorted before upper case.
`false`	No special case ordering.

The `caseFirst` property's value is set at construction time, either through the `kf` key of the locale identifier or through the `caseFirst` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `caseFirst` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the `caseFirst` value can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a caseFirst value via the locale string

In the Unicode locale string spec, `caseFirst` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the `caseFirst` value to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-kf` extension to indicate that you are adding a value for `caseFirst`. Finally, add the `caseFirst` value.

```js
const locale = new Intl.Locale("fr-Latn-FR-u-kf-upper");
console.log(locale.caseFirst); // "upper"
```

Adding a caseFirst value via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including `caseFirst`. Set the `caseFirst` property of the configuration object to your desired `caseFirst` value, and then pass it into the constructor.

```js
const locale = new Intl.Locale("en-Latn-US", { caseFirst: "lower" });
console.log(locale.caseFirst); // "lower"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.caseFirst`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`caseFirst`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.caseFirst`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- Unicode case first collation spec

Intl.Locale.prototype.collation

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/collation · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`collation`** accessor property of `Intl.Locale` instances returns the collation type for this locale, which is used to order strings according to the locale's rules.

Description

Collation is the process of ordering strings of characters. It is used whenever strings must be sorted and placed into a certain order, from search query results to ordering records in a database. While the idea of placing strings in order might seem trivial, the idea of order can vary from region to region and language to language. For a list of supported collation types, see `Intl.supportedValuesOf()`.

The `collation` property's value is set at construction time, either through the `co` key of the locale identifier or through the `collation` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `collation` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the collation type can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a collation type via the locale string

In the Unicode locale string spec, `collation` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the collation type to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-co` extension to indicate that you are adding a collation type. Finally, add the collation type.

```js
const locale = new Intl.Locale("zh-Hant-u-co-zhuyin");
console.log(locale.collation); // "zhuyin"
```

Adding a collation type via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including collation types. Set the `collation` property of the configuration object to your desired collation type, and then pass it into the constructor.

```js
const locale = new Intl.Locale("zh-Hant", { collation: "zhuyin" });
console.log(locale.collation); // "zhuyin"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.collation`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`collation`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.collation`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.Locale`
- `Intl.Locale.prototype.getCollations()`

Intl.Locale.prototype.hourCycle

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/hourCycle · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`hourCycle`** accessor property of `Intl.Locale` instances returns the hour cycle type for this locale.

Description

There are 2 main types of time keeping conventions (clocks) used around the world: the 12 hour clock and the 24 hour clock. For a list of supported hour cycle types, see `Intl.Locale.prototype.getHourCycles()`.

The `hourCycle` property's value is set at construction time, either through the `hc` key of the locale identifier or through the `hourCycle` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `hourCycle` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the hour cycle type can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding an hour cycle via the locale string

In the Unicode locale string spec, `hourCycle` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the hour cycle type to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-hc` extension to indicate that you are adding an hour cycle. Finally, add the hour cycle type.

```js
const locale = new Intl.Locale("fr-FR-u-hc-h23");
console.log(locale.hourCycle); // "h23"
```

Adding an hour cycle via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including hour cycle types. Set the `hourCycle` property of the configuration object to your desired hour cycle type, and then pass it into the constructor.

```js
const locale = new Intl.Locale("en-US", { hourCycle: "h12" });
console.log(locale.hourCycle); // "h12"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.hourCycle`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`hourCycle`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.hourCycle`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.Locale`
- `Intl.Locale.prototype.getHourCycles()`
- Unicode Hour Cycle Identifier in the Unicode locale data markup language spec

---

Intl.Locale.prototype.language

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/language · MDN page last modified 2025-10-30

Baseline: Widely available — available across major browsers since September 2020.

The **`language`** accessor property of `Intl.Locale` instances returns the language associated with this locale.

Description

Language is one of the core attributes of a locale. The Unicode specification treats the language identifier of a locale as the language and the region together (to make a distinction between dialects and variations, e.g., British English vs. American English). The `language` property of an `Intl.Locale` returns strictly the locale's language subtag.

The `language` property's value is set at construction time, either through the first part of the locale identifier or through the `language` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present.

The set accessor of `language` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the language can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Setting the language via the locale string

In order to be a valid Unicode locale identifier, a string must start with the language subtag. The main argument to the `Intl.Locale()` constructor must be a valid Unicode locale identifier, so whenever the constructor is used, it must be passed an identifier with a language subtag.

```js
const locale = new Intl.Locale("en-Latn-US");
console.log(locale.language); // "en"
```

Overriding language via the configuration object argument

While the language subtag must be specified, the `Intl.Locale()` constructor has an optional configuration object argument, which can override the language subtag.

```js
const locale = new Intl.Locale("en-Latn-US", { language: "es" });
console.log(locale.language); // "es"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.language`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`language`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.language`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- Unicode language subtag in the Unicode locale data markup language spec

---

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.numberingSystem

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/numberingSystem · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`numberingSystem`** accessor property of `Intl.Locale` instances returns the numeral system for this locale.

Description

A numeral system is a system for expressing numbers. For a list of supported numbering system types, see `Intl.supportedValuesOf()`.

The `numberingSystem` property's value is set at construction time, either through the `nu` key of the locale identifier or through the `numberingSystem` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `numberingSystem` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the numbering system type can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a numbering system via the locale string

In the Unicode locale string spec, `numberingSystem` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the numbering system type to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-nu` extension to indicate that you are adding a numbering system. Finally, add the numbering system type.

```js
const locale = new Intl.Locale("fr-Latn-FR-u-nu-mong");
console.log(locale.numberingSystem); // "mong"
```

Adding a numbering system via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including numbering system types. Set the `numberingSystem` property of the configuration object to your desired numbering system type, and then pass it into the constructor.

```js
const locale = new Intl.Locale("en-Latn-US", { numberingSystem: "latn" });
console.log(locale.numberingSystem); // "latn"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.numberingSystem`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`numberingSystem`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.numberingSystem`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.Locale`
- `Intl.Locale.prototype.getNumberingSystems()`
- Details on the standard Unicode numeral systems

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.numeric

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/numeric · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`numeric`** accessor property of `Intl.Locale` instances returns whether this locale has special collation handling for numeric characters.

Description

Like `caseFirst`, `numeric` represents a modification to the collation rules utilized by the locale. `numeric` is a boolean value, which means that it can be either `true` or `false`. If `numeric` is set to `false`, there will be no special handling of numeric values in strings. If `numeric` is set to `true`, then the locale will take numeric characters into account when collating strings. This special numeric handling means that sequences of decimal digits will be compared as numbers. For example, the string "A-21" will be considered less than "A-123".

The `numeric` property's value is set at construction time, either through the `kn` key of the locale identifier or through the `numeric` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `numeric` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the `numeric` value can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a `numeric` value via the locale string

In the Unicode locale string spec, `numeric` is an "extension subtag". These subtags add additional data about the locale, and are added to locale identifiers using the `-u` extension key. To add the `numeric` value to the initial locale identifier string passed into the `Intl.Locale()` constructor, first add the `-u` extension key if it doesn't exist. Next, add the `-kn` extension to indicate that you are adding a value for `numeric`. Finally, add the `numeric` value. If you want to set `numeric` to `true`, adding the `kn` key will suffice. To set the value to `false`, you must specify in by adding `"false"` after the `kn` key.

```js
const locale = new Intl.Locale("fr-Latn-FR-u-kn-false");
console.log(locale.numeric); // "false"
```

Adding a `numeric` value via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument, which can contain any of several extension types, including `numeric`. Set the `numeric` property of the configuration object to your desired `numeric` value, and then pass it into the constructor.

```js
const locale = new Intl.Locale("en-Latn-US", { numeric: true });
console.log(locale.numeric); // "true"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.numeric`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`numeric`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.numeric`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`

Intl.Locale.prototype.region

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/region · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`region`** accessor property of `Intl.Locale` instances returns the region of the world (usually a country) associated with this locale.

Description

Region is one of the core attributes of a locale. It allows selection for differences between the same language in, say, different countries. For example, English is spoken in the United Kingdom and the United States of America, but there are differences in spelling and other language conventions between those two countries. Knowing the locale's region helps JavaScript programmers make sure that the content from their sites and applications is correctly displayed when viewed from different areas of the world.

The `region` property's value is set at construction time, either through the part of the locale identifier after `script` or through the `region` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `region` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the region can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a region via the locale string

The region, if present, is the third part (if `script` is present, second part otherwise) of a valid Unicode language identifier string, and can be added to the initial locale identifier string that is passed into the `Intl.Locale()` constructor. Note that the region is not a required part of a locale identifier.

```js
const locale = new Intl.Locale("en-Latn-US");
console.log(locale.region); // "US"
```

Adding a region via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument. Set the `region` property of the configuration object to your desired region, and then pass it into the constructor.

```js
const locale = new Intl.Locale("fr-Latn", { region: "FR" });
console.log(locale.region); // "FR"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.region`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`region`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.region`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- Unicode region chart

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.script

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/script · MDN page last modified 2025-07-22

Baseline: Widely available — available across major browsers since September 2020.

The **`script`** accessor property of `Intl.Locale` instances returns the script used for writing the particular language used in this locale.

Description

Script, sometimes called writing system, is one of the core attributes of a locale. It indicates the set of symbols, or glyphs, that are used to write a particular language. For instance, the script associated with English is Latin, whereas the script typically associated with Korean is Hangul. In many cases, denoting a script is not strictly necessary, since the language (which is necessary) is only written in a single script. There are exceptions to this rule, however, and it is important to indicate the script when multiple scripts are applicable.

The `script` property's value is set at construction time, either through the part of the locale identifier after `language` or through the `script` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `script` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the script can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding a script via the locale string

The script, if present, is the second part of a valid Unicode language identifier string, and can be added to the initial locale identifier string that is passed into the `Intl.Locale()` constructor. Note that the script is not a required part of a locale identifier.

```js
const locale = new Intl.Locale("en-Latn-US");
console.log(locale.script); // "Latn"
```

Adding a script via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument. Set the `script` property of the configuration object to your desired script, and then pass it into the constructor.

```js
const locale = new Intl.Locale("fr-FR", { script: "Latn" });
console.log(locale.script); // "Latn"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.script`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`script`	74	79	75	62	14	74	79	53	14	11.0	74	14	1.0.0	1.8	12.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.script`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- Unicode script subtag in the Unicode locale data markup language spec

---

JavaScript › Reference › Standard built-in objects › Intl › Intl.Locale

Intl.Locale.prototype.variants

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Locale/variants · MDN page last modified 2025-07-22

The **`variants`** accessor property of `Intl.Locale` instances returns the variants associated with this locale, as a string of dash (`-`) separated identifiers in the originally specified order.

Description

Variants are a part of the main language ID. They select variants of a language that the (language, region, script) triple cannot differentiate. Usually, they represent the same language in different eras or different orthographies. For example, German has the `1901` and `1996` orthography variants, which are written as `de-1901` and `de-1996`; the "Early Modern English (1500-1700)" variant is written as `en-emodeng`. The subtag can contain multiple identifiers separated by dashes (`-`). These identifiers are technically unordered, although in practice they often have a semantic hierarchy—for example, the Resian dialect of Slovenian is written as `sl-rozaj`, and the San Giorgio/Bila dialect of Resian is written as `sl-rozaj-biske`.

The `variants` property's value is set at construction time, either through the part of the locale identifier after `region` or through the `variants` option of the `Intl.Locale()` constructor. The latter takes priority if they are both present; and if neither is present, the property has value `undefined`.

The set accessor of `variants` is `undefined`. You cannot change this property directly.

Examples

Like other locale subtags, the variants can be added to the `Intl.Locale` object via the locale string, or a configuration object argument to the constructor.

Adding variants via the locale string

The variants, if present, are the last parts of a valid Unicode language identifier string, and can be added to the initial locale identifier string that is passed into the `Intl.Locale()` constructor. Note that the

variants are not a required part of a locale identifier.

```js
const locale = new Intl.Locale("sl-rozaj-biske");
console.log(locale.variants); // "rozaj-biske"
```

Adding variants via the configuration object argument

The `Intl.Locale()` constructor has an optional configuration object argument. Set the `variants` property of the configuration object to your desired variants, and then pass it into the constructor.

```js
const locale = new Intl.Locale("sl", { variants: "rozaj-biske" });
console.log(locale.variants); // "rozaj-biske"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.Locale.prototype.variants`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`variants`	149	149	141	133	26	149	141	100	26	**No**	149	26	1.2.18	2.8	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Locale.variants`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Locale`
- Unicode variant subtag in the Unicode locale data markup language spec
- IANA language subtag registry

JavaScript › Reference › Standard built-in objects › Intl

Intl.NumberFormat

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl.NumberFormat`** object enables language-sensitive number formatting.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.NumberFormat". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const number = 123456.789;

console.log(
  new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
    number,
  ),
);
// Expected output: "123.456,79 €"

// The Japanese yen doesn't use a minor unit
console.log(
  new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" }).format(
    number,
  ),
);
// Expected output: "￥123,457"

// Limit to three significant digits
console.log(
  new Intl.NumberFormat("en-IN", { maximumSignificantDigits: 3 }).format(
    number,
  ),
);
// Expected output: "1,23,000"
```

Constructor

`Intl.NumberFormat()`

Creates a new `NumberFormat` object.

Static methods

`Intl.NumberFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.NumberFormat.prototype` and shared by all `Intl.NumberFormat` instances.

`Intl.NumberFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.NumberFormat` instances, the initial value is the `Intl.NumberFormat` constructor.

`Intl.NumberFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.NumberFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.NumberFormat.prototype.format()`

Getter function that formats a number according to the locale and formatting options of this `Intl.NumberFormat` object.

`Intl.NumberFormat.prototype.formatRange()`

Getter function that formats a range of numbers according to the locale and formatting options of the `Intl.NumberFormat` object from which the method is called.

`Intl.NumberFormat.prototype.formatRangeToParts()`

Returns an `Array` of objects representing the range of number strings in parts that can be used for custom locale-aware formatting.

`Intl.NumberFormat.prototype.formatToParts()`

Returns an `Array` of objects representing the number string in parts that can be used for custom locale-aware formatting.

`Intl.NumberFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and collation options computed during initialization of the object.

Examples

Basic usage

In basic use without specifying a locale, a formatted string in the default locale and with default options is returned.

```js
const number = 3500;

console.log(new Intl.NumberFormat().format(number));
// '3,500' if in US English locale
```

Using locales

This example shows some of the variations in localized number formats. In order to get the format of the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the `locales` argument:

```js
const number = 123456.789;

// German uses comma as decimal separator and period for thousands
console.log(new Intl.NumberFormat("de-DE").format(number));
// 123.456,789
```

(code continues on the next page)

(code continued from the previous page)

```
// Arabic in most Arabic speaking countries uses real Arabic digits
console.log(new Intl.NumberFormat("ar-EG").format(number));
// ١٢٣٤٥٦٫٧٨٩

// India uses thousands/lakh/crore separators
console.log(new Intl.NumberFormat("en-IN").format(number));
// 1,23,456.789

// the nu extension key requests a numbering system, e.g. Chinese decimal
console.log(new Intl.NumberFormat("zh-Hans-CN-u-nu-hanidec").format(number));
// 一二三,四五六.七八九

// when requesting a language that may not be supported, such as
// Balinese, include a fallback language, in this case Indonesian
console.log(new Intl.NumberFormat(["ban", "id"]).format(number));
// 123.456,789
```

Using options

The results can be customized using the options argument:

js

```
const number = 123456.789;

// request a currency format
console.log(
  new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
    number,
  ),
);
// 123.456,79 €

// the Japanese yen doesn't use a minor unit
console.log(
  new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" }).format(
    number,
  ),
);
// ￥123,457

// limit to three significant digits
console.log(
  new Intl.NumberFormat("en-IN", { maximumSignificantDigits: 3 }).format(
    number,
  ),
);
// 1,23,000

// Formatting with units
console.log(
  new Intl.NumberFormat("pt-PT", {
    style: "unit",
    unit: "kilometer-per-hour",
  }).format(50),
);
// 50 km/h
```

(code continues on the next page)

(code continued from the previous page)

```
console.log(
  (16).toLocaleString("en-GB", {
    style: "unit",
    unit: "liter",
    unitDisplay: "long",
  }),
);
// 16 litres
```

For an exhaustive list of options, see the `Intl.NumberFormat()` constructor page.

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#numberformat-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`NumberFormat`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]
`NumberFormat()` constructor	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
Supports normative optional `ChainNumberFormat` behavior	91 [2,3]	91 [4,5]	54	77 [6,7]	14.1	91 [2,3]	56	64 [8,9]	14.5	16.0 [10,11]	91 [2,3]	14.5	1.0.0	1.8	16.0.0 [12,13]
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [14,15]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.compactDisplay` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.currencyDisplay` parameter	77	79	42	64	14.1 [16,17]	77	56	55	14.5 [18,19]	12.0	77	14.5 [18,19]	1.0.0	1.8	12.11.0
`options.currencySign` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.currency` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.localeMatcher` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.maximumFractionDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.maximumSignificantDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.minimumFractionDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.minimumIntegerDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.minimumSignificantDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.notation` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`options.numberingSystem` parameter	24	12	76	15	14.1	25	56	14	14.5	1.5	4.4	14.5	1.0.0	1.8	0.12.0
`options.roundingIncrement` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.roundingMode` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.roundingPriority` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.signDisplay` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`negative` value	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.style` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.trailingZeroDisplay` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.unitDisplay` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.unit` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.useGrouping` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [14,15]
`options.useGrouping` parameter accepts: `'always'`, `'auto'`, `'min2'` (in addition to: `true` and `false`)	106	106	116	92	16.4	106	116	72	16.4	20.0	106	16.4	1.0.0	**No**	19.0.0
`format`	24	12 [20]	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]
`number` param string value is decimal (not `Number`)	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	1.25	19.0.0
`formatRange`	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	19.0.0
`formatRangeToParts`	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	19.0.0
`formatToParts`	64	12	58	51	13	64	58	47	13	9.0	64	13	1.0.0	1.8	10.0.0 [1]
`resolvedOptions`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]
`supportedLocalesOf`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [14,21]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `NumberFormat()` constructor for more details.
2. Also from version 57–91: partial support.
3. Also from version 57–91: The fallback symbol property has description `IntlFallback`.
4. Also from version 79–91: partial support.
5. Also from version 79–91: The fallback symbol property has description `IntlFallback`.
6. Also from version 44–77: partial support.
7. Also from version 44–77: The fallback symbol property has description `IntlFallback`.
8. Also from version 43–64: partial support.
9. Also from version 43–64: The fallback symbol property has description `IntlFallback`.
10. Also from version 7.0–16.0: partial support.
11. Also from version 7.0–16.0: The fallback symbol property has description `IntlFallback`.
12. Also from version 8.0.0–16.0.0: partial support.
13. Also from version 8.0.0–16.0.0: The fallback symbol property has description `IntlFallback`.
14. Also from version 0.12.0–13.0.0: partial support.
15. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `NumberFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see

Node.js documentation on the `--with-intl` option and how to provide the data.

16. Also from version 10–14.1: partial support.
17. Also from version 10–14.1: Doesn't support `currencyDisplay: 'narrowSymbol'`.
18. Also from version 10–14.5: partial support.
19. Also from version 10–14.5: Doesn't support `currencyDisplay: 'narrowSymbol'`.
20. Before Edge 18, numbers are rounded to 15 decimal digits. For example, `new Intl.NumberFormat('en-US').format(1000000000000005)` returns `"1,000,000,000,000,010"`.
21. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.NumberFormat`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Intl.NumberFormat` in FormatJS
- `Intl`
- `Number.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.NumberFormat

Intl.NumberFormat() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat/NumberFormat · MDN page last modified 2026-08-21

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl.NumberFormat()`** constructor creates `Intl.NumberFormat` objects.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.NumberFormat() constructor". It runs live in the browser there; its source code follows.

js · interactive example source

```
const number = 123456.789;

console.log(
  new Intl.NumberFormat("de-DE", { style: "currency", currency: "EUR" }).format(
    number,
  ),
);
// Expected output: "123.456,79 €"

// The Japanese yen doesn't use a minor unit
console.log(
  new Intl.NumberFormat("ja-JP", { style: "currency", currency: "JPY" }).format(
    number,
  ),
);
// Expected output: "￥123,457"

// Limit to three significant digits
console.log(
  new Intl.NumberFormat("en-IN", { maximumSignificantDigits: 3 }).format(
    number,
  ),
);
// Expected output: "1,23,000"
```

Syntax

```js
new Intl.NumberFormat()
new Intl.NumberFormat(locales)
new Intl.NumberFormat(locales, options)

Intl.NumberFormat()
Intl.NumberFormat(locales)
Intl.NumberFormat(locales, options)
```

> **Note:** `Intl.NumberFormat()` can be called with or without `new`. Both create a new `Intl.NumberFormat` instance. However, there's a special behavior when it's called without `new` and the `this` value is another `Intl.NumberFormat` instance; see Return value.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

The following Unicode extension key is allowed:

`nu`

See `numberingSystem`.

This key can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object. For ease of reading, the property list is broken into sections based on their purposes, including locale options, style options, digit options, and other options.

Locale options

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`numberingSystem`

The numbering system to use for number formatting, such as `"arab"`, `"hans"`, `"mathsans"`, and so on. For a list of supported numbering system types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `nu` Unicode extension key; if both are provided, this `options` property takes precedence.

Style options

Depending on the `style` used, some of them may be ignored, and others may be required:

`style`

The formatting style to use.

`"decimal"` (default)

For plain number formatting.

`"currency"`
: For currency formatting.

`"percent"`
: For percent formatting.

`"unit"`
: For unit formatting.

`currency`
: The currency to use in currency formatting. Possible values are the ISO 4217 currency codes, such as `"USD"` for the US dollar, `"EUR"` for the euro, or `"CNY"` for the Chinese RMB — see `Intl.supportedValuesOf()`. There is no default value; if the `style` is `"currency"`, the `currency` property must be provided. It is normalized to uppercase.

`currencyDisplay`
: How to display the currency in currency formatting.

`"code"`
: Use the ISO currency code.

`"symbol"` (default)
: Use a localized currency symbol such as €.

`"narrowSymbol"`
: Use a narrow format symbol ("$100" rather than "US$100").

`"name"`
: Use a localized currency name such as `"dollar"`.

`currencySign`
: In many locales, accounting format means to wrap the number with parentheses instead of appending a minus sign. Possible values are `"standard"` and `"accounting"`; the default is `"standard"`.

`unit`
: The unit to use in `unit` formatting. Possible values are listed in `Intl.supportedValuesOf()`. Pairs of simple units can be concatenated with "-per-" to make a compound unit. There is no default value; if the `style` is `"unit"`, the `unit` property must be provided.

`unitDisplay`
: The unit formatting style to use in `unit` formatting. Possible values are:

`"short"` (default)
: E.g., `16 l`.

`"narrow"`
: E.g., `16l`.

`"long"`
: E.g., `16 litres`.

Digit options

The following properties are also supported by `Intl.PluralRules`.

`minimumIntegerDigits`

The minimum number of integer digits to use. A value with a smaller number of integer digits than this number will be left-padded with zeros (to the specified length) when formatted. Possible values are from `1` to `21`; the default is `1`.

`minimumFractionDigits`

The minimum number of fraction digits to use. Possible values are from `0` to `100`; the default for plain number and percent formatting is `0`; the default for currency formatting is the number of minor unit digits provided by the ISO 4217 currency code list (2 if the list doesn't provide that information). See SignificantDigits/FractionDigits default values for when this default gets applied.

`maximumFractionDigits`

The maximum number of fraction digits to use. Possible values are from `0` to `100`; the default for plain number formatting is the larger of `minimumFractionDigits` and `3`; the default for currency formatting is the larger of `minimumFractionDigits` and the number of minor unit digits provided by the ISO 4217 currency code list (2 if the list doesn't provide that information); the default for percent formatting is the larger of `minimumFractionDigits` and 0. See SignificantDigits/FractionDigits default values for when this default gets applied.

`minimumSignificantDigits`

The minimum number of significant digits to use. Possible values are from `1` to `21`; the default is `1`. See SignificantDigits/FractionDigits default values for when this default gets applied.

`maximumSignificantDigits`

The maximum number of significant digits to use. Possible values are from `1` to `21`; the default is `21`. See SignificantDigits/FractionDigits default values for when this default gets applied.

`roundingPriority`

Specify how rounding conflicts will be resolved if both "FractionDigits" (`minimumFractionDigits`/`maximumFractionDigits`) and "SignificantDigits" (`minimumSignificantDigits`/`maximumSignificantDigits`) are specified. Possible values are:

`"auto"` (default)

The result from the significant digits property is used.

`"morePrecision"`

The result from the property that results in more precision is used.

`"lessPrecision"`

The result from the property that results in less precision is used.

The value `"auto"` is normalized to `"morePrecision"` if `notation` is `"compact"` and none of the four "FractionDigits"/"SignificantDigits" options are set.

Note that for values other than `auto` the result with more precision is calculated from the `maximumSignificantDigits` and `maximumFractionDigits` (minimum fractional and significant digit settings are ignored).

`roundingIncrement`

Indicates the increment at which rounding should take place relative to the calculated rounding magnitude. Possible values are `1`, `2`, `5`, `10`, `20`, `25`, `50`, `100`, `200`, `250`, `500`, `1000`, `2000`, `2500`, and `5000`; the default is `1`. It cannot be mixed with significant-digits rounding or any setting of `roundingPriority` other than `auto`.

`roundingMode`

How decimals should be rounded. Possible values are:

`"ceil"`
: Round toward +∞. Positive values round up. Negative values round "more positive".

`"floor"`
: Round toward -∞. Positive values round down. Negative values round "more negative".

`"expand"`
: Round away from 0. The *magnitude* of the value is always increased by rounding. Positive values round up. Negative values round "more negative".

`"trunc"`
: Round toward 0. This *magnitude* of the value is always reduced by rounding. Positive values round down. Negative values round "less negative".

`"halfCeil"`
: Ties toward +∞. Values above the half-increment round like `"ceil"` (towards +∞), and below like `"floor"` (towards -∞). On the half-increment, values round like `"ceil"`.

`"halfFloor"`
: Ties toward -∞. Values above the half-increment round like `"ceil"` (towards +∞), and below like `"floor"` (towards -∞). On the half-increment, values round like `"floor"`.

`"halfExpand"` (default)
: Ties away from 0. Values above the half-increment round like `"expand"` (away from zero), and below like `"trunc"` (towards 0). On the half-increment, values round like `"expand"`.

`"halfTrunc"`
: Ties toward 0. Values above the half-increment round like `"expand"` (away from zero), and below like `"trunc"` (towards 0). On the half-increment, values round like `"trunc"`.

`"halfEven"`
: Ties towards the nearest even integer. Values above the half-increment round like `"expand"` (away from zero), and below like `"trunc"` (towards 0). On the half-increment values round towards the nearest even digit.

These options reflect the ICU user guide, where "expand" and "trunc" map to ICU "UP" and "DOWN", respectively. The rounding modes example below demonstrates how each mode works.

`trailingZeroDisplay`

The strategy for displaying trailing zeros on whole numbers. Possible values are:

`"auto"` (default)
: Keep trailing zeros according to `minimumFractionDigits` and `minimumSignificantDigits`.

`"stripIfInteger"`
: Remove the fraction digits *if* they are all zero. This is the same as `"auto"` if any of the fraction digits is non-zero.

SignificantDigits/FractionDigits default values

For the four options above (the `FractionDigits` and `SignificantDigits` options), we mentioned their defaults; however, these defaults are *not unconditionally applied*. They are only applied when the property is actually going to be used, which depends on the `roundingPriority` and `notation` settings. Specifically:

- If `roundingPriority` is not `"auto"`, then all four options apply.
- If `roundingPriority` is `"auto"` and at least one `SignificantDigits` option is set, then the `SignificantDigits` options apply and the `FractionDigits` options are ignored.
- If `roundingPriority` is `"auto"`, and either at least one `FractionDigits` option is set or `notation` is not `"compact"`, then the `FractionDigits` options apply and the `SignificantDigits` options are ignored.
- If `roundingPriority` is `"auto"`, `notation` is `"compact"`, and none of the four options are set, then they are set to `{ minimumFractionDigits: 0, maximumFractionDigits: 0, minimumSignificantDigits: 1, maximumSignificantDigits: 2 }`, regardless of the defaults mentioned above, and `roundingPriority` is set to `"morePrecision"`.

Other options

`notation`

The formatting that should be displayed for the number. Possible values are:

`"standard"` (default)

Plain number formatting.

`"scientific"`

Return the order-of-magnitude for formatted number.

`"engineering"`

Return the exponent of ten when divisible by three.

`"compact"`

String representing exponent; defaults to using the "short" form.

`compactDisplay`

Only used when `notation` is `"compact"`. Possible values are `"short"` and `"long"`; the default is `"short"`.

`useGrouping`

Whether to use grouping separators, such as thousands separators or thousand/lakh/crore separators.

`"always"`

Display grouping separators even if the locale prefers otherwise.

`"auto"`

Display grouping separators based on the locale preference, which may also be dependent on the currency.

`"min2"`

Display grouping separators when there are at least 2 digits in a group.

`true`

Same as `"always"`.

`false`

Display no grouping separators.

The default is `"min2"` if `notation` is `"compact"`, and `"auto"` otherwise. The string values `"true"` and `"false"` are accepted, but are always converted to the default value.

`signDisplay`

When to display the sign for the number. Possible values are:

`"auto"` (default)

Sign display for negative numbers only, including negative zero.

`"always"`

Always display sign.

`"exceptZero"`

Sign display for positive and negative numbers, but not zero.

`"negative"`

Sign display for negative numbers only, excluding negative zero.

`"never"`

Never display sign.

Return value

A new `Intl.NumberFormat` object.

> **Note:** The text below describes behavior that is marked by the specification as "optional". It may not work in all environments. Check the browser compatibility table.

Normally, `Intl.NumberFormat()` can be called with or without `new`, and a new `Intl.NumberFormat` instance is returned in both cases. However, if the `this` value is an object that is `instanceof Intl.NumberFormat` (doesn't necessarily mean it's created via `new Intl.NumberFormat`; just that it has `Intl.NumberFormat.prototype` in its prototype chain), then the value of `this` is returned instead, with the newly created `Intl.NumberFormat` object hidden in a `[Symbol(IntlLegacyConstructedSymbol)]` property (a unique symbol that's reused between instances).

```js
const formatter = Intl.NumberFormat.call(
  { __proto__: Intl.NumberFormat.prototype },
  "en-US",
  { notation: "scientific" },
);
console.log(Object.getOwnPropertyDescriptors(formatter));
// {
//   [Symbol(IntlLegacyConstructedSymbol)]: {
//     value: NumberFormat [Intl.NumberFormat] {},
//     writable: false,
//     enumerable: false,
//     configurable: false
//   }
// }
```

Note that there's only one actual `Intl.NumberFormat` instance here: the one hidden in `[Symbol(IntlLegacyConstructedSymbol)]`. Calling the `format()` and `resolvedOptions()` methods on `formatter` would correctly use the options stored in that instance, but calling all other methods (e.g., `formatRange()`) would fail with "TypeError: formatRange method called on incompatible Object", because those methods don't consult the hidden instance's options.

This behavior, called `ChainNumberFormat`, does not happen when `Intl.NumberFormat()` is called without `new` but with `this` set to anything else that's not an `instanceof Intl.NumberFormat`. If you call

it directly as `Intl.NumberFormat()`, the `this` value is `Intl`, and a new `Intl.NumberFormat` instance is created normally.

Exceptions

`RangeError`

Thrown in one of the following cases:

- A property that takes enumerated values (such as `style`, `units`, `currency`, and so on) is set to an invalid value.
- Both `maximumFractionDigits` and `minimumFractionDigits` are set, and they are set to different values. Note that depending on various formatting options, these properties can have default values. It is therefore possible to get this error even if you only set one of the properties.

`TypeError`

Thrown if the `options.style` property is set to "unit" or "currency", and no value has been set for the corresponding property `options.unit` or `options.currency`.

Examples

Basic usage

In basic use without specifying a locale, a formatted string in the default locale and with default options is returned.

```js
const amount = 3500;

console.log(new Intl.NumberFormat().format(amount));
// '3,500' if in US English locale
```

Decimal and percent formatting

```js
const amount = 3500;

new Intl.NumberFormat("en-US", {
  style: "decimal",
}).format(amount); // '3,500'
new Intl.NumberFormat("en-US", {
  style: "percent",
}).format(amount); // '350,000%'
```

Unit formatting

If the `style` is `'unit'`, a `unit` property must be provided. Optionally, `unitDisplay` controls the unit formatting.

```js
const amount = 3500;

new Intl.NumberFormat("en-US", {
  style: "unit",
  unit: "liter",
}).format(amount); // '3,500 L'

new Intl.NumberFormat("en-US", {
  style: "unit",
  unit: "liter",
  unitDisplay: "long",
}).format(amount); // '3,500 liters'
```

Currency formatting

If the `style` is `'currency'`, a `currency` property must be provided. Optionally, `currencyDisplay` and `currencySign` control the unit formatting.

```js
const amount = -3500;
new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
}).format(amount); // '-$3,500.00'

new Intl.NumberFormat("bn", {
  style: "currency",
  currency: "USD",
  currencyDisplay: "name",
}).format(amount); // '-3,500.00 US dollars'

new Intl.NumberFormat("bn", {
  style: "currency",
  currency: "USD",
  currencySign: "accounting",
}).format(amount); // '($3,500.00)'
```

Scientific, engineering or compact notations

Scientific and compact notation are represented by the `notation` option and can be formatted like this:

```js
new Intl.NumberFormat("en-US", {
  notation: "scientific",
}).format(987654321);
// 9.877E8

new Intl.NumberFormat("pt-PT", {
  notation: "scientific",
}).format(987654321);
// 9,877E8

new Intl.NumberFormat("en-GB", {
  notation: "engineering",
}).format(987654321);
// 987.654E6
```

(code continues on the next page)

(code continued from the previous page)

```
new Intl.NumberFormat("de", {
  notation: "engineering",
}).format(987654321);
// 987,654E6

new Intl.NumberFormat("zh-CN", {
  notation: "compact",
}).format(987654321);
// 9.9亿

new Intl.NumberFormat("fr", {
  notation: "compact",
  compactDisplay: "long",
}).format(987654321);
// 988 millions

new Intl.NumberFormat("en-GB", {
  notation: "compact",
  compactDisplay: "short",
}).format(987654321);
// 988M
```

Displaying signs

Display a sign for positive and negative numbers, but not zero:

js

```
new Intl.NumberFormat("en-US", {
  style: "percent",
  signDisplay: "exceptZero",
}).format(0.55);
// '+55%'
```

Note that when the currency sign is "accounting", parentheses might be used instead of a minus sign:

js

```
new Intl.NumberFormat("bn", {
  style: "currency",
  currency: "USD",
  currencySign: "accounting",
  signDisplay: "always",
}).format(-3500);
// '($3,500.00)'
```

FractionDigits, SignificantDigits and IntegerDigits

You can specify the minimum or maximum number of fractional, integer or significant digits to display when formatting a number.

> **Note:** If both significant and fractional digit limits are specified, then the actual formatting depends on the `roundingPriority`.

Using FractionDigits and IntegerDigits

The integer and fraction digit properties indicate the number of digits to display before and after the decimal point, respectively. If the value to display has fewer integer digits than specified, it will be left-padded with zeros to the expected number. If it has fewer fractional digits, it will be right-padded with zeros. Both cases are shown below:

```js
// Formatting adds zeros to display minimum integers and fractions
console.log(
  new Intl.NumberFormat("en", {
    minimumIntegerDigits: 3,
    minimumFractionDigits: 4,
  }).format(4.33),
);
// "004.3300"
```

If a value has more fractional digits than the specified maximum number, it will be rounded. The *way* that it is rounded depends on the `roundingMode` property (more details are provided in the rounding modes section). Below the value is rounded from five fractional digits (`4.33145`) to two (`4.33`):

```js
// Display value shortened to maximum number of digits
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 2,
  }).format(4.33145),
);
// "4.33"
```

The minimum fractional digits have no effect if the value already has more than 2 fractional digits:

```js
// Minimum fractions have no effect if value is higher precision.
console.log(
  new Intl.NumberFormat("en", {
    minimumFractionDigits: 2,
  }).format(4.33145),
);
// "4.331"
```

> **Warning:** Watch out for default values as they may affect formatting even if not specified in your code. The default maximum digit value is `3` for plain values, `2` for currency, and may have different values for other predefined types.

The formatted value above is rounded to 3 digits, even though we didn't specify the maximum digits! This is because a default value of `maximumFractionDigits` is set when we specify `minimumFractionDigits`, and visa versa. The default values of `maximumFractionDigits` and `minimumFractionDigits` are `3` and `0`, respectively.

You can use `resolvedOptions()` to inspect the formatter.

```js
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 2,
  }).resolvedOptions(),
);
// {
//   …
//   minimumIntegerDigits: 1,
//   minimumFractionDigits: 0,
//   maximumFractionDigits: 2,
```

(code continues on the next page)

(code continued from the previous page)

```
//   …
// }

console.log(
  new Intl.NumberFormat("en", {
    minimumFractionDigits: 2,
  }).resolvedOptions(),
);
// {
//   …
//   minimumIntegerDigits: 1,
//   minimumFractionDigits: 2,
//   maximumFractionDigits: 3,
//   …
// }
```

Using SignificantDigits

The number of *significant digits* is the total number of digits including both integer and fractional parts. The `maximumSignificantDigits` is used to indicate the total number of digits from the original value to display.

The examples below show how this works. Note in particular the last case: only the first digit is retained and the others are discarded/set to zero.

js

```
// Display 5 significant digits
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 5,
  }).format(54.33145),
);
// "54.331"

// Max 2 significant digits
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(54.33145),
);
// "54"

// Max 1 significant digits
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 1,
  }).format(54.33145),
);
// "50"
```

The `minimumSignificantDigits` ensures that at least the specified number of digits are displayed, adding zeros to the end of the value if needed.

```js
// Minimum 10 significant digits
console.log(
  new Intl.NumberFormat("en", {
    minimumSignificantDigits: 10,
  }).format(54.33145),
);
// "54.33145000"
```

> **Warning:** Watch out for default values as they may affect formatting. If only one `SignificantDigits` property is used, then its counterpart will automatically be applied with the default value. The default maximum and minimum significant digit values are 21 and 1, respectively.

Specifying significant and fractional digits at the same time

The fraction digits (`minimumFractionDigits`/`maximumFractionDigits`) and significant digits (`minimumSignificantDigits`/`maximumSignificantDigits`) are both ways of controlling how many fractional and leading digits should be formatted. If both are used at the same time, it is possible for them to conflict.

These conflicts are resolved using the `roundingPriority` property. By default, this has a value of `"auto"`, which means that if either `minimumSignificantDigits` or `maximumSignificantDigits` is specified, the fractional and integer digit properties will be ignored.

For example, the code below formats the value of `4.33145` with `maximumFractionDigits: 3`, and then `maximumSignificantDigits: 2`, and then both. The value with both is the one set with `maximumSignificantDigits`.

```js
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 3,
  }).format(4.33145),
);
// "4.331"
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(4.33145),
);
// "4.3"
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 3,
    maximumSignificantDigits: 2,
  }).format(4.33145),
);
// "4.3"
```

Using `resolvedOptions()` to inspect the formatter, we can see that the returned object does not include `maximumFractionDigits` when `maximumSignificantDigits` or `minimumSignificantDigits` are specified.

```js
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 3,
    maximumSignificantDigits: 2,
```

(code continues on the next page)

(code continued from the previous page)

```
  }).resolvedOptions(),
);
// {
//   …
//   minimumIntegerDigits: 1,
//   minimumSignificantDigits: 1,
//   maximumSignificantDigits: 2,
//   …
// }
console.log(
  new Intl.NumberFormat("en", {
    maximumFractionDigits: 3,
    minimumSignificantDigits: 2,
  }).resolvedOptions(),
);
// {
//   …
//   minimumIntegerDigits: 1,
//   minimumSignificantDigits: 2,
//   maximumSignificantDigits: 21,
//   …
// }
```

In addition to `"auto"`, you can resolve conflicts by specifying `roundingPriority` as `"morePrecision"` or `"lessPrecision"`. The formatter calculates the precision using the values of `maximumSignificantDigits` and `maximumFractionDigits`.

The code below shows the format being selected for the three different rounding priorities:

js

```
const maxFracNF = new Intl.NumberFormat("en", {
  maximumFractionDigits: 3,
});
console.log(`maximumFractionDigits:3 - ${maxFracNF.format(1.23456)}`);
// "maximumFractionDigits:2 - 1.235"

const maxSigNS = new Intl.NumberFormat("en", {
  maximumSignificantDigits: 3,
});
console.log(`maximumSignificantDigits:3 - ${maxSigNS.format(1.23456)}`);
// "maximumSignificantDigits:3 - 1.23"

const bothAuto = new Intl.NumberFormat("en", {
  maximumSignificantDigits: 3,
  maximumFractionDigits: 3,
});
console.log(`auto - ${bothAuto.format(1.23456)}`);
// "auto - 1.23"

const bothLess = new Intl.NumberFormat("en", {
  roundingPriority: "lessPrecision",
  maximumSignificantDigits: 3,
  maximumFractionDigits: 3,
});
console.log(`lessPrecision - ${bothLess.format(1.23456)}`);
// "lessPrecision - 1.23"
```

(code continues on the next page)

(code continued from the previous page)

```
const bothMore = new Intl.NumberFormat("en", {
  roundingPriority: "morePrecision",
  maximumSignificantDigits: 3,
  maximumFractionDigits: 3,
});
console.log(`morePrecision - ${bothMore.format(1.23456)}`);
// "morePrecision - 1.235"
```

Note that the algorithm can behave in an unintuitive way if a minimum value is specified without a maximum value. The example below formats the value `1` specifying `minimumFractionDigits: 2` (formatting to `1.00`) and `minimumSignificantDigits: 2` (formatting to `1.0`). Since `1.00` has more digits than `1.0`, this should be the result when prioritizing `morePrecision`, but in fact the opposite is true:

js

```
const bothLess = new Intl.NumberFormat("en", {
  roundingPriority: "lessPrecision",
  minimumFractionDigits: 2,
  minimumSignificantDigits: 2,
});
console.log(`lessPrecision - ${bothLess.format(1)}`);
// "lessPrecision - 1.00"

const bothMore = new Intl.NumberFormat("en", {
  roundingPriority: "morePrecision",
  minimumFractionDigits: 2,
  minimumSignificantDigits: 2,
});
console.log(`morePrecision - ${bothMore.format(1)}`);
// "morePrecision - 1.0"
```

The reason for this is that only the "maximum precision" values are used for the calculation, and the default value of `maximumSignificantDigits` is much higher than `maximumFractionDigits`.

> **Note:** The working group have proposed a modification of the algorithm where the formatter should evaluate the result of using the specified fractional and significant digits independently (taking account of both minimum and maximum values). It will then select the option that displays more fractional digits if `morePrecision` is set, and fewer if `lessPrecision` is set. This will result in more intuitive behavior for this case.

Rounding modes

If a value has more fractional digits than allowed by the constructor options, the formatted value will be *rounded* to the specified number of fractional digits. The *way* in which the value is rounded depends on the `roundingMode` property.

Number formatters use `halfExpand` rounding by default, which rounds values "away from zero" at the half-increment (in other words, the *magnitude* of the value is rounded up).

For a positive number, if the fractional digits to be removed are closer to the next increment (or on the half way point) then the remaining fractional digits will be rounded up, otherwise they are rounded down. This is shown below: 2.23 rounded to two significant digits is truncated to 2.2 because 2.23 is less than the half increment 2.25, while values of 2.25 and greater are rounded up to 2.3:

```js
// Value below half-increment: round down.
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(2.23),
);
// "2.2"

// Value on or above half-increment: round up.
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(2.25),
);
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(2.28),
);
// "2.3"
// "2.3"
```

A negative number on or below the half-increment point is also rounded away from zero (becomes more negative):

```js
// Value below half-increment: round down.
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(-2.23),
);
// "-2.2"

// Value on or above half-increment: round up.
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(-2.25),
);
console.log(
  new Intl.NumberFormat("en", {
    maximumSignificantDigits: 2,
  }).format(-2.28),
);
// "-2.3"
// "-2.3"
```

The table below shows the effect of different rounding modes for positive and negative values that are on and around the half-increment.

rounding mode	2.23	2.25	2.28	-2.23	-2.25	-2.28
`ceil`	2.3	2.3	2.3	-2.2	-2.2	-2.2
`floor`	2.2	2.2	2.2	-2.3	-2.3	-2.3
`expand`	2.3	2.3	2.3	-2.3	-2.3	-2.3
`trunc`	2.2	2.2	2.2	-2.2	-2.2	-2.2

rounding mode	2.23	2.25	2.28	-2.23	-2.25	-2.28
`halfCeil`	2.2	2.3	2.3	-2.2	-2.2	-2.3
`halfFloor`	2.2	2.2	2.3	-2.2	-2.3	-2.3
`halfExpand`	2.2	2.3	2.3	-2.2	-2.3	-2.3
`halfTrunc`	2.2	2.2	2.3	-2.2	-2.2	-2.3
`halfEven`	2.2	2.2	2.3	-2.2	-2.2	-2.3

When using `halfEven`, its behavior also depends on the parity (odd or even) of the last digit of the rounded number. For example, the behavior of `halfEven` in the table above is the same as `halfTrunc`, because the magnitudes of all numbers are between a smaller "even" number (2.2) and a larger "odd" number (2.3). If the numbers are between ±2.3 and ±2.4, `halfEven` will behave like `halfExpand` instead. This behavior avoids consistently under- or over-estimating half-increments in a large data sample.

Using roundingIncrement

Sometimes we want to round the remaining fractional digits to some other increment than the next integer. For example, currencies for which the smallest coin is 5 cents might want to round the value to increments of 5, reflecting amounts that can actually be paid in cash.

This kind of rounding can be achieved with the `roundingIncrement` property.

For example, if `maximumFractionDigits` is 2 and `roundingIncrement` is 5, then the number is rounded to the nearest 0.05:

```js
const nf = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 2,
  roundingIncrement: 5,
});

console.log(nf.format(11.29)); // "$11.30"
console.log(nf.format(11.25)); // "$11.25"
console.log(nf.format(11.22)); // "$11.20"
```

This particular pattern is referred to as "nickel rounding", where nickel is the colloquial name for a USA 5 cent coin. To round to the nearest 10 cents ("dime rounding"), you could change `roundingIncrement` to `10`.

```js
const nf = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 2,
  roundingIncrement: 10,
});

console.log(nf.format(11.29)); // "$11.30"
console.log(nf.format(11.25)); // "$11.30"
console.log(nf.format(11.22)); // "$11.20"
```

You can also use `roundingMode` to change the rounding algorithm. The example below shows how `halfCeil` rounding can be used to round the value "less positive" below the half-rounding increment

and "more positive" if above or on the half-increment. The incremented digit is "0.05" so the half-increment is at .025 (below, this is shown at 11.225).

```js
const nf = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 2,
  roundingIncrement: 5,
  roundingMode: "halfCeil",
});

console.log(nf.format(11.21)); // "$11.20"
console.log(nf.format(11.22)); // "$11.20"
console.log(nf.format(11.224)); // "$11.20"
console.log(nf.format(11.225)); // "$11.25"
console.log(nf.format(11.23)); // "$11.25"
```

If you need to change the number of digits, remember that `minimumFractionDigits` and `maximumFractionDigits` must both be set to the same value, or a `RangeError` is thrown.

`roundingIncrement` cannot be mixed with significant-digits rounding or any setting of `roundingPriority` other than `auto`.

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl-numberformat-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`NumberFormat()` constructor	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
Supports normative optional `ChainNumberFormat` behavior	91[1,2]	91[3,4]	54	77[5,6]	14.1	91[1,2]	56	64[7,8]	14.5	16.0[9,10]	91[1,2]	14.5	1.0.0	1.8	16.0.0[11,12]
`locales` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0[13,14]
`options` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.compactDisplay` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.currencyDisplay` parameter	77	79	42	64	14.1[15,16]	77	56	55	14.5[17,18]	12.0	77	14.5[17,18]	1.0.0	1.8	12.11.0
`options.currencySign` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.currency` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.localeMatcher` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`options.maximumFractionDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.maximumSignificantDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.minimumFractionDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.minimumIntegerDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.minimumSignificantDigits` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.notation` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.numberingSystem` parameter	24	12	76	15	14.1	25	56	14	14.5	1.5	4.4	14.5	1.0.0	1.8	0.12.0
`options.roundingIncrement` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.roundingMode` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.roundingPriority` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.signDisplay` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`negative` value	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.style` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0
`options.trailingZeroDisplay` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.unitDisplay` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.unit` parameter	77	79	78	64	14.1	77	79	55	14.5	12.0	77	14.5	1.0.0	1.8	12.11.0
`options.useGrouping` parameter	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0[13,14]
`options.useGrouping` parameter accepts: `'always'`, `'auto'`, `'min2'` (in addition to: `true` and `false`)	106	106	116	92	16.4	106	116	72	16.4	20.0	106	16.4	1.0.0	**No**	19.0.0

1. Also from version 57–91: partial support.
2. Also from version 57–91: The fallback symbol property has description `IntlFallback`.
3. Also from version 79–91: partial support.
4. Also from version 79–91: The fallback symbol property has description `IntlFallback`.
5. Also from version 44–77: partial support.
6. Also from version 44–77: The fallback symbol property has description `IntlFallback`.
7. Also from version 43–64: partial support.
8. Also from version 43–64: The fallback symbol property has description `IntlFallback`.
9. Also from version 7.0–16.0: partial support.
10. Also from version 7.0–16.0: The fallback symbol property has description `IntlFallback`.
11. Also from version 8.0.0–16.0.0: partial support.
12. Also from version 8.0.0–16.0.0: The fallback symbol property has description `IntlFallback`.
13. Also from version 0.12.0–13.0.0: partial support.
14. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `NumberFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

15. Also from version 10–14.1: partial support.
16. Also from version 10–14.1: Doesn't support `currencyDisplay: 'narrowSymbol'`.
17. Also from version 10–14.5: partial support.
18. Also from version 10–14.5: Doesn't support `currencyDisplay: 'narrowSymbol'`.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.NumberFormat.NumberFormat`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.NumberFormat`
- `Intl.supportedValuesOf()`
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl › Intl.NumberFormat

Intl.NumberFormat.supportedLocalesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat/supportedLocalesOf · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2017.

The **`Intl.NumberFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in number formatting without having to fall back to the runtime's default locale.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.NumberFormat.supportedLocalesOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };

console.log(Intl.NumberFormat.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

js

```
Intl.NumberFormat.supportedLocalesOf(locales)
Intl.NumberFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in number formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in number formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to number formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```js
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.NumberFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl.numberformat.supportedlocalesof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedLocalesOf`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	13.0.0 [1,2]

1. Also from version 0.12.0–13.0.0: partial support.
2. Also from version 0.12.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.NumberFormat.supportedLocalesOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.NumberFormat`

JavaScript › Reference › Standard built-in objects › Intl › Intl.NumberFormat

Intl.NumberFormat.prototype.format()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat/format · MDN page last modified 2026-07-20

Baseline: Widely available — available across major browsers since September 2017.

The **`format()`** method of `Intl.NumberFormat` instances formats a number according to the locale and formatting options of this `Intl.NumberFormat` object.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.NumberFormat.prototype.format()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const amount = 654321.987;

const options1 = { style: "currency", currency: "RUB" };
const numberFormat1 = new Intl.NumberFormat("ru-RU", options1);

console.log(numberFormat1.format(amount));
// Expected output: "654 321,99 ₽"

const options2 = { style: "currency", currency: "USD" };
const numberFormat2 = new Intl.NumberFormat("en-US", options2);

console.log(numberFormat2.format(amount));
// Expected output: "$654,321.99"
```

Syntax

js

```
format(number)
```

Parameters

`number`

A `Number`, `BigInt`, or string, to format. Strings are parsed in the same way as in number conversion, except that `format()` will use the exact value that the string represents, avoiding loss of precision during implicitly conversion to a number.

> **Note:** Older versions of the specification parsed strings as a `Number`. Check the compatibility table for your browser.

Return value

A string representing the given `number` formatted according to the locale and formatting options of this `Intl.NumberFormat` object.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Description

`Number` values in JavaScript suffer from loss of precision if they are too big or too small, making the text representation inaccurate. If you are performing calculations with integers larger than `Number.MAX_SAFE_INTEGER` you should use a `BigInt` instead, which will format correctly:

```js
new Intl.NumberFormat("en-US").format(1234567891234567891); // 1,234,567,891,234,568,000
new Intl.NumberFormat("en-US").format(1234567891234567891n); // 1,234,567,891,234,567,891
```

You can also pass through very large strings to be formatted as an arbitrary-precision decimal string (if you're performing calculations on the data you will still need to work with `BigInt`):

```js
new Intl.NumberFormat("en-US").format("1234567891234567891"); // 1,234,567,891,234,567,891
```

Examples

Using format

Use the `format` getter function for formatting a single currency value. The code below shows how to format the roubles currency for a Russian locale:

```js
const options = { style: "currency", currency: "RUB" };
const numberFormat = new Intl.NumberFormat("ru-RU", options);
console.log(numberFormat.format(654321.987));
// "654 321,99 ₽"
```

Using format with map

Use the `format` getter function for formatting all numbers in an array. Note that the function is bound to the `Intl.NumberFormat` from which it was obtained, so it can be passed directly to `Array.prototype.map`. This is considered a historical artifact, as part of a convention which is no longer followed for new features, but is preserved to maintain compatibility with existing programs.

```js
const a = [123456.789, 987654.321, 456789.123];
const numberFormat = new Intl.NumberFormat("es-ES");
const formatted = a.map((n) => numberFormat.format(n));
console.log(formatted.join("; "));
// "123.456,789; 987.654,321; 456.789,123"
```

Using format with a string

Using a string we can specify numbers that are larger than `Number.MAX_SAFE_INTEGER` without losing precision.

```js
const numberFormat = new Intl.NumberFormat("en-US");

// Here the value is converted to a Number
console.log(numberFormat.format(987654321987654321));
// 987,654,321,987,654,300

// Here we use a string and don't lose precision
console.log(numberFormat.format("987654321987654321"));
// 987,654,321,987,654,321
```

We can also use the general "E" exponent syntax for decimal strings: `#.#E#`. The code below creates a `BigInt`, coerces it to a string with the suffix `E-6`, and then formats it.

```js
const numberFormat = new Intl.NumberFormat("en-US");
const bigNum = 10000000000000000110000n;
console.log(numberFormat.format(bigNum));
// "1,000,000,000,000,000,110,000"

// Format as a string using the `E` syntax:
console.log(numberFormat.format(`${bigNum}E-6`));
// "1,000,000,000,000,000.11"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.numberformat.prototype.format`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`format`	24	12[1]	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0[2]
`number` param string value is decimal (not `Number`)	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	1.25	19.0.0

1. Before Edge 18, numbers are rounded to 15 decimal digits. For example, `new Intl.NumberFormat('en-US').format(1000000000000005)` returns `"1,000,000,000,000,010"`.
2. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `NumberFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.NumberFormat.format`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.NumberFormat`
- `Number.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.NumberFormat

Intl.NumberFormat.prototype.formatRange()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat/formatRange · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since August 2023.

The **`formatRange()`** method of `Intl.NumberFormat` instances formats a range of numbers according to the locale and formatting options of this `Intl.NumberFormat` object.

Syntax

```js
formatRange(startRange, endRange)
```

Parameters

`startRange`

A `Number`, `BigInt`, or string, to format. Strings are parsed in the same way as in number conversion, except that `formatRange()` will use the exact value that the string represents, avoiding loss of precision during implicitly conversion to a number.

`endRange`

A `Number`, `BigInt`, or string, to format.

Return value

A string representing the given range of numbers formatted according to the locale and formatting options of this `Intl.NumberFormat` object. If the start and end values are formatted to the same string, the output will only contain a single value, possibly prefixed with an "approximately equals" symbol (e.g., "~$3"). The insertion of this symbol only depends on the locale settings, and is inserted even when `startRange === endRange`.

Exceptions

`RangeError`

Thrown if either `startRange` or `endRange` is `NaN` or an inconvertible string.

`TypeError`

Thrown if either `startRange` or `endRange` is undefined.

Description

The `formatRange` getter function formats a range of numbers into a string according to the locale and formatting options of this `Intl.NumberFormat` object from which it is called.

Examples

Using formatRange

Use the `formatRange` getter function for formatting a range of currency values:

```js
const nf = new Intl.NumberFormat("en-US", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 0,
});

console.log(nf.formatRange(3, 5)); // "$3 – $5"

// Note: the "approximately equals" symbol is added if
// startRange and endRange round to the same values.
console.log(nf.formatRange(2.9, 3.1)); // "~$3"
```

```js
const nf = new Intl.NumberFormat("es-ES", {
  style: "currency",
  currency: "EUR",
  maximumFractionDigits: 0,
});

console.log(nf.formatRange(3, 5)); // "3-5 €"
console.log(nf.formatRange(2.9, 3.1)); // "~3 €"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.numberformat.prototype.formatrange`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`formatRange`	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	19.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.NumberFormat.formatRange`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.NumberFormat`
- `Number.prototype.toLocaleString()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.NumberFormat

Intl.NumberFormat.prototype.formatRangeToParts()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat/formatRangeToParts · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since August 2023.

The **`formatRangeToParts()`** method of `Intl.NumberFormat` instances returns an `Array` of objects containing the locale-specific tokens from which it is possible to build custom strings while preserving the locale-specific parts. This makes it possible to provide locale-aware custom formatting ranges of number strings.

Syntax

```js
formatRangeToParts(startRange, endRange)
```

Parameters

`startRange`

A `Number`, `BigInt`, or string, to format. Strings are parsed in the same way as in number conversion, except that `formatRangeToParts()` will use the exact value that the string represents, avoiding loss of precision during implicitly conversion to a number.

`endRange`

A `Number`, `BigInt`, or string, to format.

Return value

An `Array` of objects containing the formatted range in parts. Each object has three properties, `type`, `value`, and `source`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `formatRange()`. The `type` may have the same values as `formatToParts()`, or the additional value `"approximatelySign"` (see below). The `source` can be one of the following:

`startRange`

The token is a part of the start number.

`endRange`

The token is a part of the end number.

`shared`

The token is shared between the start and end; for example, the currency symbol. All literals that are part of the range pattern itself, such as the `"–"` separator, are also marked as `shared`.

If the start and end numbers are formatted to the same string, then the output has the same list of tokens as calling `formatToParts()` on the start number, with all tokens marked as `source: "shared"`. In addition, the first token may be an "approximately equals" symbol (e.g., "∼") with `type: "approximatelySign"`. The insertion of this symbol only depends on the locale settings, and is inserted even when `startRange === endRange`.

Exceptions

`RangeError`

Thrown if either `startRange` or `endRange` is `NaN` or an inconvertible string.

`TypeError`

Thrown if either `startRange` or `endRange` is undefined.

Examples

Using formatRangeToParts()

The `formatRange()` method outputs localized, opaque strings that cannot be manipulated directly:

```js
const startRange = 3500;
const endRange = 9500;

const formatter = new Intl.NumberFormat("de-DE", {
  style: "currency",
  currency: "EUR",
});

console.log(formatter.formatRange(startRange, endRange));
// "3.500,00–9.500,00 €"
```

However, in many user interfaces you may want to customize the formatting of this string, or interleave it with other texts. The `formatRangeToParts()` method produces the same information in parts:

```js
console.log(formatter.formatRangeToParts(startRange, endRange));

// return value:
[
  { type: "integer", value: "3", source: "startRange" },
  { type: "group", value: ".", source: "startRange" },
  { type: "integer", value: "500", source: "startRange" },
  { type: "decimal", value: ",", source: "startRange" },
  { type: "fraction", value: "00", source: "startRange" },
  { type: "literal", value: "–", source: "shared" },
  { type: "integer", value: "9", source: "endRange" },
  { type: "group", value: ".", source: "endRange" },
  { type: "integer", value: "500", source: "endRange" },
  { type: "decimal", value: ",", source: "endRange" },
  { type: "fraction", value: "00", source: "endRange" },
  { type: "literal", value: " ", source: "shared" },
  { type: "currency", value: "€", source: "shared" },
];
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.numberformat.prototype.formatrangetoparts`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
formatRangeToParts	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	19.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.NumberFormat.formatRangeToParts`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.NumberFormat`
- `Intl.NumberFormat.prototype.format()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.NumberFormat

Intl.NumberFormat.prototype.formatToParts()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat/formatToParts · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2019.

The **`formatToParts()`** method of `Intl.NumberFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.NumberFormat.prototype.formatToParts()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const amount = 654321.987;
const options = { style: "currency", currency: "USD" };
const numberFormat = new Intl.NumberFormat("en-US", options);

const parts = numberFormat.formatToParts(amount);
const partValues = parts.map((p) => p.value);

console.log(partValues);
// Expected output: "["$", "654", ",", "321", ".", "99"]"
```

Syntax

js

```
formatToParts(number)
```

Parameters

`number`

A `Number`, `BigInt`, or string, to format. Strings are parsed in the same way as in number conversion, except that `formatToParts()` will use the exact value that the string represents, avoiding loss of precision during implicitly conversion to a number.

Return value

An `Array` of objects containing the formatted number in parts. Each object has two properties, `type` and `value`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The `type` may be one of the following:

`literal`

Any string that's a part of the format pattern; for example `" "`. Note that common tokens like the decimal separator or the plus/minus signs have their own token types.

`integer`

The integral part of the number, or a segment of it if using grouping (controlled by `options.useGrouping`).

`group`

The group separator string, such as `","`. Only present when using grouping (controlled by `options.useGrouping`).

`decimal`

The decimal separator string, such as `"."`. Only present when `fraction` is present.

`fraction`

The fractional part of the number.

`compact`

The compact exponent, such as `"M"` or `"thousands"`. Only present when `options.notation` is `"compact"`. The form (`"short"` or `"long"`) can be controlled via `options.compactDisplay`.

`exponentSeparator`

The exponent separator, such as `"E"`. Only present when `options.notation` is `"scientific"` or `"engineering"`.

`exponentMinusSign`

The exponent minus sign string, such as `"-"`. Only present when `options.notation` is `"scientific"` or `"engineering"` and the exponent is negative.

`exponentInteger`

The exponent's integer value. Only present when `options.notation` is `"scientific"` or `"engineering"`.

`nan`

A string representing `NaN`, such as `"NaN"`. This is the sole token representing the number itself when the number is `NaN`.

`infinity`

A string representing `Infinity` or `-Infinity`, such as `"∞"`. This is the sole token representing the number itself when the number is `Infinity` or `-Infinity`.

`plusSign`

The plus sign, such as `"+"`.

`minusSign`

The minus sign, such as `"-"`.

`percentSign`

The percent sign, such as `"%"`. Only present when `options.style` is `"percent"`.

`unit`

The unit string, such as `"l"` or `"litres"`. Only present when `options.style` is `"unit"`. The form (`"short"`, `"narrow"`, or `"long"`) can be controlled via `options.unitDisplay`.

`currency`

The currency string, such as `"$"`, `"€"`, `"Dollar"`, or `"Euro"`. Only present when `options.style` is `"currency"`. The form (`"code"`, `"symbol"`, `"narrowSymbol"`, or `"name"`) can be controlled via `options.currencyDisplay`.

`unknown`

Reserved for any token that's not recognized as one of the above; should be rarely encountered.

Examples

Using formatToParts()

The `format()` method outputs localized, opaque strings that cannot be manipulated directly:

```js
const number = 3500;

const formatter = new Intl.NumberFormat("de-DE", {
  style: "currency",
  currency: "EUR",
});

formatter.format(number);
// "3.500,00 €"
```

However, in many user interfaces you may want to customize the formatting of this string, or interleave it with other texts. The `formatToParts()` method produces the same information in parts:

```js
formatter.formatToParts(number);

// return value:
[
  { type: "integer", value: "3" },
  { type: "group", value: "." },
  { type: "integer", value: "500" },
  { type: "decimal", value: "," },
  { type: "fraction", value: "00" },
  { type: "literal", value: " " },
  { type: "currency", value: "€" },
];
```

Now the information is available separately and it can be formatted and concatenated again in a customized way. For example by using `Array.prototype.map()`, arrow functions, a switch statement, template literals, and `Array.prototype.join()`, to insert additional markup for certain components.

```js
const numberString = formatter
  .formatToParts(number)
  .map(({ type, value }) => {
    switch (type) {
      case "currency":
        return `<strong>${value}</strong>`;
      default:
        return value;
    }
  })
  .join("");

console.log(numberString);
// "3.500,00 <strong>€</strong>"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl.numberformat.prototype.formattoparts

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
formatToParts	64	12	58	51	13	64	58	47	13	9.0	64	13	1.0.0	1.8	10.0.0 [1]

1. Before version 13.0.0, only the locale data for en-US is available by default. See the NumberFormat() constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.NumberFormat.formatToParts`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.NumberFormat`
- `Intl.NumberFormat.prototype.format()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.NumberFormat

Intl.NumberFormat.prototype.resolvedOptions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/NumberFormat/resolvedOptions · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2017.

The **`resolvedOptions()`** method of `Intl.NumberFormat` instances returns a new object with properties reflecting the options computed during initialization of this `NumberFormat` object.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.NumberFormat.prototype.resolvedOptions()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const numberFormat = new Intl.NumberFormat("de-DE");
const options = numberFormat.resolvedOptions();

console.log(options.locale);
// Expected output: "de-DE"

console.log(options.numberingSystem);
// Expected output: "latn"

console.log(options.style);
// Expected output: "decimal"
```

Syntax

js

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `NumberFormat` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `nu` Unicode extension key, if requested, may be included in the output.

`numberingSystem`

The value provided for this property in the `options` argument, or using the Unicode extension key `"nu"`, with default filled in as needed. It is a supported numbering system for this locale. The default is locale dependent.

`style`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"decimal"`, `"percent"`, `"currency"`, or `"unit"`. The default is `"decimal"`.

`currency` Optional

The value provided for this property in the `options` argument. It is only present if `style` is `"currency"`. It is an ISO 4217 currency code; see `Intl.supportedValuesOf()`. It is required if `style` is `"currency"` so there is no default.

`currencyDisplay` Optional

The value provided for this property in the `options` argument, with default filled in as needed. It is only present if `style` is `"currency"`. It is either `"code"`, `"symbol"`, `"narrowSymbol"`, or `"name"`. The default is `"symbol"`.

`currencySign` Optional

The value provided for this property in the `options` argument, with default filled in as needed. It is only present if `style` is `"currency"`. It is either `"standard"` or `"accounting"`. The default is `"standard"`.

`unit` Optional

The value provided for this property in the `options` argument. It is only present if `style` is `"unit"`. It is a sanctioned unit identifier from the full CLDR list. It is required if `style` is `"unit"` so there is no default.

`unitDisplay` Optional

The value provided for this property in the `options` argument, with default filled in as needed. It is only present if `style` is `"unit"`. It is either `"short"`, `"narrow"`, or `"long"`. The default is `"short"`.

`minimumIntegerDigits`

The value provided for this property in the `options` argument, with default filled in as needed. It is an integer between `1` and `21`. The default is `1`.

`minimumFractionDigits`, `maximumFractionDigits` Optional

The value provided for these properties in the `options` argument, with defaults filled in as needed. They are only present if necessary; see digit options. It is an integer between `0` and `100`.

`minimumSignificantDigits`, `maximumSignificantDigits` Optional

The value provided for these properties in the `options` argument, with defaults filled in as needed. They are only present if necessary; see digit options. It is an integer between `1` and `21`.

`useGrouping`

The value provided for this property in the `options` argument, with default filled in as needed, and with some values normalized. It is either `"always"`, `"auto"`, `"min2"`, or the boolean `false`. The default is `"min2"` if `notation` is `"compact"`, and `"auto"` otherwise.

`notation`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"standard"`, `"scientific"`, `"engineering"`, or `"compact"`. The default is `"standard"`.

`compactDisplay` Optional

The value provided for this property in the `options` argument, with default filled in as needed. It is only present if `notation` is `"compact"`. It is either `"short"` or `"long"`. The default is `"short"`.

`signDisplay`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"`, `"always"`, `"exceptZero"`, `"negative"`, or `"never"`. The default is `"auto"`.

`roundingIncrement`

The value provided for this property in the `options` argument, with default filled in as needed. It is one of `1`, `2`, `5`, `10`, `20`, `25`, `50`, `100`, `200`, `250`, `500`, `1000`, `2000`, `2500`, and `5000`. The default is `1`.

`roundingMode`

The value provided for this property in the `options` argument, with default filled in as needed. It is one of `"ceil"`, `"floor"`, `"expand"`, `"trunc"`, `"halfCeil"`, `"halfFloor"`, `"halfExpand"`, `"halfTrunc"`, and `"halfEven"`. The default is `"halfExpand"`.

`roundingPriority`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"`, `"morePrecision"`, or `"lessPrecision"`. The default is `"auto"`.

`trailingZeroDisplay`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"` or `"stripIfInteger"`. The default is `"auto"`.

Examples

Using the `resolvedOptions` method

```js
// Create a NumberFormat
const de = new Intl.NumberFormat("de-DE", {
  style: "currency",
  currency: "USD",
  maximumFractionDigits: 2,
  roundingIncrement: 5,
  roundingMode: "halfCeil",
});

// Resolve the options
const usedOptions = de.resolvedOptions();
console.log(usedOptions.locale); // "de-DE"
console.log(usedOptions.numberingSystem); // "latn"
console.log(usedOptions.compactDisplay); // undefined ("notation" not set to "compact")
console.log(usedOptions.currency); // "USD"
console.log(usedOptions.currencyDisplay); // "symbol"
console.log(usedOptions.currencySign); // "standard"
console.log(usedOptions.minimumIntegerDigits); // 1
console.log(usedOptions.minimumFractionDigits); // 2
console.log(usedOptions.maximumFractionDigits); // 2
console.log(usedOptions.minimumSignificantDigits); // undefined (maximumFractionDigits is
set)
console.log(usedOptions.maximumSignificantDigits); // undefined (maximumFractionDigits is
set)
console.log(usedOptions.notation); // "standard"
console.log(usedOptions.roundingIncrement); // 5
console.log(usedOptions.roundingMode); // halfCeil
console.log(usedOptions.roundingPriority); // auto
console.log(usedOptions.signDisplay); // "auto"
console.log(usedOptions.style); // "currency"
console.log(usedOptions.trailingZeroDisplay); // auto
console.log(usedOptions.useGrouping); // auto
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl.numberformat.prototype.resolvedoptions

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resolvedOptions`	24	12	29	15	10	25	56	14	10	1.5	4.4	10	1.0.0	1.8	0.12.0 [1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `NumberFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.NumberFormat.resolvedOptions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.NumberFormat`

JavaScript › Reference › Standard built-in objects › Intl

Intl.PluralRules

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/PluralRules · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2019.

The **`Intl.PluralRules`** object enables plural-sensitive formatting and plural-related language rules.

Description

Languages use different patterns for expressing both plural numbers of items (cardinal numbers) and for expressing the order of items (ordinal numbers). English has two forms for expressing cardinal numbers: one for the singular "item" (1 hour, 1 dog, 1 fish) and the other for zero or any other number of "items" (0 hours, 2 lemmings, 100000.5 fish), while Chinese has only one form, and Arabic has six! Similarly, English has four forms for expressing ordinal numbers: "th", "st", "nd", "rd", giving the sequence: 0th, 1st, 2nd, 3rd, 4th, 5th, ..., 21st, 22nd, 23rd, 24th, 25th, and so on, while both Chinese and Arabic only have one form for ordinal numbers.

Given a particular language and set of formatting options, the methods `Intl.PluralRules.prototype.select()` and `Intl.PluralRules.prototype.selectRange()` return a *tag* that represents the plural form of a single or a range of numbers, cardinal or ordinal. Code can use the returned tags to represent numbers appropriately for the given language. The full set of tags that might be returned are: `zero`, `one`, `two`, `few`, `many`, and `other` (the "general" plural form, also used if the language only has one form).

As English only has two forms for cardinal numbers, the `select()` method returns only two tags: `"one"` for the singular case, and `"other"` for all other cardinal numbers. This allows construction of sentences that make sense in English for each case, such as: "1 dog is happy; do you want to play with it?" and "10 dogs are happy; do you want to play with them?".

Creating appropriate sentences for each form depends on the language, and even in English may not be as simple as just adding "s" to a noun to make the plural form. Using the example above, we see that the form may affect:

- **Nouns**: 1 dog, 2 dogs (but not "fish" or "sheep", which have the same singular and plural form).
- **Verbs**: 1 dog *is* happy, 2 dogs *are* happy.
- **Pronouns** (and other referents): Do you want to play with *it* / *them*.

Other languages have more forms, and choosing appropriate sentences can be even more complex.

`select()` can return any of four tags for ordinal numbers in English, representing each of the allowed forms: `one` for "st" numbers (1, 21, 31, ...), `two` for "nd" numbers (2, 22, 32, ...), `few` for "rd" numbers (3, 33, 43, ...), and `other` for "th" numbers (0, 4-20, etc.). Again, the returned tags allow appropriate formatting of strings describing an ordinal number.

For more information about the rules and how they are used, see Plural Rules. For a list of the rules and how they apply for different languages, see the LDML Language Plural Rules.

Constructor

`Intl.PluralRules()`

Creates a new `Intl.PluralRules` object.

Static methods

`Intl.PluralRules.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.PluralRules.prototype` and shared by all `Intl.PluralRules` instances.

`Intl.PluralRules.prototype.constructor`

The constructor function that created the instance object. For `Intl.PluralRules` instances, the initial value is the `Intl.PluralRules` constructor.

`Intl.PluralRules.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.PluralRules"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.PluralRules.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and collation options computed during initialization of the object.

`Intl.PluralRules.prototype.select()`

Returns a string indicating which plural rule to use for locale-aware formatting.

`Intl.PluralRules.prototype.selectRange()`

This method receives two values and returns a string indicating which plural rule to use for locale-aware formatting.

Examples

Using locales

This example shows some of the variations in localized plural rules for cardinal numbers.

In order to get the format for the language used in the user interface of your application, make sure to specify that language (and possibly some fallback languages) using the constructor `locales` argument:

```js
// US English
const enCardinalRules = new Intl.PluralRules("en-US");
console.log(enCardinalRules.select(0)); // "other"
console.log(enCardinalRules.select(1)); // "one"
console.log(enCardinalRules.select(2)); // "other"
console.log(enCardinalRules.select(3)); // "other"

// Arabic
const arCardinalRules = new Intl.PluralRules("ar-EG");
console.log(arCardinalRules.select(0)); // "zero"
console.log(arCardinalRules.select(1)); // "one"
console.log(arCardinalRules.select(2)); // "two"
console.log(arCardinalRules.select(6)); // "few"
console.log(arCardinalRules.select(18)); // "many"
```

Using options

The plural form of the specified number may also depend on constructor `options`, such as how the number is rounded, and whether it is cardinal or ordinal.

This example shows how you can set the type of rules to "ordinal", and how this affects the form for some numbers in US English.

```js
// US English - ordinal
const enOrdinalRules = new Intl.PluralRules("en-US", { type: "ordinal" });
console.log(enOrdinalRules.select(0)); // "other" (0th)
console.log(enOrdinalRules.select(1)); // "one"   (1st)
console.log(enOrdinalRules.select(2)); // "two"   (2nd)
console.log(enOrdinalRules.select(3)); // "few"   (3rd)
console.log(enOrdinalRules.select(4)); // "other" (4th)
console.log(enOrdinalRules.select(21)); // "one"  (21st)
```

Formatting text using the returned tag

The code below extends the previous example, showing how you might use the returned tag for an ordinal number to format text in English.

```js
const enOrdinalRules = new Intl.PluralRules("en-US", { type: "ordinal" });

const suffixes = new Map([
  ["one", "st"],
  ["two", "nd"],
  ["few", "rd"],
  ["other", "th"],
]);
const formatOrdinals = (n) => {
  const rule = enOrdinalRules.select(n);
  const suffix = suffixes.get(rule);
  return `${n}${suffix}`;
};
```

(code continues on the next page)

(code continued from the previous page)

```
formatOrdinals(0); // '0th'
formatOrdinals(1); // '1st'
formatOrdinals(2); // '2nd'
formatOrdinals(3); // '3rd'
formatOrdinals(4); // '4th'
formatOrdinals(11); // '11th'
formatOrdinals(21); // '21st'
formatOrdinals(42); // '42nd'
formatOrdinals(103); // '103rd'
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#pluralrules-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PluralRules`	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [1]
`PluralRules()` constructor	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	13.0.0 [2,3]
`options` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.roundingIncrement` parameter	**No**	**No**	116	**No**	17.2	**No**	116	**No**	17.2	**No**	**No**	17.2	1.0.3	**No**	**No**
`options.roundingMode` parameter	117	117	116	103	17.2	117	116	78	17.2	24.0	117	17.2	1.0.3	**No**	21.0.0
`options.roundingPriority` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.trailingZeroDisplay` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	**No**
`resolvedOptions`	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [1]
`select`	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [1]
`selectRange`	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	19.0.0
`supportedLocalesOf`	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `PluralRules()` constructor for more details.
2. Also from version 10.0.0–13.0.0: partial support.
3. Also from version 10.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `PluralRules` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.PluralRules`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.PluralRules` in FormatJS
- `Intl`

Intl.PluralRules() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/PluralRules/PluralRules · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2019.

The **`Intl.PluralRules()`** constructor creates `Intl.PluralRules` objects.

Syntax

```js
new Intl.PluralRules()
new Intl.PluralRules(locales)
new Intl.PluralRules(locales, options)
```

> **Note:** `Intl.PluralRules()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`type`

The type to use. Possible values are:

`"cardinal"` (default)

For cardinal numbers (referring to the quantity of things).

`"ordinal"`

For ordinal number (referring to the ordering or ranking of things, e.g., "1st", "2nd", "3rd" in English).

`Intl.PluralRules` also supports the `Intl.NumberFormat()` digit options (see `Intl.NumberFormat()` for details):

- `minimumIntegerDigits`

- `minimumFractionDigits`
- `maximumFractionDigits`
- `minimumSignificantDigits`
- `maximumSignificantDigits`
- `roundingPriority`
- `roundingIncrement`
- `roundingMode`

These options are interpreted as if the `notation` option from `Intl.NumberFormat` is `"standard"` and `style` is `"decimal"`.

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Basic usage

In basic use without specifying a locale, a formatted string in the default locale and with default options is returned. This is useful to distinguish between singular and plural forms, e.g., "dog" and "dogs".

```js
const pr = new Intl.PluralRules();

pr.select(0); // 'other' if in US English locale

pr.select(1); // 'one' if in US English locale

pr.select(2); // 'other' if in US English locale
```

Using options

The results can be customized using the `options` argument, which has one property called `type` which you can set to `ordinal`. This is useful to figure out the ordinal indicator, e.g., "1st", "2nd", "3rd", "4th", "42nd", and so forth.

```js
const pr = new Intl.PluralRules("en-US", { type: "ordinal" });

const suffixes = new Map([
  ["one", "st"],
  ["two", "nd"],
  ["few", "rd"],
  ["other", "th"],
]);
const formatOrdinals = (n) => {
  const rule = pr.select(n);
  const suffix = suffixes.get(rule);
  return `${n}${suffix}`;
};
```

(code continues on the next page)

(code continued from the previous page)

```
formatOrdinals(0); // '0th'
formatOrdinals(1); // '1st'
formatOrdinals(2); // '2nd'
formatOrdinals(3); // '3rd'
formatOrdinals(4); // '4th'
formatOrdinals(11); // '11th'
formatOrdinals(21); // '21st'
formatOrdinals(42); // '42nd'
formatOrdinals(103); // '103rd'
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl-pluralrules-constructor`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`PluralRules()` constructor	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	13.0.0 [1,2]
`options` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.roundingIncrement` parameter	**No**	**No**	116	**No**	17.2	**No**	116	**No**	17.2	**No**	**No**	17.2	1.0.3	**No**	**No**
`options.roundingMode` parameter	117	117	116	103	17.2	117	116	78	17.2	24.0	117	17.2	1.0.3	**No**	21.0.0
`options.roundingPriority` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.0	**No**	19.0.0
`options.trailingZeroDisplay` parameter	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	**No**

1. Also from version 10.0.0–13.0.0: partial support.
2. Also from version 10.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `PluralRules` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.PluralRules.PluralRules`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.PluralRules`
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl › Intl.PluralRules

Intl.PluralRules.supportedLocalesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/PluralRules/supportedLocalesOf · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2019.

The **`Intl.PluralRules.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in plural rules without having to fall back to the runtime's default locale.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.PluralRules.supportedLocalesOf()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const locales = ["en-US", "ban", "ar-OM", "de-DE"];
const options = { localeMatcher: "lookup" };

console.log(Intl.PluralRules.supportedLocalesOf(locales, options));
// Expected output: Array ["en-US", "ar-OM", "de-DE"]
```

Syntax

js

```
Intl.PluralRules.supportedLocalesOf(locales)
Intl.PluralRules.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in plural rules without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in plural rules, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to plural rules nor used with Indonesian, and a specialized German for

Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```js
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.PluralRules.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl.pluralrules.supportedlocalesof

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
supportedLocalesOf	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [1]

1. Before version 13.0.0, only the locale data for en-US is available by default. See the PluralRules() constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.PluralRules.supportedLocalesOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.PluralRules`

JavaScript › Reference › Standard built-in objects › Intl › Intl.PluralRules

Intl.PluralRules.prototype.resolvedOptions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/PluralRules/resolvedOptions · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2019.

The **`resolvedOptions()`** method of `Intl.PluralRules` instances returns a new object with properties reflecting the options computed during initialization of this `PluralRules` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.PluralRules.prototype.resolvedOptions()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const pluralRules1 = new Intl.PluralRules("uk");
const options1 = pluralRules1.resolvedOptions();

const pluralRules2 = new Intl.PluralRules("bn");
const options2 = pluralRules2.resolvedOptions();

console.log(options1.pluralCategories);
// Expected output: Array ["few", "many", "one", "other"]

console.log(options2.pluralCategories);
// Expected output: Array ["one", "other"]
```

Syntax

js

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `PluralRules` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. No Unicode extension key will be included in the output.

`type`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"cardinal"` or `"ordinal"`. The default is `"cardinal"`.

`minimumIntegerDigits`, `minimumFractionDigits`, `maximumFractionDigits` Optional

The value provided for these properties in the `options` argument, with defaults filled in as needed. These properties are present only if neither `minimumSignificantDigits` nor `maximumSignificantDigits` was provided in the `options` argument.

`minimumSignificantDigits`, `maximumSignificantDigits` Optional

The value provided for these properties in the `options` argument, with defaults filled in as needed. These properties are present only if at least one of them was provided in the `options` argument.

`pluralCategories`

An `Array` of plural categories used by the given locale, selected from the list `"zero"`, `"one"`, `"two"`, `"few"`, `"many"` and `"other"`.

`roundingIncrement`

The value provided for this property in the `options` argument, with default filled in as needed. It is one of `1`, `2`, `5`, `10`, `20`, `25`, `50`, `100`, `200`, `250`, `500`, `1000`, `2000`, `2500`, and `5000`. The default is `1`.

`roundingMode`

The value provided for this property in the `options` argument, with default filled in as needed. It is one of `"ceil"`, `"floor"`, `"expand"`, `"trunc"`, `"halfCeil"`, `"halfFloor"`, `"halfExpand"`, `"halfTrunc"`, and `"halfEven"`. The default is `"halfExpand"`.

`roundingPriority`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"`, `"morePrecision"`, or `"lessPrecision"`. The default is `"auto"`.

`trailingZeroDisplay`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"auto"` or `"stripIfInteger"`. The default is `"auto"`.

Examples

Using the resolvedOptions() method

The code below shows the construction of a `PluralRules` object, followed by logging of each of the resolved options.

```js
// Create a PluralRules instance
const de = new Intl.PluralRules("de-DE", {
  maximumSignificantDigits: 2,
  trailingZeroDisplay: "auto",
});

// Resolve the options
const usedOptions = de.resolvedOptions();
console.log(usedOptions.locale); // "de-DE"
console.log(usedOptions.pluralCategories); // Array ["one", "other"]
console.log(usedOptions.type); // "cardinal"
console.log(usedOptions.minimumIntegerDigits); // 1
console.log(usedOptions.minimumFractionDigits); // undefined (maximumSignificantDigits is
set)
console.log(usedOptions.maximumFractionDigits); // undefined (maximumSignificantDigits is
set)
console.log(usedOptions.minimumSignificantDigits); // 1
console.log(usedOptions.maximumSignificantDigits); // 2
console.log(usedOptions.roundingIncrement); // 1
console.log(usedOptions.roundingMode); // "halfExpand"
console.log(usedOptions.roundingPriority); // "auto"
console.log(usedOptions.trailingZeroDisplay); // "auto"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.pluralrules.prototype.resolvedoptions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resolvedOptions`	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `PluralRules()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.PluralRules.resolvedOptions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.PluralRules`

JavaScript › Reference › Standard built-in objects › Intl › Intl.PluralRules

Intl.PluralRules.prototype.select()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/PluralRules/select · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2019.

The **`select()`** method of `Intl.PluralRules` instances returns a string indicating which plural rule to use for locale-aware formatting of a number.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.PluralRules.prototype.select()". It runs live in the browser there; its source code follows.

js · interactive example source

```
console.log(new Intl.PluralRules("ar-EG").select(0));
// Expected output: "zero"

console.log(new Intl.PluralRules("ar-EG").select(5));
// Expected output: "few"

console.log(new Intl.PluralRules("ar-EG").select(55));
// Expected output: "many"

console.log(new Intl.PluralRules("en").select(0));
// Expected output: "other"
```

Syntax

js

```
select(number)
```

Parameters

`number`

The number to get a plural rule for.

Return value

A string representing the pluralization category of the `number`. This can be one of `zero`, `one`, `two`, `few`, `many`, or `other`.

Description

This function selects a pluralization category according to the locale and formatting options of an `Intl.PluralRules` object. These options are set in the `Intl.PluralRules()` constructor.

Examples

Using select()

First, create an `Intl.PluralRules` object, passing the appropriate `locales` and `options` parameters. Here we create a plural rules object for Arabic in the Egyptian dialect. Because the `type` is not specified the rules object will provide formatting for cardinal numbers (the default).

```
js
const pr = new Intl.PluralRules("ar-EG");
```

Then call `select()` on the rules object, specifying the number for which the plural form is required. Note that Arabic has 5 forms for cardinal numbers, as shown.

```
js
pr.select(0); // 'zero'
pr.select(1); // 'one'
pr.select(2); // 'two'
pr.select(6); // 'few'
pr.select(18); // 'many'
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.pluralrules.prototype.select`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`select`	63	18	58	50	13	63	58	46	13	8.0	63	13	1.0.0	1.8	10.0.0 [1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `PluralRules()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.PluralRules.select`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.PluralRules`

JavaScript › Reference › Standard built-in objects › Intl › Intl.PluralRules

Intl.PluralRules.prototype.selectRange()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/PluralRules/selectRange · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since August 2023.

The **`selectRange()`** method of `Intl.PluralRules` instances receives two values and returns a string indicating which plural rule to use for locale-aware formatting of the indicated range.

Syntax

```js
selectRange(startRange, endRange)
```

Parameters

`startRange`

A number representing the start of the range.

`endRange`

A number representing the end of the range.

Return value

A string representing the pluralization category of the specified range. This can be one of `zero`, `one`, `two`, `few`, `many` or `other`, that are relevant for the locale whose localization is specified in LDML Language Plural Rules.

Description

This function selects a pluralization category according to the locale and formatting options of an `Intl.PluralRules` object.

Conceptually the behavior is the same as getting plural rules for a single cardinal or ordinal number. Languages have one or more forms for describing ranges, and this method returns the appropriate form given the supplied locale and formatting options. In English there is only one plural form, such as "1–10 apples", and the method will return `other`. Other languages can have many forms.

Examples

Using selectRange()

```js
new Intl.PluralRules("sl").selectRange(102, 201); // 'few'

new Intl.PluralRules("pt").selectRange(102, 102); // 'other'
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.pluralrules.prototype.selectrange`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`selectRange`	106	106	116	92	15.4	106	116	72	15.4	20.0	106	15.4	1.0.3	**No**	19.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.PluralRules.selectRange`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.PluralRules`

JavaScript › Reference › Standard built-in objects › Intl

Intl.RelativeTimeFormat

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/RelativeTimeFormat · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2020.

The **`Intl.RelativeTimeFormat`** object enables language-sensitive relative time formatting.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.RelativeTimeFormat". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const rtf1 = new Intl.RelativeTimeFormat("en", { style: "short" });

console.log(rtf1.format(3, "quarter"));
// Expected output: "in 3 qtrs."

console.log(rtf1.format(-1, "day"));
// Expected output: "1 day ago"

const rtf2 = new Intl.RelativeTimeFormat("es", { numeric: "auto" });

console.log(rtf2.format(2, "day"));
// Expected output: "pasado mañana"
```

Constructor

`Intl.RelativeTimeFormat()`

Creates a new `Intl.RelativeTimeFormat` object.

Static methods

`Intl.RelativeTimeFormat.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.RelativeTimeFormat.prototype` and shared by all `Intl.RelativeTimeFormat` instances.

`Intl.RelativeTimeFormat.prototype.constructor`

The constructor function that created the instance object. For `Intl.RelativeTimeFormat` instances, the initial value is the `Intl.RelativeTimeFormat` constructor.

`Intl.RelativeTimeFormat.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.RelativeTimeFormat"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.RelativeTimeFormat.prototype.format()`

Formats a `value` and a `unit` according to the locale and formatting options of the given `Intl.RelativeTimeFormat` object.

`Intl.RelativeTimeFormat.prototype.formatToParts()`

Returns an `Array` of objects representing the relative time format in parts that can be used for custom locale-aware formatting.

`Intl.RelativeTimeFormat.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and formatting options computed during initialization of the object.

Examples

Basic format usage

The following example shows how to use a relative time formatter for the English language.

```js
// Create a relative time formatter in your locale
// with default values explicitly passed in.
const rtf = new Intl.RelativeTimeFormat("en", {
  localeMatcher: "best fit", // other values: "lookup"
  numeric: "always", // other values: "auto"
  style: "long", // other values: "short" or "narrow"
});

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "1 day ago"

// Format relative time using positive value (1).
rtf.format(1, "day"); // "in 1 day"
```

Using formatToParts

The following example shows how to create a relative time formatter returning formatted parts.

```js
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });

// Format relative time using the day unit.
rtf.formatToParts(-1, "day");
// [{ type: "literal", value: "yesterday"}]

rtf.formatToParts(100, "day");
// [
//   { type: "literal", value: "in " },
//   { type: "integer", value: "100", unit: "day" },
//   { type: "literal", value: " days" }
// ]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#relativetimeformat-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`RelativeTimeFormat`	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [1]
`RelativeTimeFormat()` constructor	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0
`locales` parameter	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	13.0.0 [2,3]
`options.localeMatcher` parameter	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0
`options.numberingSystem` parameter	71	79	76	58	14	71	79	50	14	10.0	71	14	1.0.0	1.8	12.0.0
`options.numeric` parameter	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0
`format`	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [1]
`formatToParts`	71	79	70	58	14	71	79	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [1]
`resolvedOptions`	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [1]
`supportedLocalesOf`	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	13.0.0 [2,4]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `RelativeTimeFormat()` constructor for more details.
2. Also from version 12.0.0–13.0.0: partial support.
3. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `RelativeTimeFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.
4. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.RelativeTimeFormat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.RelativeTimeFormat` in FormatJS
- `Intl`
- `Intl.RelativeTimeFormat` on v8.dev (2018)

JavaScript › Reference › Standard built-in objects › Intl › Intl.RelativeTimeFormat

Intl.RelativeTimeFormat() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/RelativeTimeFormat/RelativeTimeFormat · MDN
page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2020.

The **`Intl.RelativeTimeFormat()`** constructor creates `Intl.RelativeTimeFormat` objects.

Syntax

```js
new Intl.RelativeTimeFormat()
new Intl.RelativeTimeFormat(locales)
new Intl.RelativeTimeFormat(locales, options)
```

> **Note:** `Intl.RelativeTimeFormat()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

The following Unicode extension key is allowed:

`nu`

See `numberingSystem`.

This key can also be set with `options` (as listed below). When both are set, the `options` property takes precedence.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`numberingSystem`

The numbering system to use for number formatting, such as `"arab"`, `"hans"`, `"mathsans"`, and so on. For a list of supported numbering system types, see `Intl.supportedValuesOf()`; the default is locale dependent. This option can also be set through the `nu` Unicode extension key; if both are provided, this `options` property takes precedence.

`style`

The style of the formatted relative time. Possible values are:

`"long"` (default)

E.g., "in 1 month"

`"short"`
: E.g., "in 1 mo."

`"narrow"`
: E.g., "in 1 mo.". The narrow style could be similar to the short style for some locales.

`numeric`
: Whether to use numeric values in the output. Possible values are `"always"` and `"auto"`; the default is `"always"`. When set to `"auto"`, the output may use more idiomatic phrasing such as `"yesterday"` instead of `"1 day ago"`.

Exceptions

`RangeError`
: Thrown if `locales` or `options` contain invalid values.

Examples

Basic format usage

The following example shows how to create a relative time formatter using the English language.

```js
// Create a relative time formatter in your locale
// with default values explicitly passed in.
const rtf = new Intl.RelativeTimeFormat("en-US", {
  numeric: "always", // other values: "auto"
  style: "long", // other values: "short" or "narrow"
});

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "1 day ago"

// Format relative time using positive value (1).
rtf.format(1, "day"); // "in 1 day"
```

Using the auto option

If the `numeric: "auto"` option is passed, it will produce the string `yesterday` or `tomorrow` instead of `1 day ago` or `in 1 day`. This is useful when you don't want to use numeric values in the output.

```js
// Create a relative time formatter in your locale
// with numeric: "auto" option value passed in.
const rtf = new Intl.RelativeTimeFormat("en-US", { numeric: "auto" });

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "yesterday"

// Format relative time using positive day unit (1).
rtf.format(1, "day"); // "tomorrow"
```

When the value is `0`, the output may be dependent on the unit. "0 seconds" is represented by the localized version of "now".

```js
rtf.format(0, "second"); // "now"
rtf.format(0, "day"); // "today"
rtf.format(0, "minute"); // "this minute"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl-relativetimeformat-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`RelativeTimeFormat()` constructor	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0
`locales` parameter	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	13.0.0 [1,2]
`options.localeMatcher` parameter	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0
`options.numberingSystem` parameter	71	79	76	58	14	71	79	50	14	10.0	71	14	1.0.0	1.8	12.0.0
`options.numeric` parameter	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0

1. Also from version 12.0.0–13.0.0: partial support.
2. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. When other locales are specified, the `RelativeTimeFormat` instance silently falls back to `en-US`. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.RelativeTimeFormat.RelativeTimeFormat`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.RelativeTimeFormat`
- `Intl`
- `Intl.RelativeTimeFormat` on v8.dev (2018)

JavaScript › Reference › Standard built-in objects › Intl › Intl.RelativeTimeFormat

Intl.RelativeTimeFormat.supportedLocalesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/RelativeTimeFormat/supportedLocalesOf · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2020.

The **`Intl.RelativeTimeFormat.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in relative time formatting without having to fall back to the runtime's default locale.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.RelativeTimeFormat.supportedLocalesOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };

console.log(Intl.RelativeTimeFormat.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

js

```
Intl.RelativeTimeFormat.supportedLocalesOf(locales)
Intl.RelativeTimeFormat.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in relative time formatting without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in relative time formatting, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to relative time formatting nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

js

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.RelativeTimeFormat.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.RelativeTimeFormat.supportedLocalesOf`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedLocalesOf`	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	13.0.0 [1,2]

1. Also from version 12.0.0–13.0.0: partial support.
2. Also from version 12.0.0–13.0.0: Before version 13.0.0, only the locale data for `en-US` is available by default. To make full ICU (locale) data available before version 13, see Node.js documentation on the `--with-intl` option and how to provide the data.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.RelativeTimeFormat.supportedLocalesOf`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.RelativeTimeFormat`

JavaScript › Reference › Standard built-in objects › Intl › Intl.RelativeTimeFormat

Intl.RelativeTimeFormat.prototype.format()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/RelativeTimeFormat/format · MDN page last modified 2026-08-27

Baseline: Widely available — available across major browsers since September 2020.

The **`format()`** method of `Intl.RelativeTimeFormat` instances formats a `value` and `unit` according to the locale and formatting options of this `Intl.RelativeTimeFormat` object.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.RelativeTimeFormat.prototype.format()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const rtf = new Intl.RelativeTimeFormat("en", { style: "short" });

console.log(rtf.format(3, "quarter"));
// Expected output: "in 3 qtrs."

console.log(rtf.format(-1, "day"));
// Expected output: "1 day ago"

console.log(rtf.format(10, "seconds"));
// Expected output: "in 10 sec."
```

Syntax

```js
format(value, unit)
```

Parameters

`value`

Numeric value to use in the internationalized relative time message.

`unit`

Unit to use in the relative time internationalized message. Possible values are: `"year"`, `"quarter"`, `"month"`, `"week"`, `"day"`, `"hour"`, `"minute"`, `"second"`. Plural forms are also permitted.

Return value

A string representing the given `value` and `unit` formatted according to the locale and formatting options of this `Intl.RelativeTimeFormat` object.

> **Note:** Most of the time, the formatting returned by `format()` is consistent. However, the output may vary between implementations, even within the same locale — output variations are by design and allowed by the specification. It may also not be what you expect. For example, the string may use non-breaking spaces or be surrounded by bidirectional control characters. You should not compare the results of `format()` to hardcoded constants.

Examples

Basic format usage

The following example shows how to create a relative time formatter using the English language.

```js
// Create a relative time formatter in your locale
// with default values explicitly passed in.
const rtf = new Intl.RelativeTimeFormat("en", {
  localeMatcher: "best fit", // other values: "lookup"
  numeric: "always", // other values: "auto"
  style: "long", // other values: "short" or "narrow"
});

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "1 day ago"

// Format relative time using positive value (1).
rtf.format(1, "day"); // "in 1 day"
```

Using the auto option

If the `numeric: "auto"` option is passed, it will produce the string `yesterday`, `today`, or `tomorrow` instead of `1 day ago`, `in 0 days`, or `in 1 day`. This allows you to avoid using numeric values in the output.

```js
// Create a relative time formatter in your locale
// with numeric: "auto" option value passed in.
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });

// Format relative time using negative value (-1).
rtf.format(-1, "day"); // "yesterday"

rtf.format(0, "day"); // "today"

// Format relative time using positive day unit (1).
rtf.format(1, "day"); // "tomorrow"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-Intl.RelativeTimeFormat.prototype.format`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`format`	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `RelativeTimeFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.RelativeTimeFormat.format`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.RelativeTimeFormat`

JavaScript › Reference › Standard built-in objects › Intl › Intl.RelativeTimeFormat

Intl.RelativeTimeFormat.prototype.formatToParts()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/RelativeTimeFormat/formatToParts · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since September 2020.

The **`formatToParts()`** method of `Intl.RelativeTimeFormat` instances returns an array of objects representing each part of the formatted string that would be returned by `format()`. It is useful for building custom strings from the locale-specific tokens.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.RelativeTimeFormat.prototype.formatToParts()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });
const parts = rtf.formatToParts(10, "seconds");

console.log(parts[0].value);
// Expected output: "in "

console.log(parts[1].value);
// Expected output: "10"

console.log(parts[2].value);
// Expected output: " seconds"
```

Syntax

js

```
formatToParts(value, unit)
```

Parameters

`value`

Numeric value to use in the internationalized relative time message.

`unit`

Unit to use in the relative time internationalized message. Possible values are: `"year"`, `"quarter"`, `"month"`, `"week"`, `"day"`, `"hour"`, `"minute"`, `"second"`. Plural forms are also permitted.

Return value

An `Array` of objects containing the formatted relative time in parts. Each object has two or three properties, `type`, `value`, and optionally `unit`, each containing a string. The string concatenation of `value`, in the order provided, will result in the same string as `format()`. The parts can be thought of as directly obtained from calling `Intl.NumberFormat.prototype.formatToParts()` with the numeric value, passing only the `numberingSystem` option, and then adding additional `type: "literal"` tokens, such as `"in "`, `" days ago"`, etc. All tokens that are produced by the `NumberFormat` have an additional `unit` property, which is the singular form of the input `unit`; this is for programmatic usage and is not localized. The localized unit is output as a part of a literal token.

When `options.numeric` is `"auto"`, and there's a special string for the value, the returned array is a single literal token.

Examples

Using formatToParts()

```js
const rtf = new Intl.RelativeTimeFormat("en", { numeric: "auto" });

// Format relative time using the day unit
rtf.formatToParts(-1, "day");
// [{ type: "literal", value: "yesterday"}]

rtf.formatToParts(100, "day");
// [
//   { type: "literal", value: "in " },
//   { type: "integer", value: "100", unit: "day" },
//   { type: "literal", value: " days" }
// ]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-Intl.RelativeTimeFormat.prototype.formatToParts

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
formatToParts	71	79	70	58	14	71	79	50	14	10.0	71	14	1.0.0	1.8	12.0.0[1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `RelativeTimeFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.RelativeTimeFormat.formatToParts`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.RelativeTimeFormat`
- `Intl.RelativeTimeFormat.prototype.format()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.RelativeTimeFormat

Intl.RelativeTimeFormat.prototype.resolvedOptions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/RelativeTimeFormat/resolvedOptions · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since September 2020.

The **`resolvedOptions()`** method of `Intl.RelativeTimeFormat` instances returns a new object with properties reflecting the options computed during initialization of this `RelativeTimeFormat` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.RelativeTimeFormat.prototype.resolvedOptions()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const rtf1 = new Intl.RelativeTimeFormat("en", { style: "narrow" });
const options1 = rtf1.resolvedOptions();

const rtf2 = new Intl.RelativeTimeFormat("es", { numeric: "auto" });
const options2 = rtf2.resolvedOptions();

console.log(`${options1.locale}, ${options1.style}, ${options1.numeric}`);
// Expected output: "en, narrow, always"

console.log(`${options2.locale}, ${options2.style}, ${options2.numeric}`);
// Expected output: "es, long, auto"
```

Syntax

js

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `RelativeTimeFormat` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. Only the `nu` Unicode extension key, if requested, may be included in the output.

`style`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"long"`, `"short"`, or `"narrow"`. The default is `"long"`.

`numeric`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"always"` or `"auto"`. The default is `"always"`.

`numberingSystem`

The value provided for this property in the `options` argument, or using the Unicode extension key `"nu"`, with default filled in as needed. It is a supported numbering system for this locale. The default is locale dependent.

Examples

Using the resolvedOptions() method

```js
const de = new Intl.RelativeTimeFormat("de-DE");
const usedOptions = de.resolvedOptions();

usedOptions.locale; // "de-DE"
usedOptions.style; // "long"
usedOptions.numeric; // "always"
usedOptions.numberingSystem; // "latn"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.relativetimeformat.prototype.resolvedoptions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resolvedOptions`	71	79	65	58	14	71	65	50	14	10.0	71	14	1.0.0	1.8	12.0.0 [1]

1. Before version 13.0.0, only the locale data for `en-US` is available by default. See the `RelativeTimeFormat()` constructor for more details.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.RelativeTimeFormat.resolvedOptions`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.RelativeTimeFormat`

JavaScript › Reference › Standard built-in objects › Intl

Intl.Segmenter

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since April 2024.

The **`Intl.Segmenter`** object enables locale-sensitive text segmentation, enabling you to get meaningful items (graphemes, words or sentences) from a string.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.Segmenter". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const segmenterFr = new Intl.Segmenter("fr", { granularity: "word" });
const string = "Que ma joie demeure";

const iterator = segmenterFr.segment(string)[Symbol.iterator]();

console.log(iterator.next().value.segment);
// Expected output: 'Que'

console.log(iterator.next().value.segment);
// Expected output: ' '
```

Constructor

`Intl.Segmenter()`

Creates a new `Intl.Segmenter` object.

Static methods

`Intl.Segmenter.supportedLocalesOf()`

Returns an array containing those of the provided locales that are supported without having to fall back to the runtime's default locale.

Instance properties

These properties are defined on `Intl.Segmenter.prototype` and shared by all `Intl.Segmenter` instances.

`Intl.Segmenter.prototype.constructor`

The constructor function that created the instance object. For `Intl.Segmenter` instances, the initial value is the `Intl.Segmenter` constructor.

`Intl.Segmenter.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Intl.Segmenter"`. This property is used in `Object.prototype.toString()`.

Instance methods

`Intl.Segmenter.prototype.resolvedOptions()`

Returns a new object with properties reflecting the locale and granularity options computed during initialization of this `Intl.Segmenter` object.

`Intl.Segmenter.prototype.segment()`

Returns a new iterable `Segments` instance representing the segments of a string according to the locale and granularity of this `Intl.Segmenter` instance.

Examples

Basic usage and difference from String.prototype.split()

If we were to use `String.prototype.split(" ")` to segment a text in words, we would not get the correct result if the locale of the text does not use whitespaces between words (which is the case for Japanese, Chinese, Thai, Lao, Khmer, Myanmar, etc.).

js · ✗ bad example

```
const str = "吾輩は猫である。名前はたぬき。";
console.table(str.split(" "));
// ['吾輩は猫である。名前はたぬき。']
// The two sentences are not correctly segmented.
```

js · ✓ good example

```
const str = "吾輩は猫である。名前はたぬき。";
const segmenterJa = new Intl.Segmenter("ja-JP", { granularity: "word" });

const segments = segmenterJa.segment(str);
console.table(Array.from(segments));
// [{segment: '吾輩', index: 0, input: '吾輩は猫である。名前はたぬき。', isWordLike: true},
// etc.
// ]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#segmenter-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Segmenter`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
`Segmenter()` constructor	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
`resolvedOptions`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
`segment`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
`supportedLocalesOf`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Segmenter`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Intl.Segmenter` in FormatJS
- `Intl`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Segmenter

Intl.Segmenter() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter/Segmenter · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since April 2024.

The **`Intl.Segmenter()`** constructor creates `Intl.Segmenter` objects.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.Segmenter() constructor". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const segmenterFr = new Intl.Segmenter("fr", { granularity: "word" });
const string = "Que ma joie demeure";

const iterator = segmenterFr.segment(string)[Symbol.iterator]();

console.log(iterator.next().value.segment);
// Expected output: 'Que'

console.log(iterator.next().value.segment);
// Expected output: ' '
```

Syntax

js

```
new Intl.Segmenter()
new Intl.Segmenter(locales)
new Intl.Segmenter(locales, options)
```

> **Note:** `Intl.Segmenter()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`.

Parameters

`locales` Optional

A string with a BCP 47 language tag or an `Intl.Locale` instance, or an array of such locale identifiers. The runtime's default locale is used when `undefined` is passed or when none of the specified locale identifiers is supported. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object containing the following properties, in the order they are retrieved (all of them are optional):

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see Locale identification and negotiation.

`granularity`

How granularly should the input be split. Possible values are:

`"grapheme"` (default)

Split the input into segments at grapheme cluster (user-perceived character) boundaries, as determined by the locale.

`"word"`

Split the input into segments at word boundaries, as determined by the locale.

`"sentence"`

Split the input into segments at sentence boundaries, as determined by the locale.

Return value

A new `Intl.Segmenter` instance.

Exceptions

`RangeError`

Thrown if `locales` or `options` contain invalid values.

Examples

Basic usage

The following example shows how to count words in a string using the Japanese language (where splitting the string using `String` methods would have given an incorrect result).

```js
const text = "吾輩は猫である。名前はたぬき。";
const japaneseSegmenter = new Intl.Segmenter("ja-JP", { granularity: "word" });
console.log(
  [...japaneseSegmenter.segment(text)].filter((segment) => segment.isWordLike)
    .length,
);
// 8, as the text is segmented as '吾輩'|'は'|'猫'|'で'|'ある'|'。'|'名前'|'は'|'たぬき'|'。'
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-intl-segmenter-constructor

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Segmenter()` constructor	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Segmenter.Segmenter`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

JavaScript › Reference › Standard built-in objects › Intl › Intl.Segmenter

Intl.Segmenter.supportedLocalesOf()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter/supportedLocalesOf · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since April 2024.

The **`Intl.Segmenter.supportedLocalesOf()`** static method returns an array containing those of the provided locales that are supported in segmentation without having to fall back to the runtime's default locale.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Intl.Segmenter.supportedLocalesOf()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup", granularity: "string" };

console.log(Intl.Segmenter.supportedLocalesOf(locales, options));
// Expected output: Array ["id-u-co-pinyin", "de-ID"]
// (Note: the exact output may be browser-dependent)
```

Syntax

js

```
Intl.Segmenter.supportedLocalesOf(locales)
Intl.Segmenter.supportedLocalesOf(locales, options)
```

Parameters

`locales`

A string with a BCP 47 language tag, or an array of such strings. For the general form and interpretation of the `locales` argument, see the parameter description on the `Intl` main page.

`options` Optional

An object that may have the following property:

`localeMatcher`

The locale matching algorithm to use. Possible values are `"lookup"` and `"best fit"`; the default is `"best fit"`. For information about this option, see the Intl page.

Return value

An array of strings representing a subset of the given locale tags that are supported in segmentation without having to fall back to the runtime's default locale.

Examples

Using supportedLocalesOf()

Assuming a runtime that supports Indonesian and German but not Balinese in segmentation, `supportedLocalesOf` returns the Indonesian and German language tags unchanged, even though `pinyin` collation is neither relevant to segmentation nor used with Indonesian, and a specialized German for Indonesia is unlikely to be supported. Note the specification of the `"lookup"` algorithm here — a `"best fit"` matcher might decide that Indonesian is an adequate match for Balinese since most Balinese speakers also understand Indonesian, and therefore return the Balinese language tag as well.

```js
const locales = ["ban", "id-u-co-pinyin", "de-ID"];
const options = { localeMatcher: "lookup" };
console.log(Intl.Segmenter.supportedLocalesOf(locales, options));
// ["id-u-co-pinyin", "de-ID"]
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.segmenter.supportedlocalesof`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`supportedLocalesOf`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Segmenter.supportedLocalesOf`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- `Intl.Segmenter`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Segmenter

Intl.Segmenter.prototype.resolvedOptions()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter/resolvedOptions · MDN page last modified 2025-09-24

Baseline: Widely available — available across major browsers since April 2024.

The **`resolvedOptions()`** method of `Intl.Segmenter` instances returns a new object with properties reflecting the options computed during initialization of this `Segmenter` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.Segmenter.prototype.resolvedOptions()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const segmenter = new Intl.Segmenter("fr-FR");
const options = segmenter.resolvedOptions();

console.log(options.locale);
// Expected output: "fr-FR"

console.log(options.granularity);
// Expected output: "grapheme"
```

Syntax

js

```
resolvedOptions()
```

Parameters

None.

Return value

A new object with properties reflecting the options computed during the initialization of this `Segmenter` object. The object has the following properties, in the order they are listed:

`locale`

The BCP 47 language tag for the locale actually used, determined by the locale negotiation process. No Unicode extension key will be included in the output.

`granularity`

The value provided for this property in the `options` argument, with default filled in as needed. It is either `"grapheme"`, `"word"`, or `"sentence"`. The default is `"grapheme"`.

Examples

Basic usage

```js
const spanishSegmenter = new Intl.Segmenter("es", { granularity: "sentence" });
const options = spanishSegmenter.resolvedOptions();
console.log(options.locale); // "es"
console.log(options.granularity); // "sentence"
```

Default granularity

```js
const spanishSegmenter = new Intl.Segmenter("es");
const options = spanishSegmenter.resolvedOptions();
console.log(options.locale); // "es"
console.log(options.granularity); // "grapheme"
```

Fallback locale

```js
const banSegmenter = new Intl.Segmenter("ban");
const options = banSegmenter.resolvedOptions();
console.log(options.locale);
// "fr" on a runtime where the Balinese locale
// is not supported and French is the default locale
console.log(options.granularity); // "grapheme"
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.segmenter.prototype.resolvedoptions`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`resolvedOptions`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Segmenter.resolvedOptions`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

JavaScript › Reference › Standard built-in objects › Intl › Intl.Segmenter

Intl.Segmenter.prototype.segment()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter/segment · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2024.

The **`segment()`** method of `Intl.Segmenter` instances segments a string according to the locale and granularity of this `Intl.Segmenter` object.

Try it

▷ Interactive example on MDN: "JavaScript Demo: Intl.Segmenter.prototype.segment()". It runs live in the browser there; its source code follows.

js · interactive example source

```
const string1 = "Que ma joie demeure";

const segmenterFrGrapheme = new Intl.Segmenter("fr", {
  granularity: "grapheme",
});
const graphemeSegments = segmenterFrGrapheme.segment(string1);

console.log(Array.from(graphemeSegments)[0]);
// Expected output:
// Object {segment: 'Q', index: 0, input: 'Que ma joie demeure'}
```

Syntax

js

```
segment(input)
```

Parameters

`input`

The text to be segmented as a string.

Return value

A new iterable `Segments` object containing the segments of the input string, using the segmenter's locale and granularity.

Examples

js

```
// Create a locale-specific word segmenter
const segmenter = new Intl.Segmenter("fr", { granularity: "word" });

// Use it to get an iterator over the segments of a string
const input = "Moi ? N'est-ce pas ?";
const segments = segmenter.segment(input);

// Use that for segmentation
for (const { segment, index, isWordLike } of segments) {
  console.log(
    "segment at code units [%d, %d]: «%s»%s",
```

(code continues on the next page)

(code continued from the previous page)

```
    index,
    index + segment.length,
    segment,
    isWordLike ? " (word-like)" : "",
  );
}
// segment at code units [0, 3]: «Moi» (word-like)
// segment at code units [3, 4]: « »
// segment at code units [4, 5]: «?»
// segment at code units [5, 6]: « »
// segment at code units [6, 11]: «N'est» (word-like)
// segment at code units [11, 12]: «-»
// segment at code units [12, 14]: «ce» (word-like)
// segment at code units [14, 15]: « »
// segment at code units [15, 18]: «pas» (word-like)
// segment at code units [18, 19]: « »
// segment at code units [19, 20]: «?»
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-intl.segmenter.prototype.segment`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`segment`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Segmenter.segment`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

JavaScript › Reference › Standard built-in objects › Intl › Intl.Segmenter › segment()

Segments

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter/segment/Segments · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since April 2024.

A **`Segments`** object is an iterable collection of the segments of a text string. It is returned by a call to the `segment()` method of an `Intl.Segmenter` object.

Instance methods

`Segments.prototype.containing()`

Returns an object describing the segment in the original string that includes the code unit at a specified index.

`Segments.prototype[Symbol.iterator]()`

Returns an iterator to iterate over the segments.

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-segments-objects`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Segments`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
[Symbol.iterator]	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0
`containing`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Segments`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Segmenter`
- `Intl.Segmenter.prototype.segment()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Segmenter › segment() › Segments

Segments.prototype.containing()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter/segment/Segments/containing · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since April 2024.

The **`containing()`** method of `Segments` instances returns an object describing the segment in the string that includes the code unit at the specified index.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Segments.prototype.containing()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const segmenterFr = new Intl.Segmenter("fr", { granularity: "word" });
const string = "Que ma joie demeure";

const segments = segmenterFr.segment(string);

console.log(segments.containing(5));
// Expected output:
// Object {segment: 'ma', index: 4, input: 'Que ma joie demeure', isWordLike: true}
```

Syntax

js

```
containing(codeUnitIndex)
```

Parameters

`codeUnitIndex` Optional

A number specifying the index of the code unit in the original input string. If the value is omitted, it defaults to `0`.

Return value

An object describing the segment of the original string with the following properties, or `undefined` if the supplied index value is out of bounds.

`segment`

A string containing the segment extracted from the original input string.

`index`

The code unit index in the original input string at which the segment begins.

`input`

The complete input string that was segmented.

`isWordLike`

A boolean value only if `granularity` is `"word"`; otherwise, `undefined`. If `granularity` is `"word"`, then `isWordLike` is `true` when the segment is word-like (i.e., consists of letters/numbers/ideographs/etc.); otherwise, `false`.

Examples

```js
// ┃0 1 2 3 4 5┃6┃7┃8┃9  ← code unit index
// ┃A 1 1 o n s┃-┃y┃!┃   ← code unit
const input = "Allons-y!";

const segmenter = new Intl.Segmenter("fr", { granularity: "word" });
const segments = segmenter.segment(input);

let current = segments.containing();
// { index: 0, segment: "Allons", isWordLike: true }

current = segments.containing(4);
// { index: 0, segment: "Allons", isWordLike: true }

current = segments.containing(6);
// { index: 6, segment: "-", isWordLike: false }

current = segments.containing(current.index + current.segment.length);
// { index: 7, segment: "y", isWordLike: true }

current = segments.containing(current.index + current.segment.length);
// { index: 8, segment: "!", isWordLike: false }

current = segments.containing(current.index + current.segment.length);
// undefined
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification `https://tc39.es/ecma402/#sec-%intlsegmentsprototype%.containing`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`containing`	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Segments.containing`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Segmenter`
- `Intl.Segmenter.prototype.segment()`

JavaScript › Reference › Standard built-in objects › Intl › Intl.Segmenter › segment() › Segments

Segments.prototype[Symbol.iterator]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter/segment/Segments/Symbol.iterator · MDN page last modified 2025-07-20

Baseline: Widely available — available across major browsers since April 2024.

The **`[Symbol.iterator]()`** method of `Segments` instances implements the iterable protocol and allows `Segments` objects to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns a segments iterator object that yields data about each segment.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: Segments.prototype[Symbol.iterator]()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
const segmenterFr = new Intl.Segmenter("fr", { granularity: "word" });
const string = "Que ma joie demeure";

const iterator = segmenterFr.segment(string)[Symbol.iterator]();

for (const segment of iterator) {
  if (segment.segment.length > 4) {
    console.log(segment.segment);
  }
}

// Expected output: "demeure"
```

Syntax

js

```
segments[Symbol.iterator]()
```

Parameters

None.

Return value

A new iterable iterator object that yields data about each segment. Each yielded object has the same properties as the object returned by the `containing()` method.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes `Segments` objects iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```js
const segmenter = new Intl.Segmenter("zh-CN", { granularity: "word" });
const input = "你好，世界！我爱编程。";

for (const value of segmenter.segment(input)) {
  console.log(value);
}

/*
{segment: '你好', index: 0, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '，', index: 2, input: '你好，世界！我爱编程。', isWordLike: false}
{segment: '世界', index: 3, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '！', index: 5, input: '你好，世界！我爱编程。', isWordLike: false}
{segment: '我', index: 6, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '爱', index: 7, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '编', index: 8, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '程', index: 9, input: '你好，世界！我爱编程。', isWordLike: true}
{segment: '。', index: 10, input: '你好，世界！我爱编程。', isWordLike: false}
*/
```

Manually hand-rolling the iterator

You may still manually call the `next()` method of the returned iterator object to achieve maximum control over the iteration process.

```js
const segmenter = new Intl.Segmenter("fr", { granularity: "word" });
const input = "Moi ? N'est-ce pas ?";
const segments = segmenter.segment(input);
const iterator = segments[Symbol.iterator]();

let result = iterator.next();

while (!result.done) {
  console.log(result.value);
  result = iterator.next();
}

/*
{segment: 'Moi', index: 0, input: "Moi ? N'est-ce pas ?", isWordLike: true}
{segment: ' ', index: 3, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: '?', index: 4, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: ' ', index: 5, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: "N'est", index: 6, input: "Moi ? N'est-ce pas ?", isWordLike: true}
{segment: '-', index: 11, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: 'ce', index: 12, input: "Moi ? N'est-ce pas ?", isWordLike: true}
{segment: ' ', index: 14, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: 'pas', index: 15, input: "Moi ? N'est-ce pas ?", isWordLike: true}
{segment: ' ', index: 18, input: "Moi ? N'est-ce pas ?", isWordLike: false}
{segment: '?', index: 19, input: "Moi ? N'est-ce pas ?", isWordLike: false}
*/
```

Specifications

Specification
ECMAScript® 2027 Internationalization API Specification https://tc39.es/ecma402/#sec-%intlsegmentsprototype%-%symbol.iterator%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.iterator]	87	87	125	73	14.1	87	125	62	14.5	14.0	87	14.5	1.0.0	1.8	16.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Intl.Segments.@@iterator`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Intl.Segmenter`
- `Intl.Segmenter.prototype.segment()`
- `Symbol.iterator`
- Iteration protocols

JavaScript › Reference › Standard built-in objects

isFinite()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/isFinite · MDN page last modified 2026-10-27

Baseline: Widely available — available across major browsers since July 2015.

The **`isFinite()`** function determines whether a value is finite, first converting the value to a number if necessary. A finite number is one that's not `NaN` or `±Infinity`. Because coercion inside the `isFinite()` function can be surprising, you may prefer to use `Number.isFinite()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: isFinite()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function div(x) {
  if (isFinite(1000 / x)) {
    return "Number is NOT Infinity.";
  }
  return "Number is Infinity!";
}

console.log(div(0));
// Expected output: "Number is Infinity!""

console.log(div(1));
// Expected output: "Number is NOT Infinity."
```

Syntax

js

```
isFinite(value)
```

Parameters

`value`

The value to be tested.

Return value

`true`, except when the given value is either `Infinity`, `-Infinity`, or `NaN` after being coerced to a number.

Description

`isFinite()` is a function property of the global object.

When the argument to the `isFinite()` function is not of type Number, the value is first coerced to a number, and the resulting value is then compared against `NaN` and ±Infinity. This is as confusing as the behavior of `isNaN` — for example, `isFinite("1")` is `true`.

`Number.isFinite()` is a more reliable way to test whether a value is a finite number value, because it returns `false` for any non-number input.

Examples

Using isFinite()

```js
isFinite(Infinity); // false
isFinite(NaN); // false
isFinite(-Infinity); // false

isFinite(0); // true
isFinite(2e64); // true
isFinite(910); // true

// Following would've been false with the more robust Number.isFinite():

isFinite(null); // true, because null is converted to 0
isFinite(""); // true, because empty string is converted to 0
isFinite("123"); // true, because string can be parsed as a number
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/global-object.html#sec-isfinite-number`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`isFinite`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.isFinite`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Number.isFinite()`
- `Number.NaN`
- `Number.POSITIVE_INFINITY`
- `Number.NEGATIVE_INFINITY`

JavaScript › Reference › Standard built-in objects

isNaN()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/isNaN · MDN page last modified 2026-10-27

Baseline: Widely available — available across major browsers since July 2015.

The **`isNaN()`** function determines whether a value is `NaN`, first converting the value to a number if necessary. Because coercion inside the `isNaN()` function can be surprising, you may prefer to use `Number.isNaN()`.

Try it

▷ *Interactive example on MDN: "JavaScript Demo: isNaN()". It runs live in the browser there; its source code follows.*

js · interactive example source

```
function milliseconds(x) {
  if (isNaN(x)) {
    return "Not a Number!";
  }
  return x * 1000;
}

console.log(milliseconds("100F"));
// Expected output: "Not a Number!"

console.log(milliseconds("0.0314E+2"));
// Expected output: 3140
```

Syntax

js

```
isNaN(value)
```

Parameters

`value`

The value to be tested.

Return value

`true` if the given value is `NaN` after being coerced to a number.

Description

`isNaN()` is a function property of the global object.

For number values, `isNaN()` tests if the number is the value `NaN`. When the argument to the `isNaN()` function is not of type Number, the value is first coerced to a number, and the resulting value is then compared against `NaN`.

This behavior of `isNaN()` for non-numeric arguments can be confusing! For example:

- `null` is coerced to 0
- an empty string is coerced to 0
- a boolean is coerced to 0 or 1

These values are intuitively "not numbers", but they don't evaluate to `NaN`, so `isNaN()` returns `false`. Therefore, `isNaN()` answers neither the question "is the input the floating point `NaN` value" nor the question "is the input not a number".

`Number.isNaN()` is a more reliable way to test whether a value is the number value `NaN` or not. Alternatively, the expression `x !== x` can be used, and neither of the solutions is subject to the false positives that make the global `isNaN()` unreliable. To test if a value is a number, use `typeof x === "number"`.

The `isNaN()` function answers the question "is the input functionally equivalent to `NaN` when used in a number context". If `isNaN(x)` returns `false`, you can use `x` in an arithmetic expression as if it's a valid number that's not `NaN`. If `isNaN(x)` returns `true`, `x` will get coerced to `NaN` and make most arithmetic expressions return `NaN` (because `NaN` propagates). You can use this, for example, to test whether an argument to a function is arithmetically processable (usable "like" a number), and handle values that are not number-like by throwing an error, providing a default value, etc. This way, you can have a function that makes use of the full versatility JavaScript provides by implicitly converting values depending on context.

> **Note:** The + operator performs both number addition and string concatenation. Therefore, even if `isNaN()` returns `false` for both operands, the + operator may still return a string, because it's not used as an arithmetic operator. For example, `isNaN("1")` returns `false`, but `"1" + 1` returns `"11"`. To be sure that you are working with numbers, coerce the value to a number and use `Number.isNaN()` to test the result.

Examples

Note how `isNaN()` returns `true` for values that are not the value `NaN` but are not numbers either:

```js
isNaN(NaN); // true
isNaN(undefined); // true
isNaN({}); // true

isNaN(true); // false
isNaN(null); // false
isNaN(37); // false

// Strings
isNaN("37"); // false: "37" is converted to the number 37 which is not NaN
isNaN("37.37"); // false: "37.37" is converted to the number 37.37 which is not NaN
isNaN("37,5"); // true
isNaN("123ABC"); // true: Number("123ABC") is NaN
isNaN(""); // false: the empty string is converted to 0 which is not NaN
isNaN(" "); // false: a string with spaces is converted to 0 which is not NaN

// Dates
isNaN(new Date()); // false; Date objects can be converted to a number (timestamp)
isNaN(new Date().toString()); // true; the string representation of a Date object cannot be
parsed as a number

// Arrays
isNaN([]); // false; the primitive representation is "", which converts to the number 0
isNaN([1]); // false; the primitive representation is "1"
isNaN([1, 2]); // true; the primitive representation is "1,2", which cannot be parsed as
number
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/global-object.html#sec-isnan-number`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`isNaN`	1	12	1	3	1	18	4	10.1	1	1.0	4.4	1	1.0.0	1.0	0.10.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.isNaN`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `NaN`
- `Number.isNaN()`

JavaScript › Reference › Standard built-in objects

Iterator

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator · MDN page last modified 2026-10-26

Baseline: Widely available — available across major browsers since September 2016.

An **`Iterator`** object is an object that conforms to the iterator protocol by providing a `next()` method that returns an iterator result object. All built-in iterators inherit from the `Iterator` class. The `Iterator` class provides a `[Symbol.iterator]()` method that returns the iterator object itself, making the iterator also iterable. It also provides some helper methods for working with iterators.

Description

The following are all built-in JavaScript iterators:

- The *Array Iterator* returned by `Array.prototype.values()`, `Array.prototype.keys()`, `Array.prototype.entries()`, `Array.prototype[Symbol.iterator]()`, `TypedArray.prototype.values()`, `TypedArray.prototype.keys()`, `TypedArray.prototype.entries()`, `TypedArray.prototype[Symbol.iterator]()`, and `arguments[Symbol.iterator]()`.
- The *String Iterator* returned by `String.prototype[Symbol.iterator]()`.
- The *Map Iterator* returned by `Map.prototype.values()`, `Map.prototype.keys()`, `Map.prototype.entries()`, and `Map.prototype[Symbol.iterator]()`.
- The *Set Iterator* returned by `Set.prototype.values()`, `Set.prototype.keys()`, `Set.prototype.entries()`, and `Set.prototype[Symbol.iterator]()`.
- The *RegExp String Iterator* returned by `RegExp.prototype[Symbol.matchAll]()` and `String.prototype.matchAll()`.
- The `Generator` object returned by generator functions.
- The *Segments Iterator* returned by the `[Symbol.iterator]()` method of the `Segments` object returned by `Intl.Segmenter.prototype.segment()`.
- The *Iterator Helper* returned by iterator helper methods such as `Iterator.prototype.filter()` and `Iterator.prototype.map()`.

Web APIs may return iterators too. Some reuse core JavaScript iterators while others define their own iterators. For example:

- Array-Like objects such as `NodeList` return an *Array Iterator* from their respective methods `keys()`, `values()`, `entries()`, and `[Symbol.iterator]()`.
- Map-Like objects from Web APIs such as `Headers` return their own iterator type like *Headers Iterator* from their respective methods `keys()`, `values()`, `entries()`, and `[Symbol.iterator]()`.
- Set-Like objects from Web APIs such as `FontFaceSet` return their own iterator type like *FontFaceSet Iterator* from their respective methods `keys()`, `values()`, `entries()`, and `[Symbol.iterator]()`.

> **Note:** `NodeIterator` and other old interfaces are named as such but do not conform to the iterator protocol or iterable protocol.

Each of these iterators have a distinct prototype object, which defines the `next()` method used by the particular iterator. For example, all string iterator objects inherit from a hidden object `StringIteratorPrototype`, which has a `next()` method that iterates this string by code points. `StringIteratorPrototype` also has a `[Symbol.toStringTag]` property whose initial value is the string

`"String Iterator"`. This property is used in `Object.prototype.toString()`. Similarly, other iterator prototypes also have their own `[Symbol.toStringTag]` values, which are the same as the names given above.

All of these prototype objects inherit from `Iterator.prototype`, which provides a `[Symbol.iterator]()` method that returns the iterator object itself, making the iterator also iterable.

Iterator helper methods

> **Note:** These methods are *iterator* helpers, not *iterable* helpers, because the only requirement for an object to be iterable is just the presence of a `[Symbol.iterator]()` method. There is no shared prototype to install these methods on.

The `Iterator` class itself provides some helper methods for working with iterators. For example, you may be tempted to do the following:

```js
const nameToDeposit = new Map([
  ["Anne", 1000],
  ["Bert", 1500],
  ["Carl", 2000],
]);

const totalDeposit = [...nameToDeposit.values()].reduce((a, b) => a + b);
```

This first converts the iterator returned by `Map.prototype.values()` to an array, then uses the `Array.prototype.reduce()` method to calculate the sum. However, this both creates an intermediate array and iterates the array twice. Instead, you can use the `reduce()` method of the iterator itself:

```js
const totalDeposit = nameToDeposit.values().reduce((a, b) => a + b);
```

This method may be more efficient, especially memory-wise, because it only iterates the iterator once, without memorizing any intermediate values. Iterator helper methods are necessary to work with infinite iterators:

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const seq = fibonacci();
const firstThreeDigitTerm = seq.find((n) => n >= 100);
```

You cannot convert `seq` to an array, because it is infinite. Instead, you can use the `find()` method of the iterator itself, which only iterates `seq` as far as necessary to find the first value that satisfies the condition.

You will find many iterator methods analogous to array methods, such as:

Iterator method	Array method
`Iterator.prototype.every()`	`Array.prototype.every()`
`Iterator.prototype.filter()`	`Array.prototype.filter()`
`Iterator.prototype.find()`	`Array.prototype.find()`
`Iterator.prototype.flatMap()`	`Array.prototype.flatMap()`
`Iterator.prototype.forEach()`	`Array.prototype.forEach()`
`Iterator.prototype.includes()`	`Array.prototype.includes()`
`Iterator.prototype.join()`	`Array.prototype.join()`
`Iterator.prototype.map()`	`Array.prototype.map()`
`Iterator.prototype.reduce()`	`Array.prototype.reduce()`
`Iterator.prototype.some()`	`Array.prototype.some()`

`Iterator.prototype.drop()` and `Iterator.prototype.take()` combined are somewhat analogous to `Array.prototype.slice()`.

Iterator helper objects

> **Note:** *Iterator helper objects* and *iterator helper methods* are two different concepts. An Iterator helper object is detectable at runtime, while "iterator helper method" is just a name for a set of methods for comprehension. *Iterator helper* may refer to either the object or the method, depending on the context.

Among the iterator helper methods, `filter()`, `flatMap()`, `map()`, `drop()`, and `take()` return a new *Iterator Helper* object. The iterator helper is also an `Iterator` instance, making these helper methods chainable. All iterator helper objects inherit from a common prototype object, which implements the iterator protocol:

`next()`
: Calls the `next()` method of the underlying iterator, applies the helper method to the result, and returns the result.

`return()`
: Calls the `return()` method of the underlying iterator, and returns the result.

The iterator helper shares the same data source as the underlying iterator, so iterating the iterator helper causes the underlying iterator to be iterated as well. There is no way to "fork" an iterator to allow it to be iterated multiple times.

```js
const it = [1, 2, 3].values();
const it2 = it.drop(0); // Essentially a copy
console.log(it.next().value); // 1
console.log(it2.next().value); // 2
console.log(it.next().value); // 3
```

Proper iterators

There are two kinds of "iterators": objects that conform to the iterator protocol (which, at its minimum, only requires the presence of a `next()` method), and objects that inherit from the `Iterator` class, which enjoy the helper methods. They do not entail each other — objects that inherit from `Iterator` do not automatically become iterators, because the `Iterator` class does not define a `next()` method. Instead,

the object needs to define a `next()` method itself. A *proper iterator* is one that both conforms to the iterator protocol and inherits from `Iterator`, and most code expect iterators to be proper iterators and iterables to return proper iterators. To create proper iterators, define a class that extends `Iterator`, or use the `Iterator.from()` method.

```js
class MyIterator extends Iterator {
  next() {
    // …
  }
}

const myIterator = Iterator.from({
  next() {
    // …
  },
});
```

Constructor

`Iterator()`

Intended to be extended by other classes that create iterators. Throws an error when constructed by itself.

Static methods

`Iterator.concat()`

Creates a new `Iterator` object from a list of iterable objects. The new iterator yields the values from each of the input iterables in sequence.

`Iterator.from()`

Creates a new `Iterator` object from an iterator or iterable object.

`Iterator.zip()`

Creates a new `Iterator` object that aggregates elements from multiple iterable objects by yielding arrays containing elements at the same position.

`Iterator.zipKeyed()`

Creates a new `Iterator` object that aggregates elements from multiple iterable objects by yielding objects containing elements at the same position, with keys specified by the input.

Instance properties

These properties are defined on `Iterator.prototype` and shared by all `Iterator` instances.

`Iterator.prototype.constructor`

The constructor function that created the instance object. For `Iterator` instances, the initial value is the `Iterator` constructor.

`Iterator.prototype[Symbol.toStringTag]`

The initial value of the `[Symbol.toStringTag]` property is the string `"Iterator"`. This property is used in `Object.prototype.toString()`.

> **Note:** Unlike the `[Symbol.toStringTag]` on most built-in classes, `Iterator.prototype[Symbol.toStringTag]` is writable for web compatibility reasons.

Instance methods

`Iterator.prototype.chunks()` Experimental

Returns a new iterator helper object that splits the elements from the original iterator into consecutive array chunks. Each time the helper is iterated, it gets the specified number of elements from the underlying iterator and yields them together.

`Iterator.prototype.drop()`

Returns a new iterator helper object that skips the given number of elements at the start of this iterator.

`Iterator.prototype.every()`

Returns `false` if it finds an element that does not satisfy the provided testing function. Otherwise, if the iterator is exhausted without finding such an element, it returns `true`.

`Iterator.prototype.filter()`

Returns a new iterator helper object that yields only those elements of the iterator for which the provided callback function returns `true`.

`Iterator.prototype.find()`

Returns the first element produced by the iterator that satisfies the provided testing function. If no values satisfy the testing function, `undefined` is returned.

`Iterator.prototype.flatMap()`

Returns a new iterator helper object that takes each element in the original iterator, runs it through a mapping function, and yields elements returned by the mapping function (which are contained in another iterator or iterable).

`Iterator.prototype.forEach()`

Executes a provided function once for each element produced by the iterator.

`Iterator.prototype.includes()`

Returns `true` if an element produced by the iterator is equal to the given value. Otherwise, if the iterator is exhausted without finding such an element, it returns `false`.

`Iterator.prototype.join()`

Returns a string that is the concatenation of all elements produced by the iterator, separated by commas or a specified separator string.

`Iterator.prototype.map()`

Returns a new iterator helper object that yields elements of the iterator, each transformed by a mapping function.

`Iterator.prototype.reduce()`

Executes a user-supplied "reducer" callback function on each element produced by the iterator, passing in the return value from the calculation on the preceding element. The final result of running the reducer across all elements is a single value.

`Iterator.prototype.some()`

Returns `true` if it finds an element that satisfies the provided testing function. Otherwise, if the iterator is exhausted without finding such an element, it returns `false`.

`Iterator.prototype.take()`

Returns a new iterator helper object that yields the given number of elements in this iterator and then terminates.

`Iterator.prototype.toArray()`

Creates a new `Array` instance populated with the elements yielded from the iterator.

`Iterator.prototype.windows()` Experimental

Returns a new iterator helper object that yields a sliding window of elements. Each time the helper is iterated, it yields an array removing the first element from the previous iteration and adding the next element from the original iterator.

`Iterator.prototype[Symbol.dispose]()`

Calls the `return()` method of `this`, if it exists. This implements the *disposable protocol* and allows it to be disposed when used with `using` or `await using`.

`Iterator.prototype[Symbol.iterator]()`

Returns the iterator object itself. This allows iterator objects to also be iterable.

Examples

Using an iterator as an iterable

All built-in iterators are also iterable, so you can use them in a `for...of` loop:

```js
const arrIterator = [1, 2, 3].values();
for (const value of arrIterator) {
  console.log(value);
}
// Logs: 1, 2, 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator-objects

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Iterator`	38	12	17	25	10	38	17	25	10	3.0	38	10	1.0.0	1.0	0.12.0
`@@dispose`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0
[Symbol.iterator]	38	12	36[1,2,3,4]	25	10	38	36[1,2,3,4]	25	10	3.0	38	10	1.0.0	1.0	0.12.0
`Iterator()` constructor	122	122	131	108	18.4	122	131	81	18.4	26.0	122	18.4	1.1.31	1.39	22.0.0
`chunks` Experimental	**No**	**No**	154	**No**	Prev.	**No**	154	**No**	**No**	**No**	**No**	**No**	1.4.0	?	?
`concat`	146	146	147	130	26.4	146	147	97	26.4	**No**	146	26.4	1.3.7	2.7.2	26.0.0

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`drop`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`every`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`filter`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`find`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`flatMap`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`forEach`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`from`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`includes`	154	154	154	138	Prev.	154	154	103	**No**	**No**	154	**No**	1.4.0	**No**	**No**
`join`	153	153	154	137	Prev.	153	154	103	**No**	**No**	153	**No**	1.4.0	?	?
`map`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`reduce`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`some`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`take`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`toArray`	122[5]	122[5]	131	108[6]	18.4	122[5]	131	81[7]	18.4	26.0[8]	122[5]	18.4	1.1.31	1.39	22.0.0
`windows` Experimental	**No**	**No**	154	**No**	Prev.	**No**	154	**No**	**No**	**No**	**No**	**No**	1.4.0	?	?
`zip`	153	153	148	137	Prev.	153	148	103	**No**	**No**	153	**No**	1.4.0	**No**	**No**
`zipKeyed`	153	153	148	137	Prev.	153	148	103	**No**	**No**	153	**No**	1.4.0	**No**	**No**

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.
5. Also from version 117–119: supported.
6. Also from version 103–105: supported.
7. Also from version 78–79: supported.
8. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator` in `core-js`
- es-shims polyfill of `Iterator` and associated helpers
- `function*`
- Iteration protocols

JavaScript › Reference › Standard built-in objects › Iterator

Iterator() constructor

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/Iterator · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`Iterator()`** constructor is intended to be used as the superclass of other classes that create iterators. It throws an error when constructed by itself.

Syntax

```js
new Iterator()
```

> **Note:** `Iterator()` can only be constructed with `new`. Attempting to call it without `new` throws a `TypeError`. In addition, `Iterator()` cannot actually be constructed itself — it's usually implicitly constructed through `super()` calls inside the constructor of a subclass.

Parameters

None.

Return value

A new `Iterator` object.

Exceptions

`TypeError`

When `new.target` is the `Iterator` function itself, i.e., when the `Iterator` constructor itself is constructed.

Description

`Iterator` represents an *abstract class* — a class that provides common utilities for its subclasses, but is not intended to be instantiated itself. It is the superclass of all other iterator classes, and is used to create subclasses that implement specific iteration algorithms — namely, all subclasses of `Iterator` must implement a `next()` method as required by the iterator protocol. Because `Iterator` doesn't actually provide the `next()` method, it doesn't make sense to construct an `Iterator` directly.

You can also use `Iterator.from()` to create an `Iterator` instance from an existing iterable or iterator object.

Examples

Subclassing Iterator

The following example defines a custom data structure, `Range`, which allows iteration. To make an object iterable, we can provide a `[Symbol.iterator]()` method in the form of a generator function:

```js
class Range {
  #start;
  #end;
  #step;

  constructor(start, end, step = 1) {
    this.#start = start;
    this.#end = end;
    this.#step = step;
  }

  *[Symbol.iterator]() {
    for (let value = this.#start; value <= this.#end; value += this.#step) {
      yield value;
    }
  }
}

const range = new Range(1, 5);
for (const num of range) {
  console.log(num);
}
```

This works, but it isn't as nice as how built-in iterators work. There are two problems:

- The returned iterator inherits from `Generator`, which means modifications to `Generator.prototype` are going to affect the returned iterator, which is a leak of abstraction.
- The returned iterator does not inherit from a custom prototype, which makes it harder if we intend to add extra methods to the iterator.

We can mimic the implementation of built-in iterators, such as map iterators, by subclassing `Iterator`. This enables us to define extra properties, such as `[Symbol.toStringTag]`, while making the iterator helper methods available on the returned iterator.

```js
class Range {
  #start;
  #end;
  #step;

  constructor(start, end, step = 1) {
    this.#start = start;
    this.#end = end;
    this.#step = step;
  }

  static #RangeIterator = class extends Iterator {
    #cur;
    #s;
    #e;
    constructor(range) {
      super();
      this.#cur = range.#start;
      this.#s = range.#step;
      this.#e = range.#end;
    }
    static {
      Object.defineProperty(this.prototype, Symbol.toStringTag, {
```

(code continues on the next page)

(code continued from the previous page)

```
        value: "Range Iterator",
        configurable: true,
        enumerable: false,
        writable: false,
      });

      // Avoid #RangeIterator from being accessible outside
      delete this.prototype.constructor;
    }
    next() {
      if (this.#cur > this.#e) {
        return { value: undefined, done: true };
      }
      const res = { value: this.#cur, done: false };
      this.#cur += this.#s;
      return res;
    }
  };

  [Symbol.iterator]() {
    return new Range.#RangeIterator(this);
  }
}

const range = new Range(1, 5);
for (const num of range) {
  console.log(num);
}
```

The subclassing pattern is useful if you want to create many custom iterators. If you have an existing iterable or iterator object which doesn't inherit from `Iterator`, and you just want to call iterator helper methods on it, you can use `Iterator.from()` to create a one-time `Iterator` instance.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator-constructor`

Browser compatibility

	Desktop					Mobile							Server		
Feature	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`Iterator()` constructor	122	122	131	108	18.4	122	131	81	18.4	26.0	122	18.4	1.1.31	1.39	22.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.Iterator`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator` in `core-js`
- es-shims polyfill of `Iterator` and associated helpers
- `Iterator`
- `Iterator.from()`

Iterator.concat()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/concat · MDN page last modified 2026-10-26

Baseline 2026: Newly available — works across the latest browser versions since March 2026; may not work in older ones.

The **`Iterator.concat()`** static method creates a new `Iterator` object from a list of iterable objects. The new iterator yields the values from each of the input iterables in sequence.

Syntax

```js
Iterator.concat(it)
Iterator.concat(it1, it2)
Iterator.concat(it1, it2, /* …, */ itN)
```

Parameters

`it1`, `it2`, ..., `itN`

An object that implements the iterable protocol. Iterators which are not also iterable cannot be passed directly to this method; they must first be wrapped using `Iterator.from()`.

Return value

A new `Iterator` object that yields the values from each of the input iterables in sequence.

Description

The `Iterator.concat()` method is conceptually similar to `Array`'s `concat()` method, but it operates on any kind of iterable, and returns an iterator instead of an array. This means that the iterables can be lazily iterated, avoiding unnecessary allocation or computation. It also means that, technically, you can concatenate *infinite iterables*, but results from iterables after the first infinite iterable will never be reached.

While each iterable can be infinite, the list of iterables must be finite—and quite limited in number because engines impose a very low limit on the number of function arguments. If you need to concatenate a large—even infinite—number of iterables, use `Iterator.prototype.flatMap()` instead.

```js
function* infiniteIterables() {
  for (let i = 1; ; i++) {
    yield Array(i).fill(i);
  }
}

// BAD:
// Iterator.concat(...infiniteIterables());
// The spread operator never finishes

// GOOD:
const it = infiniteIterables().flatMap((x) => x);
// Infinite sequence of numbers: 1, 2, 2, 3, 3, 3, 4, 4, 4, 4, ...
```

The `Iterator.concat()` method is similar in functionality to the following function, which uses the `yield*` operator to yield values from each of the input iterables in sequence:

```js
function* concat(...iterables) {
  for (const iterable of iterables) {
    yield* iterable;
  }
}
```

Like `yield*`, `Iterator.concat()` doesn't support arguments that are not iterable (i.e., don't have the `[Symbol.iterator]()` method). This is because `Iterator.concat()` always takes ownership over its iterators and closes open iterators when the method exits. With iterable arguments, `Iterator.concat()` acquires iterators one-by-one and just closes the current iterator when the iteration is stopped. With iterator arguments, it's not clear whether the caller or `Iterator.concat()` should be responsible for closing the iterators, especially the ones that `Iterator.concat()` hasn't reached, so the method simply disallows non-iterable arguments.

Examples

Combining maps

In this example, we create a new `Map` that's the union of three other maps. The `Map()` constructor accepts an iterable of key-value pairs, while the Map iterator yields key-value pairs from the map, so just using `Iterator.concat()` on the maps is sufficient to create the new map.

```js
const map1 = new Map([
  ["a", 1],
  ["b", 2],
]);
const map2 = new Map([
  ["c", 3],
  ["d", 4],
]);
const map3 = new Map([
  ["a", 5],
  ["e", 6],
]);
```

(code continues on the next page)

(code continued from the previous page)

```
const map = new Map(Iterator.concat(map1, map2, map3));
console.log(map);
// Map(5) {'a' => 5, 'b' => 2, 'c' => 3, 'd' => 4, 'e' => 6}
```

Note that the value of key `"a"` in the resulting map is `5`. This is because the `Map` constructor uses the last value for each key.

Concatenating infinite iterables

When any of the input iterables is infinite, the resulting iterator will also be infinite. This isn't immediately a problem because iterators can be consumed lazily and closed at any time, but it does mean that iterables after the first infinite iterable will never be reached.

js

```
function* it1() {
  yield 1;
  yield 2;
}

function* it2() {
  let i = 3;
  while (true) {
    yield i++;
  }
}

function* it3() {
  yield "done";
}

const it = Iterator.concat(it1(), it2(), it3());
for (const value of it.take(10)) {
  console.log(value); // 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
}
// "done" is never reached
```

Concatenating iterables of different types

You can concatenate iterables of different types.

js

```
const array = [1, 2, 3];
const set = new Set([4, 5, 6]);
function* gen() {
  yield 7;
  yield 8;
  yield 9;
}

const it = Iterator.concat(array, set, gen());
console.log([...it]); // [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

The elements of each iterable can also be of different types, just like arrays.

```js
const array = [1, "two", 3];
const set = new Set([true, {}]);

const it = Iterator.concat(array, set);
console.log([...it]); // [1, "two", 3, true, {}]
```

Concatenating non-iterable objects

Non-iterable objects throw a `TypeError` when passed to `Iterator.concat()` because they don't have the `[Symbol.iterator]()` method.

```js
const nonIterable = {
  next() {
    return { done: true };
  },
};

const it = Iterator.concat(nonIterable); // TypeError: object is not iterable
```

Because all built-in iterators extend the `Iterator` class, they are all iterable and can be passed to `Iterator.concat()`.

```js
const arrayIterator = [1, 2, 3][Symbol.iterator]();

const it = Iterator.concat(arrayIterator); // No error
```

To pass an iterator that isn't also iterable, you can wrap it using `Iterator.from()`.

```js
const nonIterable = {
  next() {
    return { done: true };
  },
};

const it = Iterator.concat(Iterator.from(nonIterable)); // No error
```

Another option is to use `Iterator.prototype.flatMap()` instead, which automatically calls `Iterator.from()`. But be careful: you need to call `flatMap()` on an iterator, not on an array, because `Array.prototype.flatMap()` only supports array return values.

```js
const nonIterable = {
  next() {
    return { done: true };
  },
};

const it = [nonIterable].values().flatMap((x) => x); // No error
```

When implementing your own iterators, consider making them iterable by or subclassing `Iterator` or adding a `[Symbol.iterator]()` method that returns `this`.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.concat`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`concat`	146	146	147	130	26.4	146	147	97	26.4	**No**	146	26.4	1.3.7	2.7.2	26.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.concat`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.concat` in `core-js`
- es-shims polyfill of `Iterator.concat`
- `Iterator`
- `Iterator.from()`
- `Array.prototype.concat()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.from()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/from · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`Iterator.from()`** static method creates a new `Iterator` object from an iterator or iterable object.

Syntax

```js
Iterator.from(object)
```

Parameters

`object`

An object that implements the iterable protocol or the iterator protocol.

Return value

If `object` is an iterable, its `[Symbol.iterator]()` method is called to obtain the iterator. Otherwise, `object` is assumed to be an iterator. If the iterator is already `instanceof Iterator` (which means it has `Iterator.prototype` in its prototype chain), it is returned directly. Otherwise, a new `Iterator` object is created that wraps the original iterator.

Description

This method exists to convert custom iterators, probably exported by libraries, to proper iterators. All iterator objects returned by `Iterator.from()` inherit from a common prototype object, which has the following methods:

`next()`
Calls the underlying iterator's `next()` method and returns the result.

`return()`
Calls the underlying iterator's `return()` method and returns the result, or returns `{ value: undefined, done: true }` if the underlying iterator doesn't have a `return()` method.

Examples

Converting an iterable to a proper iterator

Because `obj` is already an iterable that returns a proper iterator when its `[Symbol.iterator]()` method is called, `Iterator.from(obj)` returns the same iterator.

```js
const iterator = (function* () {
  yield 1;
  yield 2;
  yield 3;
})();

const obj = {
  [Symbol.iterator]() {
    return iterator;
  },
};

const iterator2 = Iterator.from(obj);
console.log(iterator2 === iterator); // true
```

Because `obj2` is an iterable that returns a non-proper iterator when its `[Symbol.iterator]()` method is called, `Iterator.from(obj2)` returns a new iterator that wraps the original iterator.

```js
const iterator = {
  current: 0,
  next() {
    return { value: this.current++, done: false };
  },
};

const obj2 = {
  [Symbol.iterator]() {
    return iterator;
  },
};

const iterator2 = Iterator.from(obj2);
console.log(iterator2 === iterator); // false
console.log(iterator2.next()); // { value: 0, done: false }
console.log(iterator.next()); // { value: 1, done: false }
```

Converting an iterator to a proper iterator

Because `obj` is already a proper iterator, `Iterator.from(obj)` returns itself.

```js
const obj = (function* () {
  yield 1;
  yield 2;
  yield 3;
})();

const iterator = Iterator.from(obj);
console.log(iterator === obj); // true
```

Because `obj2` is a non-proper iterator, `Iterator.from(obj2)` returns a new iterator that wraps the original iterator.

```js
const obj2 = {
  current: 0,
  next() {
    return { value: this.current++, done: false };
  },
};

const iterator = Iterator.from(obj2);
console.log(iterator === obj2); // false
console.log(iterator.next()); // { value: 0, done: false }
console.log(obj2.next()); // { value: 1, done: false }
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.from`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`from`	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.from`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Iterator.from` in `core-js`
- es-shims polyfill of `Iterator.from`
- `Iterator`
- `Array.from()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.zip()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/zip · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Iterator.zip()`** static method creates a new `Iterator` object that aggregates elements from multiple iterable objects by yielding arrays containing elements at the same position. It essentially "zips" the input iterables together, allowing simultaneous iteration over them.

The `Iterator.zipKeyed()` method is similar, but yields objects instead of arrays with keys you can specify.

Syntax

```js
Iterator.zip(iterables)
Iterator.zip(iterables, options)
```

Parameters

`iterables`

An iterable of iterables whose elements are aggregated. It must be iterable and cannot be an iterator. It should be finite, although its elements can be infinite iterables. Each element must implement either the iterable protocol or, failing that, the iterator protocol. Strings are rejected: to zip strings, convert them to iterators explicitly using `Iterator.from()`.

`options` Optional

An object specifying behavior in case of inconsistent input lengths. It may have the following properties:

`mode` Optional

One of the following:

- `"shortest"` (default): The resulting iterator stops when one input iterable is exhausted.
- `"longest"`: The resulting iterator stops when all input iterables are exhausted. Missing values from shorter iterables are filled according to the `padding` option.
- `"strict"`: A `TypeError` is thrown if not all input iterables finish at the same time.

`padding` Optional

An iterable object (not iterator). Only retrieved and validated when `mode` is `"longest"`. If `undefined` or absent, missing values from shorter iterables are filled with `undefined` (which is equivalent to passing an empty iterable). If an iterable is provided, it is iterated for the number of times equal to the number of elements in `iterables` *as soon as `Iterator.zip()` is called*. `padding[i]` is used for missing values for `iterables[i]` (assuming `padding` and `iterables` are

provided as arrays; they don't have to be). If `padding` is shorter than `iterables`, `undefined` is used for the remaining iterables.

Return value

A new `Iterator` object. Each of its elements is an array with length equal to the number of input iterables, containing the elements from each input iterable at the corresponding position. If the `iterables` object is empty, the resulting iterator is created as completed.

Description

The `Iterator.zip()` function behaves like a transpose operation, yielding arrays containing the elements at matching positions in each of the inputs. If we represent iterables as arrays, the input may looks like this:

```js
[
  [a1, a2, a3, a4], // Iterable a
  [b1, b2, b3], // Iterable b
  [c1, c2, c3, c4, c5], // Iterable c
];
```

The resulting iterator, regardless of the options, will start by yielding the following arrays:

```js
[a1, b1, c1];
[a2, b2, c2];
[a3, b3, c3];
```

After the first three arrays are yielded, the input iterable `b` is exhausted on the fourth `next()` call—it returns `{ done: true }`. What happens next depends on the `mode` option. If `mode` is `"shortest"` (the default), the resulting iterator stops here: the other two input iterators are closed. If `mode` is `"strict"`, an error is thrown because the other two iterables are *not* finished when the second one yields the result `{ done: true }`. If `mode` is `"longest"`, the resulting iterator continues yielding arrays, filling missing values. For example, if `padding` is not provided, it defaults to `undefined`:

```js
[a4, undefined, c4];
[undefined, undefined, c5];
```

If `padding` is provided as an iterable, because there are three input iterables, the first three values from the `padding` iterable are used to fill missing values. Let's suppose that `padding` is an array with values `[p1, p2, p3]`. Then `p2` is used to fill the missing value from the input iterable `b`, and `p1` is used to fill the missing value from the input iterable `a`:

```js
[a4, p2, c4];
[p1, p2, c5];
```

If the `padding` iterable has fewer than three values, the remaining missing values are filled with `undefined`.

Examples

Iteration over a map with indices

Using `Iterator.zip()`, you can iterate over any iterable object (strings aren't supported by default) while also having access to an incrementing counter:

```js
const ages = new Map([
  ["Caroline", 30],
  ["Danielle", 25],
  ["Evelyn", 35],
]);

const numbers = (function* () {
  let n = 0;
  while (true) {
    yield n++;
  }
})();
for (const [index, [name, age]] of Iterator.zip([numbers, ages])) {
  console.log(`${index}: ${name} is ${age} years old.`);
}

// Output:
// 0: Caroline is 30 years old.
// 1: Danielle is 25 years old.
// 2: Evelyn is 35 years old.
```

`numbers` is an infinite iterator that generates incrementing numbers starting from `0`. Because `Iterator.zip()` stops by default when the shortest input iterable is exhausted, the loop iterates exactly three times. The `numbers` iterator is properly closed after the loop ends; it doesn't cause an infinite loop.

Creating a Map from lists of keys and values

Suppose you have two arrays: one with keys and another with values. You can use `Iterator.zip()` to combine them into a `Map`:

```js
const days = ["Mon", "Tue", "Wed", "Thu", "Fri"];
const temperatures = [22, 21, 23, 20, 19];

const dayTemperatureMap = new Map(Iterator.zip([days, temperatures]));
console.log(dayTemperatureMap);
// Map(5) { 'Mon' => 22, 'Tue' => 21, 'Wed' => 23, 'Thu' => 20, 'Fri' => 19 }
```

Joint iteration over multiple data sources

Suppose you have data coming from multiple sources, such as multiple microservices or databases. You know that each source provides related data in the same order, and you want to process them together. You can use `Iterator.zip()` to achieve this:

```js
const names = fetchNames(); // e.g., ["Caroline", "Danielle", "Evelyn"]
const ages = fetchAges(); // e.g., [30, 25, 35]
const cities = fetchCities(); // e.g., ["New York", "London", "Hong Kong"]

for (const [name, age, city] of Iterator.zip([names, ages, cities])) {
  console.log(`${name}, aged ${age}, lives in ${city}.`);
}

// Output:
// Caroline, aged 30, lives in New York.
// Danielle, aged 25, lives in London.
// Evelyn, aged 35, lives in Hong Kong.
```

Providing padding for uneven iterables

When zipping iterables of different lengths with `mode` set to `"longest"`, you can provide a `padding` iterable to specify the values used to fill in missing entries:

```js
const letters = ["a", "b", "c", "d", "e"];
const numbers = [1, 2, 3];

// One padding value per iterable
const padding = ["[Letter missing]", "[Number missing]"];
const it = Iterator.zip([letters, numbers], { mode: "longest", padding });
for (const [letter, number] of it) {
  console.log(`${letter}: ${number}`);
}
// Output:
// a: 1
// b: 2
// c: 3
// d: [Number missing]
// e: [Number missing]
```

Zipping strings

Strings are not accepted as input iterables to `Iterator.zip()`, because it is now considered a mistake to make strings implicitly iterable. To zip strings, convert them to iterators explicitly using `Iterator.from()`:

```js
const str1 = "abc";
const str2 = "1234";
const it = Iterator.zip([Iterator.from(str1), Iterator.from(str2)]);
for (const [char1, char2] of it) {
  console.log(`${char1} - ${char2}`);
}
// Output:
// a - 1
// b - 2
// c - 3
```

In some cases, you may wish to split by grapheme clusters instead of code units. In that case, you can use the `Intl.Segmenter` API:

```js
const segmenter = new Intl.Segmenter("en-US", { granularity: "grapheme" });
const str1 = "👨‍👩‍👧";
const str2 = "123";
const it = Iterator.zip([
  segmenter.segment(str1).map(({ segment }) => segment),
  segmenter.segment(str2).map(({ segment }) => segment),
]);
for (const [char1, char2] of it) {
  console.log(`${char1} - ${char2}`);
}
// Output:
// 👨 - 1
// 👩 - 2
// 👧 - 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.zip

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
zip	153	153	148	137	Prev.	153	148	103	**No**	**No**	153	**No**	1.4.0	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.zip`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.zip` in `core-js`
- es-shims polyfill of `Iterator.zip`
- `Iterator`
- `Iterator.zipKeyed()`
- `Iterator.from()`
- `Iterator.concat()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.zipKeyed()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/zipKeyed · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`Iterator.zipKeyed()`** static method creates a new `Iterator` object that aggregates elements from multiple iterable objects by yielding objects containing elements at the same position, with keys specified by the input. It essentially "zips" the input iterables together, allowing simultaneous iteration over them.

The `Iterator.zip()` method is similar, but yields arrays instead of objects.

Syntax

```js
Iterator.zipKeyed(iterables)
Iterator.zipKeyed(iterables, options)
```

Parameters

`iterables`

An object. Each property's key is used as the key in the resulting objects. The property's value must implement either the iterable protocol or, failing that, the iterator protocol. These iterables may be infinite. Strings are rejected: to zip strings, convert them to iterators explicitly using `Iterator.from()`.

`options` Optional

An object specifying behavior in case of inconsistent input lengths. It may have the following properties:

`mode` Optional

One of the following:

- `"shortest"` (default): The resulting iterator stops when one input iterable is exhausted.
- `"longest"`: The resulting iterator stops when all input iterables are exhausted. Missing values from shorter iterables are filled according to the `padding` option.
- `"strict"`: A `TypeError` is thrown if not all input iterables finish at the same time.

`padding` Optional

An object. Only retrieved and validated when `mode` is `"longest"`. If `undefined` or absent, missing values from shorter iterables are filled with `undefined` (which is equivalent to passing an empty object). If an object is provided, each key of the `iterables` argument is retrieved *as soon as `Iterator.zipKeyed()` is called*. `padding[key]` is used for missing values for `iterables[key]`. If the `padding` object is missing some keys, those keys are filled with `undefined`.

Return value

A new `Iterator` object. Each of its elements is an object with the same keys as the `iterables` argument, containing the elements from each input iterable at the corresponding position.

Description

The `Iterator.zipKeyed()` function behaves like `Iterator.zip()`; the only difference is that you can specify the keys used in the resulting objects, while `Iterator.zip()` always uses numeric indices (by yielding arrays).

If we represent iterables as arrays, the input may looks like this:

```js
({
  a: [a1, a2, a3, a4],
  b: [b1, b2, b3],
  c: [c1, c2, c3, c4, c5],
});
```

The resulting iterator, regardless of the options, will start by yielding the following arrays:

```js
({ a: a1, b: b1, c: c1 });
({ a: a2, b: b2, c: c2 });
({ a: a3, b: b3, c: c3 });
```

After the first three objects are yielded, the input iterable `b` is exhausted on the fourth `next()` call—it returns `{ done: true }`. What happens next depends on the `mode` option. If `mode` is `"shortest"` (the default), the resulting iterator stops here: the other two input iterators are closed. If `mode` is `"strict"`, an error is thrown because the other two iterables are *not* finished when the second one yields the result `{ done: true }`. If `mode` is `"longest"`, the resulting iterator continues yielding objects, filling missing values. For example, if `padding` is not provided, it defaults to `undefined`:

```js
({ a: a4, b: undefined, c: c4 });
({ a: undefined, b: undefined, c: c5 });
```

If `padding` is provided as an object, it may look like `{ a: p1, b: p2, c: p3 }`. Then `p2` is used to fill the missing value from the input iterable `b`, and `p1` is used to fill the missing value from the input iterable `a`:

```js
({ a: a4, b: p2, c: c4 });
({ a: p1, b: p2, c: c5 });
```

Examples

Transposing tabular data

There are two common ways to represent tabular data: as an object where each property is a column, or as an array of objects where each object is a row. This example shows how you can iterate the column-based representation by rows using `Iterator.zipKeyed()`.

```js
const table = {
  name: ["Caroline", "Danielle", "Evelyn"],
  age: [30, 25, 35],
  city: ["New York", "London", "Hong Kong"],
};

for (const { name, age, city } of Iterator.zipKeyed(table)) {
  console.log(`${name}, aged ${age}, lives in ${city}.`);
}

// Output:
// Caroline, aged 30, lives in New York.
// Danielle, aged 25, lives in London.
// Evelyn, aged 35, lives in Hong Kong.
```

Most use cases of `Iterator.zipKeyed()` are identical to those of `Iterator.zip()`. Which one to use depends on whether you already have an object of iterables (use `zipKeyed()`) or an array of iterables (use `zip()`). We recommend using `zipKeyed()` when possible, because specifying explicit keys makes it harder to accidentally mix up the order of iterables.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.zipkeyed`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`zipKeyed`	153	153	148	137	Prev.	153	148	103	**No**	**No**	153	**No**	1.4.0	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.zipKeyed`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Iterator.zipKeyed` in `core-js`
- es-shims polyfill of `Iterator.zipKeyed`
- `Iterator`
- `Iterator.zip()`
- `Iterator.from()`
- `Iterator.concat()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.chunks()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/chunks · MDN page last modified 2026-10-26

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`chunks()`** method of `Iterator` instances returns a new iterator helper object that splits the elements from the original iterator into consecutive array chunks. Each time the helper is iterated, it gets the specified number of elements from the underlying iterator and yields them together.

To yield overlapping sequences (i.e., sliding window), see `Iterator.prototype.windows()`.

Syntax

```
chunks(chunkSize)
```

Parameters

`chunkSize`

The number of elements in each chunk. Must be a positive integer less than 2^{32} (the maximum array length).

Return value

A new iterator helper object. Each time the returned iterator helper's `next()` method is called, the original iterator is advanced by `chunkSize` elements, and those elements are yielded together as an array.

If the original iterator has some but fewer than `chunkSize` elements remaining, those elements are still yielded as an array (so the length is less than `chunkSize`), and the iterator helper will be immediately completed the next time `next()` is called.

If the original iterator has no elements remaining, the iterator helper is immediately completed without yielding an empty array.

Exceptions

`TypeError`

Thrown if `chunkSize` is not an integer.

`RangeError`

Thrown if `chunkSize` is less than 1 or greater than $2^{32} - 1$.

Examples

Printing elements in a grid

The following example creates an iterator that yields terms in the Fibonacci sequence. Then, they are printed in a grid layout by retrieving 5 elements each time.

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const rows = fibonacci().chunks(5);

for (const row of rows.take(5)) {
  console.log(row.join("\t"));
}
```

Output:

```
1       1       2       3       5
8       13      21      34      55
89      144     233     377     610
987     1597    2584    4181    6765
10946   17711   28657   46368   75025
```

Specifications

Specification
Iterator Chunking `https://tc39.es/proposal-iterator-chunking/#sec-iterator.prototype.chunks`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
chunks Experimental	**No**	**No**	154	**No**	Prev.	**No**	154	**No**	**No**	**No**	**No**	**No**	1.4.0	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.chunks`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.chunks` in `core-js`
- es-shims polyfill of `Iterator.prototype.chunks`
- `Iterator`
- `Iterator.prototype.windows()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.drop()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/drop · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`drop()`** method of `Iterator` instances returns a new iterator helper object that skips the given number of elements at the start of this iterator.

Syntax

```
drop(limit)
```

Parameters

`limit`

The number of elements to drop from the start of the iteration.

Return value

A new iterator helper object. The first time the returned iterator helper's `next()` method is called, the current iterator is immediately advanced by `limit` elements, and then the next element (the `limit+1`-th element) is yielded. The iterator helper then yields the remaining elements one-by-one. If the current iterator has fewer than `limit` elements, the new iterator helper will be immediately completed the first time `next()` is called.

Exceptions

`RangeError`

Thrown if `limit` becomes `NaN` or negative when converted to an integer.

Examples

Using drop()

The following example creates an iterator that yields terms in the Fibonacci sequence, starting from the 3rd term by dropping the first two terms:

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const seq = fibonacci().drop(2);
console.log(seq.next().value); // 2
console.log(seq.next().value); // 3
```

This is equivalent to:

```js
const seq = fibonacci();
seq.next();
seq.next();
```

Using drop() with a for...of loop

`drop()` is most convenient when you are not hand-rolling the iterator. Because iterators are also iterable, you can iterate the returned helper with a `for...of` loop:

```js
for (const n of fibonacci().drop(2)) {
  console.log(n);
  if (n > 30) {
    break;
  }
}

// Logs:
// 2
// 3
// 5
// 8
// 13
// 21
// 34
```

Combining drop() with take()

You can combine `drop()` with `Iterator.prototype.take()` to get a slice of an iterator:

```js
for (const n of fibonacci().drop(2).take(5)) {
  // Drops the first two elements, then takes the next five
  console.log(n);
}
// Logs:
// 2
// 3
// 5
// 8
// 13

for (const n of fibonacci().take(5).drop(2)) {
  // Takes the first five elements, then drops the first two
  console.log(n);
}
// Logs:
// 2
// 3
// 5
```

Lower and upper bounds of drop count

When the `limit` is negative or `NaN`, a `RangeError` is thrown:

```js
fibonacci().drop(-1); // RangeError: -1 must be positive
fibonacci().drop(undefined); // RangeError: undefined must be positive
```

When the `limit` is larger than the total number of elements the iterator can produce (such as `Infinity`), the returned iterator helper will instantly drop all elements and then be completed the first time `next()` is called. If the current iterator is infinite, the returned iterator helper will never complete.

```js
fibonacci().drop(Infinity).next(); // Never ends
new Set([1, 2, 3]).values().drop(Infinity).next(); // { value: undefined, done: true }
new Set([1, 2, 3]).values().drop(4).next(); // { value: undefined, done: true }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.drop

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
drop	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.drop`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.drop` in `core-js`
- es-shims polyfill of `Iterator.prototype.drop`
- `Iterator`
- `Iterator.prototype.take()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.every()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/every · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`every()`** method of `Iterator` instances is similar to `Array.prototype.every()`: it returns `false` if it finds an element that does not satisfy the provided testing function. Otherwise, if the iterator is exhausted without finding such an element, it returns `true`.

Syntax

```js
every(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

`true` if `callbackFn` returns a truthy value for every element. Otherwise, `false`.

Description

`every()` iterates the iterator and invokes the `callbackFn` function once for each element. It returns `false` immediately if the callback function returns a falsy value. Otherwise, it iterates until the end of the iterator and returns `true`. If `every()` returns `false`, the underlying iterator is closed by calling its `return()` method.

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. With infinite iterators, `every()` returns `false` as soon as the first falsy value is found. If the `callbackFn` always returns a truthy value, the method never returns.

Calling `every()` always closes the underlying iterator, even if the method early-returns. The iterator is never left in a half-way state.

Examples

Using every()

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}
```

(code continues on the next page)

(code continued from the previous page)

```
const isEven = (x) => x % 2 === 0;
console.log(fibonacci().every(isEven)); // false

const isPositive = (x) => x > 0;
console.log(fibonacci().take(10).every(isPositive)); // true
console.log(fibonacci().every(isPositive)); // Never completes
```

The method closes the iterator after returning.

js

```
const seq = fibonacci();
console.log(seq.every(isEven)); // false
console.log(seq.next()); // { value: undefined, done: true }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.every

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
every	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.every`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.every` in `core-js`
- es-shims polyfill of `Iterator.prototype.every`
- `Iterator`
- `Iterator.prototype.find()`
- `Iterator.prototype.some()`
- `Array.prototype.every()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.filter()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/filter · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`filter()`** method of `Iterator` instances returns a new iterator helper object that yields only those elements of the iterator for which the provided callback function returns `true`.

Syntax

```
filter(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. It should return a truthy value to make the element yielded by the iterator helper, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

A new iterator helper object. Each time the iterator helper's `next()` method is called, it returns the next element in the iterator for which the callback function returns `true`. When the underlying iterator is completed, the iterator helper object is also completed (the `next()` method produces `{ value: undefined, done: true }`).

Description

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators.

Examples

Using filter()

The following example creates an iterator that yields terms in the Fibonacci sequence, and then reads the first few terms that are even:

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const seq = fibonacci().filter((x) => x % 2 === 0);
console.log(seq.next().value); // 2
console.log(seq.next().value); // 8
console.log(seq.next().value); // 34
```

Using filter() with a for...of loop

`filter()` is most convenient when you are not hand-rolling the iterator. Because iterators are also iterable, you can iterate the returned helper with a `for...of` loop:

```js
for (const n of fibonacci().filter((x) => x % 2 === 0)) {
  console.log(n);
  if (n > 30) {
    break;
  }
}

// Logs:
// 2
// 8
// 34
```

This is equivalent to:

```js
for (const n of fibonacci()) {
  if (n % 2 !== 0) {
    continue;
  }
  console.log(n);
  if (n > 30) {
    break;
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.filter

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`filter`	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.filter`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.filter` in `core-js`
- es-shims polyfill of `Iterator.prototype.filter`
- `Iterator`
- `Iterator.prototype.forEach()`
- `Iterator.prototype.every()`
- `Iterator.prototype.map()`
- `Iterator.prototype.some()`
- `Iterator.prototype.reduce()`
- `Array.prototype.filter()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.find()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/find · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`find()`** method of `Iterator` instances is similar to `Array.prototype.find()`: it returns the first element produced by the iterator that satisfies the provided testing function. If no values satisfy the testing function, `undefined` is returned.

Syntax

```
find(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. It should return a truthy value to indicate a matching element has been found, and a falsy value otherwise. The function is called with the following arguments:

`element`
The current element being processed.

`index`
The index of the current element being processed.

Return value

The first element produced by the iterator that satisfies the provided testing function. Otherwise, `undefined` is returned.

Description

`find()` iterates the iterator and invokes the `callbackFn` function once for each element. It returns the element immediately if the callback function returns a truthy value. Otherwise, it iterates until the end of the iterator and returns `undefined`. If `find()` returns an element, the underlying iterator is closed by calling its `return()` method.

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. With infinite iterators, `find()` returns the first satisfying element as soon as it is found. If the `callbackFn` always returns a falsy value, the method never returns.

Calling `find()` always closes the underlying iterator, even if the method early-returns. The iterator is never left in a half-way state.

Examples

Using find()

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const isEven = (x) => x % 2 === 0;
console.log(fibonacci().find(isEven)); // 2

const isNegative = (x) => x < 0;
console.log(fibonacci().take(10).find(isNegative)); // undefined
console.log(fibonacci().find(isNegative)); // Never completes
```

The method closes the iterator after returning.

```js
const seq = fibonacci();
console.log(seq.find(isEven)); // 2
console.log(seq.next()); // { value: undefined, done: true }
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.find`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`find`	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.find`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.find` in `core-js`
- es-shims polyfill of `Iterator.prototype.find`
- `Iterator`
- `Iterator.prototype.every()`
- `Iterator.prototype.some()`
- `Array.prototype.find()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.flatMap()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/flatMap · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`flatMap()`** method of `Iterator` instances returns a new iterator helper object that takes each element in the original iterator, runs it through a mapping function, and yields elements returned by the mapping function (which are contained in another iterator or iterable).

Syntax

```js
flatMap(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. It should return an iterator or iterable that yields elements to be yielded by `flatMap()`. Note that unlike `Array.prototype.flatMap()`, you cannot return single non-iterator/iterable values. The function is called with the following arguments:

`element`

The current element being processed in the array.

`index`

The index of the current element being processed in the array.

Return value

A new iterator helper object. The first time the iterator helper's `next()` method is called, it calls `callbackFn` on the first element produced by the underlying iterator, and the return value, which should be an iterator or iterable, is yielded one-by-one by the iterator helper (like `yield*`). The next element is fetched from the underlying iterator when the previous one returned by `callbackFn` is completed. When the underlying iterator is completed, the iterator helper is also completed (the `next()` method produces `{ value: undefined, done: true }`).

Exceptions

`TypeError`

Thrown if `callbackFn` returns a non-iterator/iterable value or a string primitive.

Description

`flatMap` accepts two kinds of return values from `callbackFn`: an iterator or iterable. They are handled in the same way as `Iterator.from()`: if the return value is iterable, the `[Symbol.iterator]()` method is called and the return value is used; otherwise, the return value is treated as an iterator and its `next()` method is called.

js

```
[1, 2, 3]
  .values()
  .flatMap((x) => {
    let itDone = false;
    const it = {
      next() {
        if (itDone) {
          return { value: undefined, done: true };
        }
        itDone = true;
        return { value: x, done: false };
      },
    };
    switch (x) {
      case 1:
        // An iterable that's not an iterator
        return { [Symbol.iterator]: () => it };
      case 2:
        // An iterator that's not an iterable
        return it;
      case 3:
```

(code continues on the next page)

(code continued from the previous page)

```
        // An iterable iterator is treated as an iterable
        return {
          ...it,
          [Symbol.iterator]() {
            console.log("Symbol.iterator called");
            return it;
          },
        };
      default:
        return undefined;
    }
  })
  .toArray();
// Logs "Symbol.iterator called"
// Returns [1, 2, 3]
```

Examples

Merging maps

The following example merges two `Map` objects into one:

js

```
const map1 = new Map([
  ["a", 1],
  ["b", 2],
  ["c", 3],
]);
const map2 = new Map([
  ["d", 4],
  ["e", 5],
  ["f", 6],
]);

const merged = new Map([map1, map2].values().flatMap((x) => x));
console.log(merged.get("a")); // 1
console.log(merged.get("e")); // 5
```

This avoids creating any temporary copies of the map's content. Note that the array `[map1, map2]` must first be converted to an iterator (using `Array.prototype.values()`), because `Array.prototype.flatMap()` only flattens arrays, not iterables.

js

```
new Map([map1, map2].flatMap((x) => x)); // Map(1) {undefined => undefined}
```

Returning strings

Strings are iterable, but `flatMap()` specifically rejects string primitives returned from `callbackFn`, this is because the behavior of iterating by code points is often not what you want.

js · ✗ bad example

```
[1, 2, 3]
  .values()
  .flatMap((x) => String(x))
  .toArray(); // TypeError: Iterator.prototype.flatMap called on non-object
```

You may want to wrap it in an array instead so the entire string is yielded as one:

```js
[1, 2, 3]
  .values()
  .flatMap((x) => [String(x)])
  .toArray(); // ['1', '2', '3']
```

Or, if the behavior of iterating by code points is intended, you can use `Iterator.from()` to convert it to a proper iterator:

```js
[1, 2, 3]
  .values()
  .flatMap((x) => Iterator.from(String(x * 10)))
  .toArray();
// ['1', '0', '2', '0', '3', '0']
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.flatmap

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`flatMap`	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.flatMap`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.flatMap` in `core-js`
- es-shims polyfill of `Iterator.prototype.flatMap`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.forEach()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/forEach · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`forEach()`** method of `Iterator` instances is similar to `Array.prototype.forEach()`: it executes a provided function once for each element produced by the iterator.

Syntax

```js
forEach(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. Its return value is discarded. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

`undefined`.

Description

`forEach()` iterates the iterator and invokes the `callbackFn` function once for each element. Unlike most other iterator helper methods, it does not work with infinite iterators, because it is not lazy.

Examples

Using forEach()

```js
new Set([1, 2, 3]).values().forEach((v) => console.log(v));

// Logs:
// 1
// 2
// 3
```

This is equivalent to:

```js
for (const v of new Set([1, 2, 3]).values()) {
  console.log(v);
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.foreach`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`forEach`	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.forEach`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.forEach` in `core-js`
- es-shims polyfill of `Iterator.prototype.forEach`
- `Iterator`
- `Iterator.prototype.find()`
- `Iterator.prototype.map()`
- `Iterator.prototype.filter()`
- `Iterator.prototype.every()`
- `Iterator.prototype.some()`
- `Array.prototype.forEach()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.includes()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/includes · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`includes()`** method of `Iterator` instances is similar to `Array.prototype.includes()`: it returns `true` if it finds an element that is equal to the given value. Otherwise, if the iterator is exhausted without finding such an element, it returns `false`.

Syntax

```js
includes(searchElement)
includes(searchElement, fromIndex)
```

Parameters

`searchElement`

The value to search for.

`fromIndex` Optional

Zero-based index at which to start searching. Must be a non-negative integer, `Infinity`, or `undefined`. If `fromIndex` is greater than or equal to the number of elements produced by the iterator (including when `fromIndex` is `Infinity`), the method always returns `false` after exhausting the iterator. If `fromIndex` is `undefined`, it defaults to `0`.

Return value

A boolean value which is `true` if the value `searchElement` is found within the iterator (or the part of the iterator starting at `fromIndex`, if specified).

Exceptions

`TypeError`

Thrown if `fromIndex` is not one of: an integer, `Infinity`, `-Infinity`, or `undefined`.

`RangeError`

Thrown if `fromIndex` is negative.

Description

The `includes()` method compares `searchElement` to elements of the array using the SameValueZero algorithm. This algorithm works like strict equality `===` (where `-0` and `+0` are considered equal), with the exception that `NaN` is considered equal to itself.

Unlike `Array.prototype.includes()`, the `fromIndex` parameter of `Iterator.prototype.includes()` is not allowed to be negative, because the iterator does not have a known length. The type validation is also stricter: non-integer values are not truncated to integers.

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. With infinite iterators, `includes()` returns `true` as soon as the first match is found. If the value is never encountered, the method never returns.

Calling `includes()` always closes the underlying iterator, even if the method early-returns. The iterator is never left in a half-way state.

Examples

Using includes()

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

console.log(fibonacci().includes(8)); // true
console.log(fibonacci().take(10).includes(7)); // false
console.log(fibonacci().includes(7)); // Never completes
```

The method closes the iterator after returning.

```js
const seq = fibonacci();
console.log(seq.includes(8)); // true
console.log(seq.next()); // { value: undefined, done: true }
```

Using fromIndex

`fromIndex` specifies the number of elements to be skipped from the beginning. It is equivalent to calling `drop(fromIndex).includes(searchElement)`.

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

console.log(fibonacci().includes(8, 3)); // true
console.log(fibonacci().includes(8, 10)); // false
```

If `fromIndex` is greater than or equal to the elements available, `false` is returned.

```js
["a", "b", "c"].values().includes("a", 3); // false
```

Specifications

Specification
Iterator Includes https://tc39.es/proposal-iterator-includes/#sec-iterator.prototype.includes

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`includes`	154	154	154	138	Prev.	154	154	103	**No**	**No**	154	**No**	1.4.0	**No**	**No**

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.includes`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.includes` in `core-js`
- es-shims polyfill of `Iterator.prototype.includes`
- `Iterator`
- `Iterator.prototype.every()`
- `Iterator.prototype.find()`
- `Iterator.prototype.some()`
- `Array.prototype.includes()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.join()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/join · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`join()`** method of `Iterator` instances is similar to `Array.prototype.join()`: it returns a string that is the concatenation of all elements produced by the iterator, separated by commas or a specified separator string. If the iterator has only one item, that item's stringification is returned without using the separator.

Syntax

```
join()
join(separator)
```

Parameters

`separator` Optional

A string to separate each pair of adjacent elements of the iterator. If omitted, the elements are separated with a comma (",").

Return value

A string joining all yielded elements. The elements are converted to strings. If an element is `undefined` or `null`, it is converted to an empty string instead of the string `"null"` or `"undefined"`. If the iterator is empty, the empty string is returned.

Description

See `Array.prototype.join()` for details about how `join()` works. Unlike most other iterator helper methods, it does not work well with infinite iterators, because it is not lazy.

Examples

Using join()

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

console.log(fibonacci().take(5).join()); // "1,1,2,3,5"
console.log(fibonacci().take(5).join(" - ")); // "1 - 1 - 2 - 3 - 5"
```

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.join`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`join`	153	153	154	137	Prev.	153	154	103	**No**	**No**	153	**No**	1.4.0	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.join`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Iterator.prototype.join` in `core-js`
- es-shims polyfill of `Iterator.prototype.join`
- `Iterator`
- `Iterator.prototype.reduce()`
- `Array.prototype.join()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.map()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/map · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`map()`** method of `Iterator` instances returns a new iterator helper object that yields elements of the iterator, each transformed by a mapping function.

Syntax

```js
map(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. Its return value is yielded by the iterator helper. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

A new iterator helper object. Each time the iterator helper's `next()` method is called, it gets the next element from the underlying iterator, applies `callbackFn`, and yields the return value. When the underlying iterator is completed, the iterator helper is also completed (the `next()` method produces `{ value: undefined, done: true }`).

Description

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. The `map()` method allows you to create a new iterator that, when iterated, produces transformed elements.

Examples

Using map()

The following example creates an iterator that yields terms in the Fibonacci sequence, transforms it into a new sequence with each term squared, and then reads the first few terms:

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const seq = fibonacci().map((x) => x ** 2);
console.log(seq.next().value); // 1
console.log(seq.next().value); // 1
console.log(seq.next().value); // 4
```

Using map() with a for...of loop

`map()` is most convenient when you are not hand-rolling the iterator. Because iterators are also iterable, you can iterate the returned helper with a `for...of` loop:

```js
for (const n of fibonacci().map((x) => x ** 2)) {
  console.log(n);
  if (n > 30) {
    break;
  }
}

// Logs:
// 1
// 1
// 4
// 9
// 25
// 64
```

This is equivalent to:

```js
for (const n of fibonacci()) {
  const n2 = n ** 2;
  console.log(n2);
  if (n2 > 30) {
    break;
  }
}
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.map

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`map`	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.map`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.map` in `core-js`
- es-shims polyfill of `Iterator.prototype.map`
- `Iterator`
- `Iterator.prototype.flatMap()`
- `Array.prototype.reduce()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.reduce()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/reduce · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`reduce()`** method of `Iterator` instances is similar to `Array.prototype.reduce`: it executes a user-supplied "reducer" callback function on each element produced by the iterator, passing in the return value from the calculation on the preceding element. The final result of running the reducer across all elements is a single value.

Syntax

```js
reduce(callbackFn)
reduce(callbackFn, initialValue)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. Its return value becomes the value of the `accumulator` parameter on the next invocation of `callbackFn`. For the last invocation, the return value becomes the return value of `reduce()`. The function is called with the following arguments:

`accumulator`

The value resulting from the previous call to `callbackFn`. On the first call, its value is `initialValue` if the latter is specified; otherwise its value is the first element of the iterator.

`currentValue`

The value of the current element. On the first call, its value is the first element of the iterator if `initialValue` is specified; otherwise its value is the second element.

`currentIndex`

The index position of `currentValue`. On the first call, its value is `0` if `initialValue` is specified, otherwise `1`.

`initialValue` Optional

A value to which `accumulator` is initialized the first time the callback is called. If `initialValue` is specified, `callbackFn` starts executing with the first element as `currentValue`. If `initialValue` is *not* specified, `accumulator` is initialized to the first element, and `callbackFn` starts executing with the second element as `currentValue`. In this case, if the iterator is empty (so that there's no first value to return as `accumulator`), an error is thrown.

Return value

The value that results from running the "reducer" callback function to completion over the entire iterator.

Exceptions

`TypeError`

Thrown if the iterator contains no elements and `initialValue` is not provided.

Description

See `Array.prototype.reduce()` for details about how `reduce()` works. Unlike most other iterator helper methods, it does not work well with infinite iterators, because it is not lazy.

Examples

Using reduce()

The following example creates an iterator that yields terms in the Fibonacci sequence, and then sums the first ten terms:

js

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

console.log(
  fibonacci()
    .take(10)
    .reduce((a, b) => a + b),
); // 143
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.reduce

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
reduce	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.reduce`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.reduce` in `core-js`
- es-shims polyfill of `Iterator.prototype.reduce`
- `Iterator`
- `Iterator.prototype.map()`
- `Iterator.prototype.flatMap()`
- `Array.prototype.reduce()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.some()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/some · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`some()`** method of `Iterator` instances is similar to `Array.prototype.some()`: it returns `true` if it finds an element that satisfies the provided testing function. Otherwise, if the iterator is exhausted without finding such an element, it returns `false`.

Syntax

```js
some(callbackFn)
```

Parameters

`callbackFn`

A function to execute for each element produced by the iterator. It should return a truthy value to indicate the element passes the test, and a falsy value otherwise. The function is called with the following arguments:

`element`

The current element being processed.

`index`

The index of the current element being processed.

Return value

`true` if the callback function returns a truthy value for at least one element. Otherwise, `false`.

Description

`some()` iterates the iterator and invokes the `callbackFn` function once for each element. It returns `true` immediately if the callback function returns a truthy value. Otherwise, it iterates until the end of the iterator and returns `false`. If `some()` returns `true`, the underlying iterator is closed by calling its `return()` method.

The main advantage of iterator helpers over array methods is that they are lazy, meaning that they only produce the next value when requested. This avoids unnecessary computation and also allows them to be used with infinite iterators. With infinite iterators, `some()` returns `true` as soon as the first truthy value is found. If the `callbackFn` always returns a falsy value, the method never returns.

Calling `some()` always closes the underlying iterator, even if the method early-returns. The iterator is never left in a half-way state.

Examples

Using some()

js

```
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const isEven = (x) => x % 2 === 0;
console.log(fibonacci().some(isEven)); // true

const isNegative = (x) => x < 0;
console.log(fibonacci().take(10).some(isNegative)); // false
console.log(fibonacci().some(isNegative)); // Never completes
```

The method closes the iterator after returning.

```js
const seq = fibonacci();
console.log(seq.some(isEven)); // true
console.log(seq.next()); // { value: undefined, done: true }
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.some

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
some	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.some`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.some` in `core-js`
- es-shims polyfill of `Iterator.prototype.some`
- `Iterator`
- `Iterator.prototype.every()`
- `Iterator.prototype.find()`
- `Array.prototype.some()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.take()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/take · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`take()`** method of `Iterator` instances returns a new iterator helper object that yields the given number of elements in this iterator and then terminates.

Syntax

```js
take(limit)
```

Parameters

`limit`

The number of elements to take from the start of the iteration.

Return value

A new iterator helper object. The returned iterator helper yields the elements in the original iterator one-by-one, and then completes (the `next()` method produces `{ value: undefined, done: true }`) once `limit` elements have been yielded, or when the original iterator is exhausted, whichever comes first.

Exceptions

`RangeError`

Thrown if `limit` becomes `NaN` or negative when converted to an integer.

Examples

Using take()

The following example creates an iterator that yields terms in the Fibonacci sequence, and then logs the first three terms:

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const seq = fibonacci().take(3);
console.log(seq.next().value); // 1
console.log(seq.next().value); // 1
console.log(seq.next().value); // 2
console.log(seq.next().value); // undefined
```

Using take() with a for...of loop

`take()` is most convenient when you are not hand-rolling the iterator. Because iterators are also iterable, you can iterate the returned helper with a `for...of` loop:

```js
for (const n of fibonacci().take(5)) {
  console.log(n);
}

// Logs:
// 1
// 1
// 2
// 3
// 5
```

Because `fibonacci()` is an infinite iterator, using a `for` loop to iterate it without any logic to exit early (such as a `break` statement) would result in an infinite loop.

Combining drop() with take()

You can combine `take()` with `Iterator.prototype.drop()` to get a slice of an iterator:

```js
for (const n of fibonacci().drop(2).take(5)) {
  // Drops the first two elements, then takes the next five
  console.log(n);
}
// Logs:
// 2
// 3
// 5
// 8
// 13

for (const n of fibonacci().take(5).drop(2)) {
  // Takes the first five elements, then drops the first two
  console.log(n);
}
// Logs:
// 2
// 3
// 5
```

Lower and upper bounds of take count

When the `limit` is negative or `NaN`, a `RangeError` is thrown:

```js
fibonacci().take(-1); // RangeError: -1 must be positive
fibonacci().take(undefined); // RangeError: undefined must be positive
```

When the `limit` is larger than the total number of elements the iterator can produce (such as `Infinity`), the returned iterator helper has essentially the same behavior as the original iterator:

```js
for (const n of new Set([1, 2, 3]).values().take(Infinity)) {
  console.log(n);
}

// Logs:
// 1
// 2
// 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.take

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`take`	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.take`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.take` in `core-js`
- es-shims polyfill of `Iterator.prototype.take`
- `Iterator`
- `Iterator.prototype.drop()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.toArray()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/toArray · MDN page last modified 2026-10-26

Baseline 2025: Newly available — works across the latest browser versions since March 2025; may not work in older ones.

The **`toArray()`** method of `Iterator` instances creates a new `Array` instance populated with the elements yielded from the iterator.

Syntax

```js
toArray()
```

Parameters

None.

Return value

A new `Array` instance containing the elements from the iterator in the order they were produced.

Examples

Using toArray()

`iterator.toArray()` is equivalent to `Array.from(iterator)` and `[...iterator]`, except that it's easier to chain when multiple iterator helper methods are involved. The following example creates an iterator that yields terms in the Fibonacci sequence, takes the first 10 terms, filters out the odd numbers, and converts the result to an array:

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const array = fibonacci()
  .take(10)
  .filter((x) => x % 2 === 0)
  .toArray();

console.log(array); // [2, 8, 34]
```

Note that it's a good idea to call `toArray()` as a last step of your processing. For example, `fibonacci().take(10).toArray().filter(...)` is less efficient, because iterator helpers are lazy and avoids creating a temporary array.

Specifications

Specification
ECMAScript® 2027 Language Specification `https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype.toarray`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`toArray`	122[1]	122[1]	131	108[2]	18.4	122[1]	131	81[3]	18.4	26.0[4]	122[1]	18.4	1.1.31	1.39	22.0.0

1. Also from version 117–119: supported.
2. Also from version 103–105: supported.
3. Also from version 78–79: supported.
4. Also from version 24.0–25.0: supported.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.toArray`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.toArray` in `core-js`
- es-shims polyfill of `Iterator.prototype.toArray`
- `Iterator`
- `Array.from()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype.windows()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/windows · MDN page last modified 2026-10-26

> **Experimental: This is an experimental technology**
> Check the Browser compatibility table carefully before using this in production.

The **`windows()`** method of `Iterator` instances returns a new iterator helper object that yields a sliding window of elements. Each time the helper is iterated, it yields an array removing the first element from the previous iteration and adding the next element from the original iterator.

To yield distinct chunks, see `Iterator.prototype.chunks()`.

Syntax

```js
windows(windowSize)
windows(windowSize, underSized)
```

Parameters

`windowSize`

The number of elements in the sliding window. Must be a positive integer less than 2^{32} (the maximum array length).

`underSized` Optional

A string specifying what to do when the number of elements in the iterator is less than `windowSize` but greater than 0. It can be one of the following:

`"only-full"` (default)

Ignore the partial window. The returned iterator immediately completes as if the input is empty.

`"allow-partial"`

Yield all elements in the partial window as an array (with length less than `windowSize`).

Note that when the input is empty, the returned iterator always immediately completes without yielding an empty array.

Return value

A new iterator helper object. The first time the returned iterator helper's `next()` method is called, the current iterator is immediately advanced by `windowSize` elements. If that many elements are available, they are yielded together as an array. Otherwise, the behavior depends on the `underSized` argument: the iterator either immediately completes without yielding anything (with `"only-full"` or with a fully empty input), or an array of length less than `windowSize` is yielded.

Afterwards, each time, the yielded array consists of the previous array, the leftmost element removed, a new element retrieved from the input iterable appended. When the input iterable is exhausted, the iterator helper also completes.

Exceptions

`TypeError`

Thrown if `limit` is not an integer, or if `underSized` is not one of the allowed values.

`RangeError`

Thrown if `limit` is less than 1 or greater than $2^{32} - 1$.

Examples

Moving average

The following example creates an iterator that yields terms in the Fibonacci sequence. Then, a new iterator is created, each time yielding the average of two adjacent elements.

```js
function* fibonacci() {
  let current = 1;
  let next = 1;
  while (true) {
    yield current;
    [current, next] = [next, current + next];
  }
}

const averaged = fibonacci()
  .windows(2)
  .map((window) => (window[0] + window[1]) / 2);

console.log(averaged.next().value); // 1 (1, 1)
console.log(averaged.next().value); // 1.5 (1, 2)
console.log(averaged.next().value); // 2.5 (2, 3)
console.log(averaged.next().value); // 4 (3, 5)
```

This process, known as a moving average, is very useful in signal processing.

Enumerating substrings

The following example enumerates all substrings of a given length in a string. The `windows()` method is used to create a sliding window of characters, which are then joined together to form substrings.

> **Note:** This is probably much less efficient than manually iterating a number index and calling the `String.prototype.substring()` method.

```js
const chars = Iterator.from("MASSACHUSETTS");

const substrings = chars.windows(3).map((window) => window.join(""));

for (const substring of substrings) {
  console.log(substring);
}
// MAS
// ASS
// SSA
// ...
```

Specifications

Specification
Iterator Chunking `https://tc39.es/proposal-iterator-chunking/#sec-iterator.prototype.windows`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Nodejs
`windows` Experimental	**No**	**No**	154	**No**	Prev.	**No**	154	**No**	**No**	**No**	**No**	**No**	1.4.0	?	?

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.windows`*. Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- Polyfill of `Iterator.prototype.windows` in `core-js`
- es-shims polyfill of `Iterator.prototype.windows`
- `Iterator`
- `Iterator.prototype.chunks()`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype[Symbol.dispose]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/Symbol.dispose · MDN page last modified 2026-10-26

Limited availability — not Baseline: does not work in some of the most widely-used browsers.

The **`[Symbol.dispose]()`** method of `Iterator` instances implements the *disposable protocol* and allows it to be disposed when used with `using`. It calls the `return()` method of `this`, if it exists.

Syntax

```js
iterator[Symbol.dispose]()
```

Parameters

None.

Return value

None (`undefined`).

Examples

Declaring an iterator with `using`

The `Symbol.dispose` method is intended to be automatically called in a `using` declaration. This is useful if you have an iterator that you manually iterate over by calling its `next()` method; if you iterate it with `for...of` or something similar, then error handling and cleanup is done automatically.

```js
function* generateNumbers() {
  try {
    yield 1;
    yield 2;
    yield 3;
  } finally {
    console.log("Cleaning up");
  }
}

function doSomething() {
  using numbers = generateNumbers();
  const res1 = numbers.next();
  // Not iterating the rest of the numbers
  // Before the function exits, the async iterator is disposed
  // Logs "Cleaning up"
}

doSomething();
```

Specifications

Specification
ECMAScript Async Explicit Resource Management `https://tc39.es/proposal-async-explicit-resource-management/#sec-%iteratorprototype%-@@dispose`

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
`@@dispose`	134	134	141	119	Prev.	134	141	88	**No**	29.0	134	**No**	1.3.0	2.2.10	24.0.0

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.@@dispose`. Numbers are the first version with support; see "How to read this book" for the full legend.

See also

- Polyfill of `Iterator.prototype[Symbol.dispose]` in `core-js`
- JavaScript resource management
- `Symbol.dispose`
- `using`

JavaScript › Reference › Standard built-in objects › Iterator

Iterator.prototype[Symbol.iterator]()

Source: developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Iterator/Symbol.iterator · MDN page last modified 2025-07-10

Baseline: Widely available — available across major browsers since September 2016.

The **`[Symbol.iterator]()`** method of `Iterator` instances implements the iterable protocol and allows built-in iterators to be consumed by most syntaxes expecting iterables, such as the spread syntax and `for...of` loops. It returns the value of `this`, which is the iterator object itself.

Syntax

```
iterator[Symbol.iterator]()
```

Parameters

None.

Return value

The value of `this`, which is the iterator object itself.

Examples

Iteration using for...of loop

Note that you seldom need to call this method directly. The existence of the `[Symbol.iterator]()` method makes built-in iterators iterable, and iterating syntaxes like the `for...of` loop automatically call this method to obtain the iterator to loop over.

```
const arrIterator = [1, 2, 3].values();
for (const value of arrIterator) {
  console.log(value);
}
// Logs: 1, 2, 3
```

Specifications

Specification
ECMAScript® 2027 Language Specification https://tc39.es/ecma262/multipage/control-abstraction-objects.html#sec-iterator.prototype-%symbol.iterator%

Browser compatibility

Feature	Desktop					Mobile							Server		
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Browser	WebView Android	WebView on iOS	Bun	Deno	Node.js
[Symbol.iterator]	38	12	36[1,2,3,4]	25	10	38	36[1,2,3,4]	25	10	3.0	38	10	1.0.0	1.0	0.12.0

1. Also from version 27–36: uses the non-standard name `@@iterator`.
2. Also from version 27–36: A placeholder property named `@@iterator` is used.
3. Also from version 17–27: uses the non-standard name `iterator`.
4. Also from version 17–27: A placeholder property named `iterator` is used.

Compatibility data: MDN browser-compat-data 8.1.8, `javascript.builtins.Iterator.@@iterator`. *Numbers are the first version with support; see "How to read this book" for the full legend.*

See also

- `Iterator`
- `Symbol.iterator`
- Iteration protocols